Oblique-Projector Recurrences, Multiplicative Units and an Analytic Discriminant Block

Exact expressive limits, capacity results and calibrated rank selection for recursive oblique projection networks (ROPN)

A technical report: theorems, worked examples, experiments and code

Olivier Caspary
Université de Lorraine, LCOMS, F-57000 Metz, France
olivier.caspary@univ-lorraine.fr

November 12, 2026

Abstract

A recursive oblique projection network (ROPN) is a residual network whose linear maps are oblique projectors $P = U(W^\top U)^{-1}W^\top$: a *representation* subspace $\operatorname{im} U$ and a *measurement* subspace $\operatorname{im} W$ that need not coincide. We characterise exactly the expressive limits of recurrences built on oblique projectors, establish capacity results for multiplicative units, and construct an analytic discriminant realisation with calibrated rank selection. *Recurrences.* For a shared oblique block, the linear recursion collapses to $I + c_L P$; the oblique code is invariant under the scalar-gated recursion, whose gate is constant along the orbit when it reads inside $\operatorname{span}(W)$; the gated block has an explicit Jacobian determinant; and with r code coordinates and m independent gate directions in the kernel, the non-linearity has at most $r + m$ linear functionals of the input as arguments, whatever the number of steps. Depth therefore does not enlarge the information accessible to the non-linearity; it can still change the function computed on it. *Multiplicative units.* A gated unit with decoupled gate and value directions (the triceptron, a gated linear unit) contains SiLU units, has ReLU units as uniform limits, approximates every quadric of inertia (p, n) at fixed width $\max(p, n)$, and obeys the cell-count bound of a ReLU layer; a ReLU layer needs a width that grows with the accuracy required on a disc. With the same trainer for both families, the minimal widths predicted by the theorem are observed, and no accuracy difference between the two families is established beyond about thirty parameters. *Analytic discriminant block.* The Fisher generalised eigenproblem gives a biorthogonal pair (U, W) and an exactly idempotent projector; with a centroid read-out the block is exactly linear discriminant analysis (LDA) with the same covariance estimate, an equivalence stated explicitly and used throughout. Its specific content is an exact $C \times C$ factorisation with a dual form for $n < d$, a bound on the projector norm through the condition number of the metric, and a rank selection in which the structural, tested and used ranks are kept apart: the permutation test in a label-free metric is calibrated under the global null hypothesis, a sequential variant is evaluated under alternatives of every rank, and a min-p procedure controls the family-wise error across blocks. The tested rank is a detection statement, not a risk-optimal choice: in our experiments, using the structural rank $C - 1$ for prediction was within 0.2 accuracy point of using the true rank, whereas using a tested rank cost up to nine points when the test lacked power. *Evidence.* The theorems are proved; the experiments use synthetic data and small public datasets, with paired comparisons, confidence intervals and a pre-specified equivalence margin. In them the block is equivalent to shrinkage LDA, as it must be, ahead of logistic regression and MLP baselines on small near-Gaussian samples, tuned or not, and behind the MLP on the real digits data from thirty samples per class (behind logistic regression at one hundred). Gradient fine-tuning of the block, including variants in which both subspaces move, showed no net gain in the protocol tested. No experiment on real EEG data is reported. Exploratory extensions

(lattices, Kronecker metrics, kernels, the SPD manifold, Fourier operators) are collected in appendices and carry no general claim. The complete `numpy` code and the logs of every experiment are included.

Contents

1 Introduction **5**
- 1.1 Scope and positioning . . . 5
- 1.2 Status of the claims . . . 5
- 1.3 Terminology . . . 6
- 1.4 Notation . . . 6

2 Oblique projectors: definitions and geometry **6**

3 Exact expressive limits of oblique-projector recurrences **7**
- 3.1 Collapse of the linear recursion . . . 7
- 3.2 Invariance of the oblique code under a gated recursion . . . 8
- 3.3 An exact Jacobian determinant for the gated block . . . 9
- 3.4 Reduction of gated depth to $r + m$ coordinates . . . 10

4 Capacity of multiplicative units: the triceptron **13**
- 4.1 Quadric capacity of the triceptron . . . 13
- 4.2 Experiments aligned with the theorem . . . 17
- 4.3 Summary of the two capacity results . . . 19

5 An analytic discriminant block **19**
- 5.1 From Fisher's rank-one direction to a rank-r projector . . . 19
- 5.2 Re-whitening: exact idempotence . . . 20
- 5.3 Equivalence with linear discriminant analysis, and the read-out . . . 20
- 5.4 An analytic gate . . . 21
- 5.5 The reduced architecture . . . 22

6 Exact rank-$(C-1)$ factorisation of the Fisher problem **22**
- 6.1 The equivalent $C \times C$ problem . . . 22
- 6.2 Dual form for $n < d$: no $d \times d$ matrix . . . 23
- 6.3 Batched permutation test . . . 23
- 6.4 Further options . . . 23
- 6.5 Against a specialised LDA solver . . . 24

7 A complete worked example of the analytic block **25**

8 Rank selection: structural, tested and used ranks **27**
- 8.1 The rules . . . 28
- 8.2 Calibration under the global null hypothesis . . . 28
- 8.3 Rank recovery under alternatives . . . 29
- 8.4 Rank error is not predictive risk . . . 30
- 8.5 Families of tests: per-block and family-wise control . . . 31

9 Limits of oblique projectors and their control in the analytic block **31**
- 9.1 Noise amplification: norm and angle . . . 32
- 9.2 The interpolation threshold: where the raw projector's norm peaks . . . 32
- 9.3 What bounds the obliquity: the metric . . . 33

9.4 Best approximation: not Euclidean, but in the Fisher metric . . . 34
9.5 Information loss and the residual form . . . 35
9.6 Regularising the cross-Gram versus regularising the metric . . . 35
9.7 Expressivity and depth . . . 36

10 What each ingredient contributes 36

11 Central benchmark: symmetric comparisons with paired uncertainty 37
11.1 Protocol . . . 37
11.2 Results . . . 37

12 The small-sample regime: diagnosis, quantification, remedies 39
12.1 Ablation: where is the accuracy lost? . . . 40
12.2 Geometric quantification of the loss . . . 41
12.3 Gradient fine-tuning of the block: what is free, and what is observed . . . 41
12.4 The recipe for accuracy . . . 44
12.5 Anisotropy sweep . . . 44

13 Audit of the classical risks of oblique blocks with rank selection 45

14 Limitations 47

15 Conclusion 48

A Exploratory extensions: status and reading guide 49

B Benchmark of the reference implementation on small datasets 49

C Lattices of analytic blocks with fixed non-linearities 50
C.1 Architecture . . . 51
C.2 Verdict . . . 51
C.3 Advantages and drawbacks relative to the MLP . . . 52

D Doubly oblique geometry: Kronecker metrics for matrix-valued inputs 52

E Alternatives to the MLP 53
E.1 The kernel oblique block . . . 53
E.2 Verdict . . . 53
E.3 What remains unexplored . . . 54

F Language modelling 54

G The Euclidean cross-Gram 55

H Riemannian geometry: data on the SPD manifold 56
H.1 Data on the SPD manifold: the analytic block in tangent-space coordinates . . . 56
H.2 What each algorithm becomes . . . 58
H.3 Experiment: covariances versus means, and session change . . . 59

I Fourier operators as banks of per-mode analytic problems 60
I.1 Structure: a spectral block is a bank of K independent $C \times C$ problems . . . 60
I.2 Experiments . . . 61
I.3 Interest and contribution . . . 63

J Python code **63**

Acknowledgments **133**

References **133**

1 Introduction

1.1 Scope and positioning

Oblique projection networks replace the linear maps of a residual network by oblique projectors and are normally trained by gradient descent, with gates that select the rank of each block and the depth of the stack. This report studies what can be said *exactly* about such networks, and what can be built *without* back-propagation. It does not propose a new architecture, and it does not claim an empirical advantage over established classifiers. Its three contributions are of different natures and are kept apart throughout.

1. **Exact expressive limits of oblique-projector recurrences** (Section 3). For a shared block built on one projector, four results describe what repeated application can and cannot do: collapse of the linear recursion (Theorem 1), invariance of the code and constancy of a gate reading inside $\mathrm{span}(W)$ (Proposition 1), an explicit Jacobian determinant (Theorem 2), and the reduction of the arguments of the non-linearity to $r+m$ linear functionals (Theorem 3). The precise conclusion is: *depth does not enlarge the subspace of information accessible to the non-linearity; it can nevertheless modify the function computed on that subspace.*

2. **Capacity results for multiplicative units** (Section 4). Theorem 4 gives the inclusion of SiLU/ReLU units, the exact number of units needed for a quadric, a cell count, and a necessary width for ReLU layers on a disc. The experiments of that section use the same trainer for gated and ReLU networks, with and without the linear skip of the theoretical construction.

3. **An analytic discriminant realisation with calibrated rank selection** (Sections 5–8). The block is the Fisher discriminant written as an oblique projector. With a centroid read-out it is *exactly* LDA with the same covariance estimate (Proposition 3); what is specific is the exact $C \times C$ factorisation and its dual form (Proposition 4), the control of the projector norm by the metric (Proposition 6), and a rank selection whose calibration is established under the global null hypothesis and measured under alternatives (Section 8).

Three ingredients of the block must not be confused (Section 10): the *Fisher metric* is what brings prediction accuracy; the *factorisation* is what brings computational savings; *idempotence* brings exact structural statements (the theorems above, the norm identities) and no accuracy by itself.

1.2 Status of the claims

Proved	**Measured, in the stated protocols**	**Suggested, not established**
Theorems 1–4, Propositions 1–6 and their corollaries; exact level of the permutation test under the global null hypothesis; exact equivalence of the block with LDA.	Calibration and rank recovery (Section 8); accuracy relative to LDA, logistic regression and MLPs with paired confidence intervals (Section 11); timings; small-sample diagnosis and fine-tuning (Section 12); parity of gated and ReLU networks under one trainer (Section 4).	Usefulness on real EEG data (not tested); behaviour of the extensions of Appendices A–I beyond the favourable synthetic cases examined; any statement about well-tuned large networks.

An absence of measured gain is reported as such ("no net gain observed in the protocol tested"), never as a proof of uselessness; a difference smaller than its uncertainty is reported as undetermined, not as an equality (Section 11 gives the convention).

1.3 Terminology

Back-propagation-free qualifies the programme as a whole: no parameter is obtained by gradient descent. *Analytic* is reserved for sub-problems solved by a finite sequence of standard linear-algebra operations with no tuning loop: the block given its covariance estimate, the $C \times C$ reduction, the centroid read-out. Several components of the appendices are back-propagation-free without being analytic (the Karcher mean, the alternating estimation of a Kronecker metric, internal cross-validation). *Rank selection* replaces the looser "self-sizing": the *structural* rank is the bound $\min(C-1, d)$, the *tested* rank is the output of a statistical rule and may be zero, the *used* rank is the one the classifier actually employs.

1.4 Notation

Data $x \in \mathbb{R}^d$, n training samples, C classes with sizes n_c, class means μ_c, grand mean $\bar{\mu}$. Within-class scatter $S_W = \frac{1}{n}\sum_c \sum_{i \in c}(x_i - \mu_c)(x_i - \mu_c)^\top$ (pooled; the unshrunk baseline of Section 12 divides by $n - C$ instead of n), between-class scatter $S_B = \sum_c n_c(\mu_c - \bar{\mu})(\mu_c - \bar{\mu})^\top/(C-1)$ (of rank at most $C-1$), total scatter $S_T = \frac{1}{n}\sum_i (x_i - \bar{\mu})(x_i - \bar{\mu})^\top = S_W + \frac{C-1}{n} S_B$, which does not depend on the labels. An oblique projector of rank r is $P = U(W^\top U)^{-1} W^\top$ with $U, W \in \mathbb{R}^{d \times r}$ of full rank and $W^\top U$ invertible; in biorthogonal form $W^\top U = I_r$ and $P = UW^\top$. The *code* of h is $c = W^\top h \in \mathbb{R}^r$ and its *residual* is $\rho = (I - P)h \in \ker P$, so that $h = Uc + \rho$ uniquely. A projector is M-orthogonal for a positive definite M when $W = MU$; the Fisher block is S_W^{-1}-orthogonal, hence oblique in the Euclidean sense and orthogonal in the Fisher metric. σ denotes the logistic sigmoid; $\kappa(S_W)$ the condition number of the within-class scatter, used as an anisotropy knob in the synthetic experiments; "LW" the Ledoit–Wolf shrinkage estimator. All experiments use `numpy`, `scipy` and `scikit-learn` only; every gradient used in the comparisons is written by hand and verified by finite differences. Unless stated otherwise, accuracies are test accuracies on stratified 70/30 splits, reported as mean ± standard deviation over seeds (the number of seeds is given with each table). In Sections 8, 11, 12, 9 and 13, and in Table 1, every seed draws a new synthetic dataset; in the benchmark tables of Appendices B, C, E and G and in Table 2 each dataset (synthetic or real) is fixed and the seeds are five different splits, so that the small synthetic cases rest on 5×18 test predictions and their standard deviations measure split variability only. Appendices H and I use unstratified 50/50 splits of independent draws. Comparisons between methods in Sections 4, 8, 11 and 12 are paired on the same realisations and reported with a 95% interval and a verdict relative to an equivalence margin of one accuracy point (convention in Section 11). The tables of the appendices report means and standard deviations only: there, differences smaller than the quoted standard deviations must be read as undetermined. Timings are wall-clock times on the CPU of the execution sandbox, vary from run to run, and are indicative only. Shrinkage follows Ledoit and Wolf [1]. A few symbols are reused with a local meaning, always stated where it applies: L is the residual scale (and number of steps) of a recursion, T a number of recursion steps (the number of time samples of a trial in Appendices D and H), M a generic metric (the $C \times C$ matrix $D^\top S^{-1} D$ in Section 6), μ without index the Ledoit–Wolf target scale $\operatorname{tr} S_W/d$, and a either the gate value or the Ledoit–Wolf intensity.

2 Oblique projectors: definitions and geometry

Figure 1 shows the object this report is about, in the plane. An oblique projector is determined by two subspaces: its image $\operatorname{span}(u)$, where the projected points land, and its kernel $\operatorname{span}(w)^\perp$, the direction *along which* points are moved to the image. The measurement direction w is orthogonal to the kernel, not to the image: the code $w^\top x$ reads how much of x lies "across" the kernel, and $Px = u\, w^\top x$ places that amount on the image. When $w \propto u$ the projector is orthogonal and Px is the nearest point of the image; otherwise Px is further from x than

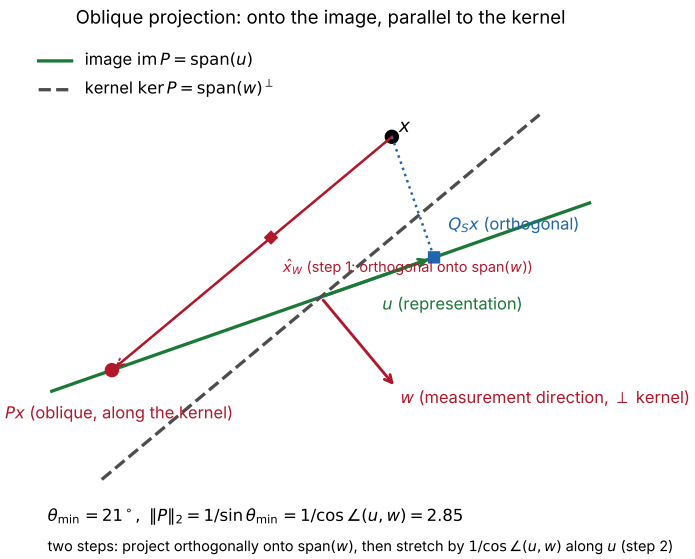


Figure 1: An oblique projector in the plane: image (green), kernel (grey, dashed), representation direction u and measurement direction w (orthogonal to the kernel). The point x is sent to Px along the kernel direction; the orthogonal projection $Q_S x$ onto the same image is shown for comparison. The two-step reading: x is first projected orthogonally onto span(w) (diamond, $\hat{x}_W$), then carried to the image with the stretch factor $1/\cos\angle(u,w)$. The norm $\|P\|_2 = 1/\sin\theta_{\min} = 1/\cos\angle(u,w)$ grows as the kernel approaches the image.

the orthogonal projection by at most the factor $\|P\|_2 = 1/\sin\theta_{\min}$ (Section 9), and the same factor bounds the amplification of input perturbations. In a classifier the image is where the class means differ and the kernel is where the within-class noise lives: an oblique projector can discard noise directions that are *not* orthogonal to the signal, which an orthogonal projector cannot.

A second reading, due to Rocks and Mehta [2] in the context of over-parameterised regression, is useful throughout this report. Write $P = Q_U(Q_W^\top Q_U)^{-1}Q_W^\top$ with orthonormal bases Q_U, Q_W of the image and of the measurement subspace. Since $Q_W^\top x = Q_W^\top(Q_W Q_W^\top x)$, the projection proceeds in *two steps*: x is first projected *orthogonally* onto the measurement subspace span(W), giving $\hat{x}_W = Q_W Q_W^\top x$ (the only part of x the block reads: the code $W^\top x$ depends on x through $\hat{x}_W$ alone), and $\hat{x}_W$ is then re-expressed in the representation subspace span(U). The second step is where obliquity acts: if $\theta_1 \le \cdots \le \theta_r$ are the principal angles between span(U) and span(W), the non-zero singular values of P are exactly $1/\cos\theta_i$, so that each measured direction is stretched by the factor $1/\cos\theta_i$ on its way to the image, and $\|P\|_2 = 1/\cos\theta_r = 1/\sin\theta_{\min}$ (the two angles are complementary, since $\ker P = \text{span}(W)^\perp$). When $U = W$ all angles vanish, every factor is one and the projector is orthogonal. Verified on the analytic block ($d = 20$, $C = 4$, $\kappa = 200$): singular values $(1.5469, 1.4340, 1.1861)$ against $1/\cos\theta_i$ for principal angles $(49.7°, 45.8°, 32.5°)$, and $W^\top x = W^\top \hat{x}_W$ to 3×10^{-16}. Figure 1 shows both steps.

3 Exact expressive limits of oblique-projector recurrences

3.1 Collapse of the linear recursion

Theorem 1 (Collapse of a recurrent idempotent oblique block)**.** *Let $P = UW^\top$ be idempotent ($W^\top U = I_r$) and $\mathcal{T}_L = (I + P/L)^L$ the operator obtained by applying L times the* same *residual*

block $h \mapsto h + Ph/L$. *Then, for every* $L \geq 1$,

$$\mathcal{T}_L = I + c_L\, P, \qquad c_L = (1 + \tfrac{1}{L})^L - 1, \tag{1}$$

exactly.

Proof. P idempotent induces the oblique direct sum $\mathbb{R}^d = \operatorname{im} P \oplus \ker P$. On $\ker P$ the block is the identity; on $\operatorname{im} P$ it is multiplication by $1+1/L$. Hence $\mathcal{T}_L x = (I-P)x + (1+1/L)^L Px$. □

Numerically, the brute-force iteration agrees with (1) to a relative error below 1.2×10^{-15} for $L \in \{1, \ldots, 256\}$ ($c_1 = 1$, $c_2 = 1.25$, $c_4 = 1.441406$, $c_8 = 1.565785$, $c_{16} = 1.637928$, $c_{64} = 1.697345$, $c_{256} = 1.712992$, converging to $e - 1$).

3.2 Invariance of the oblique code under a gated recursion

It is tempting to believe that a non-linear gate restores genuine depth, because $\|h_T - h_0\|$ keeps evolving with T. The evolution is real, but it is also in closed form.

Proposition 1 (Code invariance and gate collapse)**.** *Let* $h_{t+1} = (1 - a_t)h_t + a_t P h_t$ *with* $a_t = \sigma(s^\top h_t + b)$ *(shared weights).*

(i) ***Code invariance.*** *For every* T *and every sequence* (a_t)*:* $W^\top h_T = W^\top h_0$*. The recursion moves* h *only along* $\ker W^\top$*, the direction* along which *one projects.*

(ii) ***Gate collapse.*** *If* $s \in \operatorname{span}(W)$ *(the analytic gate,* $s \propto w_1$*), then* $a_t = a_0 =: a(x)$ *for all* t *and*

$$h_T = Px + \big(1 - a(x)\big)^T (I - P)x. \tag{2}$$

(iii) ***Converse.*** *The gate value* a_t *is constant along the orbit of* every *input if and only if* $s \in \operatorname{span}(W)$*. Otherwise the gate reads the residual* $(I-P)h_t \in \ker P$*, the component that the recursion contracts, and the recursion reduces to the scalar recursion of Theorem 3(vi), for which no closed form in* T *is given here.*

Proof. (i) $h_{t+1} - h_t = a_t(P - I)h_t$ and $W^\top(P - I) = W^\top U W^\top - W^\top = 0$. (ii) $s \in \operatorname{span}(W) \Rightarrow s^\top(P - I) = 0 \Rightarrow s^\top h_{t+1} = s^\top h_t$, so a_t is constant; by (i) $Ph_t = Px$ and $(I - P)h_{t+1} = (1 - a)(I - P)h_t$. (iii) $s^\top(P - I) = 0 \iff P^\top s = s \iff s \in \operatorname{im} P^\top = \operatorname{span}(W)$. If $s \notin \operatorname{span}(W)$, choose h_0 with $s^\top(P-I)h_0 \neq 0$; then $s^\top h_1 - s^\top h_0 = a_0\, s^\top(P-I)h_0 \neq 0$ because $a_0 > 0$, and $a_1 \neq a_0$ because σ is injective. □

Verified ($d = 12$, $r = 3$, $T \in \{1, 4, 16\}$): $\|W^\top h_T - W^\top x\| \leq 7 \times 10^{-14}$ for gates inside and outside $\operatorname{span}(W)$, and $\|h_T - [Px + (1 - a)^T(I - P)x]\| \leq 3 \times 10^{-15}$ for a gate inside $\operatorname{span}(W)$; with a gate direction that reads only the residual, a_t varies by 0.34 along the orbit ($T = 16$).

Consequences. (a) A read-out connected to the code $c = W^\top h_T$ sees *exactly the same thing* for every T. (b) With the analytic gate, T only sets how strongly the residual is damped: $(1 - a(x))^T \to 0$ for every x, faster where $a(x)$ is large, which pushes h_T towards Px and hurts a read-out on h_T in small samples (conditioning, Section 12). (c) For this block, a dependence of the gate on t requires a gate reading outside $\operatorname{span}(W)$: a design choice, not an automatic property of the non-linearity. (d) The answer to the parameter-saving question is therefore two-sided: shared-weight recursion does save a factor L ($\mathcal{O}(Ldr) \to \mathcal{O}(dr)$), but with this gate the T steps are equal to a *single* step with the gate $a_T(x) = 1 - (1 - a(x))^T$ (equation (2)). The number of steps therefore changes the function computed on the residual, through a scalar that depends on the code only; it gives the non-linearity access to no new coordinate of the input, and it does not change a read-out on the code at all.

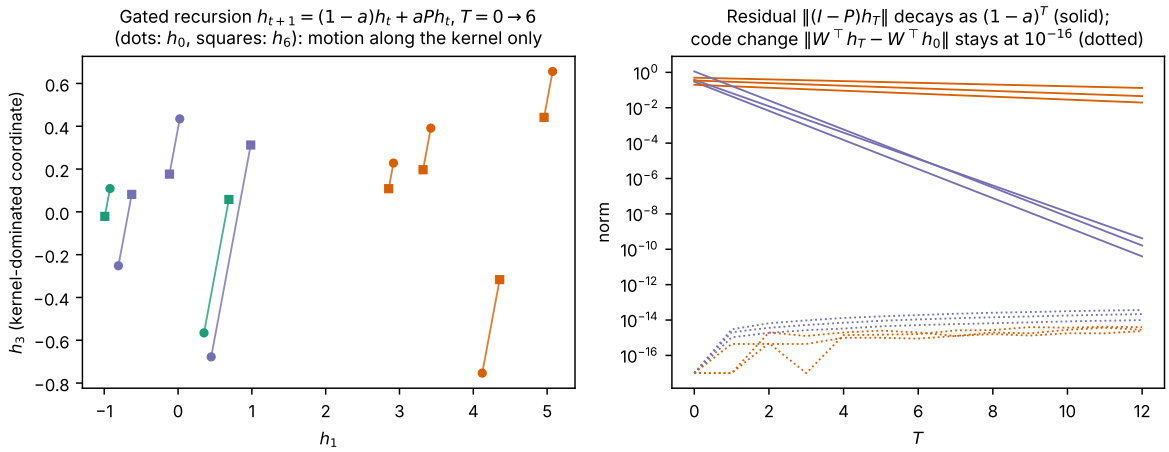


Figure 2: Proposition 1 on the worked example of Section 7. Left: trajectories $h_0 \to h_6$ of nine test points under the gated recursion, plotted in the coordinates (h_1, h_3); all segments are parallel, since the motion is confined to the one-dimensional kernel. Right: the residual norm decays geometrically with a point-dependent rate $1-a(x)$ (solid), while the change of the code stays at machine precision (dotted) for every T.

3.3 An exact Jacobian determinant for the gated block

For the linear stack, $\det \mathcal{T} \neq 0$ is the Liouville–Jacobi guarantee against representational collapse; with a non-linearity the guarantee is usually only local. For the gated oblique block the determinant is available in closed form.

Theorem 2 (Exact determinant of the gated oblique block)**.** *Let $P = UW^\top$ be idempotent of rank r and $F(h) = h + \frac{a(h)}{L}(P-I)h$, $a(h) = \sigma(s^\top h + b) \in (0,1)$, $L \geq 1$. Then*

$$\det \nabla F(h) = \left(1-\tfrac{a}{L}\right)^{d-r}\left(1-\frac{a(1-a)}{L-a}\, s^\top (I-P)h\right) = \left(1-\tfrac{a}{L}\right)^{d-r-1}\left(1-\tfrac{a}{L}-\tfrac{a(1-a)}{L}\, s^\top (I-P)h\right). \tag{3}$$

In particular, for $L = 1$, $\det \nabla F = (1-a)^{d-r}(1-a\, s^\top (I-P)h)$, and F is a local diffeomorphism at h if and only if $a\, s^\top (I-P)h \neq 1$.

Proof. $\nabla F = B + \frac{1}{L}(P-I)h\,(\nabla a)^\top$ with $B = (1-\frac{a}{L})I + \frac{a}{L}P$ and $\nabla a = a(1-a)s$. B has eigenvalue 1 on $\operatorname{im} P$ and $1-\frac{a}{L}$ on $\ker P$. Since $(P-I)h \in \ker P$, $B^{-1}(P-I)h = (P-I)h/(1-\frac{a}{L})$, and the matrix determinant lemma [3] gives $\det \nabla F = \det B\,(1 + \frac{1}{L}(\nabla a)^\top B^{-1}(P-I)h) = (1-\frac{a}{L})^{d-r}(1-\frac{a(1-a)}{L-a}s^\top(I-P)h)$. □

Verified by central finite-difference Jacobians ($d = 12$, $r = 3$): agreement to the eight digits displayed for general s ($-5.1138954 \times 10^{-3}$ for $L = 1$, 5.2176658×10^{-2} for $L = 2$); $\det = (1-a)^{d-r} = 0.76077713$ when $s \in \operatorname{span}(W)$; $|\det| < 10^{-14}$ on a case constructed with $a\, s^\top (I-P)h = 1$. The theorem quantifies the trade-off left qualitative by Proposition 1: for the gate to vary along the orbit it must read the residual ($s^\top (I-P)h \neq 0$), but for $L = 1$ it must not read it *too much* ($a\, s^\top (I-P)h < 1$) if the orientation is to be preserved. The residual normalisation $1/L$ relaxes the condition ($|s^\top (I-P)h| < L/a$ suffices for a positive determinant), and suggests a possible regulariser, $\max(0,\, a\, s^\top (I-P)h - 1 + \epsilon)$, which is not tested here.

Example 1 (Theorems 1 and 2 in dimension three). Take $d = 3$, $r = 1$, $u = (1,0,0)$, $w = (1,1,0)$, so $w^\top u = 1$, $P = uw^\top$ maps h to $(h_1 + h_2, 0, 0)$, $\ker P = \{h :\ h_1 + h_2 = 0\} = \operatorname{span}\{(-1,1,0), e_3\}$ and $(I-P)h = (-h_2, h_2, h_3)$. Gate $s = (0,0,1)$, $b = 0$: it reads h_3, entirely in the kernel ($s^\top (I-P)h = h_3$). At $h = (1, \frac{1}{2}, 2)$, $a = \sigma(2) = 0.8808$.

- $L = 1$: $\det \nabla F = (1-a)^2(1-2a) = 0.0142 \times (-0.7616) = -0.010822$ (finite differences: -0.010822). The determinant is *negative*: at $h_3 = 2$ the gate has already crossed the threshold $\sigma(h_3)h_3 = 1$, reached at $h_3^* = 1.2785$ (numerical determinant 2.4×10^{-11}); the block reverses orientation and is not injective near the plane $h_3 = h_3^*$ (its third coordinate, $h_3(1-\sigma(h_3))$, has a maximum there).

- $L = 2$: $\det \nabla F = (1-\frac{a}{2})^2(1-\frac{a(1-a)}{2-a}\cdot 2) = 0.3132 \times 0.8124 = +0.254399$ (finite differences: $+0.254399$). For this value of a the determinant would vanish only at $s^\top(I-P)h = (L-a)/(a(1-a)) = 10.66$, and the sufficient condition $|s^\top(I-P)h| < L/a = 2.27$ is met. In fact, since $a = \sigma(h_3)$ here and $\sup_x x\,\sigma'(x) = 0.224 < 1 \le L - a$, the second factor of (3) stays above 0.80 for every h_3: with $L = 2$ the determinant of this block never vanishes.

The same projector illustrates Theorem 1: $(I+P/2)^2 - I = 1.25P$ and $(I+P/2)^3 - I = 2.375P$ exactly; on $h = (1, \frac{1}{2}, 2)$ two linear steps give $(2.875, \frac{1}{2}, 2) = h + 1.25\,(1.5, 0, 0)$: only the code coordinate $h_1 + h_2 = 1.5$ has been touched, and only by a constant factor.

3.4 Reduction of gated depth to $r + m$ coordinates

Consider the block gated per code coordinate (the triceptron of Section 4 in residual form, i.e. $h + Ph/L$ with an input-dependent rank mask),

$$F(h) = h + \tfrac{1}{L}\,U\,\operatorname{diag}\big(a(h)\big)W^\top h, \qquad a(h) = \sigma(S^\top h + b) \in (0,1)^r, \quad S \in \mathbb{R}^{d\times r}, \tag{4}$$

and its shared-weight iterates $h_{t+1} = F(h_t)$, $h_T = F^{(T)}(h_0)$.

Theorem 3 (Reduction of gated depth)**.** *Let $A = S^\top U \in \mathbb{R}^{r\times r}$ and $\beta = S^\top(I-P)h_0 \in \mathbb{R}^r$.*

(i) ***Invariant residual****: $(I-P)h_t = (I-P)h_0$ for all t.*

(ii) ***Code recursion in dimension*** *r: $c_{t+1} = c_t \odot \big(1 + \frac{1}{L}\sigma(Ac_t + \beta + b)\big)$.*

(iii) ***Structure in*** *T: $h_T = U\Phi_T(c_0, \beta) + (I-P)h_0$, where Φ_T is the T-fold iterate of $\varphi_\beta(c) = c \odot (1 + \sigma(Ac + \beta + b)/L)$; the code $c_T = W^\top h_T$ depends on the input only through the $2r$ numbers $(W^\top h_0,\ S^\top(I-P)h_0)$, and the residual $(I-P)h_0$ is carried unchanged.*

(iv) ***Read-out****: for every linear read-out $h \mapsto V_{\mathrm{ro}}h + v_0$, $V_{\mathrm{ro}}h_T + v_0 = V_{\mathrm{ro}}U\,\Phi_T(W^\top h,\ S^\top(I-P)h) + V_{\mathrm{ro}}(I-P)h + v_0$: the non-linearity acts on $r+m$ linear functionals of the input only, with $m = \operatorname{rank}(S^\top(I-P)) \le r$, whatever T.*

(v) ***Case*** *$\operatorname{im} S \subset \operatorname{span}(W)$: $\beta = 0$, $\Phi_T(c, 0)$ is a fixed reparametrisation of the code and the kernel $\ker W^\top$ enters the output only linearly, for every T (this generalises Proposition 1(ii)).*

(vi) ***Scalar variant*** *$F(h) = h + \frac{a(h)}{L}(P-I)h$, $a = \sigma(s^\top h + b)$: the code is invariant and $(I-P)h_T = \Lambda_T(c_0,\ s^\top(I-P)h_0)\,(I-P)h_0$ with Λ_T scalar: the* direction *of the residual is fixed and the kernel enters through a single functional, whatever s. For $s \notin \operatorname{span}(W)$ no closed form in T is given, but the recursion is a scalar one.*

Proof. (i) $(I-P)U = U - UW^\top U = 0$, so $(I-P)F(h) = (I-P)h$. (ii) $W^\top F(h) = c + \frac{1}{L}W^\top U\,\operatorname{diag}(a)W^\top h = c + \frac{1}{L}a \odot c$, and $S^\top h = S^\top Uc + S^\top\rho = Ac + \beta$ with ρ constant by (i). (iii)–(v) follow: $h_T = Uc_T + \rho_0$ with $c_T = \Phi_T(c_0, \beta)$; if $S = WB$ then $S^\top(I-P) = B^\top W^\top(I-P) = B^\top(W^\top - W^\top UW^\top) = 0$. (vi) $W^\top(P-I) = 0$ gives code invariance; $(I-P)(P-I)h = -(I-P)h$ gives $(I-P)F(h) = (1-a/L)(I-P)h$, and $a(h_t) = \sigma(s^\top Uc_0 + \lambda_t s^\top\rho_0 + b)$ with $\lambda_{t+1} = \lambda_t(1-a_t/L)$, $\lambda_0 = 1$. □

Proposition 2 (Exact determinant of the block (4))**.** $\det \nabla F(h) = \det\big(I_r + \frac{1}{L}[\mathrm{diag}(a) + \mathrm{diag}(c \odot a \odot (1-a))A]\big)$*, an* $r \times r$ *determinant.*

Proof. $\nabla F = I_d + UM$ with $M = \frac{1}{L}[\mathrm{diag}(a)W^\top + \mathrm{diag}(c \odot a \odot (1-a))S^\top]$; $\det(I_d + UM) = \det(I_r + MU)$ (Sylvester) and $MU = \frac{1}{L}[\mathrm{diag}(a) + \mathrm{diag}(c \odot a \odot (1-a))A]$. □

Verified by finite differences ($d = 10$, $r = 3$, $L = 2$, three random points): -3.33654787, 5.35252229 and 1.54207626, in agreement to at least seven significant digits. A sufficient condition for a non-zero determinant is strict row diagonal dominance of this $r \times r$ matrix, which holds as soon as $|c_i|\, a_i(1-a_i)\, \|A_{i\cdot}\|_1 < L + a_i$ for every i.

Corollary 1 (Depth acts on at most $r+m$ coordinates)**.** *The set of functions realisable by* T *shared steps of the block* (4) *followed by a linear read-out is contained in* $\{h \mapsto g(W^\top h,\, S^\top(I - P)h) + \ell(h)\}$ *with* $g : \mathbb{R}^{r+m} \to \mathbb{R}^C$ *and* ℓ *linear. Two inputs with the same code and the same* m *residual functionals are separated only linearly, for every* T*. The number of steps changes* g *within the family of iterates of* φ_β*; it does not increase the number* $r+m$ *of coordinates on which the non-linearity acts.*

Remark 1 (What the reduction does and does not say)*.* Theorem 3 and Corollary 1 are statements about the *arguments* of the non-linearity, not about its *form*: *depth does not enlarge the subspace of information accessible to the non-linearity; it can nevertheless modify the function computed on that subspace.* The function $g_T = V_{\mathrm{ro}} U\, \Phi_T$ is a T-fold iterate, and iterates of a fixed map can be markedly more complex functions of their $r+m$ arguments than the map itself (Example 4 gives numbers). Whether this additional complexity is useful on a given problem is an empirical question that the theorem leaves open. What the theorem excludes is that additional steps make the non-linearity sensitive to directions of the input that the code and the m gate functionals do not contain: such directions enter the output linearly at every T.

Corollary 2 (Rank one: under a monotonicity condition, depth does not change the topology of the boundary)**.** *For* $r = 1$, $\varphi_\beta(c) = c(1 + \sigma(\alpha c + \beta + b)/L)$ *with* $\alpha = s^\top u$*, and* $\varphi'_\beta(c) > 1$ *when* $\alpha c \geq 0$, $\varphi'_\beta(c) \geq 1 - |\alpha c|/(4L)$ *when* $\alpha c < 0$*. Let* I *be an interval of initial codes such that* $\alpha c_t > -4L$ *for* $t = 0, \dots, T-1$ *along every orbit starting in* I*, for the values of* β *considered (this holds automatically where* $\alpha c_0 \geq 0$*, and it is implied, uniformly in* β*, by* $|\alpha c_0|(1 + 1/L)^{T-1} < 4L$*). Then* $\Phi_T(\cdot, \beta)$ *is strictly increasing on* I *for these* β*, so every linear threshold on* Φ_T *defines, within* I*, a region* $\{c > \psi_T(\beta)\}$ *(or* $\{c < \psi_T(\beta)\}$*) for a single function* ψ_T*, possibly equal to an endpoint of* I *(empty or full region): depth deforms one boundary curve (a graph over* β*) and does not create a second one.*

Proof. $\varphi'_\beta = 1 + \sigma/L + c\alpha\sigma'/L$ with $\sigma > 0$ and $0 < \sigma' \leq 1/4$. Since $c_{t+1} = c_t(1 + \sigma/L)$, the sign of c_t is constant and $|c_t| \leq |c_0|(1 + 1/L)^t$. By the chain rule, $\partial c_T / \partial c_0 = \prod_{t<T} \varphi'_\beta(c_t) > 0$ under the stated condition, and the threshold $v\, \Phi_T(c, \beta) > \theta$ inverts to $c > \psi_T(\beta)$ if $v > 0$ and to $c < \psi_T(\beta)$ if $v < 0$. □

The condition cannot simply be dropped: for $L = 1$, $\alpha = 1$, $b = 0$ and $\beta = 12$, $\varphi_\beta(-16) = -16.29$, $\varphi_\beta(-12) = -18.00$ and $\varphi_\beta(-8) = -15.86$, so that $\{\varphi_\beta > -17\}$ has two components after a single step. The constant $4L$ is sufficient, not sharp: since $\varphi'_\beta = 1 + \sigma/L + \alpha c\, \sigma(1-\sigma)/L$ and $\inf_{\sigma \in (0,1)} \frac{L+\sigma}{\sigma(1-\sigma)} = 2L + 1 + 2\sqrt{L(L+1)}$ (attained at $\sigma = \sqrt{L(L+1)} - L$), $\varphi'_\beta(c) > 0$ for every β if and only if $\alpha c > -(2L + 1 + 2\sqrt{L(L+1)})$, i.e. -5.83, -9.90, -17.94 for $L = 1, 2, 4$ (confirmed numerically).

Numerical verification ($d = 12$, $r = 3$, $L = 4$, $T = 6$). Invariant residual to 2.3×10^{-16}; code recursion in dimension r to 4.6×10^{-16}; two inputs with the same $(c_0, S^\top \rho_0)$ whose residuals differ by 3.0 in norm give the same c_T to 1.7×10^{-14}; with $\mathrm{im}\, S \subset \mathrm{span}(W)$, c_T ignores the

residual to 2.7×10^{-15}; at rank one ($\alpha = 0.4$, $c_0 \in [-3, 3]$, so that the condition of Corollary 2 holds), $c_0 \mapsto c_T$ is strictly increasing; for the scalar variant, $\cos\angle(\rho_T, \rho_0) = 1$ to 10^{-12}.

Example 2 (The gate reads the kernel: two different inputs, one code). Same geometry as Example 1, $L = 2$, $T = 3$, $b = 0$: code $c = h_1 + h_2$, residual $(-h_2, h_2, h_3)$. Gate $s = (0, 0, 1)$: $s^\top(I - P) = (0, 0, 1) \neq 0$, so $m = 1$, and $A = s^\top u = 0$. By Theorem 3(ii), $a_t = \sigma(\beta)$ with $\beta = h_3$ independent of t, so the recursion solves in closed form, $c_T = c_0(1 + \sigma(h_3)/L)^T$. For $h = (1, \frac{1}{2}, 2)$ and $h' = (0.2, 1.3, 2)$, same code $c_0 = 1.5$ and same $\beta = 2$ but different residuals, $(-0.5, 0.5, 2)$ and $(-1.3, 1.3, 2)$. Three steps give $h_T = (3.9827, 0.5, 2)$ and $h'_T = (3.1827, 1.3, 2)$: different vectors, the *same code* $c_T = 4.4827$, and the closed form returns $1.5\,(1 + 0.8808/2)^3 = 4.4827$. The residuals are unchanged. Any linear read-out separates h from h' only through the linear term $V_{\mathrm{ro}}(I - P)h_0$, whatever T: Corollary 1 in dimension three. The determinant of Proposition 2 is a scalar here, $A = 0$, so $\det \nabla F = 1 + a/L = 1.4404$, recovered by finite differences. With $s = w = (1, 1, 0)$ instead ($s \in \mathrm{span}(W)$, $m = 0$), the inputs $(1, \frac{1}{2}, 2)$ and $(1, \frac{1}{2}, -7)$ give the same $c_T = 4.5151$: the kernel is ignored by the non-linearity (Theorem 3(v)).

Example 3 (Rank one with $A \neq 0$: a single boundary curve). Same geometry, $L = 2$, gate $s = (1, 0, 1)$: $A = s^\top u = 1$ and $\beta = s^\top(I - P)h = h_3 - h_2$. No closed form is known for the recursion $c \leftarrow c(1 + \sigma(c + \beta)/L)$, but it is strictly increasing in c_0 on the range used below (Corollary 2: positive codes satisfy the condition trivially, and the negative orbits stay above $-2.93 > -4L = -8$). At $\beta = 0.5$: $c_0 = -2, -1, 0, 1, 2$ give $c_1 = -2.1824, -1.1888, 0, 1.4088, 2.9241$ and $c_6 = -2.9261, -2.1664, 0, 9.8550, 21.9073$. Depth amplifies positive codes and leaves $c = 0$ fixed; the order is preserved. The threshold $c_T > 1$ inverts to $c_0 > \psi_T(\beta)$, computed by bisection: $\psi_1(\beta) = 0.8897, 0.7467, 0.6812$ and $\psi_6(\beta) = 0.5425, 0.2055, 0.1055$ for $\beta = -2, 0, 2$. Going from $T = 1$ to $T = 6$ shifts and tilts the curve in the (c, β) plane; it does not create a second component.

Example 4 (Rank two: it is m, not T, that counts). $d = 4$, $U = [e_1, e_2]$, $W = [e_1 + e_3, e_2 + e_4]$ ($W^\top U = I_2$); $Ph = (h_1 + h_3, h_2 + h_4, 0, 0)$ and $(I - P)h = (-h_3, -h_4, h_3, h_4)$. Two inputs with the same code $(1.3, 1.3)$ and the same h_3: $h = (1, 0.5, 0.3, 0.8)$ and $h' = (1, 3.3, 0.3, -2)$, differing in h_4 (and in h_2 to compensate the code). With $L = 2$, $T = 4$: if both gates read h_3 ($S = [e_3, e_3]$, $m = 1$) then $c_T(h) = c_T(h') = (3.5691, 3.5691)$: four steps have not distinguished h from h', because they differ in a kernel direction the gate does not read. If the gates read h_3 and h_4 ($S = [e_3, e_4]$, $m = 2$) then $c_T(h) = (3.5691, 4.2542)$ and $c_T(h') = (3.5691, 1.6388)$: the second coordinate now separates them. With $m = 1$, additional steps do not separate h from h' through the non-linearity: $c_T(h) = c_T(h')$ at $T = 4$, 8 and 40, as Theorem 3(iv) requires, since the two inputs share their $r + m = 3$ functionals. Additional steps do change the function computed on those functionals: for a third input $h'' = (2, 0.5, -1, 0.8)$, with code $(1.0, 1.3)$ and $h_3 = -1$, the ratio of the first code coordinates $c_T(h)_1/c_T(h'')_1$ is 1.3 at $T = 0$ and 2.1547, 3.5713 and 203.39 at $T = 4$, 8 and 40. Sensitivity to a new direction of the input is thus bought with independent gate directions in the kernel (m), whereas the number of steps (T) reshapes the response to the directions already read.

Design rule. For the two gate families analysed above, the non-linearity of a shared block built on a *single* projector has at most $r + m$ linear functionals of the input as arguments, whatever the number of steps (for the scalar variant it multiplies the residual by a scalar); the function of these arguments does depend on the number of steps (Remark 1). Within this reduction, independent gate directions in the kernel raise m up to r (the triceptron, Section 4); outside it, several non-aligned projectors with a non-linearity between them (the lattice, Appendix C) are one route, not claimed to be the only one.

4 Capacity of multiplicative units: the triceptron

A perceptron has one direction (w); the oblique perceptron has two (u for representation, w for measurement). The *triceptron* has three: a *gate* direction s that is free to differ from w,

$$y = \sigma(s^\top x + b_s) \cdot \frac{w^\top x}{w^\top u} \ \text{(rank one)}, \qquad h = \sigma(S^\top x + b_s) \odot (W^\top x + b_w), \ \text{output } V_{\text{ro}} h \ \text{(rank } r\text{)}. \tag{5}$$

The motivation comes from Proposition 1: in the recursion of the oblique block, a gate that reads inside span(W) is constant along the orbit and brings nothing that the code does not already contain; a gate direction outside span(W) reads the residual. (With $s = w$ the unit is still a genuine non-linearity of x: it is the SiLU unit, Theorem 4(i).) At rank r the unit is a gated linear unit [4]: W is the "up" projection, S the gate, V_{ro} the "down" projection of the SwiGLU feed-forward blocks [5] used in recent language models. The gate×code product makes each unit a *quadratic* function of x to second order in the gate pre-activation, where a ReLU unit is piecewise linear.

4.1 Quadric capacity of the triceptron

Unit: $\tau(x) = \sigma(\gamma(s^\top x + b_s))\,(w^\top x + b_w)$, with a gate sharpness $\gamma > 0$ (a notational convenience: it can be absorbed in s and b_s); layer: $f(x) = \sum_{i=1}^{k} v_i \tau_i(x) + u^\top x + u_0$ (the linear term $u^\top x$ is a *skip*; the experiments of Section 4.2 are run with and without it). The *inertia* (p, n) of a real symmetric matrix is its number of positive and of negative eigenvalues.

Theorem 4 (Capacity of the triceptron)**.** *(i)* ***Inclusion.*** *For $s_i = w_i$, $b_{s,i} = b_{w,i} = b_i$ and $\gamma = 1$, $\tau_i = \mathrm{SiLU}(w_i^\top x + b_i)$: every one-hidden-layer SiLU (swish) network with k units is a network of k triceptrons. For general γ, $\sup_z |\sigma(\gamma z)z - \mathrm{ReLU}(z)| = c_0/\gamma$ with $c_0 = \max_{t>0} t/(1+e^t) \approx 0.2785$: every one-hidden-layer ReLU network with k units is the uniform limit, as $\gamma \to \infty$, of networks of k triceptrons. Decoupling s from w is, for the activation, what obliquity is for the linear layer.*

(ii) ***Soft regime: quadrics.*** *As $\gamma \to 0$, with read-out weights v_i/γ and a skip chosen to cancel the linear terms (its coefficients are of order $1/\gamma$), f converges, uniformly on every compact set and with an error $O(\gamma^2)$, to a polynomial of degree two whose quadratic part has matrix $Q = \frac{1}{8}\sum_i v_i(s_i w_i^\top + w_i s_i^\top)$. Conversely, every real symmetric matrix of inertia (p, n) can be written in this way with $k = \max(p, n)$ units, and not with fewer. Hence, on any compact set, k soft triceptrons plus a skip approximate to $O(\gamma^2)$ every quadratic function whose quadratic part satisfies $\max(p, n) \le k$; where the gradient of that function does not vanish on its zero set, the decision boundary converges to the quadric (in Hausdorff distance, on compact sets) at the same rate. Special cases: the XOR pattern $\mathrm{sign}(x_1 x_2)$ (inertia $(1, 1)$): one unit, exact in sign for every γ; an ellipsoid of $\mathbb{R}^d$ (inertia $(d, 0)$): d units; a hyperbola: one unit.*

(iii) ***Hard regime: the cell count of a ReLU layer.*** *As $\gamma \to \infty$ the layer tends, pointwise away from the gate hyperplanes, to a function that is affine on each cell of the arrangement of the k gate hyperplanes, of which there are at most $\sum_{j \le d} \binom{k}{j}$ (Zaslavsky [6]); the value hyperplanes w_i create no cells. This is the bound that holds for a k-unit ReLU layer; the difference is that the affine piece of each cell combines the w_i freely instead of being tied to the gate directions.*

(iv) ***A necessary width for ReLU on a curved boundary.*** *Let $\mathcal{D} \subset \mathbb{R}^2$ be the open region assigned to one class by a one-hidden-layer ReLU network with m units and an affine read-out (with or without a linear skip), and call it ε-accurate for the disc of radius R if $B(0, R-\varepsilon) \subset \mathcal{D} \subset \bar{B}(0, R+\varepsilon)$, $0 < \varepsilon < R$. The boundary $\partial\mathcal{D}$ is contained in at*

most $N(m) = 1 + m + \binom{m}{2}$ segments or half-lines, and in at most $2m$ segments if $\mathcal{D}$ is convex and bounded. A region whose boundary is contained in N segments or half-lines is ε-accurate only if

$$N \;\geq\; \frac{\pi}{\arccos\frac{R-\varepsilon}{R+\varepsilon}}, \qquad \textit{i.e.} \qquad \varepsilon \;\geq\; R\tan^2\frac{\pi}{2N} \;\sim\; \frac{\pi^2 R}{4N^2}, \tag{6}$$

with equality, for $N \geq 3$, for the regular N-gon of apothem $R-\varepsilon$ and circumradius $R+\varepsilon$. Hence no such network is exact, and ε-accuracy requires $m \gtrsim (\pi^2 R/\varepsilon)^{1/4}$ units in general and $m \geq \frac{\pi}{4}\sqrt{R/\varepsilon}\,(1+o(1))$ units when $\mathcal{D}$ is convex. These are necessary conditions; whether a network can actually place $N(m)$ segments around a circle is not established here, and the convex case suggests that it cannot. By contrast, the two soft triceptrons of (ii) with a skip are ε-accurate with $\varepsilon = \gamma^2 R^3/24 + O(\gamma^4)$ at fixed width two, at the price of read-out and skip weights of order $1/\gamma$.

Proof. (i) With $t = -\gamma z$ for $z < 0$, $|\sigma(\gamma z)z| = \gamma^{-1}t/(1+e^t)$, and for $z > 0$, $|\sigma(\gamma z)z - z| = \gamma^{-1}t/(1+e^t)$ with $t = \gamma z$; the maximum of $t/(1+e^t)$ over $t > 0$ is c_0. The uniform convergence of the network follows from $|\sum_i v_i(\tau_i - \mathrm{ReLU}_i)| \leq c_0 \sum_i |v_i|/\gamma$. (ii) $\sigma(\gamma z) = \frac{1}{2} + \frac{\gamma z}{4} + O(\gamma^3 z^3)$, so $\tau_i = \frac{1}{2}(w_i^\top x + b_{w,i}) + \frac{\gamma}{4}(s_i^\top x + b_{s,i})(w_i^\top x + b_{w,i}) + O(\gamma^3)$ uniformly on compact sets; the skip absorbs the linear terms and the read-out v_i/γ isolates the quadratic term, of symmetric matrix $\frac{1}{8}\sum v_i(s_i w_i^\top + w_i s_i^\top)$, with a remainder $O(\gamma^2)$. Each $\mathrm{sym}(sw^\top) = \frac{1}{2}(sw^\top + ws^\top)$ has inertia $(1,1)$ if s, w are independent (its non-zero eigenvalues are $\frac{1}{2}(s^\top w \pm \|s\|\|w\|)$), and $(1,0)$ or $(0,1)$ if they are collinear. Conversely, write the spectral decomposition $Q = \sum_{j\leq p} \lambda_j e_j e_j^\top - \sum_{j\leq n} \nu_j f_j f_j^\top$ with $\lambda_j, \nu_j > 0$ and orthonormal e_j, f_j; for $j \leq \min(p,n)$, $\lambda_j e_j e_j^\top - \nu_j f_j f_j^\top = \mathrm{sym}\big((\sqrt{\lambda_j}e_j + \sqrt{\nu_j}f_j)(\sqrt{\lambda_j}e_j - \sqrt{\nu_j}f_j)^\top\big)$, and the remaining $|p-n|$ terms are collinear units. Hence $\max(p,n)$ units suffice. They are necessary because the numbers of positive and of negative eigenvalues are subadditive under addition of symmetric matrices (a consequence of Weyl's inequalities [3]) and each unit contributes at most one eigenvalue of each sign. For XOR, $\sigma(\gamma x_1)x_2 - \frac{1}{2}x_2 = \frac{1}{2}x_2\tanh(\gamma x_1/2)$ has the sign of $x_1 x_2$ for every $\gamma > 0$. (iii) In the limit τ_i is affine on each open half-space of its gate hyperplane; the layer is affine on each cell of the arrangement of the k gate hyperplanes, whose number of cells in $\mathbb{R}^d$ is at most $\sum_{j=0}^{d}\binom{k}{j}$. (iv) The network function f is continuous and affine on each closed cell of the arrangement of the m kink lines, and $\mathcal{D} = \{f < \theta\}$ (or $\{f > \theta\}$). A point of $\partial\mathcal{D}$ satisfies $f = \theta$ and is a limit of points of $\mathcal{D}$ lying in one closed cell on which $f - \theta$ is not identically zero; it therefore belongs to the zero line of the affine piece of that cell: at most one segment or half-line per cell, and at most $N(m)$ cells. If $\mathcal{D}$ is convex, $\partial\mathcal{D}$ is a convex polygon whose vertices lie on kink lines, and a line contains at most two vertices of a convex polygon: at most $2m$ edges. If $\mathcal{D}$ is ε-accurate, $\partial\mathcal{D}$ lies in the closed annulus $R - \varepsilon \leq \|x\| \leq R + \varepsilon$, and every ray from the origin meets $\partial\mathcal{D}$ (it starts in $\mathcal{D}$ and leaves it). The part of a segment or half-line lying in that annulus is seen from the origin under a total angle of at most $2\arccos\frac{R-\varepsilon}{R+\varepsilon}$: if its supporting line is at distance $h \geq R - \varepsilon$ from the origin the angle is at most $2\arccos\frac{h}{R+\varepsilon}$; if $h < R - \varepsilon$ that part consists of at most two pieces, one on each side of the foot of the perpendicular, each seen under at most $\arccos\frac{h}{R+\varepsilon} - \arccos\frac{h}{R-\varepsilon} \leq \arccos\frac{R-\varepsilon}{R+\varepsilon}$, this difference being increasing in h. N such angles must cover 2π, which is (6) since $\cos\frac{\pi}{N} \geq \frac{R-\varepsilon}{R+\varepsilon}$ is equivalent to $\varepsilon \geq R\tan^2\frac{\pi}{2N}$. For the two triceptrons, $\sum_i \frac{x_i}{2}\tanh\frac{\gamma x_i}{2} = \frac{\gamma}{4}\|x\|^2 - \frac{\gamma^3}{48}(x_1^4 + x_2^4) + O(\gamma^5)$, whose level set at $\frac{\gamma}{4}R^2$ has polar radius $R + \frac{\gamma^2 R^3}{24}(\cos^4\theta + \sin^4\theta) + O(\gamma^4)$. □

Numerical verification. (i) $\sup_z |\sigma(\gamma z)z - \mathrm{ReLU}(z)| = 5.569 \times 10^{-3}$ at $\gamma = 50$, equal to $0.2785/\gamma$. (ii) Circle with two soft units and a skip: the largest radial gap between the level set and the unit circle is 1.05×10^{-2}, 1.67×10^{-3} and 1.04×10^{-4} for $\gamma = 0.5$, 0.2 and 0.05, i.e. $\gamma^2/24$; XOR with one unit plus skip, and with two hard units ($\gamma = 50$, $x_2\tanh(25x_1)$): 100%

sign agreement off the axes on a 301×301 grid. (iv) Disc *without margin* (class $= \|x\| > 1$, 4000 training points uniform in $[-1.5, 1.5]^2$): ReLU MLPs with m units trained by L-BFGS (three initialisations, the one with the best *training* accuracy is kept) are evaluated on a 601×601 grid of the same square, by the fraction of misclassified area and by the accuracy $\hat{\varepsilon}$ in the sense of (iv), estimated as the largest $|\,\|x\| - 1|$ over misclassified grid points:

m	$N(m)$	misclassified area	measured $\hat{\varepsilon}$	bound $\tan^2 \frac{\pi}{2N(m)}$	convex bound $\tan^2 \frac{\pi}{4m}$
2	4	32.08%	1.121	0.1716	0.1716
3	7	2.76%	0.136	0.0521	0.0718
4	11	1.55%	0.070	0.0207	0.0396
6	22	1.09%	0.070	0.0051	0.0173
10	56	0.75%	0.050	7.9×10^{-4}	0.0062
20	211	0.51%	0.027	5.5×10^{-5}	0.0015
40	821	0.35%	0.024	3.7×10^{-6}	3.9×10^{-4}

The measured accuracies are above the general bound, as they must be, and also above the convex bound (which is binding only for convex regions); they decrease much more slowly than either: the trained networks are far from the best polygon with the same number of units, so that the bounds say nothing about what optimisation achieves in practice. ($\hat{\varepsilon}$ is itself a lower estimate, computed on a grid of the training square; at $m = 2$ it exceeds R, i.e. that network is not ε-accurate for any admissible ε.) The two soft triceptrons with explicit weights ($\gamma = 0.05$) misclassify 0.014% of the grid, with a radial gap of 1.04×10^{-4}. With a margin (`make_circles`, factor 0.5), three ReLU units reach 100% training accuracy (two units: 83.3%): it is the margin, not the curvature, that sets the unit requirement; the capacity advantage of the triceptron concerns tight curved boundaries, and Section 4.2 measures how much of it is visible in trained models.

Example 5 (The inclusion in numbers). With $s = w$ and $z = w^\top x + b = 2$: $\gamma = 1$ gives $\sigma(2) \cdot 2 = 1.7616 = \mathrm{SiLU}(2)$; $\gamma = 50$ gives $\sigma(100) \cdot 2 = 2.0000$ and, at $z = -2$, $\sigma(-100) \cdot (-2) = -7 \times 10^{-44}$, i.e. $\mathrm{ReLU}(\pm 2)$ to machine precision. The largest gap, $0.2785/\gamma = 5.6 \times 10^{-3}$, is reached near $z = \pm 1.28/\gamma$. A ReLU neuron is the $\gamma \to \infty$ limit of a triceptron whose gate direction is *aligned* with its value direction.

Example 6 (XOR with one unit and a skip). Take $s = e_1$, $w = e_2$, $b_s = b_w = 0$, $\gamma = 0.05$, read-out $v = 1$ and skip $u = -\frac{1}{2}e_2$: $f(x) = \sigma(0.05x_1)x_2 - \frac{1}{2}x_2 = 0.0125\, x_1 x_2 + O(10^{-4})$ on $[-2, 2]^2$. The skip cancels the linear term $\frac{1}{2}x_2$ of the expansion of σ; only the cross term remains, with matrix $Q = \frac{\gamma}{8}(e_1 e_2^\top + e_2 e_1^\top)$, inertia $(1, 1)$: a rectangular hyperbola whose asymptotes are the axes, i.e. the XOR pattern. At $(\pm 1, \pm 1)$: $f = \pm 0.01250$ with the sign of $x_1 x_2$ in all four cases; at $(\pm 2, 0.5)$: ± 0.01249 against ± 0.01250. With the same skip, a single ReLU unit can separate the four canonical XOR points ($|x_1 + x_2| - 1$ does), but not the pattern $\mathrm{sign}(x_1 x_2)$ on the whole plane: $v\,\mathrm{ReLU}(\cdot)$ plus an affine function is convex or concave, so one of its two decision regions is convex, whereas neither $\{x_1 x_2 > 0\}$ nor $\{x_1 x_2 < 0\}$ is. One triceptron realises it because gate and value look in orthogonal directions. The small values (10^{-2}) are only a matter of scale: the read-out v/γ of the theorem brings them back to $x_1 x_2/4$.

Example 7 (The unit circle with two units). Units $(s_1, w_1) = (e_1, e_1)$ and $(s_2, w_2) = (e_2, e_2)$, $\gamma = 0.05$, skip $-\frac{1}{2}(x_1 + x_2)$: $f(x) = \sum_i [\sigma(0.05x_i)x_i - \frac{1}{2}x_i] = 0.0125(x_1^2 + x_2^2) + O(\gamma^3)$, inertia $(2, 0)$; decide "outside" if $f > 0.0125R^2$ with $R = 1$. Then $f(0.9, 0) = 0.01012$ (inside), $f(1.1, 0) = 0.01512$ (outside), $f(0.7, 0.7) = 0.01225$ (inside, $\|x\|^2 = 0.98$), $f(0.6, 0.9) = 0.01462$ (outside, $\|x\|^2 = 1.17$), $f(-0.5, 0.5) = 0.00625$ (inside): all correct. The point $(0.7, 0.7)$, at 0.02 of the circle in squared norm, is classified correctly: the boundary is a smooth closed curve within $\gamma^2/24 \approx 10^{-4}$ of the unit circle, not a polygon.

Example 8 (Reading the inertia on $\mathrm{sym}(sw^\top)$). Hyperbola $x_1^2 - x_2^2$: one unit, $s = (1, 1)$, $w = (1, -1)$: $\frac{1}{2}(sw^\top + ws^\top) = \mathrm{diag}(1, -1)$, eigenvalues $\frac{1}{2}(0 \pm 2) = \pm 1$, inertia $(1, 1)$. Ellipse $2x_1^2 + x_2^2$:

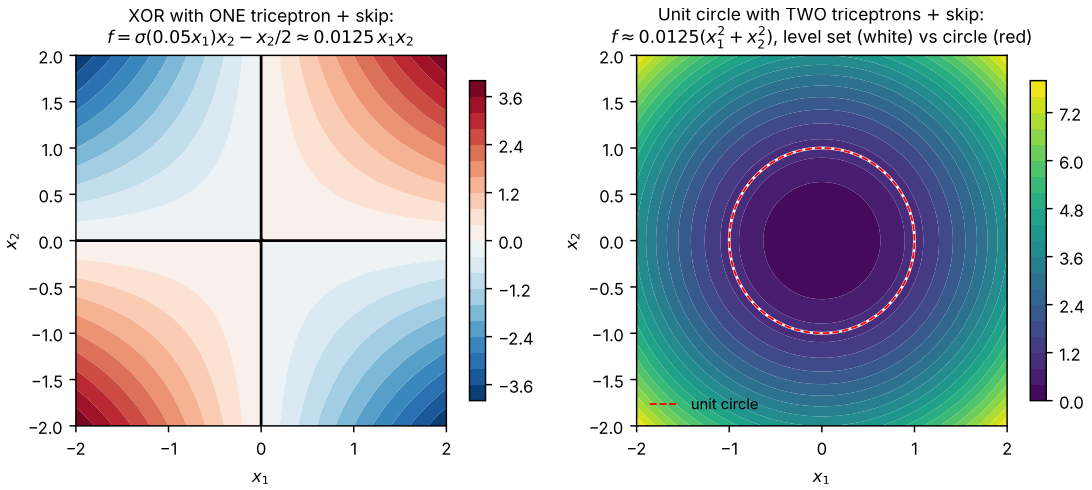


Figure 3: The two soft-regime constructions of Theorem 4(ii) as functions of the plane (values divided by $\gamma/4 = 0.0125$). Left: one triceptron plus a linear skip realises the saddle x_1x_2 to second order, i.e. the XOR pattern, with the axes as zero set. Right: two triceptrons plus a skip realise $x_1^2 + x_2^2$ to second order; the level set at 0.0125 (white) is indistinguishable at this scale from the unit circle (red dashed; the radial gap is 10^{-4}), a boundary that a ReLU layer approaches only by increasing its width.

inertia $(2, 0)$, two collinear units $s = w = (\sqrt{2}, 0)$ and $s = w = (0, 1)$, whose sum is $\mathrm{diag}(2, 1)$; in the soft limit one unit cannot make an ellipse, since $\mathrm{sym}(sw^\top)$ has at most one positive eigenvalue. Saddle $x_1^2 + x_2^2 - x_3^2$ in $\mathbb{R}^3$: inertia $(2, 1)$, so two units: $s = (1, 0, 1)$, $w = (1, 0, -1)$ (contributing $x_1^2 - x_3^2$) and $s = w = e_2$ (contributing x_2^2); the sum is $\mathrm{diag}(1, 1, -1)$. This is the construction of the proof of (ii): one positive and one negative eigenvalue are paired per unit, the $|p - n|$ remaining ones go into collinear units.

Example 9 (The necessary width in numbers). For the disc $R = 1$, a ReLU layer with $m = 2, 3, 4, 6, 10, 20$ units has at most $N(m) = 4, 7, 11, 22, 56, 211$ boundary segments, hence by (6) an accuracy $\varepsilon \geq 0.1716$, 0.0521, 0.0207, 0.0051, 7.9×10^{-4}, 5.5×10^{-5}. Conversely, an accuracy $\varepsilon = 10^{-2}, 10^{-3}, 10^{-4}$ needs $N \geq 16, 50, 158$ segments, i.e. at least $m = 5, 10, 18$ ReLU units ($N(5) = 16$, $N(10) = 56$, $N(18) = 172$), and at least $8, 25, 79$ units if the decision region is convex. Two soft triceptrons reach these accuracies at fixed width, with $\gamma \leq 0.48$, 0.15, 0.048. The growth in $\varepsilon^{-1/4}$ is a necessary condition, independent of the optimiser; it is not claimed to be attainable.

Example 10 (Hard regime: a switch). $s = e_1$, $w = e_2$, $\gamma = 50$: $\tau(x) = \sigma(50x_1)x_2$ equals 3.0000 at $(1, 3)$ and 0.0000 at $(-1, 3)$: it is close to $\mathbf{1}[x_1 > 0]\, x_2$, one variable *switching* another, which is what a gated linear unit does. The transition has width $\sim 1/\gamma$: 1.8674 at $(0.01, 3)$, 1.1326 at $(-0.01, 3)$. A ReLU layer realises this switch approximately on bounded sets, as $\mathrm{ReLU}(x_2 + Mx_1) - \mathrm{ReLU}(Mx_1)$ with large M, with two units; in both cases a large weight (M or γ) sets the width of the transition, and the difference is that the triceptron has the product inside a single unit.

Remark 2 (Squared-norm units). A unit of the form $\|P_{U,W}x\|^2 = x^\top P^\top Px$ is a positive semi-definite quadratic form of rank r: thresholded alone, it bounds a convex (cylindrical) set. Signed combinations of several such units can realise indefinite forms, so the difference with the triceptron is one of economy per unit: a single triceptron already brings inertia $(1, 1)$, an indefinite (hyperbolic) quadric.

4.2 Experiments aligned with the theorem

Theorem 4 is a statement about what a layer *with a linear skip* can represent, with explicit weights in a soft regime. An experiment can only be said to illustrate it if it (a) compares gated and ReLU layers under the same optimiser, (b) includes the skip, and (c) separates the weights constructed in the proof from the weights found by training. The three points are treated in turn. Comparing a hand-trained gated layer with a library-trained MLP would confound the architecture with the optimiser (last column of Table 1), and is avoided.

Models and trainer. *GLU*: the layer (5) with r units, parameters $(W, b_w, S, b_s, V_{\mathrm{ro}}, b_0)$, i.e. $2dr + 2r + rC + C$ parameters. *MLP*: one hidden ReLU layer whose width h gives the nearest number of parameters. Both exist with and without a linear skip Kx added to the output (dC additional parameters). All four models are trained by the *same* code: full-batch Adam, 2000 steps, learning rate 10^{-2}, L_2 penalty 10^{-4}, the same initialisation scale, hand-written gradients verified by finite differences (relative errors between 7.6×10^{-8} and 1.9×10^{-7} for the four models). The two-dimensional problems (two moons with noise 0.15, concentric circles with radius ratio 0.5 and noise 0.1, XOR pattern $\mathrm{sign}(x_1 x_2)$ on Gaussian inputs; $n = 400$, 70/30 split) are drawn independently ten times; differences are paired by draw and reported with the conventions of Section 11.

Table 1: Gated (GLU, r units) against ReLU (MLP, h units) layers at nearly equal parameter counts, same trainer, without and with a linear skip. Test accuracy in % (mean ± standard deviation over 10 independent draws) and paired difference GLU − MLP in points with its 95% interval ($\approx$ equivalent within one point, $\uparrow$ GLU better, $\downarrow$ GLU worse, ? undetermined). Last column: the MLP without skip trained by scikit-learn's L-BFGS, a different optimiser, for reference only.

		without skip				with linear skip				
problem	r/h	param.	GLU	MLP	GLU − MLP	param.	GLU	MLP	GLU − MLP	MLP, L-BFGS
two moons	16/26	130/132	98.8 ± 0.9	98.9 ± 0.9	$-0.1\,[-0.4, +0.3]^{\approx}$	134/136	98.8 ± 1.1	98.9 ± 0.9	$-0.2\,[-0.4, +0.1]^{\approx}$	98.5 ± 0.9
two moons	4/6	34/32	98.7 ± 0.9	98.9 ± 0.9	$-0.2\,[-0.5, +0.0]^{\approx}$	38/36	98.8 ± 0.8	98.8 ± 1.1	$+0.1\,[-0.4, +0.5]^{\approx}$	96.8 ± 2.8
two moons	2/3	18/17	97.6 ± 3.8	91.0 ± 4.0	$+6.6\,[+2.8, +10.4]^{\uparrow}$	22/21	98.8 ± 0.9	95.6 ± 4.9	$+3.2\,[-0.6, +6.9]^{?}$	89.5 ± 4.0
two moons	1/2	10/12	88.2 ± 2.2	88.8 ± 2.7	$-0.7\,[-1.5, +0.1]^{?}$	14/16	94.6 ± 5.4	91.2 ± 4.4	$+3.4\,[-2.9, +9.7]^{?}$	84.8 ± 12.0
circles	16/26	130/132	99.1 ± 0.7	99.2 ± 0.7	$-0.2\,[-0.4, +0.1]^{\approx}$	134/136	99.1 ± 0.7	99.2 ± 0.6	$-0.1\,[-0.3, +0.1]^{\approx}$	99.2 ± 0.7
circles	4/6	34/32	99.0 ± 0.8	98.8 ± 0.9	$+0.2\,[-0.1, +0.4]^{\approx}$	38/36	99.0 ± 0.8	98.8 ± 0.8	$+0.2\,[-0.1, +0.4]^{\approx}$	97.9 ± 1.8
circles	2/3	18/17	98.3 ± 1.5	92.5 ± 10.7	$+5.8\,[-2.1, +13.7]^{?}$	22/21	98.6 ± 1.4	98.0 ± 2.1	$+0.6\,[-0.4, +1.6]^{?}$	82.9 ± 11.2
circles	1/2	10/12	70.6 ± 3.9	69.7 ± 6.9	$+0.9\,[-5.1, +6.9]^{?}$	14/16	70.5 ± 3.9	81.9 ± 13.9	$-11.4\,[-22.0, -0.8]^{\downarrow}$	72.2 ± 4.2
XOR	16/26	130/132	97.9 ± 1.6	99.0 ± 0.7	$-1.1\,[-2.3, +0.2]^{?}$	134/136	98.8 ± 0.8	99.2 ± 0.6	$-0.5\,[-1.4, +0.4]^{?}$	98.3 ± 1.6
XOR	4/6	34/32	98.2 ± 1.1	98.3 ± 1.2	$-0.0\,[-0.6, +0.6]^{\approx}$	38/36	98.1 ± 1.3	98.7 ± 0.9	$-0.6\,[-1.8, +0.6]^{?}$	98.4 ± 0.9
XOR	2/3	18/17	98.4 ± 1.3	88.3 ± 10.9	$+10.1\,[+1.8, +18.4]^{\uparrow}$	22/21	98.0 ± 1.2	95.5 ± 8.3	$+2.5\,[-3.9, +8.9]^{?}$	83.0 ± 12.6
XOR	1/2	10/12	74.2 ± 4.6	70.7 ± 7.1	$+3.4\,[-3.1, +10.0]^{?}$	14/16	98.7 ± 0.8	90.3 ± 11.6	$+8.3\,[-0.7, +17.3]^{?}$	70.8 ± 6.7

(a) Same trainer: parity, except at about eighteen parameters without skip. From 32 parameters upwards the two families are equivalent within one point or undetermined on the three problems, with or without skip. At 18 against 17 parameters without skip (two gated units against three ReLU units) the gated layer is ahead on two moons (+6.6 points, interval $[+2.8, +10.4]$) and on XOR (+10.1, $[+1.8, +18.4]$); on circles the difference (+5.8) is undetermined, the ReLU layer having a standard deviation of 10.7 points over the draws. With the skip the three differences at this size (+3.2, +0.6, +2.5) are undetermined. The last column shows why the optimiser must be shared: at 17 parameters the library L-BFGS training of the same MLP is 1.5 to 9.6 points below the Adam training used here, so that a comparison across optimisers would attribute to the architecture a gap that belongs largely to the optimisation.

(b) The widths of Theorem 4(ii) are observed, and the skip plays the role the proof gives it. The XOR pattern is a quadric of inertia $(1, 1)$: the theorem asks for $\max(p, n) = 1$

unit and a skip that cancels the linear term. One gated unit with skip (14 parameters) reaches $98.7 \pm 0.8\%$; the same unit without skip reaches 74.2%. The circle has inertia $(2, 0)$ and asks for two units: one gated unit reaches 70.6% without and 70.5% with skip, two units 98.3% and 98.6%. The comparison with ReLU layers at these sizes is not one-sided. On XOR, two ReLU units with skip (16 parameters) reach $90.3 \pm 11.6\%$, and the difference to the gated unit (+8.3 points) is undetermined. On circles, two ReLU units with skip reach $81.9 \pm 13.9\%$ and are *ahead* of the single gated unit (-11.4 points, interval $[-22.0, -0.8]$): a single gated unit cannot produce a definite quadratic form in the soft regime, whatever its parameters, whereas two ReLU units already bound a strip.

(c) Constructed against learned weights. The weights of the proof, used without any training on the raw data, give

gate sharpness γ	1	0.5	0.2	0.05
circles, two units + skip, accuracy (%)	99.7	99.7	99.6	99.6
XOR, one unit + skip, accuracy (%)	100.0	100.0	100.0	100.0
largest weight in normalised form ($4/\gamma$)	4	8	20	80

(accuracy on the 400 points of each of the ten draws; the circles are noisy, so that 100% is not attainable). The soft regime $\gamma \to 0$ is needed for the *approximation* statement of the theorem (the radial gap is $\gamma^2 R^3/24$), not for classification: on these data $\gamma = 1$, with weights of order 4, is as accurate as $\gamma = 0.05$, whose read-out weights are of order 80. The weights *learned* by Adam for the same two architectures reach $98.8 \pm 1.1\%$ (circles) and $98.6 \pm 0.8\%$ (XOR) on the test sets, with a largest weight of about 7.7 and gate directions of norm 3.7 and 7.5 (standardised inputs): training finds solutions with moderately sharp gates, not the soft-regime construction. The theorem therefore accounts for the *widths* at which the two problems become solvable, which is what the experiments reproduce; it does not describe the solutions reached by gradient descent, and it says nothing of the parity observed at larger widths, where both families have enough units.

Larger problems, and the analytic realisation. Two further realisations are compared on eight datasets with $r = 16$ (five 70/30 splits of a fixed dataset; the intervals measure split variability only). *TRI-CF* uses analytic features: (U, W) from the fast block, S the k leading principal components of the residual $(I - P)x$ (outside span(W) by construction), features $[c,\ c_i a_j]$, a least-squares read-out, k chosen by internal cross-validation. On the eight datasets

Table 2: Width $r = 16$: analytic block (centroid read-out), TRI-CF (least-squares read-out on its features; with $k = 0$ it reduces to the block with the least-squares read-out), and the gated and ReLU layers of Table 1 without skip (same trainer). Test accuracy in %, 5 splits of a fixed dataset. Parameters: GLU / MLP.

dataset	param.	analytic block	TRI-CF	GLU	MLP	GLU − MLP	MLP, L-BFGS
two moons	130/132	86.8 ± 3.2	86.8 ± 3.2 (k=2)	98.2 ± 1.3	98.0 ± 1.1	$+0.2\,[-0.3, +0.6]^{\approx}$	96.7 ± 1.9
circles	130/132	46.0 ± 4.1	69.5 ± 6.4 (k=2)	99.8 ± 0.3	100.0 ± 0.0	$-0.2\,[-0.6, +0.3]^{\approx}$	99.3 ± 0.3
XOR	130/132	50.0 ± 0.9	80.2 ± 6.2 (k=2)	98.0 ± 0.8	97.8 ± 0.7	$+0.2\,[-1.2, +1.5]^{?}$	96.3 ± 1.5
Gaussian n=15/cl (d=20)	740/729	96.7 ± 4.4	93.3 ± 6.5 (k=0)	90.0 ± 5.4	90.0 ± 5.4	$+0.0\,[+0.0, +0.0]^{\approx}$	90.0 ± 5.4
heteroscedastic n=60/cl	740/729	93.6 ± 1.1	92.2 ± 1.7 (k=2)	93.3 ± 1.0	92.8 ± 1.4	$+0.6\,[-0.4, +1.5]^{?}$	91.9 ± 1.6
digits	2154/2170	95.3 ± 0.2	94.6 ± 0.3 (k=7)	98.0 ± 0.6	98.0 ± 0.4	$-0.0\,[-0.3, +0.3]^{\approx}$	96.4 ± 0.8
breast	1026/1025	95.4 ± 0.9	95.6 ± 0.9 (k=4)	96.4 ± 0.9	96.6 ± 0.7	$-0.2\,[-0.9, +0.4]^{\approx}$	96.0 ± 1.2
wine	499/496	100.0 ± 0.0	99.3 ± 0.9 (k=1)	98.9 ± 0.9	99.3 ± 0.9	$-0.4\,[-1.4, +0.7]^{?}$	99.6 ± 0.7

the gated and ReLU layers are equivalent or undetermined (differences between -0.4 and $+0.6$ point): at a comfortable budget the triceptron is *different*, not better. On the small Gaussian sample the three gradient-trained models give the same accuracy on every split (90.0%, on 18

test points per split) and stay behind the analytic block (96.7%), with about 740 parameters for 42 training samples. The analytic version of the gate is not enough: gate directions taken from the residual help (XOR 50 → 80%, circles 46 → 70%) but remain far from a *learned* gate (98 to 100%). This is one of the two places in this report where a gradient-trained component was clearly better than every back-propagation-free construction tried (the learned lifting of Appendix I is the other): s is not a second-order statistic of the classes (for XOR the class means coincide in population and the Fisher block is blind).

What is established. Gated linear units [4, 5] and multiplicative (sigma–pi) units [7] are known; what is added here is the geometric reading (a third direction, the gate, decoupled from the measurement direction), the capacity statements of Theorem 4, and measurements under a common trainer. These measurements support a narrow conclusion: the widths predicted for the two quadrics are the widths at which a gated layer with skip solves them; an accuracy advantage over ReLU layers was observed only at about eighteen parameters without skip, on two problems out of three; and no advantage is observed from thirty-two parameters upwards or on the higher-dimensional datasets.

4.3 Summary of the two capacity results

What the two theorems of this section and of Section 3.4 say: Theorem 3 settles, for the two gate families analysed, the question of shared-weight depth of a single oblique block (the non-linearity acts on at most $r+m$ linear functionals, whatever T); for these blocks, recursion alone does not give the non-linearity access to new directions of the input, and sensitivity to such directions has to come from gate directions that read the kernel or from non-aligned projectors. Theorem 4 situates the triceptron: it contains SiLU networks and has ReLU networks as uniform limits; it approximates quadrics to any accuracy at fixed width $\max(p, n)$; and it has the cell-count bound of ReLU in the hard regime. The experiments reproduce the minimal widths of (ii) and show an accuracy gap to ReLU layers only at about eighteen parameters without skip; they do not show an advantage at larger widths. Neither theorem depends on how the weights are obtained: they are statements about the *architecture*.

5 An analytic discriminant block

5.1 From Fisher's rank-one direction to a rank-r projector

For two classes, the Fisher-optimal oblique perceptron [8] has representation direction $u = \mu_1 - \mu_0$ and measurement direction $w^* = S_W^{-1}u$: a result in closed form. The multi-class, multi-rank generalisation is the Fisher generalised eigenvalue problem (GEVP)

$$S_B\, v_i = \lambda_i\, S_W\, v_i, \qquad \lambda_1 \geq \lambda_2 \geq \cdots, \tag{7}$$

solved by a standard dense symmetric eigensolver (`scipy.linalg.eigh(S_B, S_W)`), with no gradient-based optimisation. The generalised eigenvectors are the *measurement* directions (at rank one and for two classes, $v_1 \propto S_W^{-1}(\mu_1 - \mu_0) = w^*$), and the representation directions follow:

$$W_{\text{raw}} = [v_1, \ldots, v_r], \qquad U_{\text{raw}} = S_W\, W_{\text{raw}} = \left[S_B v_i/\lambda_i\right]_i \subset \operatorname{span}(\mu_c - \bar{\mu}), \tag{8}$$

the inclusion coming from $S_W v_i = S_B v_i/\lambda_i \in \operatorname{im} S_B$. The oblique code $c = (W^\top U)^{-1} W^\top h$ is then exactly the vector of Fisher discriminant scores, the sufficient statistic of linear discriminant analysis (LDA); Proposition 3 states the equivalence precisely.

Remark 3 (A convention that is easy to get wrong). Setting $U = [v_i]$ and $W = S_W^{-1}U$ instead shifts both matrices by a power of S_W^{-1} relative to the natural convention (u in the space

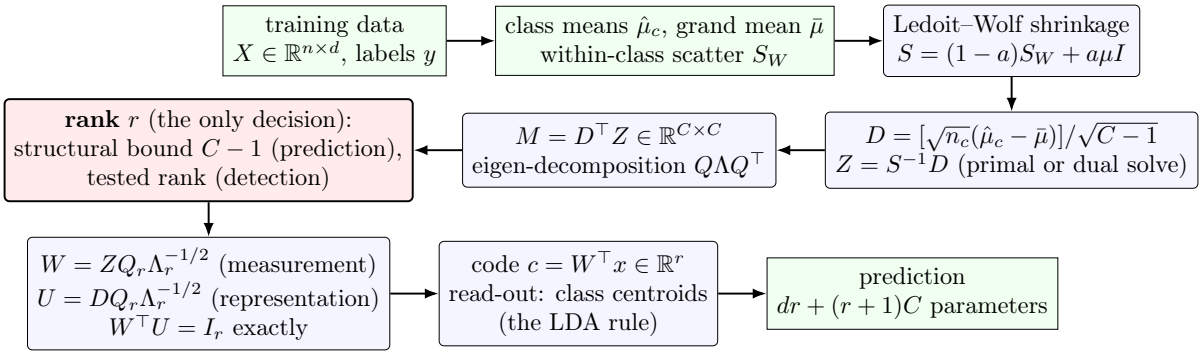


Figure 4: The analytic pipeline. Every box is a standard linear-algebra computation (factorisation, linear solves, one small symmetric eigen-decomposition); the only decision is the rank (Section 8). No parameter is learned by gradient or by any iterative optimisation. The $d \times d$ problem never appears: the eigen-decomposition is $C \times C$ (Section 6).

of means, $w = S_W^{-1}u$). The code is then $V^\top S_W^{-1}x$, which is *not* the vector of Fisher scores $V^\top x$: the resulting classifier is no longer LDA. The geometric readings (angles to the true subspaces, Kronecker and kernel variants) and the optimality statements of this report hold for convention (8), which is that of the M-orthogonal projector $P = U(U^\top MU)^{-1}U^\top M$ with $M = S_W^{-1}$.

When S_W is noisy (n small relative to Cd), it is replaced by its Ledoit–Wolf shrinkage estimator $S = (1-a)S_W + a\mu I$; Section 12 shows that this is necessary well before $n \leq d$.

5.2 Re-whitening: exact idempotence

Let $V = [v_1, \ldots, v_r]$. The Gram matrix $G = W_{\text{raw}}^\top U_{\text{raw}} = V^\top S_W V$ is symmetric positive definite, and equals I_r only if the eigenvectors are normalised to be S_W-orthonormal. This is the normalisation returned by `scipy.linalg.eigh`, so that in the implementation G differs from I_r only through the small ridge added to S_W for numerical safety and through round-off; with any other normalisation, or another solver, $G \neq I_r$. Re-whitening by its inverse square root,

$$U \leftarrow U_{\text{raw}}\, G^{-1/2}, \qquad W \leftarrow W_{\text{raw}}\, G^{-1/2}, \tag{9}$$

gives $W^\top U = G^{-1/2}GG^{-1/2} = I_r$ exactly, hence $P = UW^\top$ idempotent by construction, $\kappa(G) = 1$ and $\det(I + P/L) = (1 + 1/L)^r > 0$. Measured: $\|W^\top U - I_r\|$ and $\|P^2 - P\|_F$ of the order of 10^{-15}. (The factorisation of Section 6 makes this step unnecessary: it produces biorthogonal pairs directly.)

5.3 Equivalence with linear discriminant analysis, and the read-out

The block is not a new classifier, and the equivalence should be stated once, exactly.

Proposition 3 (The block with a centroid read-out is LDA)**.** *Let $S \succ 0$ be the within-class covariance estimate used by the block (raw or shrunk), $\pi_k = n_k/n$, and let (U, W) be the biorthogonal pair of rank $r = \operatorname{rank} S_B$ (generically $C-1$), written in the form $W = S^{-1}DQ_r\Lambda_r^{-1/2}$, $U = SW$ of Proposition 4.*

(i) $\operatorname{span}(W) = S^{-1}\operatorname{span}\{\mu_k - \bar{\mu}\}$ *and* $W^\top SW = I_r$*: the code $c = W^\top x$ is a whitened coordinate system of the discriminant subspace.*

(ii) For every x and every pair of classes, the LDA log-odds with covariance S, $\delta_k(x) - \delta_l(x) = (\mu_k - \mu_l)^\top S^{-1}\big(x - \frac{1}{2}(\mu_k + \mu_l)\big) + \log(\pi_k/\pi_l)$, is an affine function of the code; conversely every affine function of the code is an affine function of the LDA discriminant scores.

(iii) The plug-in LDA rule with covariance S is the nearest-centroid rule in code space,

$$\hat{k}(x) = \arg\max_k \; -\tfrac{1}{2}\big\|W^\top x - W^\top \mu_k\big\|_2^2 + \log \pi_k. \tag{10}$$

Hence the block followed by the centroid read-out (10) *makes exactly the predictions of LDA with the covariance estimate S; any other affine read-out of the code (for instance least squares on the class indicators) is a different classifier built on the same r coordinates.*

Proof. (i) The columns of D span $\operatorname{span}\{\mu_k - \bar{\mu}\}$, and Q_r spans the orthogonal complement of $\ker D = \ker(D^\top S^{-1} D)$, so DQ_r and D have the same column space; $W^\top S W = \Lambda_r^{-1/2} Q_r^\top D^\top S^{-1} D Q_r \Lambda_r^{-1/2} = I_r$. (ii) $S^{-1}(\mu_k - \mu_l) \in \operatorname{span}(W)$ by (i), so the log-odds depend on x through $W^\top x$ only, affinely; the $C-1$ independent log-odds and the r code coordinates span the same space of affine functions. (iii) $P = UW^\top$ is the S^{-1}-orthogonal projector onto $\operatorname{span}\{\mu_k - \bar{\mu}\}$, so $\|x - \mu_k\|_{S^{-1}}^2 = \|P(x-\mu_k)\|_{S^{-1}}^2 + \|(I-P)(x-\mu_k)\|_{S^{-1}}^2$. Since $\mu_k - \bar{\mu} \in \operatorname{im} P$, the second term equals $\|(I-P)(x-\bar{\mu})\|_{S^{-1}}^2$ for every k; the first is $(x-\mu_k)^\top W (U^\top S^{-1} U) W^\top (x - \mu_k) = \|W^\top (x-\mu_k)\|_2^2$ because $U^\top S^{-1} U = W^\top S W = I_r$. □

Verified on the digits data (ten classes, $n = 10$, 30 and 100 per class, ten subsamples each): the predictions of the block with the centroid read-out coincide with those of an LDA computed directly from the same shrunk S on 100% of the test points, and with those of scikit-learn's `LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto")` on 97.1%, 98.6% and 99.2% of them. The residual disagreement is one of covariance estimate, not of classifier: in the implementation used here, that solver applies the Ledoit–Wolf estimator to each class separately, after standardising the features within the class, and averages the resulting class covariances, whereas the block shrinks the pooled within-class scatter once.

Which read-out. Two analytic read-outs of the code are used in this report. The *centroid read-out* (10) needs no fitting (rC centroid coordinates and C priors) and is the LDA rule. The *least-squares read-out* regresses the class indicators on the code ($(r+1)C$ coefficients); it was the read-out of the reference implementation. The two are equivalent within one point on the synthetic four-class problems of Section 11, but on the ten-class digits data the least-squares read-out is 1.7 to 2.4 points *below* the centroid read-out at every sample size, with or without selection (paired 95% intervals excluding zero), the classical masking effect of indicator regression with many classes [9, Section 4.2]. The ranking is not universal: on the next-character task of Appendix F (35 classes, one-hot inputs) the least-squares read-out is 3.5 to 4.6 points above the centroid rule. The centroid read-out is used as the reference throughout the report, because it is the LDA rule, which gives the block a precise statistical meaning, and because it is the better of the two on the central benchmark. The least-squares read-out is kept only where it is itself the object of a comparison (the baseline pipeline diagnosed in Section 12 and the rows of its ablation, the paired comparisons of Sections 11 and 12, the controls of Section 13), and a regression read-out is used where the features are not the code of a single block (the triceptron features of Section 4.2, the concatenated band codes of Appendix I), since the centroid rule has the meaning of LDA only on the code of one block.

5.4 An analytic gate

For the recursion studied in Section 3, the oblique gate $y = (1 - a(x))x + a(x)Px$, $a(x) = \sigma(s^\top x + b)$, is also fixed in closed form: s is the first column of W normalised (dominant Fisher direction) and $b = -s^\top \bar{\mu}$.

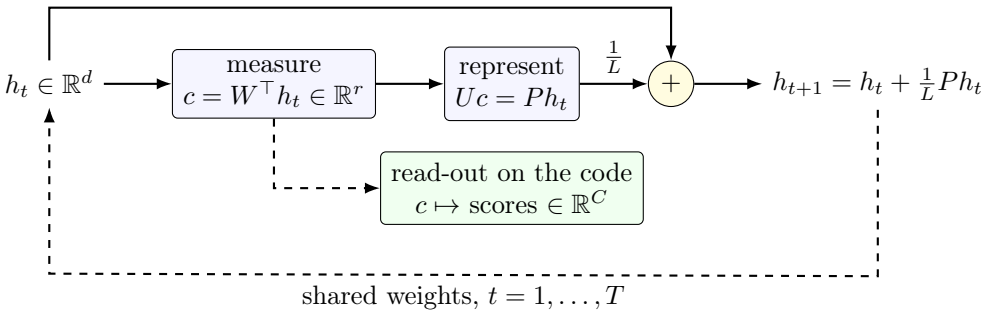


Figure 5: The reduced architecture: one analytic block (U, W) with $W^\top U = I_r$, optionally applied T times with shared weights, and an analytic read-out connected to the code $c = W^\top h$, not to the full state. Because $W^\top P = W^\top$, each pass of the dashed loop multiplies the code by the constant $1 + 1/L$ (Theorem 1), a rescaling that a linear read-out absorbs; for the gated block of Proposition 1 the code does not change at all.

5.5 The reduced architecture

The input is the raw dimension d (no input projection), the output the C classes. Stored parameters, with r the used rank: $dr + (r+1)C$ (W, the rC centroid coordinates and the C constant terms of the centroid read-out, the same count as a least-squares read-out; U is not needed at inference when the read-out acts on the code), i.e. 76 for $d = 20$, $r = 3$, $C = 4$. Nothing is optimised by gradient.

6 Exact rank-$(C-1)$ factorisation of the Fisher problem

The analytic block solves `eigh(S_B, S_W)` in dimension d: a Cholesky factorisation of S_W ($d^3/3$), reduction to standard form ($\approx d^3$), and a full diagonalisation (4–9 d^3 with vectors). But S_B has rank at most $C-1$, and this structural property brings everything down to a problem of size C without any approximation.

6.1 The equivalent $C \times C$ problem

Write $S_B = DD^\top$ with $D = [\sqrt{n_c}(\mu_c - \bar{\mu})]_c / \sqrt{C-1} \in \mathbb{R}^{d \times C}$.

Proposition 4 (Dual reduction of the Fisher GEVP)**.** *Let $S_W \succ 0$, $Z = S_W^{-1} D$ and $M = D^\top Z = D^\top S_W^{-1} D \in \mathbb{R}^{C \times C}$ (symmetric positive semi-definite, rank $\leq C-1$) with diagonalisation $M = Q \Lambda Q^\top$. The* non-zero *eigenpairs of $S_B v = \lambda S_W v$ are exactly $(\lambda_i, v_i = Z q_i / \sqrt{\lambda_i})$, with $v_i^\top S_W v_j = \delta_{ij}$. Moreover, for any $r \leq \operatorname{rank} M$, with Q_r, Λ_r the r leading eigenvectors and (positive) eigenvalues, and*

$$W = Z Q_r \Lambda_r^{-1/2} \quad (\textit{measurement}), \qquad U = D Q_r \Lambda_r^{-1/2} \quad (\textit{representation}), \tag{11}$$

one has $W^\top U = I_r$ automatically: the re-whitening (9) becomes unnecessary.

Proof. Let $S_W = RR^\top$ be a Cholesky factorisation and $E = R^{-1} D$. With $z = R^\top v$, the GEVP $DD^\top v = \lambda RR^\top v$ is the standard problem $EE^\top z = \lambda z$. The non-zero eigenvalues of $EE^\top$ and of $E^\top E = D^\top R^{-\top} R^{-1} D = M$ coincide with their multiplicities, and if $E^\top E q = \lambda q$ with $\lambda > 0$ and $\|q\| = 1$, then $z = Eq/\sqrt{\lambda}$ is a unit eigenvector of $EE^\top$ for λ ($\|z\|^2 = q^\top E^\top E q / \lambda = 1$), orthonormal eigenvectors q_i giving orthonormal z_i. Hence $v = R^{-\top} Eq/\sqrt{\lambda} = S_W^{-1} Dq/\sqrt{\lambda} = Zq/\sqrt{\lambda}$ and $v_i^\top S_W v_j = z_i^\top z_j = \delta_{ij}$. Finally, with (11) (for which $U = S_W W$), $W^\top U = \Lambda_r^{-1/2} Q_r^\top D^\top S_W^{-1} D Q_r \Lambda_r^{-1/2} = I_r$. □

Cost: one factorisation of S_W ($d^3/3$ flops), C solves (d^2C), one $C \times C$ diagonalisation and two $d \times C \times r$ products, instead of roughly $10d^3$ flops for the full generalised eigen-decomposition. Nothing is approximated: $\|P_{\text{fast}} - P_{\text{ref}}\|/\|P_{\text{ref}}\| \leq 2 \times 10^{-9}$ and the selected rank is identical (Table 3).

6.2 Dual form for $n < d$: no $d \times d$ matrix

With Ledoit–Wolf shrinkage, $S = (1-a)X_c^\top X_c/n + a\mu I$, where $X_c \in \mathbb{R}^{n \times d}$ holds the samples centred by their class means, is a multiple of the identity plus a matrix of rank at most $n - C$. For $a\mu > 0$ the Woodbury identity gives, for any right-hand side B,

$$S^{-1}B = \frac{B}{a\mu} - \frac{(1-a)/n}{a\mu} X_c^\top \left(a\mu I_n + \tfrac{1-a}{n} X_c X_c^\top \right)^{-1} X_c B, \tag{12}$$

an $n \times n$ system ($\mathcal{O}(n^2 d + n^3)$) and never a $d \times d$ matrix. The Ledoit–Wolf coefficient itself is computed without $X_c^\top X_c$ through two exact identities: $\|X_c^\top X_c\|_F^2 = \|X_c X_c^\top\|_F^2$ (an $n \times n$ Gram) and $\mathbf{1}^\top (X_c^{\circ 2})^\top X_c^{\circ 2} \mathbf{1} = \sum_i \|x_i\|^4$ ($\mathcal{O}(nd)$). Verified: coefficient identical to scikit-learn's to 10^{-17}, projector identical to 3.5×10^{-11} ($d = 200$, $n_{\text{train}} = 42$). This is the regime of high-dimensional features computed from a few dozen to a few hundred trials, as in single-subject EEG classification: at $d = 2080$, $n = 60$, the fit of the corrected block goes from 3.9 s to 2.4 ms in the run of Table 3.

6.3 Batched permutation test

The permutations change S_B but not the metric. Since $D_p = \bar{X}^\top Y_p N^{-1/2}/\sqrt{C-1}$ ($\bar{X}$ globally centred, Y_p the indicator matrix of the permuted classes), all n_{perm} matrices D_p come from *one* product $\bar{X}^\top [Y_1 \cdots Y_{n_{\text{perm}}}]$, all $S^{-1}D_p$ from *one* solve with $n_{\text{perm}}C$ right-hand sides, and the maximal eigenvalues from one `eigvalsh` on a stack of $C \times C$ matrices. Cost $\mathcal{O}(n_{\text{perm}}(nd + d^2C))$ instead of $\mathcal{O}(n_{\text{perm}}d^3)$, entirely in BLAS-3: $\times 116$ at $d = 784$ in the run of Table 3, and 0.2 s at $d = 2080$, $n = 600$, where the reference (about 200 full eigen-decompositions) was not run. As shown in Section 8, the metric of the batched test should be the label-free total scatter S_T for the null to be exchangeable; Table 3 times the shared-S_W variant, in order to compare like with like with the reference implementation, and the S_T variant costs one additional factorisation. The sequential rule of Section 8 reuses the same batch: the deflated statistics are obtained from the stack of $C \times C$ matrices without any new solve.

6.4 Further options

- **Analytic rank (Bartlett/Wilks).** The sequential Bartlett test on Wilks' Λ, $\chi^2 = (n - 1 - \frac{d+C}{2}) \sum_{i>k} \ln(1 + \tilde{\lambda}_i)$ with $(d-k)(C-1-k)$ degrees of freedom and $\tilde{\lambda}_i = \lambda_i (C-1)/(n-C)$ the eigenvalues on the scale of the unnormalised scatter matrices, costs nothing beyond the λ_i already computed. Its asymptotic calibration is approximate in small samples and with shrinkage; it is implemented as an option and evaluated in Section 8.

- **Minimal inference.** By Proposition 1, a read-out on the code depends neither on T nor on the gate: inference reduces to $c = W^\top x$ followed by an $r \times C$ map, $\mathcal{O}(dr)$ per sample, the minimum for a rank-r linear classifier.

- **Parallelism.** Everything is BLAS-3; per-subject fits are independent; the only sequential part is the $C \times C$ diagonalisation.

- **Online update.** S_W and D are sufficient statistics: a new sample updates the Cholesky factor in $\mathcal{O}(d^2)$ (rank one) and replays the C solves, with no GEVP. The rank-one update

of the within-class scatter is exact: adding x to a class with n_c samples and mean μ_c adds $\frac{n_c}{n_c+1}(x-\mu_c)(x-\mu_c)^\top$. Implemented (`OnlineBlock`) and checked ($d = 20$, 200 samples then 100 online additions): agreement with a batch recomputation at the same frozen (a, μ, n) to 2.6×10^{-15}. Two reservations: with the shrinkage (a, μ) and the normalisation n frozen between refits, the projector differs from a full refit by 0.8% in relative Frobenius norm in this test; and a pure-`numpy` rank-one Cholesky update (a Python loop) does not realise the asymptotic gain (67 ms per sample at $d = 2080$, $n = 600$, against 58 ms for a full fast refit). The $\mathcal{O}(d^2)$ versus $\mathcal{O}(d^3/3)$ gain would require a compiled update, which is not provided.

- **What not to do.** A randomised SVD or a Lanczos iteration on $S_W^{-1}S_B$ would be approximate and more costly than the exact reduction, which exploits the rank-$(C-1)$ structure instead of guessing it.

Table 3: Exactness and timing of the factorisation on a sandbox CPU (corrected block: minimum of 3 runs after warm-up; baseline with 200 permutations: one run after warm-up; the baseline reference at $d = 2080$ was not run). "ref.": full GEVD `eigh(S_B, S_W)`. Timings vary by a factor of up to two between runs; only the orders of magnitude are meaningful.

d	n_{train}	C	corrected ref. (s)	corrected fast (s)	gain	baseline ref. (+200 perm.)	baseline fast	gain
20	800	4	0.002	0.0007	2×	0.04	0.010	4×
64	1200	10	0.004	0.0016	3×	0.14	0.026	5×
200	400	4	0.016	0.0023	7×	0.72	0.013	55×
784	1000	10	0.596	0.0437	14×	20.9	0.180	116×
2080	60	2	3.899	0.0024	1644×	(not run)	0.022	—
2080	600	4	4.981	0.0468	106×	(not run)	0.202	—

Exactness ($d = 20$): $\|P_{\text{fast}} - P_{\text{ref}}\|/\|P_{\text{ref}}\| = 9 \times 10^{-11}$ (corrected), 2×10^{-9} (baseline); $\|W^\top U - I\| = 2 \times 10^{-15}$ without re-whitening; Ledoit–Wolf coefficient identical to scikit-learn ($\leq 10^{-17}$); identical r^*; dual $d = 200$, $n = 42$: 3.5×10^{-11}, same accuracy (100%).

At small d the cost is dominated by Python overheads and the gain is marginal; it becomes substantial from $d \gtrsim 200$ and above all in the $n < d$ regime, where the dual form avoids every $d \times d$ matrix. The exact reduction also makes the block *simpler*: no re-whitening, no GEVD, a single primal/dual solver.

6.5 Against a specialised LDA solver

The gains of Table 3 are measured against a *dense generalised eigen-decomposition*, which no careful implementation of LDA uses. The relevant reference for an LDA estimator is a specialised solver. Table 4 compares the fit of the block (rank $C-1$, Ledoit–Wolf shrinkage) with scikit-learn's `LinearDiscriminantAnalysis`: the `lsqr` and `eigen` solvers with `shrinkage="auto"`, which estimate the same kind of model (LDA with a shrunk covariance), and the `svd` solver, which has no shrinkage and is therefore a different estimator. Against the solver that estimates the same model, the block is 5 to 11 times faster when $n \geq d$ (these ratios vary by up to a factor of 1.5 between runs of the same script): this is the order of magnitude of the computational contribution in that regime, and part of it may come from the per-class shrinkage of the library, which calls the covariance estimator C times. The three-orders-of-magnitude figure appears only when $n \ll d$, where the library solver forms and factorises $d \times d$ matrices and the dual form of the block does not; there the `svd` solver, which also works in the sample space, is only about 8 times slower than the block, but it cannot shrink. Predictions of the block with the centroid read-out and of the `lsqr` solver agree on 98.9% of 800 test points ($d = 20$, $n = 15$ per class, 20 draws; minimum 97.5%; 98.8% with the least-squares read-out): the two estimate LDA with different shrunk covariances (pooled against per class, Section 5.3); with the same S the agreement is exact (Proposition 3).

Table 4: Fit time in seconds (minimum of three runs after warm-up, same data as Table 3) of the block and of scikit-learn's LDA solvers. The `eigen` solver was not run at $d = 2080$. Single CPU, indicative only.

d	n_{train}	C	block	`lsqr`, shrunk	`eigen`, shrunk	`svd`, no shrinkage	`lsqr` / block
20	800	4	0.0006	0.0051	0.0063	0.0015	8×
64	1200	10	0.0015	0.0163	0.0251	0.0080	11×
200	400	4	0.0033	0.0186	0.0179	0.0119	6×
784	1000	10	0.0630	0.3366	0.3538	0.3103	5×
2080	60	2	0.0017	3.172	—	0.0131	1820×
2080	600	4	0.0435	3.553	—	0.3366	82×

7 A complete worked example of the analytic block

This section follows the construction of Section 5 on a three-dimensional example small enough for every matrix to be printed, so that each step can be checked by hand (`ropn_auto_exemple_complet.py`; the script also regenerates the figures of this report).

Data. Three classes in $\mathbb{R}^3$, 30 samples per class drawn from Gaussians with a *common* covariance $\Sigma = \begin{pmatrix} 1 & 0.7 & 0.5 \\ 0.7 & 1 & 0.2 \\ 0.5 & 0.2 & 0.6 \end{pmatrix}$ and means $(0,0,0)$, $(3.5, 0.5, 0)$, $(0.5, 3.5, 0)$. The class information lives in the plane (x_1, x_2); the third coordinate has no mean difference but is correlated with x_1 (0.5), which is what will tilt the measurement directions out of the plane of the means. A random split keeps 60 samples for training (19, 22 and 19 per class) and 30 for test. The pipeline is shown in Figure 4; the geometry of the result in Figure 6.

Step 1: class statistics. Class means (rows) and grand mean:

$$\hat{\mu} = \begin{pmatrix} -0.241 & 0.017 & -0.097 \\ 3.471 & 0.461 & -0.027 \\ 0.536 & 3.383 & 0.168 \end{pmatrix}, \qquad \bar{\mu} = (1.366,\ 1.246,\ 0.012).$$

Within-class scatter (centred per class, divided by $n = 60$) and its Ledoit–Wolf shrinkage with $a = 0.196$, $\mu = \operatorname{tr} S_W / d = 0.697$:

$$S_W = \begin{pmatrix} 0.707 & 0.401 & 0.432 \\ 0.401 & 0.817 & 0.085 \\ 0.432 & 0.085 & 0.567 \end{pmatrix}, \qquad S = (1-a)S_W + a\mu I = \begin{pmatrix} 0.705 & 0.323 & 0.347 \\ 0.323 & 0.793 & 0.069 \\ 0.347 & 0.069 & 0.592 \end{pmatrix}.$$

The shrinkage pulls the off-diagonal entries towards zero by 20% and the diagonal towards 0.697; with 60 samples in dimension 3 it is mild, as it should be.

Step 2: the between-class factor and the $C \times C$ problem. $D = [\sqrt{n_c}(\hat{\mu}_c - \bar{\mu})]_c / \sqrt{C-1}$ has one column per class and $S_B = DD^\top$:

$$D = \begin{pmatrix} -4.953 & 6.981 & -2.559 \\ -3.787 & -2.603 & 6.588 \\ -0.338 & -0.130 & 0.479 \end{pmatrix}, \qquad Z = S^{-1}D = \begin{pmatrix} -8.515 & 19.969 & -12.973 \\ -1.709 & -10.479 & 12.985 \\ 4.619 & -10.713 & 6.909 \end{pmatrix}.$$

The third row of Z is not small although the third row of D is: the metric S^{-1} transfers the correlation between x_3 and x_1 into the measurement directions. The $C \times C$ matrix of

Proposition 4 and its eigenvalues are

$$M = D^\top S^{-1} D = \begin{pmatrix} 47.082 & -55.592 & 12.739 \\ -55.592 & 168.072 & -125.262 \\ 12.739 & -125.262 & 122.051 \end{pmatrix}, \qquad \lambda = (283.48,\ 53.72,\ 0.00).$$

The zero eigenvalue is structural: the columns of D satisfy $\sum_c \sqrt{n_c}\, D_{\cdot c} = 0$ (the weighted class means average to the grand mean), so $\operatorname{rank} D \le C - 1$ and $\operatorname{rank} M \le C - 1 = 2$. Nothing of dimension d was diagonalised.

Step 3: rank. The permutation test with the total-scatter metric (500 label shuffles) gives an exact level-5% threshold of 4.84 for the largest eigenvalue; the two eigenvalues in the S_T metric, 28.97 and 19.21, exceed it by far (Figure 6, right), so the tested rank is 2, equal to the structural bound, and the used rank is $r = 2$.

Step 4: measurement and representation. With Q_r the first two eigenvectors of M and Λ_r their eigenvalues,

$$W = Z Q_r \Lambda_r^{-1/2} = \begin{pmatrix} -1.483 & -0.548 \\ 0.924 & -0.856 \\ 0.794 & 0.304 \end{pmatrix}, \qquad U = D Q_r \Lambda_r^{-1/2} = \begin{pmatrix} -0.472 & -0.558 \\ 0.309 & -0.835 \\ 0.019 & -0.069 \end{pmatrix},$$

$$W^\top U = \begin{pmatrix} 1.000 & 0.000 \\ 0.000 & 1.000 \end{pmatrix}.$$

The representation directions u_i are (nearly) in the plane of the means (third components 0.019 and −0.069); the measurement directions w_i have large third components (0.794, 0.304): the block measures along directions tilted out of the plane of the means, because the noise is correlated with x_3 and the metric compensates for it. Biorthogonality is exact without any re-whitening, as Proposition 4 states. The projector and its diagnostics:

$$P = U W^\top = \begin{pmatrix} 1.005 & 0.042 & -0.544 \\ 0.000 & 1.001 & -0.008 \\ 0.010 & 0.077 & -0.006 \end{pmatrix}, \qquad \|P^2 - P\| = 10^{-15}, \quad \operatorname{rank} P = 2,$$

$$\|P\|_2 = 1.147 = \frac{1}{\sigma_{\min}(Q_W^\top Q_U)},$$

with a minimal angle of 60.7° between image and kernel; the metric bound of Proposition 6, $\frac{1}{2}(\sqrt{\kappa(S)} + 1/\sqrt{\kappa(S)}) = 1.34$ (with $\sqrt{\kappa(S)} = 2.24$), is satisfied. P is visibly not symmetric (the entry −0.544 in position $(1,3)$ against 0.010 in $(3,1)$): it is oblique. The block assembled by hand coincides with `fit_block_fast` to 4×10^{-16}.

Step 5: read-out and prediction. The read-out is the centroid rule of Proposition 3: the class means of the training codes,

$$m_0 = (0.295,\ 0.088), \qquad m_1 = (-4.743,\ -2.305), \qquad m_2 = (2.464,\ -3.140),$$

and the constant terms $-\frac{1}{2}\|m_k\|^2 + \log \pi_k = (-1.197,\ -14.907,\ -9.115)$; the score of class k is $c^\top m_k - \frac{1}{2}\|m_k\|^2 + \log \pi_k$. Nothing is fitted beyond these averages, and the rule gives 96.7% test accuracy with $dr + rC + C = 15$ parameters. An LDA computed directly from the shrunk covariance S of Step 1 makes the same predictions on every test point, as Proposition 3 states; the least-squares read-out of the same code would give 90.0% (same prediction on 93% of the test points). For the first test point, $x = (5.074, 2.205, 0.656)$ of class 1: code $c = W^\top x = (-4.967, -4.471)$, $Px = (4.836, 2.201, 0.216)$, residual $(I - P)x = (0.238, 0.004, 0.440)$; squared distances to the three centroids $\|c - m_k\|^2 = (48.48,\ 4.74,\ 56.99)$, scores $(-3.059,\ 18.957,\ -7.317)$, predicted class 1. The residual is mostly along x_3: the part of x that the block declares non-discriminant.

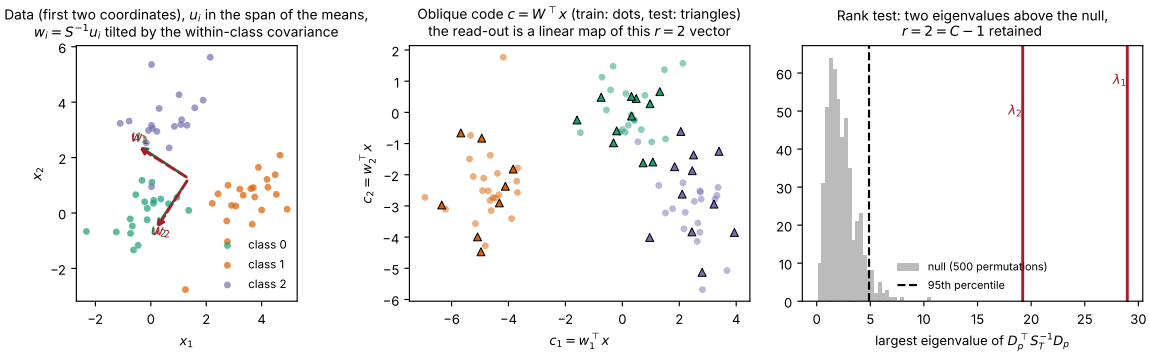


Figure 6: The worked example. Left: training data in the plane of the means with the representation directions u_i (green) and the measurement directions w_i (red, dashed); their in-plane tilt is small, the obliquity is mostly in the third coordinate (not shown), where the noise is correlated with x_1. Centre: the two-dimensional oblique code $c = W^\top x$ on which the read-out acts; test points are triangles. Right: the rank test, the null distribution of the largest eigenvalue over 500 label permutations (grey), its level-5% threshold (dashed) and the two observed eigenvalues (red), both far above the null.

Step 6: what the recursion does to this point. With the analytic gate ($s \propto w_1$, $b = -s^\top \bar{\mu}$) and the shared block applied T times:

T	h_T	code $W^\top h_T$	$\|(I-P)h_T\|$
0	(5.074, 2.205, 0.656)	(−4.967, −4.471)	0.500
1	(5.049, 2.204, 0.610)	(−4.967, −4.471)	0.447
2	(5.027, 2.204, 0.568)	(−4.967, −4.471)	0.400
4	(4.989, 2.204, 0.498)	(−4.967, −4.471)	0.320

The code does not move at all (Proposition 1(i)); the residual shrinks geometrically by the constant factor $1 - a(x) = 0.894$ per step (Proposition 1(ii)): $0.500 \times 0.894^4 = 0.320$. The read-out, which sees only the code, returns the same scores for every T. This is the content of Proposition 1, on one point.

8 Rank selection: structural, tested and used ranks

The rank is the only decision of the analytic block (Figure 4). Three different objects are often merged under the name "automatic rank", and they are kept apart here.

- The **structural rank** $r_{\text{struct}} = \min(C-1, d)$ is a bound, exact since $\operatorname{rank} S_B \le C-1$. It involves no statistics.
- The **tested rank** $r_{\text{test}} \in \{0, \dots, r_{\text{struct}}\}$ is the output of a statistical rule. It is a *detection* statement (how many discriminant directions are distinguishable from noise at a given level) and it may be zero.
- The **used rank** r_{used} is the number of directions the classifier employs. The implementation returns the three values; when a tested rule is requested the classifier uses $\max(1, r_{\text{test}})$, and the recommended default for prediction is $r_{\text{used}} = r_{\text{struct}}$, for the reasons measured in Section 8.4.

What is *guaranteed* is limited to two statements: the structural bound, and the level of the permutation test under the *global* null hypothesis (no class structure at all). Everything else

in this section is *measured*: the behaviour under alternatives of every rank, the sequential rule, the Bartlett rule, and the consequences for prediction.

8.1 The rules

Permutation test of the global null hypothesis. The labels are shuffled n_{perm} times and the leading generalised eigenvalue $\lambda_1^{(p)}$ is recomputed for each shuffle. With $p = (1+\#\{p : \lambda_1^{(p)} \geq \lambda_1\})/(n_{\text{perm}}+1)$, rejecting when $p \leq \alpha$ has level at most α under the global null hypothesis for every n, d and noise distribution, provided the statistic is computed in the same way for the observed and the shuffled labels (exchangeability [10]). The implementation uses this exact rule; an interpolated empirical quantile of the shuffled values is not the same rule and is slightly anti-conservative.

The metric of the test matters. The natural fast implementation shares one factorisation of S_W across all permutations (only S_B changes when labels are shuffled). This breaks the proviso above: S_W computed with the true labels excludes exactly the true between-class direction but not the permuted ones, so the observed statistic is not exchangeable with the shuffled ones. The remedy costs one factorisation: use the label-free *total* scatter S_T (shrunk) as the metric of the test. Without shrinkage, $S_T = S_W + \frac{C-1}{n} S_B$ implies that the two GEVPs share their eigenvectors, with $\lambda_T = \lambda_W/(1 + \frac{C-1}{n}\lambda_W)$, an increasing function of λ_W: the ordering of the directions and the number of non-zero eigenvalues are unchanged (with two separate Ledoit–Wolf shrinkages the identity holds only approximately).

Two extensions beyond rank zero, neither with a guarantee. *Max rule* (`rank="perm"`): every λ_i is compared with the level-α threshold of the *largest* shuffled eigenvalue; the tested rank is the number of eigenvalues above it. Comparing $\lambda_2, \lambda_3, \ldots$ with the null distribution of a maximum is expected to be conservative. *Sequential rule with deflation* (`rank="seq"`): at step k, the $(k+1)$-th eigenvalue is compared with the distribution, over the shuffles, of the largest eigenvalue computed after the first k estimated directions have been removed from the metric; the procedure stops at the first non-rejection. With (U_k, W_k) the biorthogonal pair of the first k directions in the metric of the test and $P_k = U_k W_k^\top$,

$$(I - P_k)^\top S^{-1} (I - P_k) = S^{-1} - W_k W_k^\top,$$

so that the deflated statistic of a shuffle is the largest eigenvalue of $M_p - B_p^\top B_p$ with $B_p = W_k^\top D_p$: no new factorisation or solve is needed. The first step is the exact test above. For $k \geq 1$ the labels are no longer exchangeable under the hypothesis "rank exactly k" and the deflation uses estimated directions: the rule is a heuristic, in the spirit of parallel analysis, whose error rates are measured below.

Bartlett's sequential test on Wilks' Λ (Section 6) [11] is analytic and asymptotic. Information-theoretic criteria such as MDL [12] play the same role in array processing; MDL is not reused here because it assumes isotropic additive noise on the eigenvalues of a covariance, an assumption foreign to the Fisher GEVP.

8.2 Calibration under the global null hypothesis

The total-scatter permutation test is at its nominal level in the nine configurations (between 3.9% and 5.5%, for a standard error of 0.5 point), as the theory requires. The shared-S_W variant is anti-conservative as soon as n is not large compared with d (up to 31.6% at $n = 30$, $d = 36$) and reaches its level only for $n \gg d$. The Bartlett rule is conservative to the point of never rejecting when n is of the order of d. Under the global null hypothesis the sequential rule coincides with the max rule (same first step).

Table 5: Probability of a non-zero tested rank on pure noise (nominal level 5%; 2000 independent draws, so that the binomial standard error is 0.5 point at 5%; 199 or 200 permutations). Upper part: isotropic Gaussian noise. Lower part: anisotropic Gaussian noise ($\kappa = 10$), the null case of Table 6.

n	d	C	permutation, S_T	sequential, S_T	permutation, shared S_W	Bartlett
30	36	3	3.9%	(same test)	31.6%	0.0%
60	36	3	4.4%		9.7%	0.0%
120	36	3	5.5%		6.8%	0.0%
40	20	4	4.5%		13.1%	0.0%
120	20	4	4.5%		5.2%	0.2%
400	20	4	4.8%		5.0%	2.8%
40	20	4	4.7%	4.7%	—	—
120	20	4	4.8%	4.8%	—	—
400	20	4	4.4%	4.4%	—	—

8.3 Rank recovery under alternatives

The global null hypothesis is the least interesting case in practice. The three rules are therefore evaluated on Gaussian data whose class means span *exactly* q dimensions, for $q = 0, 1, 2, 3$ ($C = 4$, $d = 20$, $\kappa = 10$, shared covariance), with either equal strengths per direction (between-class standard deviation 1.2 along each of the q directions, in units of the average noise standard deviation) or graded strengths (1.6, 0.8, 0.4), at 10, 30 and 100 samples per class; 200 independent draws per cell, 200 permutations, level 5%. Tables 6 and 7 report, for each rule, the probabilities of under-estimating, recovering and over-estimating q.

Table 6: Rank recovery and predictive risk, *equal* strengths $(1.2, 1.2, 1.2)$. For each rule: $P(r_{\text{test}} < q)$ / $P(r_{\text{test}} = q)$ / $P(r_{\text{test}} > q)$ in % over 200 draws (binomial standard error at most 3.5 points). Right: test accuracy (%, 2000 test points, centroid read-out) with the rank used by the sequential rule and with $C - 1$, and paired differences in points with 95% intervals (conventions of Section 11); "oracle" is the true rank q.

n/class	q	max rule	sequential rule	Bartlett	acc., seq.	acc., $C-1$	seq. $-$ $(C-1)$	$(C-1)$ $-$ oracle
10	0	0.0 / 94.0 / 6.0	0.0 / 94.0 / 6.0	0.0 / 100.0 / 0.0	–	–	–	–
	1	0.5 / 98.5 / 1.0	0.5 / 98.5 / 1.0	99.0 / 1.0 / 0.0	44.0	43.9	$+0.1\,[-0.1, +0.3]^{\approx}$	$-0.1\,[-0.3, +0.1]^{\approx}$
	2	9.5 / 90.5 / 0.0	5.0 / 94.0 / 1.0	97.0 / 3.0 / 0.0	63.3	64.0	$-0.8\,[-1.3, -0.2]^{\downarrow}$	$-0.0\,[-0.2, +0.1]^{\approx}$
	3	25.5 / 74.5 / 0.0	8.5 / 91.5 / 0.0	51.5 / 48.5 / 0.0	84.8	85.9	$-1.1\,[-1.7, -0.6]^{\downarrow}$	$+0.0\,[+0.0, +0.0]^{\approx}$
30	0	0.0 / 92.0 / 8.0	0.0 / 92.0 / 8.0	0.0 / 99.0 / 1.0	–	–	–	–
	1	0.0 / 100.0 / 0.0	0.0 / 98.5 / 1.5	0.0 / 98.5 / 1.5	46.8	46.6	$+0.2\,[+0.1, +0.4]^{\approx}$	$-0.2\,[-0.4, -0.0]^{\approx}$
	2	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	0.0 / 98.5 / 1.5	68.7	68.7	$+0.0\,[-0.1, +0.2]^{\approx}$	$-0.0\,[-0.2, +0.1]^{\approx}$
	3	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	92.3	92.3	$+0.0\,[+0.0, +0.0]^{\approx}$	$+0.0\,[+0.0, +0.0]^{\approx}$
100	0	0.0 / 93.5 / 6.5	0.0 / 93.5 / 6.5	0.0 / 97.0 / 3.0	–	–	–	–
	1	0.0 / 99.5 / 0.5	0.0 / 99.0 / 1.0	0.0 / 98.5 / 1.5	47.5	47.4	$+0.0\,[-0.1, +0.2]^{\approx}$	$-0.1\,[-0.2, +0.1]^{\approx}$
	2	0.0 / 100.0 / 0.0	0.0 / 99.5 / 0.5	0.0 / 97.0 / 3.0	70.3	70.2	$+0.1\,[-0.0, +0.2]^{\approx}$	$-0.1\,[-0.2, +0.0]^{\approx}$
	3	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	94.4	94.4	$+0.0\,[+0.0, +0.0]^{\approx}$	$+0.0\,[+0.0, +0.0]^{\approx}$

- **Over-estimation is rare for $q \geq 1$.** It never exceeds 1.5% for the two permutation rules and 3% for Bartlett. (In the rows $q = 0$ the over-estimation rate is the false-positive rate of the global null hypothesis; the values 6 to 8% rest on 200 draws and are compatible with the 4.4 to 4.8% measured on 2000 draws of the same configurations in Table 5.)
- **Under-estimation is the failure mode, and it can be total.** With equal strengths and 10 samples per class the max rule misses a direction in 25.5% of the draws at $q = 3$, the sequential rule in 8.5%, Bartlett in 51.5% (and in 97 to 99% at $q = 1$ and 2, where it

Table 7: Rank recovery and predictive risk, *graded* strengths $(1.6, 0.8, 0.4)$. Same conventions as Table 6.

n/class	q	max rule	sequential rule	Bartlett	acc., seq.	acc., $C-1$	seq. $-$ $(C-1)$	$(C-1)$ $-$ oracle
10	0	0.0 / 94.0 / 6.0	0.0 / 94.0 / 6.0	0.0 / 100.0 / 0.0	–	–	–	–
	1	0.0 / 99.0 / 1.0	0.0 / 99.0 / 1.0	89.0 / 11.0 / 0.0	47.7	47.6	$+0.1\,[-0.1, +0.2]^{\approx}$	$-0.1\,[-0.2, +0.1]^{\approx}$
	2	53.5 / 46.5 / 0.0	37.0 / 63.0 / 0.0	100.0 / 0.0 / 0.0	56.7	60.9	$-4.2\,[-5.0, -3.4]^{\downarrow}$	$+0.2\,[-0.0, +0.5]^{\approx}$
	3	100.0 / 0.0 / 0.0	98.0 / 2.0 / 0.0	100.0 / 0.0 / 0.0	57.5	66.3	$-8.9\,[-9.7, -8.0]^{\downarrow}$	$+0.0\,[+0.0, +0.0]^{\approx}$
30	0	0.0 / 92.0 / 8.0	0.0 / 92.0 / 8.0	0.0 / 99.0 / 1.0	–	–	–	–
	1	0.0 / 99.5 / 0.5	0.0 / 99.0 / 1.0	0.0 / 98.5 / 1.5	49.0	49.0	$+0.1\,[-0.1, +0.2]^{\approx}$	$-0.1\,[-0.2, +0.1]^{\approx}$
	2	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	0.0 / 98.5 / 1.5	66.0	65.9	$+0.1\,[-0.1, +0.2]^{\approx}$	$-0.1\,[-0.2, +0.1]^{\approx}$
	3	69.0 / 31.0 / 0.0	50.0 / 50.0 / 0.0	34.0 / 66.0 / 0.0	71.2	74.4	$-3.3\,[-3.8, -2.8]^{\downarrow}$	$+0.0\,[+0.0, +0.0]^{\approx}$
100	0	0.0 / 93.5 / 6.5	0.0 / 93.5 / 6.5	0.0 / 97.0 / 3.0	–	–	–	–
	1	0.0 / 99.5 / 0.5	0.0 / 99.0 / 1.0	0.0 / 98.5 / 1.5	49.3	49.3	$-0.0\,[-0.2, +0.2]^{\approx}$	$+0.0\,[-0.2, +0.2]^{\approx}$
	2	0.0 / 100.0 / 0.0	0.0 / 99.5 / 0.5	0.0 / 97.0 / 3.0	67.9	67.8	$+0.1\,[+0.0, +0.3]^{\approx}$	$-0.1\,[-0.3, -0.0]^{\approx}$
	3	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	0.0 / 100.0 / 0.0	78.5	78.5	$+0.0\,[+0.0, +0.0]^{\approx}$	$+0.0\,[+0.0, +0.0]^{\approx}$

almost never rejects). With graded strengths the weak directions are simply not detected at small n: at 10 per class and $q = 3$ no rule recovers the rank in more than 2% of the draws (mean tested rank 1.50, 1.66 and 0.82), and at 30 per class the third direction is found in 31%, 50% and 66% of the draws. At 100 per class the three rules recover every rank $q \geq 1$ in at least 97% of the draws.

- **The sequential rule dominates the max rule**: it never under-estimates more often, and less often wherever the two differ (for instance 8.5% against 25.5%, 37% against 53.5%, 50% against 69%), at the price of an over-estimation rate of at most 1.5%. Bartlett has no power when n is of the order of d and becomes the most powerful of the three at 30 per class on the weakest direction, with an over-estimation rate of 1.5 to 3% at larger n.

A tested rank is therefore a statement of the form "at least r_{test} directions are detectable at this sample size", with a small probability of over-statement; it is not an estimate of the true rank when directions are weak relative to n.

8.4 Rank error is not predictive risk

The right-hand columns of Tables 6 and 7 answer a different question: which rank should the classifier *use*? Three facts stand out.

- **Keeping all $C-1$ directions costs almost nothing when the true rank is lower.** The difference between the structural rank and the oracle rank q lies between -0.2 and $+0.2$ point in the eighteen cells, always inside the equivalence margin: the centroid read-out is barely affected by one or two pure-noise coordinates.

- **Using a tested rank gains at most** 0.2 **point** when the test is right and the true rank is below $C-1$.

- **Using a tested rank costs up to** 8.9 **points** when the test lacks power: -0.8 and -1.1 point with equal strengths at 10 per class, and -4.2, -8.9 and -3.3 points with graded strengths (at 10 per class for $q = 2, 3$, and at 30 per class for $q = 3$), all with intervals excluding zero. A weak direction that is not *detectable* at level 5% still carries information that the classifier can use.

The asymmetry is the practical conclusion of this section: for prediction the used rank should be the structural rank $C-1$, and the tested rank should be reported as what it is, a detection result. The same asymmetry is observed on the digits data (Section 11) and in the small-sample diagnosis of Section 12, where an under-estimated rank is one of the three causes of the accuracy loss.

8.5 Families of tests: per-block and family-wise control

When one block is fitted per frequency band, per sensor group or per subject, and each block is retained if its own test rejects, the tests form a family. A test at level 5% *per block* controls the rate of false selections per null block, not the probability of selecting at least one null block, which grows with the size of the family. A family-wise control is obtained without additional computation by the single-step min-p procedure [13]: the same label permutations are used for all blocks, every statistic (observed or shuffled) is converted into a p-value within its own block, and the adjusted p-value of a block is the proportion of labellings (observed one included) whose smallest p-value over the family is at most the p-value of that block. Under the global null hypothesis the probability of retaining at least one block is then at most α, again by exchangeability.

Measured on families of eight independent blocks ($d = 36$, $C = 3$, $n = 30$, 200 draws, 200 permutations):

	per-block test	min-p control
all blocks null: P(at least one block retained)	33.0%	3.5%
all blocks null: selection rate per block	5.1%	0.5%
one informative block: P(at least one null block retained)	27.5%	2.5%
one informative block: power on that block	100%	100%

The per-block test behaves as designed (5.1% per block, hence 33.0% for the family, for an expected $1 - 0.95^8 = 33.7\%$ under independence); the min-p control brings the family-wise rate to 3.5% with no loss of power in this configuration, where the informative block is strong. Which control is appropriate depends on the use: a per-block level is acceptable when false selections merely add noise coordinates to a downstream classifier, a family-wise level is required when the selection itself is the finding ("this band is discriminant"). Both are available in the implementation (`fwer_select`), and Appendix I reports both for the frequency-band experiment.

Summary of the guarantees.

Statement	**Status**	**Where it fails**
$r \leq C - 1$	exact	—
$P(r_{\text{test}} > 0) \leq \alpha$ under the global null hypothesis (S_T metric, exact p-value)	proved (exchangeability), checked on 2000 draws	shared-S_W metric; interpolated quantile
family-wise level of the min-p selection under the global null hypothesis	proved, checked on 200 draws	—
$P(r_{\text{test}} > q)$ small for a true rank $q \geq 1$	measured ($\leq$ 1.5% for the permutation rules in the cases tested)	no proof; other distributions not tested
$r_{\text{test}} = q$	measured	weak directions at small n (down to 0–2% recovery)
tested rank optimal for prediction	*not supported*	costs up to 8.9 points when the test lacks power

9 Limits of oblique projectors and their control in the analytic block

Oblique projectors have intrinsic limits that no construction removes: they can amplify noise, they discard the kernel irreversibly, they are not Euclidean best approximations, and they

are a very constrained class of linear maps. This section states these limits precisely, separates what is intrinsic from what depends on how the blocks are assembled, and shows how the analytic construction bounds or mitigates them. Every statement is verified numerically (`ropn_auto_stabilite.py`); the measurements are reported with the statements.

9.1 Noise amplification: norm and angle

Proposition 5 (Norm of an oblique projector)**.** *Let $P = U(W^\top U)^{-1}W^\top$ with $U, W \in \mathbb{R}^{d\times r}$ of full rank, $W^\top U$ invertible, $0 < r < d$, and let Q_U, Q_W be orthonormal bases of* $\operatorname{span} U$ *and* $\operatorname{span} W$*. Then*

$$\|P\|_2 = \frac{1}{\sigma_{\min}(Q_W^\top Q_U)} = \frac{1}{\sin\theta_{\min}}, \qquad \|I - P\|_2 = \|P\|_2, \tag{13}$$

where $\theta_{\min}$ is the smallest principal angle between $\operatorname{im} P = \operatorname{span} U$ *and* $\ker P = (\operatorname{span} W)^\perp$*. More precisely, the non-zero singular values of P are $1/\cos\theta_i$, $i = 1, \dots, r$, where the θ_i are the principal angles between* $\operatorname{span} U$ *and* $\operatorname{span} W$*. The amplification bound $\|P(x + \delta x) - Px\|_2 \le \|P\|_2\|\delta x\|_2$ is attained along the dominant right singular vector of P.*

Proof. P depends only on the two subspaces, so $P = Q_U(Q_W^\top Q_U)^{-1}Q_W^\top$; since Q_U has orthonormal columns and $Q_W^\top$ orthonormal rows, the singular values of P are those of $(Q_W^\top Q_U)^{-1}$, i.e. the inverses of the singular values of $Q_W^\top Q_U$, which are the cosines of the principal angles between $\operatorname{span} U$ and $\operatorname{span} W$; hence $\|P\|_2 = 1/\sigma_{\min}(Q_W^\top Q_U)$. With $Q_\perp$ an orthonormal basis of $(\operatorname{span} W)^\perp = \ker P$, $Q_U^\top Q_\perp Q_\perp^\top Q_U = I - Q_U^\top Q_W Q_W^\top Q_U$, so the largest singular value of $Q_\perp^\top Q_U$, which is the cosine of the smallest principal angle $\theta_{\min}$ between $\operatorname{span} U$ and $\ker P$, satisfies $\cos^2\theta_{\min} = 1 - \sigma_{\min}^2(Q_W^\top Q_U)$, i.e. $\sigma_{\min}(Q_W^\top Q_U) = \sin\theta_{\min}$. The identity $\|I - P\|_2 = \|P\|_2$ for $P \neq 0, I$ is classical [14, 15]. The bound is attained because $\|P\|_2$ is by definition the maximum of $\|P\delta x\|/\|\delta x\|$. □

Two consequences deserve emphasis. First, eigenvalues confined to $\{0, 1\}$ say nothing about amplification: $P_a = \begin{pmatrix} 1 & a \\ 0 & 0 \end{pmatrix}$ is idempotent with eigenvalues 0 and 1, yet $\|P_a\|_2 = \sqrt{1 + a^2}$ and the perturbation $(0, \varepsilon)$ becomes $(a\varepsilon, 0)$ (verified for $a = 0, 1, 10$: norms 1, 1.4142, 10.0499). Second, the condition number of the cross-Gram $W^\top U$ is not an obliquity measure either: at rank one it is always 1, and in the biorthogonal gauge of this report it is 1 at every rank by construction, while $\|P\|_2$ can be arbitrary (with $u = e_1$ and $w = u + t\,e_2$, $w^\top u = 1$ and $\|P\|_2 = \sqrt{1 + t^2}$: 1.41, 5.10, 50.01 for $t = 1, 5, 50$). This is why the report measures $\sigma_{\min}(Q_W^\top Q_U)$ and $\|P\|_2$, never $\kappa(W^\top U)$. On the analytic block ($d = 20$, $C = 4$, $n = 100$ per class) the three expressions of (13) agree to four digits (1.1614 at $\kappa = 10$, 1.5469 at $\kappa = 200$), $\|I - P\|_2$ equals $\|P\|_2$, and the measured amplification along the dominant singular direction equals the bound.

9.2 The interpolation threshold: where the raw projector's norm peaks

Rocks and Mehta [2] showed that in over-parameterised least-squares regression with linear random features, in the ridge-less limit, the fitted map is an oblique projector whose singular values $1/\cos\theta_i$ diverge at the interpolation threshold (number of parameters equal to the number of samples), because one subspace orientation angle approaches 90°; they relate this divergence to the double-descent peak of the test error. (That an explicit ridge penalty attenuates such a peak is a classical observation, not a result of that paper.) The analytic block has its own interpolation threshold: the within-class scatter S_W has rank at most $\min(n_{\text{train}} - C, d)$, with equality almost surely for data with a density, so at $n_{\text{train}} - C \approx d$ it becomes full rank with a very small smallest eigenvalue, and the raw Fisher projector (metric S_W^+) is strongly oblique.

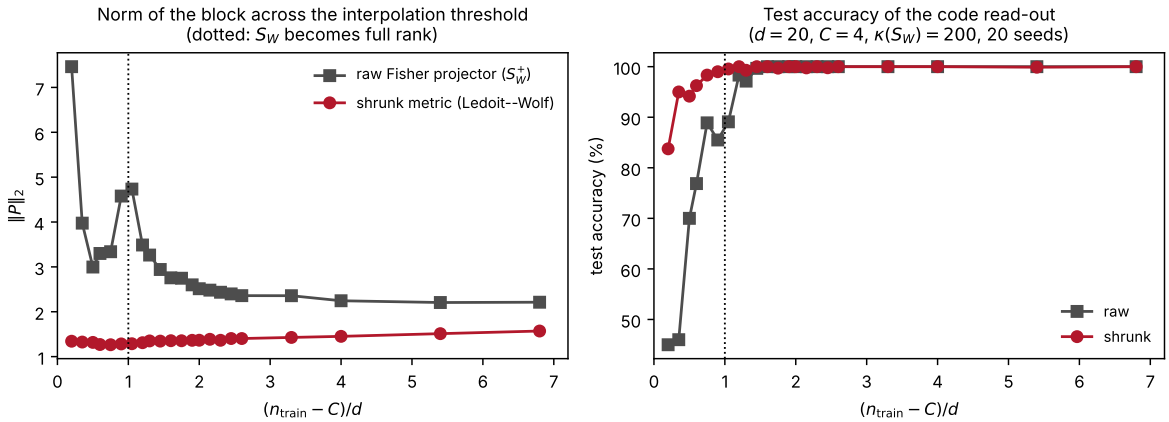


Figure 7: The interpolation threshold of the analytic block. Left: spectral norm of the raw Fisher projector (pseudo-inverse of S_W, grey) and of the shrunk projector (red) as a function of $(n_{\text{train}} - C)/d$; the raw norm peaks where S_W becomes full rank with a vanishing smallest eigenvalue (dotted line), the shrunk norm is flat. Right: test accuracy of the centroid read-out of the code for the two metrics.

Measured ($d = 20$, $C = 4$, $\kappa(S_W) = 200$, 20 seeds, Figure 7): the norm of the raw projector rises from 3.3 at $(n_{\text{train}} - C)/d = 0.75$ to 4.7 at 1.05 and falls back to 3.5 at 1.2 and 2.2 at large n; below the threshold it rises again as the pseudo-inverse acts on an ever smaller subspace (7.5 at 0.2). The shrunk projector is flat (1.26–1.57) throughout: Ledoit–Wolf shrinkage plays for the oblique block a role analogous to that of ridge regularisation in regression. The test accuracy of the raw block (centroid read-out) falls from 100% to 45% as n decreases, with a small dip at the threshold (88.9%, 85.5% and 89.1% at $(n_{\text{train}} - C)/d = 0.75$, 0.90 and 1.05) that is of the order of the standard error of these means (about 2.5 points on 20 seeds of about ten test points each) and is therefore not established; a read-out on a low-dimensional code may simply be more tolerant than a regression on the full state. The norm peak nonetheless signals the regime to avoid. The shrunk block is less oblique everywhere, much more accurate below the threshold (84–99% against 45–89% for $(n_{\text{train}} - C)/d \leq 0.9$) and equivalent well above it (within 0.3 point for $(n_{\text{train}} - C)/d \geq 1.45$).

9.3 What bounds the obliquity: the metric

The analytic block is never an arbitrary oblique projector: it is the M-orthogonal projector for $M = S^{-1}$, S the (shrunk) within-class scatter. This bounds its norm.

Proposition 6 (Metric bound on the norm)**.** *If P is M-orthogonal ($W = MU$, $M \succ 0$, $0 < r < d$) and $\kappa = \kappa(M)$, then*

$$\|P\|_2 \;\leq\; \frac{1}{2}\Big(\sqrt{\kappa} + \frac{1}{\sqrt{\kappa}}\Big) \;\leq\; \sqrt{\kappa}, \tag{14}$$

and, for $\kappa > 1$, the first inequality is an equality for a suitable rank-one projector. For the analytic block with Ledoit–Wolf shrinkage, $S = (1-a)S_W + a\mu I$ with $a\mu > 0$, $\kappa = \kappa(S)$ and

$$\sqrt{\kappa(S)} \leq \sqrt{1 + \frac{(1-a)\lambda_{\max}(S_W)}{a\mu}}. \tag{15}$$

Proof. $\ker P = (\operatorname{span} W)^{\perp} = \{y : y^{\top} M u = 0 \;\forall u \in \operatorname{span} U\}$ is the M-orthogonal complement of $\operatorname{im} P$. Wielandt's inequality [3, Section 7.4], applied to M^{-1} and to the Euclidean-orthogonal

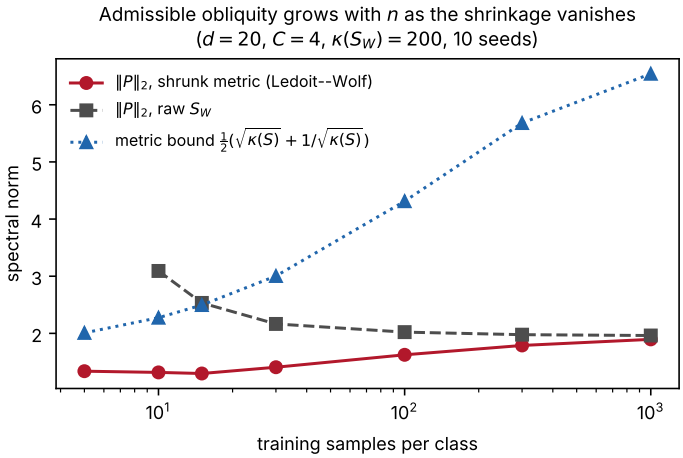


Figure 8: Admissible obliquity as a function of the sample size: spectral norm of the shrunk Fisher projector (red), of the raw Fisher projector (grey, computed only where S_W is invertible, $n_{\text{train}} > d + C$) and the metric bound $\frac{1}{2}(\sqrt{\kappa(S)} + 1/\sqrt{\kappa(S)})$ of Proposition 6 (blue).

vectors $M^{1/2}x$, $M^{1/2}y$, states that $|x^\top y| \leq \frac{\kappa-1}{\kappa+1}\|x\|\|y\|$ whenever $x^\top My = 0$. The smallest principal angle between $\operatorname{im} P$ and $\ker P$ therefore satisfies $\cos\theta_{\min} \leq \frac{\kappa-1}{\kappa+1}$, i.e. $\sin\theta_{\min} \geq \frac{2\sqrt{\kappa}}{\kappa+1}$, and Proposition 5 gives $\|P\|_2 = 1/\sin\theta_{\min} \leq \frac{\kappa+1}{2\sqrt{\kappa}}$, which is at most $\sqrt{\kappa}$ since $\kappa \geq 1$. Equality holds at rank one for $u = v_{\min}/\sqrt{\lambda_{\min}} + v_{\max}/\sqrt{\lambda_{\max}}$, with $(\lambda_{\min}, v_{\min})$ and $(\lambda_{\max}, v_{\max})$ the extreme eigenpairs of M (the equality case of the Kantorovich inequality). (The weaker bound $\sqrt{\kappa}$ also follows directly from the fact that $M^{1/2}PM^{-1/2}$ is an orthogonal projector.) Finally, with $M = S^{-1}$, $\kappa(M) = \kappa(S)$, $\lambda_{\min}(S) \geq a\mu$ and $\lambda_{\max}(S) \leq (1-a)\lambda_{\max}(S_W) + a\mu$. □

The shrinkage coefficient therefore sets the *admissible obliquity*: the data-driven choices of the block concern not only the rank but, through a, its norm. Measured ($\kappa(S_W) = 200$ at the population level, 10 seeds):

n/class	a (LW)	$\|P\|_2$ raw S_W	$\|P\|_2$ shrunk	bound (14)	$\sqrt{\kappa(S)}$	bound (15)	$\sqrt{\kappa(S_W)}$
10	0.278	3.09	1.32	2.27	4.31	4.32	68.7
15	0.217	2.53	1.30	2.50	4.78	4.85	28.1
30	0.132	2.16	1.41	3.00	5.84	6.14	18.5
100	0.045	2.02	1.62	4.31	8.51	10.33	15.0
1000	0.005	1.96	1.90	6.54	13.01	32.71	14.2

The bound (14) holds for every seed (Figure 8; it is attained to four digits by the rank-one example of the proof, and never exceeded on 2000 random pairs (M, U)), and M-orthogonality is exact ($\|MP - P^\top M\|/\|MP\| \leq 2.3 \times 10^{-15}$). As n grows the coefficient a decreases, the admissible obliquity grows, and the measured norm rises from 1.3 to 1.9, towards the value of the raw projector; at small n the raw projector ($\|P\|_2 = 3.1$ at 10 per class) is replaced by a much less oblique one (1.3). The bound is far from the measured norm at large n (the extreme eigen-directions of the metric are not aligned with the discriminant directions); what it establishes is the mechanism: shrinkage is a control of the norm, by the same coefficient that improves the accuracy at $\kappa = 10$ in Section 12.

9.4 Best approximation: not Euclidean, but in the Fisher metric

The orthogonal projector Q_S onto the subspace $\operatorname{im} P$ (here Q_S denotes a projector, not a basis) returns the Euclidean nearest point of $\operatorname{im} P$; an oblique projector onto the same subspace does

not. Since $(I-P)Q_S x = 0$, one has $x - Px = (I-P)(x - Q_S x)$ and therefore

$$\|x - Px\|_2 \le \|I - P\|_2 \, \|x - Q_S x\|_2 = \|P\|_2 \, \|x - Q_S x\|_2. \tag{16}$$

The reconstruction penalty is governed by the same norm that governs noise amplification. Measured on the block ($\kappa = 200$, $n = 100$ per class): the ratio $\|x - Px\|/\|x - Q_S x\|$ lies in $[1.005, 1.354]$ on the test set, below $\|P\|_2 = 1.547$, and exceeds one at every point. In the Fisher norm $\|v\|_M = \sqrt{v^\top M v}$, however, the same ratio lies in $[0.526, 0.986]$: the M-orthogonal projector *is* the best approximation in its own metric. Obliquity is therefore justified by the task metric, not by any general approximation property; the block minimises the reconstruction error in the (shrunk) within-class Mahalanobis norm, the natural norm for a discriminant representation under the shared-covariance model, and pays for it in Euclidean error by at most the factor $\|P\|_2$.

9.5 Information loss and the residual form

For $r < d$, two inputs that differ by a kernel component are indistinguishable after projection, and no later block can recover what a projector has removed unless another path carries it. In the vocabulary of Rocks and Mehta [2], every input perturbation splits into an *invariant* component in $\ker P$, which leaves the prediction unchanged, and an *adversarial* component in $(\ker P)^\perp = \mathrm{span}(W)$, which changes it; the invariant subspace is a consequence of the model's bias, i.e. of what the block has decided not to read. In the analytic block the read-out acts on the code, so the kernel is discarded *deliberately*: a perturbation of norm 3σ confined to $\ker W^\top$ changes the code by 3×10^{-14} and leaves 100% of the predictions unchanged, while a perturbation of the same norm inside $\mathrm{span}\, U$ changes 26% of them (centroid read-out). The most effective perturbation of a given norm is along the dominant left singular direction of W, and its effect on the code is bounded by $\sigma_{\max}(W)\|\delta x\|$. This is Proposition 1(i) read as a risk: a direction that is useless on the training distribution but informative out of distribution is lost. The justification is statistical, not architectural: on data for which LDA is Bayes-optimal the Fisher code is a sufficient statistic (Section 12.3); on other data the lattice (Appendix C) or per-class metrics (Appendix E) must re-expose the kernel through a fixed non-linearity. The residual form $I + P/L$ keeps the kernel but is no longer a projector: with $\mathcal{T} = I + P/L$, $\|\mathcal{T}^2 - \mathcal{T}\|/\|\mathcal{T}\| = 0.86,\ 0.37,\ 0.16$ for $L = 1, 2, 4$. It is, however, invertible in closed form,

$$(I + P/L)^{-1} = I - \frac{P}{L+1}, \qquad \det(I + P/L) = (1 + 1/L)^r, \tag{17}$$

(verified to 1.5×10^{-15} and to all displayed digits of the determinant), which is why the linear recursion of Section 3 does not lose rank, although it remains a single affine function of P (Theorem 1).

9.6 Regularising the cross-Gram versus regularising the metric

A tempting numerical safeguard is to replace $(W^\top U)^{-1}$ by $(W^\top U + \lambda I)^{-1}$. This destroys idempotence: on the analytic block, $\|P^2 - P\|/\|P\| = 9.9 \times 10^{-3}$, 9.1×10^{-2} and 0.50 for $\lambda = 10^{-2}$, 10^{-1} and 1, against 1.9×10^{-16} for $\lambda = 0$ (in the biorthogonal gauge this ratio is exactly $\lambda/(1+\lambda)$). Such an operator is a regularised projection-like map, and the results of Section 3, whose proofs use $P^2 = P$, do not apply to it as they stand. The analytic block never does this: its regularisation enters the *metric* ($S_W \to (1-a)S_W + a\mu I$), the cross-Gram is fixed at I_r by construction, and idempotence stays exact ($\|P^2 - P\|/\|P\| = 10^{-15}$, $\|W^\top U - I\| = 1.7 \times 10^{-15}$ with $a = 0.045$). The price, documented in Section 13, is that the operator is the shrunk Fisher projector, not the raw one.

9.7 Expressivity and depth

A projector is a very constrained linear map: identity on its image, zero on its kernel, $P^k = P$, the composition of fixed projectors without non-linearity is linear and of rank at most the smallest factor rank. The expected benefit is a geometric inductive bias, not an automatic gain in expressivity over a free linear layer; Theorems 1 and 3 make this precise for the shared-weight case, and the unconstrained low-rank control of Section 13 indicates that, on clean data, the accuracy comes from the metric, not from idempotence. In depth, the product bound $\|P_L \cdots P_1\|_2 \leq \prod_\ell \|P_\ell\|_2$ shows why moderate amplification per layer is not a global guarantee; for the shared block the exact value is available and the bound is loose ($\|(I + P/L)^L\|_2 = 2.75$, 3.03, 3.22 against $(1 + \|P\|_2/L)^L = 3.15$, 3.70, 4.11 for $L = 2, 4, 8$). Finally, invertibility, stability and conditioning are three different properties: the un-normalised gated recursion $h \leftarrow (1-a)h + aPh$ with the analytic gate ($s \in \mathrm{span}(W)$, for which Theorem 2 gives a positive determinant $(1-a)^{d-r}$ per step) has a non-zero determinant at every depth, yet its smallest singular value falls from 0.38 ($T = 1$) to 0.018 ($T = 8$) and its condition number rises from 4.3 to 84.9 on the same input. (For a gate that reads the residual, the determinant can vanish, as in Example 1.) A non-zero determinant excludes rank collapse; it says nothing about conditioning. The condition number of the global Jacobian is the quantity to report, and Section 13 does so, including for the residual-normalised gate, which behaves very differently.

10 What each ingredient contributes

A natural objection to the construction is: why speak of a network of projectors if the final predictor only uses the classical discriminant directions? At inference with a read-out on the code, U is indeed not even needed: one computes $W^\top x$ and applies a linear read-out. The answer is that three ingredients are present, that they do different things, and that the experimental gains must be attributed to the right one.

Ingredient	What it brings	Evidence in this report
Fisher metric $W = S^{-1}U$ (with shrinkage)	*Prediction accuracy.* It is the whole of the statistical content: the block is LDA (Proposition 3).	The orthogonal variant with the same image loses 4 to 6 points at $\kappa = 200$, while an unconstrained low-rank map fitted with a ridge does as well as the block on clean data in three cases out of four (Section 13, R6). The block is equivalent to shrinkage LDA in the central benchmark (Section 11).
Factorisation ($C \times C$ reduction, dual form)	*Computation.* Exactly the same estimator at a lower cost; nothing statistical.	One to three orders of magnitude against a dense generalised eigendecomposition, but a factor 5 to 11 against scikit-learn's shrinkage LDA solver when $n \geq d$, and more only when $n \ll d$ (Section 6.5); batched permutations make the rank test affordable.
Idempotence $W^\top U = I_r$, $P^2 = P$	*Exact structural statements*: the theorems of Section 3, the norm identities and the metric bound of Section 9, the invariance of the code. No accuracy by itself.	Fine-tuning with or without the idempotence constraint gives the same or a lower accuracy than the analytic block (Section 12.3); the unconstrained low-rank control matches it on clean data.

In short: the accuracy is that of the metric, the speed is that of the factorisation, and the projector structure is what makes the recurrences of Section 3 analysable in closed form. The

projector structure is used by the predictor only when the state h itself is propagated (recursion, residual, lattices of Appendix C); for a single block read on its code, the projector is an interpretation of LDA, not an additional mechanism.

11 Central benchmark: symmetric comparisons with paired uncertainty

This section is the empirical core of the report. It tests one promise: *the analytic block remains useful at small n when its assumptions (Gaussian classes with a common covariance) hold only approximately.* The other experiments of the report are diagnoses (Section 12) or demonstrations (appendices).

11.1 Protocol

Data. (a) Synthetic problems with $C = 4$, $d = 20$, $\kappa = 50$ and $n = 15$ or 60 training samples per class, each realisation being an *independent draw* of the whole problem (means, covariances, samples) evaluated on 2000 fresh test points, so that the test noise is small: Gaussian classes with a shared covariance (assumptions satisfied); heteroscedastic classes, one covariance orientation per class (shared covariance violated); Student t_3 noise with a common scatter (Gaussianity violated); proportional covariances with factors $0.4, 0.8, 1.4, 2.4$ (mild violation). (b) One real dataset, the largest available offline in the execution environment: scikit-learn's `digits` (1797 images 8×8, ten classes, 61 non-constant features), with repeated stratified subsamples of 10, 30 or 100 training images per class and all remaining images as test set. These repetitions measure the variability of the subsample; they are not independent realisations of the problem, and the intervals below must be read accordingly.

Methods. The analytic block (Ledoit–Wolf shrinkage of the pooled within-class scatter, rank $C - 1$) with the centroid read-out, i.e. LDA; the same block with the least-squares read-out; scikit-learn's shrinkage LDA (`lsqr`, which shrinks each class covariance separately before averaging, Section 5.3); multinomial logistic regression; an MLP with one hidden layer.

Two separate comparisons. *Without tuning*: one default recipe per method, fixed before the experiments (block: Ledoit–Wolf; LDA: `shrinkage="auto"`; logistic regression: $C = 1$; MLP: scikit-learn defaults, 100 hidden units, Adam). *After selection*: four configurations per method, chosen by three-fold cross-validation on the *same* folds for all methods (block: Ledoit–Wolf or a fixed ridge $\lambda \in \{0.01, 0.1, 1\}$ of Appendix G; LDA: shrinkage in $\{\text{auto}, 0.01, 0.1, 0.5\}$; logistic regression: $C \in \{0.01, 0.1, 1, 10\}$; MLP: 64 hidden units, L2 penalty 10^{-4} or 10^{-1}, Adam or L-BFGS). Features are standardised on the training set.

Statistics. Every method is compared with the block on the same realisations: mean paired difference, Student 95% confidence interval on the paired differences, and a verdict relative to an equivalence margin of one accuracy point fixed in advance: *equivalent* ($\approx$) if the interval lies within ± 1 point, *better* ($\uparrow$) or *worse* ($\downarrow$) than the block if it excludes zero without lying within the margin, *undetermined* (?) otherwise. A difference smaller than its uncertainty is thus never declared null. Twenty realisations are used without tuning and ten after selection.

11.2 Results

What the two tables support, and what they do not.

- **The block is LDA, and behaves as such.** With the centroid read-out it is equivalent to scikit-learn's shrinkage LDA on the heteroscedastic data, on the Gaussian and proportional-covariance problems at $n = 60$ and on digits (differences of at most 0.5 point, inside the margin), and everywhere after selection except for one undetermined

Table 8: Central benchmark *without tuning* (default recipes). Block accuracy in % (mean ± standard deviation over 20 realisations), then paired difference to the block in points, [95% interval] and verdict (≈ equivalent within one point, ↑ better than the block, ↓ worse, ? undetermined). n: training samples per class.

data	block (LDA rule)	least-squares read-out	scikit-learn shrinkage LDA	logistic regression	MLP
Gaussian, shared cov., n=15	98.9 ± 1.2	$-0.3\,[-0.6, -0.1]^{\approx}$	$-1.1\,[-1.4, -0.7]^{\downarrow}$	$-1.1\,[-1.5, -0.6]^{\downarrow}$	$-2.5\,[-3.2, -1.9]^{\downarrow}$
Gaussian, shared cov., n=60	99.7 ± 0.4	$-0.2\,[-0.4, -0.0]^{\approx}$	$-0.2\,[-0.4, -0.1]^{\approx}$	$-0.6\,[-0.9, -0.2]^{\approx}$	$-0.9\,[-1.4, -0.5]^{\downarrow}$
heteroscedastic, n=15	95.4 ± 2.5	$-0.5\,[-0.8, -0.2]^{\approx}$	$+0.0\,[-0.2, +0.2]^{\approx}$	$+0.0\,[-0.3, +0.4]^{\approx}$	$-0.3\,[-0.6, +0.1]^{\approx}$
heteroscedastic, n=60	97.5 ± 1.5	$-0.4\,[-0.6, -0.2]^{\approx}$	$+0.0\,[-0.1, +0.1]^{\approx}$	$+0.2\,[-0.1, +0.5]^{\approx}$	$+0.6\,[+0.3, +1.0]^{\approx}$
Student t_3, n=15	90.8 ± 4.6	$-0.7\,[-1.1, -0.3]^{\downarrow}$	$-2.2\,[-2.8, -1.7]^{\downarrow}$	$+0.5\,[-0.3, +1.4]^{?}$	$-0.9\,[-1.6, -0.1]^{\downarrow}$
Student t_3, n=60	92.7 ± 4.2	$-0.3\,[-0.5, -0.1]^{\approx}$	$-1.2\,[-1.7, -0.8]^{\downarrow}$	$+1.0\,[-0.3, +2.2]^{?}$	$+0.8\,[-0.5, +2.1]^{?}$
proportional cov., n=15	96.0 ± 2.5	$-0.3\,[-0.6, +0.0]^{\approx}$	$-1.6\,[-2.1, -1.2]^{\downarrow}$	$-0.8\,[-1.2, -0.5]^{\downarrow}$	$-2.1\,[-2.7, -1.5]^{\downarrow}$
proportional cov., n=60	98.5 ± 1.6	$-0.3\,[-0.5, -0.1]^{\approx}$	$-0.4\,[-0.6, -0.2]^{\approx}$	$-0.3\,[-0.5, -0.1]^{\approx}$	$-0.4\,[-0.5, -0.2]^{\approx}$
digits, n=10	88.5 ± 1.2	$-2.1\,[-2.4, -1.8]^{\downarrow}$	$+0.5\,[+0.4, +0.7]^{\approx}$	$+0.6\,[+0.2, +1.0]^{\approx}$	$-0.8\,[-1.3, -0.4]^{\downarrow}$
digits, n=30	92.9 ± 0.9	$-1.7\,[-2.0, -1.4]^{\downarrow}$	$+0.2\,[+0.0, +0.4]^{\approx}$	$+1.0\,[+0.6, +1.3]^{\uparrow}$	$+1.2\,[+0.8, +1.6]^{\uparrow}$
digits, n=100	95.1 ± 0.8	$-1.7\,[-2.0, -1.4]^{\downarrow}$	$+0.2\,[+0.1, +0.4]^{\approx}$	$+1.4\,[+1.1, +1.7]^{\uparrow}$	$+2.3\,[+2.0, +2.7]^{\uparrow}$

Table 9: Central benchmark *after selection* (four configurations per method, common three-fold internal cross-validation), 10 realisations. Same conventions as Table 8.

data	block (LDA rule)	least-squares read-out	scikit-learn shrinkage LDA	logistic regression	MLP
Gaussian, shared cov., n=15	99.2 ± 1.2	$-0.1\,[-0.3, +0.1]^{\approx}$	$-0.1\,[-0.3, +0.0]^{\approx}$	$-2.0\,[-3.1, -1.0]^{\downarrow}$	$-1.7\,[-2.6, -0.7]^{\downarrow}$
Gaussian, shared cov., n=60	99.8 ± 0.2	$-0.1\,[-0.3, +0.0]^{\approx}$	$-0.2\,[-0.3, +0.0]^{\approx}$	$-0.5\,[-1.0, -0.1]^{\approx}$	$-0.6\,[-1.0, -0.1]^{\downarrow}$
heteroscedastic, n=15	95.4 ± 2.3	$-0.6\,[-1.2, -0.0]^{\downarrow}$	$-0.1\,[-0.3, +0.2]^{\approx}$	$-0.3\,[-0.9, +0.3]^{\approx}$	$-0.5\,[-1.2, +0.2]^{?}$
heteroscedastic, n=60	97.4 ± 1.4	$-0.4\,[-0.7, +0.0]^{\approx}$	$-0.0\,[-0.2, +0.2]^{\approx}$	$+0.1\,[-0.3, +0.4]^{\approx}$	$+0.4\,[+0.0, +0.7]^{\approx}$
Student t_3, n=15	93.3 ± 3.5	$-0.5\,[-1.9, +0.9]^{?}$	$+0.3\,[-0.9, +1.5]^{?}$	$-1.9\,[-3.2, -0.7]^{\downarrow}$	$-1.9\,[-3.2, -0.5]^{\downarrow}$
Student t_3, n=60	96.0 ± 1.9	$-0.7\,[-1.0, -0.3]^{\downarrow}$	$+0.0\,[-0.0, +0.0]^{\approx}$	$-1.7\,[-1.9, -1.4]^{\downarrow}$	$-1.4\,[-1.7, -1.1]^{\downarrow}$
proportional cov., n=15	96.9 ± 2.2	$-0.4\,[-1.1, +0.4]^{?}$	$-0.2\,[-0.8, +0.4]^{\approx}$	$-2.4\,[-4.1, -0.8]^{\downarrow}$	$-2.1\,[-3.2, -0.9]^{\downarrow}$
proportional cov., n=60	98.9 ± 1.1	$-0.3\,[-0.6, +0.0]^{\approx}$	$-0.1\,[-0.2, +0.1]^{\approx}$	$-0.5\,[-0.9, -0.2]^{\approx}$	$-0.3\,[-0.5, -0.1]^{\approx}$
digits, n=10	88.6 ± 0.7	$-2.4\,[-3.0, -1.8]^{\downarrow}$	$+0.4\,[+0.2, +0.7]^{\approx}$	$+0.4\,[-0.4, +1.3]^{?}$	$+0.9\,[-0.3, +2.0]^{?}$
digits, n=30	93.0 ± 0.9	$-1.7\,[-2.4, -0.9]^{\downarrow}$	$+0.3\,[-0.1, +0.6]^{\approx}$	$+0.6\,[-0.1, +1.3]^{?}$	$+1.9\,[+0.9, +2.8]^{\uparrow}$
digits, n=100	95.5 ± 0.6	$-1.7\,[-2.2, -1.2]^{\downarrow}$	$+0.1\,[+0.0, +0.2]^{\approx}$	$+1.3\,[+0.8, +1.7]^{\uparrow}$	$+2.2\,[+1.8, +2.7]^{\uparrow}$

case (Student, $n = 15$). Without tuning it is 1.1 to 2.2 points above scikit-learn's default on the Gaussian, Student and proportional-covariance problems at 15 samples per class and on the Student problem at 60; the difference disappears once the shrinkage is selected, which points to the shrinkage convention (pooled against per-class), not to the projector formulation.

- **Where the assumptions hold or nearly hold, at small n, it is ahead of the discriminative baselines.** On the Gaussian and proportional-covariance problems at 15 per class the block is 0.8 to 2.5 points above logistic regression and the MLP, with or without selection (intervals excluding zero). At 60 per class the differences fall inside the margin, except for the MLP on the Gaussian problem (-0.9 and -0.6).

- **Where the shared-covariance assumption fails, nothing separates the methods.** On the heteroscedastic problems all differences between the block and the three baselines are within the margin or undetermined: the block is not worse than the baselines, and it does not capture the second-order information either.

- **With heavy tails the picture depends on tuning.** Without tuning, logistic regression is undetermined against the block (+0.5 and +1.0, intervals containing zero); after selection the block is 1.4 to 1.9 points above logistic regression and the MLP.

- **On the real data the block is behind as soon as data are not scarce.** On digits it is equivalent to shrinkage LDA at every size, 0.8 point above the default MLP at 10 images per class, and then behind: without tuning, logistic regression is 1.0 and 1.4 points above

it at 30 and 100 per class and the MLP 1.2 and 2.3 points; after selection the MLP is 1.9 and 2.2 points above it, logistic regression 1.3 points at 100 per class and undetermined at 30 (+0.6, interval $[-0.1, +1.3]$). At 10 per class the comparisons after selection are undetermined. On this dataset the classes are neither Gaussian nor of common covariance, and a linear discriminant with the LDA metric is not the best linear classifier.

- **The least-squares read-out is inferior with ten classes** (-1.7 to -2.4 points on digits) and equivalent, or within 0.7 point, with four.

The promise is therefore supported in a limited form: on small samples drawn from distributions close to the LDA model the analytic block is the best of the methods compared, tuned or not; it degrades gracefully to parity under heteroscedasticity; and on the one real dataset available it is a shrinkage LDA, with the strengths and the limits of LDA. These experiments do not show an advantage on real data, and no experiment on real EEG recordings could be run in the execution environment, which has no network access; that validation remains to be done.

Sources of variability. For the MLP with default settings on digits at 30 images per class, the standard deviation over ten training seeds at a fixed subsample is 0.46 point, against 0.9 over the subsamples; the analytic block is deterministic given the subsample.

Cost, phase by phase. The four phases are timed separately on digits at 100 images per class (one realisation, single CPU core, milliseconds). Preprocessing (standardisation) is common to all methods: 3.1 ms. Selection is the cross-validation over four configurations and three folds; the final fit and the inference on the 797 test images use the selected configuration (minimum of three runs).

	selection (4×3 fits)	final fit	inference (797 images)
analytic block (centroid read-out)	13.2	1.1	0.11
analytic block (least-squares read-out)	13.7	1.5	0.10
scikit-learn shrinkage LDA	98.5	5.2	0.29
logistic regression	116.8	12.5	0.33
MLP	2986	493	0.75

The block is the cheapest method in every phase: by a factor of about five to seven against the library LDA for selection and fit, and by more than two orders of magnitude against the MLP. At inference all methods are linear maps or one hidden layer and differ by less than a factor of ten; the absolute times are far below a millisecond per sample for all of them. These are single measurements and only the orders of magnitude are meaningful. The selection phase dominates the cost of every method, which is why the phases must not be merged into one figure.

Tested rank on the real data. With the sequential rule of Section 8 (200 permutations, 20 subsamples) the tested rank on digits is 5.75 on average (between 4 and 8) at 5 images per class, 7.80 (between 6 and 9) at 10 and always $9 = C - 1$ at 30. With the centroid read-out, using the tested rank instead of $C - 1$ changes the accuracy by -3.5 points $[-5.1, -1.9]$ at 5 per class (79.7% against 83.2%), by -1.0 point $[-1.5, -0.5]$ at 10 (87.5% against 88.5%), and by nothing at 30. Selecting the rank by test is thus harmful to prediction exactly where the test lacks power, in agreement with Section 8; the prediction results of this section therefore use the rank $C - 1$.

12 The small-sample regime: diagnosis, quantification, remedies

Protocol: $C = 4$, $d = 20$, class separation 0.8, $n = 15$ per class ($n_{\text{train}} = 42$ after a 70/30 split), 20 seeds. References: the Bayes *oracle* rule (true Σ and μ_c, an upper bound in expectation),

shrinkage LDA, and a scikit-learn MLP (32 hidden units, Adam, default learning rate, at most 3000 iterations, no hyper-parameter search: a small untuned baseline, not a tuned competitor). The *baseline* analytic block uses the raw S_W, the rank of the shared-S_W permutation test, and a least-squares read-out on the full state h_T with $T = 4$. The ablation changes one factor at a time and therefore keeps the least-squares read-out when it moves to the code; its last row then replaces it by the centroid read-out (the LDA rule). The *corrected* block of the rest of the report is that last configuration: Ledoit–Wolf shrinkage, rank $C - 1$, centroid read-out on the code.

12.1 Ablation: where is the accuracy lost?

Table 10: Small-sample ablation ($n = 15$ per class, test accuracy in %, mean $\pm$ standard deviation over 20 seeds). "Code" rows read the code with the least-squares read-out of the baseline; the last row uses the centroid read-out (the LDA rule), for which T is irrelevant since the code is invariant.

	$\kappa = 10$	$\kappa = 200$
Bayes oracle	99.2 ± 2.0	100.0 ± 0.0
shrinkage LDA (reference)	96.7 ± 4.1	98.9 ± 2.2
MLP (back-propagation)	92.8 ± 5.3	98.1 ± 3.2
analytic block, *baseline*	78.9 ± 12.1	86.4 ± 10.6
+ shrinkage only	75.3 ± 13.3	87.8 ± 11.5
+ read-out on the code only	90.3 ± 7.2	81.4 ± 17.7
+ structural rank $C - 1$ only	81.1 ± 11.7	94.7 ± 6.0
+ $T = 1$ only	91.1 ± 6.9	98.1 ± 4.0
+ shrinkage + code	93.9 ± 7.8	97.8 ± 5.9
+ code + $C - 1$ (no shrinkage)	93.1 ± 5.2	100.0 ± 0.0
+ shrinkage + code + $C - 1$	96.1 ± 5.0	99.7 ± 1.2
same with $T = 1$	96.1 ± 5.0	99.7 ± 1.2
same, centroid read-out (**corrected block**)	$\mathbf{97.5 \pm 3.7}$	$\mathbf{100.0 \pm 0.0}$

Three distinct mechanisms, none of which is an irrecoverable loss of information inside the block, and a fourth, smaller effect of the read-out:

1. **Conditioning of the read-out** (the largest item). By (2), $T = 4$ crushes the component $(I - P)h$ by a factor $(1 - a)^4$: h_T becomes almost of rank r in $\mathbb{R}^{20}$, and a least-squares read-out with 21×4 coefficients on 42 samples overfits the $d - r$ noisy residual directions. Going to $T = 1$ or reading the code $c \in \mathbb{R}^r$ removes most of the drop.

2. **Under-estimated rank.** With the shared-S_W permutation test, $r^* = 2.75$ ($\kappa = 10$) and 2.05 ($\kappa = 200$) on average out of three true directions: a discriminant direction is often discarded. The structural bound settles the point (and the total-scatter test of Section 8 recovers $r = 3$ on these data).

3. **Noise on S_W^{-1}.** Shrinkage does not help on its own (the read-out dominates). Once the first two points are fixed, its effect is isolated by the rows "code + $C - 1$" and "shrinkage + code + $C - 1$": +3.0 points at $\kappa = 10$ (93.1 $\to$ 96.1) and no measurable effect at $\kappa = 200$ (100.0 $\to$ 99.7, i.e. one test error out of 360).

4. **Read-out of the code.** Replacing the least-squares read-out of the code by the centroid read-out adds 1.4 points at $\kappa = 10$ (96.1 $\to$ 97.5; paired interval $[0.0, 2.8]$, undetermined) and 0.3 point at $\kappa = 200$ (equivalent). With four classes the masking effect of Section 5.3

is small; the change is made for the meaning of the rule (LDA) more than for these differences.

With paired differences on the 20 seeds (conventions of Section 11), the corrected block (systematic shrinkage, rank $C-1$, centroid read-out on the code) is not distinguished from scikit-learn's shrinkage LDA at $\kappa = 10$ ($+0.8$ point, interval $[-0.7, +2.4]$, undetermined) and is 1.1 points above it at $\kappa = 200$ (interval $[0.0, 2.2]$, whose lower end is zero to the displayed precision); both are LDA rules and differ only by the shrinkage convention (Section 5.3). The block is above the MLP baseline, with 76 parameters against 804, by 4.7 points ($[+2.0, +7.4]$) at $\kappa = 10$ and 1.9 points ($[+0.4, +3.5]$) at $\kappa = 200$. The intervals are wide because each seed has 18 test points, and the MLP is an untuned baseline, so these figures establish an advantage over a small default MLP on these data and nothing more general. The remaining gap to the oracle (1.7 points at $\kappa = 10$, interval $[0.2, 3.2]$; none at $\kappa = 200$) is of the same order as that of LDA, which points to sample size rather than to the architecture.

12.2 Geometric quantification of the loss

The loss *inside the block* is measured directly, since the generator provides the true quantities: the largest principal angle between $\operatorname{span}\hat{U}$ and $\operatorname{span}(\mu_c - \bar{\mu})$, and between $\operatorname{span}\hat{W}$ and $\operatorname{span}(\Sigma^{-1}(\mu_c - \bar{\mu}))$. The angles are computed at rank $C-1$ for both estimates, so that subspaces of equal dimension are compared.

	$n = 15$/class		$n = 200$/class	
largest principal angle (degrees), rank $C-1$	$\hat{U}$	$\hat{W}$	$\hat{U}$	$\hat{W}$
$\kappa = 10$, raw S_W	28.1	48.1	—	—
$\kappa = 10$, Ledoit–Wolf	28.1	36.7	—	—
$\kappa = 200$, raw S_W	28.8	36.4	7.4	8.7
$\kappa = 200$, Ledoit–Wolf	28.8	41.4	7.4	13.5

At $n = 15$ per class the measurement subspace is 36–48° away from the true Fisher subspace (against 9–14° at $n = 200$): this is the order of magnitude of the geometric error due to estimation. (At the rank selected by the shared-S_W permutation test the figures for $\hat{W}$ are 47.6° and 35.8° at $\kappa = 10$, 29.8° and 40.9° at $\kappa = 200$, but they then compare subspaces of different dimensions.) Shrinkage reduces the error on $\hat{W}$ at $\kappa = 10$ and increases it at $\kappa = 200$, at $n = 15$ and at $n = 200$; a plausible reading is that its isotropic target erases part of the anisotropy that the measurement directions must follow: what is optimal for estimating Σ in Frobenius norm need not be optimal for the task. Despite angles above 30°, the corrected accuracy is within 0–1.7 points of the oracle: on these data, classification is more tolerant to angular error than the angle itself suggests.

12.3 Gradient fine-tuning of the block: what is free, and what is observed

The corrected block is used as the initialisation of a gradient training (a Fisher warm start), in `numpy` with hand-written gradients verified by finite differences. Two parametrisations are used, and the rows of the tables are named by the parameters that are actually free.

(A) Image frozen. $U_o = \mathrm{QR}(U)$ is *fixed* and $W = U_o + (I - U_o U_o^\top)M$ with M learned, so that $W^\top U_o = I_r$ exactly for every M: the measurement subspace moves, the representation subspace does not. The gate (s, b) and the read-out are learned or frozen according to the row. The read-out acts on the full state h_T (dimension d, to leave the gradient the possibility of exploiting $\ker W^\top$) and is initialised with the centroid read-out lifted to h_T: weights $W\bar{M}^\top$ and biases $-\frac{1}{2}\|m_k\|^2 + \log \pi_k$, with $\bar{M}$ the matrix of class centroids of the code. Since $W^\top h_T = W^\top x$ along the recursion, before any gradient step the model predicts exactly as the corrected block, and its logits are the LDA discriminant scores (checked: identical predictions on every test

point of ten draws). Penalty: L_2 of weight wd on M and on the read-out. Gradient check: relative error 2.8×10^{-7}.

(B) Both subspaces free. U and W are both learned, together with the gate and the read-out, in two variants: *idempotent*, with the projector $P = U(W^\top U)^{-1}W^\top$ recomputed from the current (U, W) at every step and the gradient propagated through the inverse, so that $P^2 = P$ holds exactly throughout training without any retraction (gradient check 9.1×10^{-8}); and *unconstrained*, with the rank-r map $UW^\top$ and no relation imposed between U and W (gradient check 1.3×10^{-6}). Penalty: weight wd on the read-out and on the distance of (U, W) to their initial values. The eight combinations of initialisation (warm start, with the same centroid read-out lifted to h_T as in (A), or a random orthonormal U with W close to it and small random gate and read-out), constraint (idempotent or not) and regularisation (wd 10^{-3} for 300 steps, or wd 10^{-1} for 100 steps) form a $2 \times 2 \times 2$ factorial design, which separates the three effects.

Common settings: cross-entropy, Adam, full batch, learning rate 10^{-2}, $T = 4$, 20 seeds; each seed draws a new dataset and a 70/30 split, so that at $n = 15$ per class the test set has 18 points and the intervals are wide. Heteroscedastic data (one covariance per class), for which LDA is mis-specified and information exists outside the Fisher code (the oracle is then the QDA rule), are included. All differences are paired with the analytic block on the same seeds, with the conventions of Section 11.

Table 11: Fine-tuning with the image U_o frozen (parametrisation A). Accuracy of the analytic block in % (mean ± standard deviation over 20 seeds), and paired difference of every other row to it, in points, with its 95% interval ($\approx$ equivalent within one point, $\uparrow$ above the block, $\downarrow$ below, ? undetermined). Rows: initialisation (W: warm start from the analytic block, R: random); free parameters; regularisation (weak: wd 10^{-3}, 300 steps; strong: wd 10^{-1}, 100 steps).

	Gauss. κ=10, n=15	Gauss. κ=200, n=15	hetero. n=15	hetero. n=60
Bayes oracle	$+1.7\,[+0.2, +3.2]^{\uparrow}$	$+0.0\,[+0.0, +0.0]^{\approx}$	$+5.3\,[+2.2, +8.4]^{\uparrow}$	$+2.3\,[+1.3, +3.3]^{\uparrow}$
shrinkage LDA	$-0.8\,[-2.4, +0.7]^{?}$	$-1.1\,[-2.2, -0.0]^{\downarrow}$	$-0.6\,[-2.0, +0.9]^{?}$	$+0.1\,[-0.2, +0.5]^{\approx}$
MLP	$-4.7\,[-7.4, -2.0]^{\downarrow}$	$-1.9\,[-3.5, -0.4]^{\downarrow}$	$-0.3\,[-2.6, +2.0]^{?}$	$-0.1\,[-1.0, +0.7]^{?}$
analytic block, accuracy	97.5 ± 3.7	100.0 ± 0.0	94.7 ± 6.4	97.7 ± 2.0
W; read-out; weak	$-1.1\,[-2.2, -0.0]^{\downarrow}$	$-0.6\,[-1.4, +0.2]^{?}$	$+0.6\,[-0.6, +1.7]^{?}$	$-0.4\,[-1.1, +0.3]^{?}$
W; gate, read-out; weak	$+0.0\,[-0.8, +0.8]^{\approx}$	$-0.8\,[-1.8, +0.1]^{?}$	$+0.8\,[-0.4, +2.1]^{?}$	$-0.3\,[-1.0, +0.3]^{?}$
W; M, read-out; weak	$-2.2\,[-4.5, +0.1]^{?}$	$-0.6\,[-1.4, +0.2]^{?}$	$-1.7\,[-3.8, +0.4]^{?}$	$-1.0\,[-1.8, -0.2]^{\downarrow}$
W; M, gate, read-out; weak	$-5.8\,[-8.6, -3.1]^{\downarrow}$	$-2.8\,[-4.8, -0.8]^{\downarrow}$	$-3.3\,[-6.1, -0.6]^{\downarrow}$	$-1.1\,[-2.0, -0.3]^{\downarrow}$
W; M, gate, read-out; strong	$-2.2\,[-4.5, +0.1]^{?}$	$-3.6\,[-5.7, -1.5]^{\downarrow}$	$-0.8\,[-2.9, +1.3]^{?}$	$-0.5\,[-1.3, +0.3]^{?}$
R; M, gate, read-out; weak	$-4.7\,[-7.1, -2.3]^{\downarrow}$	$-3.9\,[-6.1, -1.6]^{\downarrow}$	$-2.8\,[-5.8, +0.2]^{?}$	$-1.2\,[-2.4, +0.0]^{?}$
R; M, gate, read-out; strong	$-8.6\,[-12.7, -4.5]^{\downarrow}$	$-8.6\,[-11.5, -5.8]^{\downarrow}$	$-5.0\,[-7.2, -2.8]^{\downarrow}$	$-4.7\,[-5.9, -3.4]^{\downarrow}$

No net gain observed in the protocol tested. Of the 60 comparisons between a gradient-trained variant and the analytic block (15 variants, four configurations), none is above the block with an interval excluding zero; 3 are equivalent within one point, 24 are undetermined, and 33 are below the block. The largest positive differences (+0.6 to +0.8 point, all on the heteroscedastic problem at $n = 15$) are undetermined. This is a statement about these four synthetic configurations, these sample sizes, this optimiser and these budgets; it is not a statement that back-propagation cannot improve an oblique block.

The three effects, separately. *Initialisation.* With both subspaces free, the warm start is 4.9 to 12.4 points above the random initialisation on average over the two other factors, with intervals excluding zero in the four configurations: at these sample sizes and budgets the analytic solution is a much better starting point than a random one. With the image frozen,

Table 12: Fine-tuning with both subspaces free (parametrisation B): U, W, gate and read-out are all learned. Paired differences to the analytic block of Table 11, same conventions and abbreviations (idem.: exactly idempotent; unc.: unconstrained). The last five rows are paired contrasts between levels of the factorial design; the first is averaged over the two other factors.

	Gauss. κ=10, n=15	Gauss. κ=200, n=15	hetero. n=15	hetero. n=60
W; idempotent; weak	$-4.4\,[-6.9, -2.0]^{\downarrow}$	$-1.7\,[-3.8, +0.4]^{?}$	$-6.9\,[-12.5, -1.4]^{\downarrow}$	$-1.9\,[-3.3, -0.6]^{\downarrow}$
W; idempotent; strong	$-0.6\,[-2.4, +1.3]^{?}$	$+0.0\,[+0.0, +0.0]^{\approx}$	$+0.8\,[-0.9, +2.6]^{?}$	$-0.4\,[-1.1, +0.2]^{?}$
W; unconstrained; weak	$-1.9\,[-3.7, -0.2]^{\downarrow}$	$-1.1\,[-2.9, +0.7]^{?}$	$+0.0\,[-1.9, +1.9]^{?}$	$-0.8\,[-1.7, +0.0]^{?}$
W; unconstrained; strong	$-0.8\,[-1.8, +0.1]^{?}$	$+0.0\,[+0.0, +0.0]^{\approx}$	$+0.0\,[-1.5, +1.5]^{?}$	$-0.5\,[-1.0, +0.0]^{?}$
R; idempotent; weak	$-9.2\,[-12.7, -5.7]^{\downarrow}$	$-9.7\,[-14.0, -5.4]^{\downarrow}$	$-14.4\,[-21.5, -7.4]^{\downarrow}$	$-3.1\,[-4.5, -1.7]^{\downarrow}$
R; idempotent; strong	$-14.4\,[-20.3, -8.6]^{\downarrow}$	$-12.2\,[-17.1, -7.3]^{\downarrow}$	$-16.9\,[-21.7, -12.2]^{\downarrow}$	$-8.4\,[-11.0, -5.8]^{\downarrow}$
R; unconstrained; weak	$-14.4\,[-19.0, -9.9]^{\downarrow}$	$-10.6\,[-15.6, -5.5]^{\downarrow}$	$-11.1\,[-14.8, -7.4]^{\downarrow}$	$-5.0\,[-6.7, -3.3]^{\downarrow}$
R; unconstrained; strong	$-18.9\,[-23.5, -14.3]^{\downarrow}$	$-13.3\,[-18.4, -8.3]^{\downarrow}$	$-13.1\,[-17.9, -8.2]^{\downarrow}$	$-6.9\,[-8.8, -4.9]^{\downarrow}$
initialisation: W − R	$+12.3\,[+9.6, +15.0]^{\uparrow}$	$+10.8\,[+7.0, +14.6]^{\uparrow}$	$+12.4\,[+8.6, +16.1]^{\uparrow}$	$+4.9\,[+3.8, +6.0]^{\uparrow}$
constraint: idem. − unc., from W	$-1.1\,[-2.6, +0.4]^{?}$	$-0.3\,[-1.7, +1.2]^{?}$	$-3.1\,[-5.9, -0.2]^{\downarrow}$	$-0.5\,[-1.5, +0.5]^{?}$
constraint: idem. − unc., from R	$+4.9\,[-0.1, +9.8]^{?}$	$+1.0\,[-2.6, +4.6]^{?}$	$-3.6\,[-8.9, +1.7]^{?}$	$+0.2\,[-1.7, +2.0]^{?}$
regularisation: weak − strong, from W	$-2.5\,[-4.6, -0.4]^{\downarrow}$	$-1.4\,[-2.7, -0.1]^{\downarrow}$	$-3.9\,[-7.1, -0.6]^{\downarrow}$	$-0.9\,[-1.6, -0.3]^{\downarrow}$
regularisation: weak − strong, from R	$+4.9\,[+1.3, +8.4]^{\uparrow}$	$+2.6\,[-0.1, +5.4]^{?}$	$+2.2\,[-2.7, +7.1]^{?}$	$+3.6\,[+2.3, +4.9]^{\uparrow}$

the warm start is 4 to 6 points above the random initialisation under the strong penalty and within about one point of it under the weak one (differences of the entries of Table 11; no interval was computed for this contrast). *Geometric constraint.* From the warm start, the exactly idempotent model is below the unconstrained one by 0.3 to 3.1 points; the difference is established in one configuration out of four (heteroscedastic, $n = 15$: -3.1, interval $[-5.9, -0.2]$) and undetermined in the three others; from a random initialisation the effect of the constraint is undetermined in the four configurations. Imposing idempotence during training therefore brought no accuracy; whether its occasional cost is statistical or comes from the conditioning of the optimisation through $(W^\top U)^{-1}$ was not investigated. The unconstrained model started from the analytic solution is within 1.9 points of it (-1.9 to 0.0), below it in one case out of eight. *Regularisation.* From the warm start, the longer, weakly penalised training is below the short, strongly penalised one in the four configurations (-0.9 to -3.9 points, intervals excluding zero): the further training moves away from the analytic solution, the lower the accuracy. From a random initialisation the weak, longer setting is better ($+2.2$ to $+4.9$ points, two intervals out of four excluding zero), as expected for a penalty that pulls (U, W) back to an uninformative starting point.

Which parameters can be trained without loss. With the image frozen (Table 11), training the gate and the read-out stays within 0.8 point of the block (equivalent or undetermined in the four configurations). Training the read-out alone, starting from the LDA logits, is 1.1 points below the block at $\kappa = 10$ (interval $[-2.2, -0.0]$) and undetermined elsewhere: the cross-entropy fit moves the read-out away from the LDA rule without improving it. Training M with the weak penalty lowers the accuracy, by 0.6 to 2.2 points without the gate (below the block at $n = 60$, undetermined elsewhere) and by 1.1 to 5.8 points with it (intervals excluding zero in the four configurations).

Reading. On Gaussian data with a shared covariance the absence of gain is expected: LDA *is* the Bayes rule, so the Fisher code is a sufficient statistic in population; there is nothing to reinject, only to estimate. On heteroscedastic data, information exists outside the code (the QDA oracle is 2.3 to 5.3 points above the block), but no method of the two tables captures it at these sizes, including the models in which both subspaces are free: covariance differences involve $\mathcal{O}(Cd^2)$ statistics, a gated block of rank $C - 1$ has no capacity for them, and gradient descent cannot create information that the parametrisation cannot represent (Appendices C

and E show constructions that do capture it). At $n = 60$ the block and scikit-learn's shrinkage LDA are equivalent (+0.1 point, interval $[-0.2, +0.5]$), as two LDA rules should be.

12.4 The recipe for accuracy

The ablation suggests the following recipe on these data: (1) read the oblique code, not the full state (alone, +11.4 points at $\kappa = 10$ but -5.0 at $\kappa = 200$, where the under-estimated rank then dominates; +15.0 and +11.4 points together with shrinkage; and the prediction no longer depends on T, by Proposition 1); (2) structural rank $C-1$ rather than a test when n is not large compared with d; (3) systematic Ledoit–Wolf shrinkage of S_W, not only when $n \leq d$ (useful at $\kappa = 10$, neutral at $\kappa = 200$); (4) the centroid read-out on the code, which makes the block the LDA rule and needs no fitting (+1.4 and +0.3 points over least squares here, not established); (5) no gradient step through the block: in Section 12.3 no trained variant was above the block, and only the gate and the read-out trained together stayed within a point of it; (6) with the analytic gate and a read-out on the code, $T > 1$ changes nothing (Proposition 1); a dependence on depth requires a gate reading outside $\mathrm{span}(W)$, and it can only be useful if $\ker W^\top$ carries information in the data.

12.5 Anisotropy sweep

Same data, $\kappa \in \{1, 10, 50, 200, 1000\}$, $T = 4$, 20 seeds. The *orthogonal* variant is the nearest-centroid classifier (the same subspace, Euclidean metric). The gain of the corrected block

Table 13: Abundant regime ($n = 200$ per class), test accuracy in %. Parameters: baseline 144 (U and the $(d+1)C$ read-out on h_T; 204 if W, also needed to compute h_T, is counted), corrected 76 (W, the centroids and the constant terms of the centroid read-out), MLP 804.

$\kappa(S_W)$	orthogonal	block, baseline	block, corrected	MLP
1	97.5	96.8	97.5	96.6
10	97.2	98.9	99.1	98.2
50	96.8	99.9	100.0	99.5
200	96.6	100.0	100.0	99.8
1000	96.4	100.0	100.0	99.9

Table 14: Small-sample regime ($n = 15$ per class). r^*: rank retained by the shared-S_W permutation test of the baseline.

$\kappa(S_W)$	orthogonal	block, baseline	block, corrected	MLP	r^*
1	93.6	69.7 ± 15.0	93.1 ± 5.2	90.6 ± 5.0	2.4
10	96.7	78.9 ± 12.1	97.5 ± 3.7	92.8 ± 5.3	2.8
50	95.6	85.0 ± 9.3	99.2 ± 2.0	95.6 ± 5.4	2.2
200	93.6	86.4 ± 10.6	100.0 ± 0.0	98.1 ± 3.2	2.0
1000	91.1	90.8 ± 8.7	100.0 ± 0.0	97.5 ± 3.7	2.0

over the orthogonal variant grows with $\kappa(S_W)$ in both regimes, as expected from the role of the metric: it is nil or undetermined at $\kappa = 1$ (isotropic noise, where the Euclidean metric is already adequate) and reaches +9 points at $\kappa = 1000$ in small samples. The paired differences (20 seeds, same data and splits) are as follows.

$\kappa(S_W)$	corrected − MLP $n = 15$/class	corrected − MLP $n = 200$/class	corrected − orthogonal $n = 15$/class	corrected − orthogonal $n = 200$/class
1	$+2.5\,[+0.2, +4.8]^{\uparrow}$	$+0.9\,[+0.5, +1.3]^{\uparrow}$	$-0.6\,[-1.4, +0.2]^{?}$	$+0.0\,[-0.0, +0.1]^{\approx}$
10	$+4.7\,[+2.0, +7.4]^{\uparrow}$	$+0.9\,[+0.6, +1.3]^{\uparrow}$	$+0.8\,[-0.9, +2.6]^{?}$	$+2.0\,[+1.4, +2.5]^{\uparrow}$
50	$+3.6\,[+0.9, +6.3]^{\uparrow}$	$+0.4\,[+0.2, +0.6]^{\approx}$	$+3.6\,[+0.9, +6.3]^{\uparrow}$	$+3.1\,[+2.3, +4.0]^{\uparrow}$
200	$+1.9\,[+0.4, +3.5]^{\uparrow}$	$+0.2\,[+0.0, +0.3]^{\approx}$	$+6.4\,[+3.6, +9.2]^{\uparrow}$	$+3.4\,[+2.4, +4.5]^{\uparrow}$
1000	$+2.5\,[+0.7, +4.3]^{\uparrow}$	$+0.1\,[-0.0, +0.2]^{\approx}$	$+8.9\,[+5.4, +12.4]^{\uparrow}$	$+3.6\,[+2.5, +4.7]^{\uparrow}$

The effect of the metric (corrected against orthogonal) is established from $\kappa = 50$ upwards in small samples and from $\kappa = 10$ in the abundant regime, and is absent at $\kappa = 1$, as it must be. Against the untuned MLP baseline the corrected block, with no gradient and 76 parameters, is above in the five small-sample cases (lower ends of the intervals between 0.2 and 2.0 points); in the abundant regime it is above by less than 1.3 point at $\kappa \leq 10$ and equivalent within one point from $\kappa = 50$. The seed-to-seed standard deviation of the corrected block is at least two times lower than that of the baseline block. With the least-squares read-out on the same code the corrected block would differ by -1.4 to $+0.3$ point in small samples (paired differences equivalent or undetermined) and by at most 0.2 point in the abundant regime (log of `ropn_auto_nobp_sweep.py`).

13 Audit of the classical risks of oblique blocks with rank selection

The risk grid used for gradient-trained oblique networks (noise amplification, nearly degenerate subspaces, capacity loss, depth instability, unstable rank or configuration selection, gains wrongly attributed to obliquity, regularisation that changes the model) is applied here to the analytic block, with the recommended measurements, on the Gaussian data of this report ($C = 4$, $d = 20$, separation 0.8, 20 seeds).

R1/R2: noise amplification and nearly degenerate subspaces. These are the same risk (Proposition 5): $\|P\|_2 = 1/\sigma_{\min}(Q_W^{\top} Q_U) = 1/\sin\theta_{\min}$, the minimal angle between $\operatorname{im} P$ and $\ker P$ (in the metric S^{-1} the projector is orthogonal and has norm one), and Proposition 6 bounds it by $\frac{1}{2}(\sqrt{\kappa(S)} + 1/\sqrt{\kappa(S)})$. Measured: $\|P\|_2 = 1.10$ ($\kappa = 10$, $n = 15$/cl) to 1.70 ($\kappa = 200$, $n = 100$/cl), $\sigma_{\min}$ from 0.91 to 0.60: moderate obliquity, because Ledoit–Wolf shrinkage flattens the metric. Robustness to perturbations of the test inputs is the point that remains (noise: isotropic Gaussian noise of standard deviation 0.5σ or σ added to the test inputs, σ the mean per-feature standard deviation of the training set; last column: test cloud dilated by a factor two about its mean). The three variants use the same read-out, the centroid rule on their code; the same table with the least-squares read-out is in the log and leads to the same conclusions:

oblique / orthogonal / unconstrained low rank (%)	clean	noise 0.5σ	noise 1.0σ	dilation $\times 2$
$\kappa = 10$, $n = 15$/cl	97.5 / 96.7 / 93.3	92.5 / 91.1 / 86.9	83.3 / 83.1 / 72.2	96.1 / 95.8 / 93.6
$\kappa = 10$, $n = 100$/cl	98.8 / 96.2 / 98.9	96.2 / 94.1 / 94.9	84.7 / 84.8 / 82.8	98.8 / 96.4 / 98.9
$\kappa = 200$, $n = 15$/cl	100.0 / 93.6 / 100.0	98.3 / 90.3 / 93.9	85.8 / 78.6 / 75.3	100.0 / 92.8 / 100.0
$\kappa = 200$, $n = 100$/cl	100.0 / 96.0 / 100.0	97.9 / 94.0 / 96.3	81.7 / 86.1 / 77.8	100.0 / 96.0 / 100.0

Under isotropic noise added at test time (1.0σ) the comparison with the orthogonal variant goes either way: the oblique block loses 4 points at $\kappa = 200$, $n = 100$/cl (81.7 vs 86.1) and gains 7 at $n = 15$/cl (85.8 vs 78.6). The metric S^{-1} is fitted to the *training* noise, and noise of a different structure can penalise it; what is bounded is the amplification of the perturbation of the state, by $\|P\|_2 \leq 1.7$ here, not the loss of accuracy. Dilating the test cloud changes nothing measurable, as expected for an arg-max over affine scores, which is nearly invariant

to such a dilation by construction: that column is a sanity check, not evidence of robustness to distribution shift. The risk is not eliminated; the amplification is bounded by $\|P\|_2$, which shrinkage keeps below two on these data.

R3: capacity loss. Fixed ranks versus tested rank ($\kappa = 200$): at 15 and 100 per class the total-scatter test retains $r = 3 = C-1$ at every seed and the accuracy is that of full rank (100% in both cases, centroid read-out). At 5 per class ($n_{\text{train}} = 14 < d$) the test lacks power (mean tested rank 1.15 ± 0.48) and costs 27 points (67.5 vs 94.2 at rank $C-1$). The risk is located at $n < d$, and the answer remains the structural bound $C-1$ by default; Section 8.4 quantifies the same asymmetry under alternatives of every rank.

R4: depth instability. Global Jacobian of T shared steps ($d = 20$, $\kappa = 200$, $n = 100$/cl, $\|P\|_2 = 1.547$; for the gated recursions, finite-difference Jacobians, worst of five test points). *Linear recursion* $(I + P/T)^T$: condition number 2.85, 3.36, 3.77, 4.04, 4.28 for $T = 1, 2, 4, 8, 32$, below the bound $(1 + c_T\|P\|_2)\big(1 + \frac{c_T}{1+c_T}\|P\|_2\big) = 4.52$, 5.45, 6.18, 6.65, 7.08 that follows from Theorem 1 and from the inverse $(I + c_T P)^{-1} = I - \frac{c_T}{1+c_T}P$. *Un-normalised gated recursion* $h \leftarrow (1-a)h + aPh$ (analytic gate): condition numbers 4.3, 8.7, 16.7, 84.9, 4.8×10^6 and determinants from 4.5×10^{-4} to 7×10^{-108}: each step contracts the kernel by $1-a$ (Theorem 2 with $L = 1$, factor $(1-a)^{d-r}$), and the global Jacobian degenerates with depth. *Residual-normalised gated recursion* $h \leftarrow h + \frac{a}{T}(P-I)h$ (Theorem 2 with $L = T$): the condition number stays between 2.8 and 4.3 and the determinant between 4.5×10^{-4} and 2.0×10^{-3} up to $T = 32$. The degeneracy therefore belongs to the un-normalised gate, not to depth as such. The question does not arise in the recommended configuration ($T = 1$, analytic gate, read-out on the code), in which the prediction is independent of T (Proposition 1).

R5: unstable selection. Tested rank over 20 seeds: standard deviation 0 from 15 per class ($\kappa = 10$ and 200, histogram $[0, 0, 0, 20]$ over the ranks 0 to 3), 0.5 at 5 per class (histograms $[0, 11, 9, 0]$ at $\kappa = 10$ and $[1, 15, 4, 0]$ at $\kappa = 200$). Lattice AUTO by cross-validation (Appendix C) over 8 seeds: the same configuration every time (linear, no quadratic stage, consistent with the lattice helping only on heteroscedastic data). Band selection (Appendix I): true band found at every seed, three false positives out of 35 null bands at each sample size with the per-band level, none with the family-wise control. On these data the selection is stable above $n \approx d$ and unstable below: the same frontier as R3.

R6: gain wrongly attributed to obliquity. The two control variants in the table above have the same rank and the same read-out. The *orthogonal* variant (projector $QQ^\top$ onto the between-class directions) loses 4–6 points at $\kappa = 200$ on clean data: the gain of the oblique block is that of the within-class metric. But the *unconstrained low-rank* variant (reduced-rank ridge regression with a fixed ridge, no idempotence, read through the same centroid rule) does *as well* as the oblique block on clean data in three cases out of four (equal at $\kappa = 200$, +0.1 point at $\kappa = 10$, $n = 100$/cl; 4.2 points lower at $\kappa = 10$, $n = 15$/cl) and 1.3 to 11 points worse under added test noise, a difference that may reflect its different regularisation (fixed ridge versus Ledoit–Wolf) rather than the absence of the constraint. (With the least-squares read-out it was within 0.3 point at $\kappa = 200$ and 1.9 points lower at $\kappa = 10$, $n = 15$/cl.) The conclusion we draw: on clean data the accuracy comes from the Fisher metric, not from the projector constraint; what the constraint brings is structure (exact rank, invariant code, recursions that solve in closed form, Proposition 1), not accuracy points. The fine-tuning experiments of Section 12.3, in which the constraint is kept or released during training, point the same way.

R7: regularisation that changes the model. Idempotence: $\|P^2 - P\|/\|P\| \leq 10^{-15}$ by construction ($W^\top U = I_r$ exact). The operator actually used is $P = UW^\top$ with $W = S^{-1}DQ\Lambda^{-1/2}$, $U = DQ\Lambda^{-1/2}$, $S = (1-a)S_W + a\mu I$ (Ledoit–Wolf), $D^\top S^{-1} D = Q\Lambda Q^\top$. And shrinkage *does* change the projector: $\|P_{\mathrm{LW}} - P_{\mathrm{raw}}\|/\|P_{\mathrm{raw}}\| = 0.66$ at 15 per class ($a = 0.22$), 0.27 at 100 ($a = 0.045$), 0.045 at 1000 ($a = 0.005$). At small n the oblique block is not the raw Fisher projector but its shrunk version, by two thirds, and it is the shrunk version that is used (Section 12). The risk is present and accepted: the model is the shrunk projector, described exactly, and its distance to the raw projector decreases with n and can be reported.

Balance. On the data examined, two of the seven risks are removed by the construction (R7 for idempotence, which is exact; R5 above $n \approx d$), two do not arise in the recommended configuration (R4, and the block-count part of R5), and three remain in bounded, measured form: noise amplification, with $\|P\|_2$ between 1.1 and 1.7 (R1/R2), capacity loss at $n < d$ when the rank is tested (R3), and the change of model by shrinkage (R7, second half). R6 is settled in a direction that does not flatter the construction: on clean data the gain is that of the metric, not of idempotence. This audit covers one synthetic Gaussian family; it is not a general robustness study.

14 Limitations

- **No real EEG data, one real dataset in the central benchmark.** The execution environment had no network access: the central benchmark uses synthetic generators and scikit-learn's `digits`, the largest dataset available offline. No experiment on real EEG recordings is reported; the generators labelled "MI" and "ERP" in the appendices are not EEG models, and whether real features exhibit the anisotropy that favours the Fisher metric has to be measured. The protocol of Section 11 (fixed defaults, common selection folds, paired intervals, equivalence margin) applies unchanged to such data.

- **The block is LDA.** Its accuracy is that of shrinkage LDA (Proposition 3); the central benchmark shows where that is an advantage (small samples close to the Gaussian shared-covariance model) and where it is not (heteroscedastic data, digits from thirty images per class). Rank at most $C - 1$ by construction.

- **Synthetic Gaussian data.** Most experiments outside the central benchmark use synthetic data with a shared covariance, on which LDA is Bayes-optimal: this structurally favours a Fisher construction.

- **Extreme regime $n < d$.** At 4 per class ($n_{\mathrm{train}} = 11 < d = 20$) the corrected block reaches 92% ($\kappa = 10$) and 95% ($\kappa = 200$), against 83% for the MLP baseline, 84–89% for shrinkage LDA and 90% and 84% for the Euclidean nearest-centroid classifier (20 seeds, five test points per seed, standard deviations of 9–17 points: none of these differences is established). At this size S_W is barely estimable and no method can be recommended on this evidence.

- **Rank selection.** The level of the permutation test is guaranteed under the global null hypothesis only. The behaviour under alternatives is measured on one Gaussian family; the sequential rule has no proof of validity; a tested rank is not a good choice for prediction when the test lacks power (Section 8.4).

- **Statistics.** Paired intervals are reported in Sections 4, 8, 11 and 12; they are Student intervals on 10 or 20 realisations, without correction for the number of comparisons in a table, and on repeated subsamples of a fixed real dataset they measure subsample

variability only. The appendices report means and standard deviations over five splits, on test sets that are sometimes very small (18 samples per split in the small Gaussian cases): differences there are undetermined unless large. Timings are single runs on a shared CPU.

- **Depth.** With a shared block and a gate inside span(W), T changes neither the code nor the classification by the code; with gates reading the kernel, the non-linearity acts on at most $r+m$ linear functionals. This bounds the information accessible to the non-linearity, not the complexity of the function computed on it, which does change with T (Remark 1); we have not studied whether that change can be exploited. Other forms of depth (distinct blocks with learned non-linearities) are outside the scope of the theorems.

- **Fine-tuning.** "No net gain observed" refers to four synthetic configurations, one optimiser, two budgets and two penalties; the idempotent parametrisation through $(W^\top U)^{-1}$ may be harder to optimise than the unconstrained one, which was not investigated.

- **Multiplicative units.** The capacity theorem concerns quadrics and a disc; the experiments that reproduce its widths are two-dimensional. No advantage of gated over ReLU layers was observed beyond about thirty parameters or on the higher-dimensional datasets.

- **Comparisons in the appendices.** The gradient baselines there (MLP, numpy FNO) are small models trained with default or lightly adjusted settings for short budgets on a CPU, without systematic hyper-parameter search, whereas some back-propagation-free variants (lattice, kernel block, triceptron TRI-CF) use an internal cross-validation; those tables concern parameter and time efficiency at these scales, not the accuracy attainable by well-tuned networks.

- **What "analytic" covers.** Several back-propagation-free components are iterative numerical procedures (the Karcher mean, the alternating estimation of the Kronecker metric, internal cross-validation) and are not analytic solutions (Section 1).

15 Conclusion

We have characterised exactly the expressive limits of recurrences built on oblique projectors, established capacity results for multiplicative units, and constructed an analytic discriminant realisation with a calibrated rank selection. The three contributions have different statuses, and the conclusion keeps them apart.

Proved. For a shared block built on one oblique projector, the linear recursion collapses to $I + c_L P$; the code is invariant under the scalar-gated recursion, and a gate reading inside span(W) is constant along the orbit; the gated block has an explicit determinant; and with m independent kernel-reading gate directions the non-linearity has at most $r+m$ linear functionals of the input as arguments, whatever the number of steps. Depth therefore does not enlarge the subspace of information accessible to the non-linearity; it can nevertheless modify the function computed on that subspace. A gated unit with decoupled gate and value directions contains SiLU units, approximates every quadric at width $\max(p, n)$, and has the cell count of a ReLU layer, while a ReLU layer needs a width growing with the accuracy required on a disc. The Fisher block with a centroid read-out is exactly LDA with the same covariance estimate; its projector is exactly idempotent, admits an exact $C \times C$ factorisation with a dual form, and has a Euclidean norm bounded through the condition number of the shrunk metric. The permutation test in the label-free metric has its nominal level under the global null hypothesis, and the min-p selection its family-wise level.

Measured, in the stated protocols. The tested rank is a detection statement: it rarely overestimates, it under-estimates weak directions at small n, and using it for prediction costs up to

nine accuracy points where using $C-1$ costs at most 0.2. The block is equivalent to shrinkage LDA, ahead of logistic regression and of an MLP on small samples close to its model, on a par with them under heteroscedasticity, and behind them on the digits data once thirty to one hundred images per class are available. Its fit is 5 to 11 times faster than a library shrinkage LDA when $n \geq d$, and much more only when $n \ll d$ or against a dense generalised eigendecomposition. The small-sample loss of the baseline pipeline was a conditioning problem of the read-out, an under-estimated rank and a noisy S_W^{-1}, all three corrected without gradients. Gradient fine-tuning of the block, with the image frozen or with both subspaces free, with or without the idempotence constraint, showed no net gain in the protocol tested; the accuracy comes from the Fisher metric and the speed from the factorisation, while idempotence contributes exact statements and no accuracy by itself. Gated and ReLU layers trained by the same optimiser reach the minimal widths predicted for two quadrics and are not distinguished in accuracy beyond about thirty parameters.

Open. The usefulness of the construction on real EEG data and against tuned baselines has not been tested and is the necessary next step. Whether the change of function produced by depth on the $r+m$ accessible coordinates can be exploited is not known. Fixed non-linearities (polynomial lifts, matrix logarithms on the SPD manifold, Fourier dictionaries; Appendices C–I) extend the back-propagation-free programme in the favourable synthetic cases examined, and learned non-linearities (the gate direction of the triceptron, the lifting of a Fourier operator, the representation of a language model) are where gradient-trained components were clearly better in our experiments; these observations are exploratory.

A Exploratory extensions: status and reading guide

Appendices B to I collect explorations carried out with the same code base: a benchmark of the reference implementation on small datasets, lattices of blocks with fixed non-linearities, Kronecker metrics for matrix-valued inputs, a kernel variant and classical alternatives, a next-character experiment, the Euclidean cross-Gram, data on the manifold of symmetric positive definite matrices, and Fourier operators. They are kept because they document what was tried, including negative results, and because several of them contain exact statements of independent interest (Proposition 7, the mode-wise idempotence of spectral blocks). They are *not* part of the contributions listed in Section 1: each of them touches a separate literature and would need its own validation; their experiments are demonstrations on synthetic generators chosen to match the structure being illustrated (shared covariance for the Fisher block, separable noise for the Kronecker metric, a polynomial non-linearity for the Fourier dictionary), on a few small public datasets, with five splits per dataset and without the paired analysis of Section 11. Wherever the read-out acts on the code of a single block they use the centroid read-out (the LDA rule), as the main text does; the exceptions (rows that compare the two read-outs, read-outs on features that are not the code of one block) are stated where they occur. The comparisons with MLPs use untuned scikit-learn defaults. Statements of the form "A is ahead of B" in these appendices describe the tables and nothing more.

B Benchmark of the reference implementation on small datasets

Four methods are compared on synthetic data and on three real datasets (scikit-learn `digits`, `breast_cancer`, `wine`), 70/30 splits, standardisation fitted on the training part (constant features removed), 5 splits per dataset: the orthogonal (centroid) classifier, the baseline block (raw S_W when $n > d$, shared-S_W permutation rank, read-out on h_T), the corrected block, and an MLP with one hidden layer of width $2d$ clipped to $[32, 256]$ (Adam, default settings, at most 500 epochs, no hyper-parameter search). Time is the fitting plus prediction time on a sandbox

CPU, from one run; parameter counts are those of the last split (the rank of the baseline and the number of non-constant features can vary by a few units between splits). The corrected block

Table 15: Benchmark (test accuracy in %, parameters, fitting time).

dataset	method	accuracy	parameters	time (s)
Gaussian κ=10, n=15/cl	orthogonal	94.4 ± 3.5	80	0.002
	block, baseline	82.2 ± 11.9	124	0.025
	block, corrected	96.7 ± 4.4	76	0.001
	MLP	91.1 ± 5.7	1004	0.095
Gaussian κ=200, n=200/cl	orthogonal	98.6 ± 0.6	80	0.002
	block, baseline	100.0 ± 0.0	144	0.054
	block, corrected	100.0 ± 0.0	76	0.003
	MLP	99.9 ± 0.2	1004	0.246
heteroscedastic κ=50, n=60/cl	orthogonal	92.5 ± 1.4	80	0.001
	block, baseline	92.2 ± 1.4	144	0.021
	block, corrected	93.6 ± 1.1	76	0.001
	MLP	92.8 ± 1.6	1004	0.142
digits (8×8, 10 classes)	orthogonal	89.3 ± 1.2	610	0.004
	block, baseline	94.6 ± 0.6	1169	0.122
	block, corrected	95.3 ± 0.2	649	0.006
	MLP	97.8 ± 0.6	8794	0.967
breast cancer (2 classes)	orthogonal	92.9 ± 1.6	60	0.002
	block, baseline	94.6 ± 1.0	92	0.032
	block, corrected	95.4 ± 0.9	34	0.002
	MLP	96.5 ± 1.2	1921	0.243
wine (3 classes)	orthogonal	96.3 ± 2.0	39	0.002
	block, baseline	97.0 ± 2.5	68	0.020
	block, corrected	100.0 ± 0.0	35	0.001
	MLP	98.5 ± 1.4	547	0.118

uses the centroid read-out; the baseline keeps its least-squares read-out on h_T. Against the MLP baseline, the corrected block is ahead or tied on the Gaussian, heteroscedastic and wine data (differences of 0.1 to 5.6 points; only the 5.6 points of the small Gaussian case exceed one standard deviation), with 13 to 57 times fewer parameters and about two orders of magnitude less time; on the small Gaussian case it is also above the nearest-centroid classifier (96.7 vs 94.4). The MLP is ahead on digits (97.8 vs 95.3, a clear difference) and on breast cancer (96.5 vs 95.4), where a linear discriminant with a shared covariance is not an adequate model. The analytic block is a regularised LDA written as an oblique projector and should not be presented as better than an MLP in general; Section 11 gives the controlled comparison and Appendix E places the block among the classical alternatives.

C Lattices of analytic blocks with fixed non-linearities

Can analytic blocks be assembled *transversally* (several blocks per stage, on different inputs) and *in depth* (several stages) to exceed the single block while remaining back-propagation-free? Two structural constraints limit what can work. Stacking *linear* blocks adds no expressivity, since a composition of linear maps followed by a linear read-out is a linear classifier; moreover, when the block has full rank $r = \operatorname{rank} S_B$ (so that $\operatorname{span}(U) = \operatorname{span}(\mu_c - \bar{\mu})$), its residual $(I - P)x$ has

identical class means, and a second linear block fitted on that residual finds nothing ($S_B = 0$). In a cascade of back-propagation-free stages, depth can therefore contribute only through a **fixed non-linearity** placed between stages. Width contributes only if the blocks of a stage see **different inputs** (groups of variables: patches, bands, channels); two blocks on the same input are identical.

C.1 Architecture

Stage 1, transversal: G groups of variables, one fast block per group (independent, parallel) giving codes $c_g \in \mathbb{R}^{C-1}$, plus one global block giving a code c_0 and a residual $r_0 = (I - P_0)x$. Fixed non-linearity: $\varphi(c) = [c, c^{\circ 2}]$ on the codes (exposes variance differences), and degree-two monomials of the k leading principal components of the residual (second-order information, class-specific covariances invisible to LDA). Stage 2: one fast block on the concatenation (often $n < d$: dual form), final code and centroid read-out, i.e. an LDA on the stage-2 features. The composite map from the input to the stage-2 code is no longer a projector: a fixed non-linearity between two back-propagation-free stages produces what Rocks and Mehta [2] call a "noisy" oblique projection (idempotence and the orthogonality of the first step are both lost), and the exact results of Section 3 apply to each stage separately, not to their composition. Model selection: (k, φ) is chosen by internal cross-validation, each fit costing milliseconds; this plays the role of the grow/prune loop of gradient-trained oblique networks, without gradients.

Table 16: Back-propagation-free lattice vs single block vs MLP (test accuracy in %, 5 splits 70/30, standardisation). quad(k): quadratic stage on k residual components; AUTO: (k, φ) by internal cross-validation on the training part. The lattice is the only method of this table that receives an internal model selection; the MLP is run with default settings (width $2d$ clipped to $[32, 256]$, at most 500 epochs). Times from one run; parameter counts averaged over the splits.

	Gaussian n=15/cl	hetero. n=60/cl	hetero. n=300/cl	digits	breast	wine
single block	**96.7**	93.6	96.8	95.3	95.4	**100.0**
transversal only (G=4)	94.4	93.6	96.7	95.4	95.3	99.6
transversal $+[c, c^2]$	95.6	93.1	95.7	95.8	95.6	99.6
+ quad(8)	92.2	95.3	96.8	94.3	93.6	98.9
+ quad(16)	72.2	**96.7**	98.8	94.7	91.0	98.9
lattice AUTO	95.6	96.1	**99.2**	95.7	95.4	98.5
MLP	91.1	92.8	97.6	**97.8**	**96.5**	98.5
time AUTO / MLP (ms)	66/87	88/179	156/511	345/964	96/261	87/168
parameters AUTO / MLP	190/1004	680/1004	909/1004	2441/8688	73/1921	244/547

C.2 Verdict

The lattice is clearly ahead on heteroscedastic data: the quadratic stage captures part of the information "outside the code" that neither the single block, nor the MLP baseline, nor back-propagation through the block captured (96.1 and 99.2 against 93.6 and 96.8 for the single block and 92.8 and 97.6 for the MLP; at $n = 60$ per class the internal cross-validation picks a configuration slightly below the best fixed one, 96.7), without gradients and with fewer parameters: second-order information is used without estimating C covariances $d \times d$, only a block on $k(k+1)/2$ monomials. Regularised QDA does better still when it is affordable (Appendix E). It does not win on rich non-linear structure (digits: 95.7 vs 97.8, and +0.4 point only over the single block): two back-propagation-free stages with fixed squares do not replace 122 learned hidden units; closing that gap would call for richer non-linearities (random features, convolutional patches), which brings the approach close to PCANet [16] and extreme learning machines [17], to which it is related. The danger is capacity: without control, the quadratic

stage overfits as soon as $k(k+1)/2 \gtrsim n_{\text{train}}$ (72% on the Gaussian $n_{\text{train}} = 42$ case, 91.0% on breast); internal cross-validation avoids the collapse on these datasets, but where the lattice does not improve on the single block it costs about one standard deviation (95.6 vs 96.7 on the small Gaussian case, 98.5 vs 100.0 on wine), at one to two orders of magnitude more time than a single fixed configuration, which remains of the order of the MLP. Beyond two stages, since a further linear stage on a residual without class means finds nothing, an additional stage can add only what its non-linearity exposes.

C.3 Advantages and drawbacks relative to the MLP

Advantages	**Drawbacks**
No gradient, no optimisation hyper-parameter (learning rate, epochs, initialisation); deterministic and reproducible.	Capacity bounded by construction: $r \leq C-1$ per block, fixed non-linearities; does not catch an MLP on data with rich structure.
Each block is interpretable (Fisher directions per group, obliquity, rank), with structural properties (exact idempotence, $\det(I + P/L) \neq 0$).	The number of stage-2 features must stay $\lesssim n$: capacity is governed by n, not by depth.
Cost $\mathcal{O}(nd^2 + d^3/3)$, or $\mathcal{O}(n^2 d)$ in dual form; blocks of a stage in parallel; inference $\mathcal{O}(dr)$.	Groups and non-linearities are priors (patches, bands, monomials); the MLP learns them.
Configuration chosen by cross-validation, each candidate in milliseconds.	On already whitened or isotropic features, nothing to gain over shrinkage LDA.

D Doubly oblique geometry: Kronecker metrics for matrix-valued inputs

"Doubly oblique" can denote three different objects; only one withstands examination, and it is directly useful for channels×time data such as EEG trials.

1. **Measurement obliquity beyond the metric** ($W \neq MU$). The analytic block is M-orthogonal with $M = S_W^{-1}$: oblique in the Euclidean sense, orthogonal in the Fisher sense. Adding a second obliquity *inside* that metric leaves the Bayes optimum; it makes sense only with a second source of information, e.g. *per-class* metrics $w_c = \Sigma_c^{-1} u_c$ (a fan of measurement directions). The object is then no longer a single linear projector but a regularised QDA, at a cost of $\mathcal{O}(Cd^2)$ statistics; the quadratic stage of Appendix C is an economical substitute, and in Appendix E regularised QDA is the most accurate back-propagation-free option on the heteroscedastic data and on digits, where $n_c > d$.

2. **Kronecker projector** $P = P_R \otimes P_L$ for inputs $X \in \mathbb{R}^{c \times t}$, one oblique projector on the left (channels) and one on the right (time), fitted alternately. Idempotent, of rank $r_L r_R$, but the restriction of the *image* to a Kronecker structure loses information and the alternation is only a heuristic: tested, worse than the vectorised block (Table 17).

3. **Kronecker metric** $M = \Sigma_T^{-1} \otimes \Sigma_S^{-1}$: keep the full representation $U = \text{vec}(M_c - \bar{M})$ (rank $C-1$, no restriction) and make the *measurement* oblique in space *and* time,

$$w_c = \text{vec}\big(\Sigma_S^{-1}(M_c - \bar{M})\Sigma_T^{-1}\big), \tag{18}$$

with Σ_S, Σ_T estimated on the residuals by a few alternating analytic updates (five passes of the alternating maximum-likelihood scheme of Dutilleul [18], with a small ridge; this estimation is a short iteration, not a single closed form). This is exactly the fast block with a Kronecker solver ($S^{-1}\,\text{vec}(A) = \text{vec}(\Sigma_S^{-1} A \Sigma_T^{-1})$): the whole framework applies (idempotence, $W^\top U = I$, $C \times C$ reduction, Theorem 1, Proposition 1), never a $ct \times ct$ matrix, and $\mathcal{O}(c^2 + t^2)$ statistics instead of $\mathcal{O}(c^2 t^2)$.

Table 17: Doubly oblique geometry on synthetic 16×32 matrix data ($d = 512$), $C = 3$, rank-2 spatio-temporal class patterns, anisotropic $\Sigma_S \otimes \Sigma_T$ noise ($\kappa = 30$ on each side). Test accuracy in %, 8 independent draws, stratified 70/30 splits; the three blocks use the centroid read-out on their code. MLP: 64 hidden units, default Adam, at most 500 epochs.

separation, n/class	vectorised block	Kronecker metric	Kronecker projector	MLP
0.25, 20	94.4 ± 6.8	$\mathbf{100.0 \pm 0.0}$	87.5 ± 13.0	81.9 ± 11.0
0.25, 60	99.8 ± 0.6	$\mathbf{100.0 \pm 0.0}$	98.4 ± 2.3	95.1 ± 5.7
0.15, 60	91.0 ± 8.9	$\mathbf{100.0 \pm 0.0}$	88.7 ± 6.6	73.8 ± 11.8
0.15, 200	99.4 ± 0.6	$\mathbf{100.0 \pm 0.0}$	93.5 ± 6.2	94.4 ± 2.6
0.10, 200	92.2 ± 4.2	$\mathbf{99.9 \pm 0.2}$	90.8 ± 6.0	79.7 ± 6.1

The gain is large because the generator's noise *is* Kronecker: the doubly oblique metric estimates of the order of $16^2 + 32^2$ coefficients where the vectorised block has to regularise a 512×512 scatter estimated from 42 trials (first row). This is a best case for the method. On real EEG the space×time separability is only approximate, so any gain is expected to be partial and has to be measured; no real-data experiment is reported here. On these data the Kronecker *projector* variant is worse than the vectorised block and is not pursued: the Kronecker structure is useful in the metric, not in the image of the projector. Separable (Kronecker) covariance models for matrix-valued data and their alternating maximum-likelihood estimation, often called flip-flop, are classical [18]; what is specific here is the reading in the oblique formalism (oblique measurement in two spaces at once, unrestricted representation) and the fact that it is obtained by changing only the solver of the fast block, so that idempotence, biorthogonality and the results of Section 3 still hold. Common spatial pattern (CSP) filters [19] are also non-orthogonal spatial measurement directions, obtained from a generalised eigenproblem between class *covariances*; (18) is a counterpart for class *mean* patterns (evoked responses), with a temporal metric in addition. Appendix H shows, accordingly, that this block is blind to covariance differences, for which tangent-space features on the SPD manifold are better suited.

E Alternatives to the MLP

E.1 The kernel oblique block

The dual form of Section 6 depends on the data only through the Gram matrix $X_c X_c^\top$. Replacing it by a kernel $K_{ij} = k(x_i, x_j)$ gives, in closed form, the same oblique block in the feature space φ: $U = \Phi^\top B$ (class means, rank $C - 1$), $W = S^{-1}U = \Phi^\top A$ by Woodbury on $n \times n$, code $W^\top \varphi(x) = \alpha^\top k(X, x)$, with $W^\top U = I_r$ automatically (verified to 4×10^{-15} in feature space). This is kernel Fisher discriminant analysis [20] read as an oblique projector: capacity governed by n rather than by depth, no gradient, hyper-parameters (γ, ridge) by internal cross-validation; its read-out is the centroid rule on the kernel code. It nearly closes the gap of the linear block on digits (97.5, against 97.8 for the MLP baseline), at a cost of $\mathcal{O}(n^3)$ plus cross-validation (about 4 s at $n = 1257$).

E.2 Verdict

On these five datasets the most accurate alternatives to the MLP are not the fast ROPN block: the RBF-kernel SVM is best or within 0.4 points of the best on digits, breast cancer and wine, in milliseconds with default settings; regularised QDA is far ahead on the heteroscedastic data (99.7) and close to the best on digits (98.1), but requires $n_c > d$; on the small Gaussian case the block with its centroid read-out and the kernel block are best (96.7), ahead of the library shrinkage LDA (95.6). The centroid read-out is above the least-squares read-out by 3.4

Table 18: Test accuracy in % (5 splits 70/30, standardisation). "CV": hyper-parameters chosen by 3-fold cross-validation inside the training part (kernel block: bandwidth and ridge; QDA: scikit-learn's `reg_param`; SVM: C and bandwidth). The other methods use fixed or default settings. QDA could not be fitted at $n = 15$ per class ($n_c < d$; the library raises an error).

	Gaussian n=15/cl	hetero. n=60/cl	digits	breast	wine
fast block, least-squares read-out	93.3	93.3	93.7	95.4	99.6
fast block, centroid read-out	**96.7**	93.6	95.3	95.4	**100.0**
shrinkage LDA (library)	95.6	93.9	95.6	95.4	99.6
lattice AUTO (CV)	95.6	96.1	95.7	95.4	98.5
kernel oblique block (CV)	**96.7**	94.7	97.5	96.6	99.3
regularised QDA (CV)	—	**99.7**	98.1	95.8	99.3
SVM, RBF kernel, $C = 1$ (default)	94.4	95.3	**98.3**	96.6	99.6
SVM, RBF kernel (CV)	93.3	95.0	**98.3**	**96.8**	99.3
histogram gradient boosting	78.9	91.4	97.3	95.9	98.5
k-NN ($k = 5$)	86.7	94.7	97.4	95.6	94.8
MLP	91.1	92.8	97.8	96.5	98.5

points on that case and by 1.6 points on digits, and within 0.4 point elsewhere (five splits, no interval computed; Section 11 gives the paired comparison). Boosting is weak on the small synthetic sets; k-NN is competitive on digits. The fast block is a regularised LDA: it does well where LDA does well (small n, anisotropic data with shared covariance), and adds a geometric reading, a low cost and exact idempotence. The per-class-metric reading of "doubly oblique" (Appendix D), i.e. regularised QDA, is above the quadratic lattice on the heteroscedastic data (99.7 vs 96.1), where Cd^2 statistics are affordable; the lattice may keep an interest at larger d, which is not tested here. QDA can be read in the oblique framework as one representation with C measurement metrics. The kernel block is the path suggested by the dual derivation: it comes within 0.3 point of the MLP baseline on digits without gradients, and it is above the SVM only on the small Gaussian case, where it ties with the linear block. Differences of a few tenths of a point in this table are within the split-to-split standard deviations (0.2–1.5 points on digits and breast cancer, up to 3 on wine, 4–7 on the small Gaussian case).

E.3 What remains unexplored

The LDA→QDA continuum (Friedman's regularised discriminant analysis [21], two parameters), i.e. C metrics $M_c = [(1 - \lambda)\Sigma_c + \lambda S_W]^{-1}$ shrunk towards the identity, is a natural back-propagation-free path between the fast block and QDA; it is not tested here. Ensembles of blocks on random subspaces are another untested option for small samples. For EEG, the Kronecker metric on raw trials or per-class metrics on tangent-space features remain to be tested on real data. Nothing in this report suggests that a back-propagation-free construction of this kind could replace learned convolutions on images, or boosting on large heterogeneous tables.

F Language modelling

A minimal "language" task is next-character prediction: a frozen corpus of 1.5×10^5 characters (Python source code with French comments, reduced to an alphabet of 35 symbols: lowercase letters, space and common punctuation; file `ropn_corpus.txt`), a context of k one-hot characters ($d = 35k$), $C = 35$ output classes, 30 000 sampled positions split 75/25 (22 500 training examples), one split. The target depends on *conjunctions* of context positions. This is

a toy experiment on an unusual corpus, meant to exhibit a structural limitation, not a language modelling benchmark. At $k = 4$ the analytic block reaches 26.5% with the centroid read-out,

Table 19: Next-character prediction: top-1 test accuracy (%) and perplexity (where probabilities are available), one split. The MLP (256 hidden units, Adam) is stopped after 60 epochs, before convergence.

	$k = 2$ ($d = 70$)		$k = 4$ ($d = 140$)	
	acc.	ppl	acc.	ppl
n-gram (counts, smoothing)	**37.0**	9.06	**52.7**	7.35
fast block, centroid read-out (LDA rule)	23.0	—	26.5	—
fast block, least-squares read-out	27.6	—	30.0	—
shrinkage LDA (probabilities)	22.0	25.5	25.2	24.0
logistic regression (linear)	29.4	11.4	33.9	10.4
lattice AUTO (quadratic)	30.8	—	—	—
MLP 256	36.6	**8.41**	51.8	**6.44**

close to the library shrinkage LDA (25.2%) as two LDA rules should be, against 51.8% for the MLP and 52.7% for a plain count table; shrinkage LDA has a perplexity more than three times that of the count table (24.0 vs 7.35). On this task, with 35 classes, sparse one-hot inputs far from the Gaussian model and a shared-covariance assumption that does not hold, the least-squares read-out of the same code is 3.5 to 4.6 points above the LDA rule: the advantage of the centroid read-out on digits (Section 5.3) is not general. The quadratic lattice recovers part of the pairwise interactions (+7.8 points over the block at $k = 2$) but stays behind the count table. The models that are *linear* in the one-hot input (LDA, logistic regression) plateau between 23% and 34%: the target depends on conjunctions of context characters, which a linear discriminant of the one-hot context cannot represent. This limitation is structural rather than a matter of tuning: all results of this report assume *given* features, and nothing here estimates a representation in closed form; the feed-forward sub-layer of a transformer acts as a non-linear key–value memory (keys W, values U, code $W^\top h$ followed by a non-linearity), of which the oblique block is the version *without* the non-linearity, hence a rank-r linear map that an adjacent linear layer can absorb; the structural rank bound is not informative when C is the vocabulary size; and the dual form, whose cost grows with n, does not scale to corpora of realistic size. The oblique geometry may remain useful for language models as a reading grid; the following are analogies and directions, not results of this report: attention as a bilinear form with query/key asymmetry, LoRA [22] as a rank-r update $UAW^\top$, determinants of residual blocks as diagnostics of rank collapse; and, without back-propagation, classification heads on frozen embeddings in small samples (the regime in which the block was above the MLP baselines in Sections 11 and 12), Fisher initialisation of heads or adapters (in Section 12.3 the warm start was above a random initialisation in every configuration when both subspaces were trained), rank selection for classification targets, and possibly an oblique version of structural compression by projection of the residual stream (the orthogonal version exists [23]), with the caveat that a non-orthogonal transformation does not commute with RMSNorm/LayerNorm, the property on which the orthogonal insertion relies.

G The Euclidean cross-Gram

Two objects carry this name. The coupling $G = W^\top U$ is a *gauge*, not a lever: $P = U(W^\top U)^{-1}W^\top$ depends only on the subspaces $\operatorname{im} U$ and $\ker W^\top$; for invertible A, B, (UA, WB) gives the same P while G becomes $B^\top GA$. The fast block fixes the gauge at $G = I_r$. Replacing G^{-1} by an arbitrary matrix gives $UAW^\top$, a rank-r map that is in general

not idempotent (the form of a low-rank adapter such as LoRA [22]); it loses the properties of a projector without adding a subspace. Improvements can only come from the choice of subspaces, i.e. from the *metric* that defines W from U.

The cross-covariance S_{XY} between inputs and class indicators is the second object. The Fisher block can be seen as a canonical correlation analysis between X and the class indicators Y: U comes from S_{XY} (class means) and $W = S_W^{-1}U$. Taking the *Euclidean* cross-Gram, with no metric ($W = U$, left singular vectors of S_{XY}), gives an **orthogonal** projector: the PLS-SVD construction, which coincides with PLS-DA [24] on the first component (later components of the usual PLS-DA algorithm are obtained after deflation and differ, hence the two distinct columns of Table 20). Between the two, the shrunk metric $M_\lambda = (S_W + \lambda \frac{\operatorname{tr} S_W}{d} I)^{-1}$ describes a continuous path whose $\lambda \to \infty$ end is the Euclidean one (verified: $\|U - W\| = 0$, $\|W^\top U - I\| \approx 10^{-15}$). On these three cases the Euclidean cross-Gram does not improve the block: it is its *orthogonal*

Table 20: Shrinkage path from the Fisher block to the Euclidean cross-Gram: test accuracy in %, 5 splits 70/30 of a fixed dataset (18 test samples per split on the Gaussian cases); every block uses the centroid read-out on its code (the LDA rule only for the Fisher metric). "λ CV": $\lambda \in \{0.01, 0.1, 1, 10\}$ chosen by 3-fold cross-validation inside the training part; the $\lambda = \ldots$ columns are fixed values.

	LW auto	λ=0.01	λ=0.1	λ=1	λ=10	Euclidean	λ CV	PLS-DA	MLP
Gaussian d=20, n=15/cl, κ=10	96.7	94.4	94.4	97.8	96.7	94.4	96.7	91.1	91.1
Gaussian d=20, n=15/cl, κ=200	100.0	100.0	100.0	100.0	97.8	91.1	100.0	86.7	93.3
digits	95.3	95.7	95.6	94.5	91.0	89.2	95.5	90.4	**97.8**

end, and it is behind when anisotropy matters (91.1 vs 100 at $\kappa = 200$; 89.2 vs 95.3 on digits). Choosing λ by internal cross-validation instead of Ledoit–Wolf gives 0.0, 0.0 and +0.2 point: with the centroid read-out the Ledoit–Wolf metric is as good as the cross-validated one on these cases, while some fixed values do better or worse by one or two test samples per split on the Gaussian cases, which is within the split-to-split variability. With continuous targets (the embedding of the next token) the same object is reduced-rank regression / CCA between context and target (the eigenwords family [25]), closed-form and useful as an initialisation or a probe, and the rank becomes a quantity that must be selected, since $C - 1$ no longer bounds anything; but it remains linear in the given features and does not learn the representation.

H Riemannian geometry: data on the SPD manifold

Everything so far is Euclidean: a vector space, a constant metric $M = S_W^{-1}$, a fixed oblique projector. Riemannian geometry enters through three doors that must be kept apart: the *data* may live on a manifold (covariance matrices of EEG trials, symmetric positive definite, the standard setting of brain–computer interfaces); the *metric* of the representation space may depend on the point; the *parameters* (U, W) live on a manifold. Only the first preserves both the closed form and the algebraic content of the theorems.

H.1 Data on the SPD manifold: the analytic block in tangent-space coordinates

Let $X \in \mathbb{R}^{p \times T}$ be a temporally centred trial and $\Sigma = XX^\top/(T-1)$ its sample covariance. We assume that Σ is positive definite, which holds when $T > p$ and the channels are not linearly dependent (all experiments below use $T > p$; the smallest covariance eigenvalue observed is 0.047 at $p = 8$ and 0.035 at $p = 32$), and which must otherwise be enforced by a regularised covariance estimator. The space $\mathcal{S}_p^{++}$ of symmetric positive definite matrices has dimension

$d = p(p+1)/2$. We equip it with the affine-invariant Riemannian metric [26]

$$\langle A, B\rangle_\Sigma = \operatorname{tr}\big(\Sigma^{-1} A \Sigma^{-1} B\big).$$

An alternative is the log-Euclidean metric [27], the pull-back of the Frobenius inner product by the matrix logarithm: it is a different geometry on the same space, with distance $d_{\mathrm{LE}}(\Sigma_1, \Sigma_2) = \|\log \Sigma_1 - \log \Sigma_2\|_F$, not a simplification of the affine-invariant one.

Following the tangent-space approach to Riemannian brain–computer interface classification [28], we compute a single reference point G, the affine-invariant Riemannian (Karcher) mean of the *training* covariances, and keep it fixed when transforming subsequent trials. In a validation protocol, G is part of the fit of each partition: "fixed once" means fixed after estimation on the training split, never computed on the pooled data before validation (this is what the scripts of this report do). Each covariance is represented by

$$\phi_G(\Sigma) = \operatorname{vec}_F\big[\log\big(G^{-1/2}\Sigma G^{-1/2}\big)\big] \in \mathbb{R}^d, \qquad d = \tfrac{p(p+1)}{2}, \tag{19}$$

where vec_F stacks the diagonal entries and the upper-triangular off-diagonal entries, multiplying the latter by $\sqrt{2}$, so that $\|\operatorname{vec}_F(S)\|_2 = \|S\|_F$ for every symmetric S. These are whitened coordinates of the affine-invariant logarithmic map at G, a local chart of the manifold, not a globally distance-preserving flattening. The log-Euclidean alternative uses $\psi(\Sigma) = \operatorname{vec}_F(\log \Sigma)$ with no reference point; both are compared in Section H.3.

The resulting vectors are then processed by the algorithms of this report: the fast block ($C \times C$ generalised eigenvalue problem, Proposition 4, Ledoit–Wolf shrinkage, rank at most $C-1$) followed by a linear read-out. Conditional on the fitted reference point G, the purely algebraic statements of Theorems 1 to 4 and of Propositions 1, 4 and 5, which hold for arbitrary Euclidean feature vectors, carry over to $\phi_G(\Sigma)$ verbatim; the statistical statements (optimality of the Fisher code, calibration of the rank test, accuracy comparisons) require their assumptions to be checked on the transformed data, and Section H.3 does so empirically. The essential non-linearity is supplied by the covariance representation and the matrix logarithm, not by the subsequent linear block: once G has been estimated, ϕ_G is a fixed non-linear feature map requiring no back-propagation, and it provides the non-linear representation on which the discriminant machinery operates, the role the shared block could not play (Theorem 3). A single tangent-space representation locally approximates the affine-invariant geometry; it does not, in general, transform Wishart-distributed covariances into Gaussian vectors nor guarantee that the assumptions of LDA hold. The usual probabilistic reading of tangent-space LDA is a wrapped-Gaussian model with a shared covariance in the tangent coordinates: a modelling assumption, not a consequence of the logarithm.

Finally, $p = 64$ channels yield $d = 2080$ tangent-space features, the feature dimension of the timing benchmark of Section 6 (Table 3). The speed-up reported there characterises the block fit at that dimension, not an end-to-end speed-up of the present pipeline: geometric preprocessing must be added, namely covariance estimation, $O(np^2T)$, the Karcher mean, $O(knp^3)$ for k iterations of the standard fixed-point scheme ($k = 8$ to 10 at $p = 8$ and $k = 15$ at $p = 32$ for a tolerance of 10^{-10} on the mean tangent vector in the experiments below), and the tangent map, $O(np^3)$. The Karcher mean is an iterative computation: this pipeline is free of back-propagation, not of iteration.

Proposition 7 (Exact congruence invariance, shrinkage included)**.** *Let $A \in GL_p(\mathbb{R})$ be arbitrary (channel gains, re-referencing, linear mixing) and $\Sigma_i \mapsto A\Sigma_i A^\top$ be applied to* all *trials, training and test. The chain {Karcher mean $\to \phi_G \to$ fast block (Ledoit–Wolf shrinkage, $C \times C$ eigenproblem, rank $C-1$) $\to$ read-out fitted on the codes (centroid or least squares)} produces* identical *codes $W^\top\phi$ (up to the sign and rotation indeterminacy of eigenvectors, which the read-out absorbs) and identical predictions. The Euclidean chain on $\operatorname{vec}_F \Sigma$ with shrinkage is not invariant in general. The log-Euclidean chain $\operatorname{vec}_F \log \Sigma$ is invariant when A is a scalar multiple of an orthogonal matrix (the codes are then shifted by a constant), and not in general.*

Proof. (a) The Karcher mean is equivariant, $G(A\Sigma A^\top) = AG(\Sigma)A^\top$, since the affine-invariant metric is invariant under congruence and so is the Fréchet functional. (b) Let $G' = AGA^\top$ and $Q = G'^{-1/2}AG^{1/2}$: $QQ^\top = G'^{-1/2}AGA^\top G'^{-1/2} = I$, so Q is orthogonal, and $G'^{-1/2}(A\Sigma A^\top)G'^{-1/2} = Q(G^{-1/2}\Sigma G^{-1/2})Q^\top$. (c) $\log(QXQ^\top) = Q(\log X)Q^\top$ for orthogonal Q, and $X \mapsto QXQ^\top$ is a Frobenius isometry of symmetric matrices; through vec_F it is an orthogonal matrix $R \in O(d)$: $\phi'(\Sigma_i) = R\phi(\Sigma_i)$ for all i. (d) Under an orthogonal change of features $h \mapsto Rh$, $S_W \mapsto RS_WR^\top$ and $D \mapsto RD$. The Ledoit–Wolf coefficient depends on the centred samples x_i only through $\operatorname{tr} S_W$, $\|S_W\|_F$ and $\sum_i \|x_i\|^4$, which are invariant, so the coefficient a and the target scale $\mu = \operatorname{tr}(S_W)/d$ are unchanged and $S = (1-a)S_W + a\mu I \mapsto RSR^\top$. The $C \times C$ matrix $D^\top S^{-1} D$ of Proposition 4 is therefore invariant, so $W \mapsto RW$, $U \mapsto RU$ and the codes $W^\top h$ are invariant; the read-out, trained on the same codes, is the same. For $\mathrm{vec}_F\,\Sigma$ the map induced by A is $\Sigma \mapsto A\Sigma A^\top$, which is not an isometry unless $A \in O(p)$: the shrinkage target is not equivariant and the codes change. For the log-Euclidean chain, if $A = cQ$ with Q orthogonal then $\log(A\Sigma A^\top) = Q(\log\Sigma)Q^\top + 2\log|c|\,I$, an orthogonal change of features followed by a constant shift, to which the class-centred statistics and the re-fitted read-out are insensitive; for a general A, $\log(A\Sigma A^\top)$ is not an affine function of $\log\Sigma$. □

Verification ($p = 8$, A = channel gains between 0.1 and 10 followed by a rotation, condition number 40.8, covariance-difference regime, one draw): Karcher mean equivariant to 2.9×10^{-15}; tangent-space chain: Gram matrix of the tangent vectors preserved to 6.3×10^{-14} (orthogonal rotation), codes preserved to 2.4×10^{-14}, predictions identical at 100% (96.7% → 96.7%, centroid read-out); log-Euclidean: codes changed (relative gap 1.3), 96.7% of the predictions unchanged (accuracy 96.7% → 93.3%); Euclidean $\mathrm{vec}_F\,\Sigma$: accuracy 95.0% → 53.3%, 48.3% of the predictions unchanged. For EEG this says something useful: with the Kronecker metric of Appendix D, or with any shrunk Euclidean block, a change of electrode gains changes the classifier; in the affine-invariant tangent space it changes nothing, provided the same transformation applies to all trials.

H.2 What each algorithm becomes

The fast block becomes a fast block on $\phi_G(\Sigma)$ at base point G: the construction is unchanged ($C \times C$ eigenproblem, Ledoit–Wolf shrinkage, rank $C-1$, permutation test), and it acquires the exact invariance above; it is shrinkage tangent-space LDA [28] re-read in biorthogonal (U, W) form. The algebraic content of Theorems 1–4 is unchanged in the tangent coordinates at a fixed G; with a *moving* base point (a projector P_h in $T_h\mathcal{M}$, transported along the recursion) the proofs of Section 3, which rely on a fixed linear projector, no longer apply, and neither does the closed form; this case is not analysed here. The Kronecker metric has no direct equivalent: it is a model of matrix *means* with separable noise ($\Sigma_S \otimes \Sigma_T$), suited to class differences in the mean (as in evoked potentials) and blind to covariance differences (as in motor imagery); the tangent space is a model of *covariances*; the two are complementary (Section H.3). Untested possibilities: a lattice of tangent spaces at several base points, a kernel block with a log-Euclidean or affine-invariant kernel, gates and triceptrons acting on tangent vectors. Session changes are handled by re-centring [29] (re-estimating the base point on the test session without labels), leaving an orthogonal residual rotation quantified below.

A position-dependent metric $M(h)$ (the second door) is the Fisher–Rao metric of the heteroscedastic Gaussian family, $M(h) = \Sigma_{k(h)}^{-1}$ near class k: the corresponding "Riemannian oblique projector" is a field of projectors whose decision rule is, up to the log-determinant and prior terms, that of regularised QDA, already met in Appendix E; a piecewise-constant field (one metric per class) is computable in closed form, and we know of no closed form for a continuous one. The parameter manifold (third door) is the manifold of rank-r oblique projectors, of dimension $2r(d-r)$, parametrised by biorthogonal pairs up to gauge; gradient fine-tuning is a descent on it with retraction $U \leftarrow U(W^\top U)^{-1}$; the Fisher GEVD gives directly the maximiser

of the generalised Rayleigh quotient on that manifold, and in Section 12.3 the descent that followed brought no measurable gain.

H.3 Experiment: covariances versus means, and session change

Synthetic trials with $p = 8$ channels, $T = 64$ samples per trial (so that the sample covariances are noisy, Wishart-distributed estimates), $C = 3$ classes, random amplitude gain per trial ($\exp(0.3\mathcal{N}(0,1))$, a simple non-stationarity). Covariance regime (labelled "MI" by analogy with motor imagery): class-specific spatial covariance $\Sigma_{\text{base}} + 0.1 u_c u_c^\top$ (rank 2), centred signals. Mean regime ("ERP", by analogy with evoked potentials): class-specific mean pattern $0.4 a_c\,\text{wave}(t)$, common covariance. These generators are deliberately simple and are not models of real EEG. Five independent draws, unstratified 50/50 split (the numbers of trials per class in the column headings are averages), test accuracy in %. In the covariance regime the Kronecker block is *at chance* (33%): it models means, and the means are zero; the tangent block is at 97–98%. In the mean regime it is the reverse: Kronecker 85–99%, tangent 74–84% (the covariance sees the mean pattern only through a rank-one term). On these two synthetic regimes the indication is: Kronecker metric when the classes differ by their mean pattern, tangent space when they differ by their covariance, both in parallel if unsure; this remains to be confirmed on real recordings. The matrix logarithm is worth +15 points over $\text{vec}_F\,\Sigma$ at 10 per class and +6 at 30. TS-AI and TS-LE give nearly identical accuracies here, although the reference point is not close to the identity ($\|\log G\|_F$ between 1.1 and 1.6); a plausible explanation is that the two charts differ, to first order around G, by an invertible linear map of the features, to which a Fisher discriminant is insensitive up to shrinkage. They separate under congruence (Proposition 7). The MLP on tangent vectors does not improve on the analytic block overall (92.0 vs 98.0 and 98.0 vs 97.6 in the covariance regime, 74.7 vs 74.0 and 82.4 vs 83.6 in the mean regime; standard deviations of 2–16 points): on these data a linear discriminant on tangent vectors is sufficient. MDM, a Riemannian classifier without within-class metric, is far behind (67–82% in the covariance regime), which suggests that the within-class metric of the block matters more here than the Riemannian distance alone.

Table 21: Tangent-space blocks (affine-invariant TS-AI, log-Euclidean TS-LE), Euclidean block, Kronecker-metric block on raw trials, minimum distance to Riemannian mean (MDM), and an MLP on tangent vectors.

	MI, 10/cl	MI, 30/cl	ERP, 10/cl	ERP, 30/cl
TS-AI + fast block	**98.0** $\pm$ 1.6	97.6 $\pm$ 3.8	74.0 $\pm$ 16.1	83.6 $\pm$ 10.8
TS-LE + fast block	97.3 $\pm$ 1.3	97.6 $\pm$ 3.8	75.3 $\pm$ 14.8	83.8 $\pm$ 10.5
$\text{vec}_F\,\Sigma$ + fast block	82.7 $\pm$ 6.5	92.0 $\pm$ 10.0	70.0 $\pm$ 12.8	81.1 $\pm$ 13.3
Kronecker metric (raw trials)	31.3 $\pm$ 6.2	34.2 $\pm$ 3.7	**84.7** $\pm$ 10.5	**98.7** $\pm$ 1.6
MDM	67.3 $\pm$ 6.5	82.4 $\pm$ 7.9	46.7 $\pm$ 13.5	60.4 $\pm$ 19.7
TS-AI + MLP(64)	92.0 $\pm$ 6.2	**98.0** $\pm$ 4.0	74.7 $\pm$ 12.6	82.4 $\pm$ 12.8

Session change. Training on a session with gains A_1 (diagonal, $e^{\pm 0.7}$), testing on a session with $A_2 = A_1 \exp(\varepsilon M)$, M random symmetric, same classes, covariance regime at 30 per class:

	$\varepsilon = 0$	$\varepsilon = 0.2$	$\varepsilon = 0.5$
TS-AI, training base point	97.6 ± 3.8	58.4 ± 28.0	42.4 ± 16.2
TS-AI, **re-centred** per session	97.3 ± 4.8	$\mathbf{97.6} \pm 4.4$	$\mathbf{97.1} \pm 5.2$
residual $\|Q - I\|_F$	0.00	0.31	0.61
predictions = no change	100%	99.8%	98.9%
TS-LE	97.3 ± 4.3	61.1 ± 26.8	51.8 ± 25.2
$\mathrm{vec}_F\,\Sigma$	90.4 ± 12.4	69.1 ± 23.4	50.0 ± 16.1
MDM	82.4 ± 7.9	67.8 ± 18.1	52.9 ± 15.4
TS-AI + MLP(64), re-centred	97.1 ± 4.7	97.3 ± 4.8	97.1 ± 5.2

Re-centring (Karcher mean of the test session, *without labels*) brings the block from 42% back to 97%. This is not an invariance theorem: after re-centring, the proof of Proposition 7 leaves an orthogonal residual rotation $Q = G_2^{-1/2}(A_2A_1^{-1})G_1^{1/2}$ of the tangent space, equal to I if and only if $(A_2A_1^{-1})G_1^{1/2}$ is symmetric positive definite. Here $\|Q-I\|_F = 0.31$ and 0.61, and yet 99% of the predictions are those of the unperturbed session: the residual rotation is small relative to the class separation. Compensating it requires a further alignment step such as Riemannian Procrustes analysis [30], which uses some labelled trials of the new session and was not done here. At a larger size ($p = 32$, $d = 528$, $T = 100$, 30 trials per class, one draw) the tangent block reaches 100% in 0.2 s including the Karcher mean, the log-Euclidean one in 26 ms and MDM 98.9% in 0.2 s; the Euclidean $\mathrm{vec}_F\,\Sigma$ block also reaches 100% (in 2 ms) on this easier configuration, which therefore shows feasibility at this size but does not discriminate between the methods.

Associated with Riemannian geometry, the family of algorithms does not become deeper; it becomes *invariant* under congruence, and on covariance data it coincides with the established tangent-space pipeline without back-propagation: the non-linearity is provided by the covariance representation and the logarithmic map at a fitted reference point, not by a network. Tangent-space shrinkage LDA [28] and session re-centring [29] are established methods; what is added here is the oblique reading (biorthogonal pair in the tangent space, $C \times C$ eigenproblem, rank by permutation), the proof that the invariance is exact with shrinkage included, and the comparison with the Kronecker block on synthetic data.

I Fourier operators as banks of per-mode analytic problems

A Fourier neural operator (FNO [31]) composes layers $v \mapsto \sigma(\mathcal{W}v + \mathcal{F}^{-1}(R \cdot \mathcal{F}v))$ with a lifting $\mathcal{P}$ on input and a projection $\mathcal{Q}$ on output; the spectral convolution learns, for each of the K retained modes, a complex matrix $R(k) \in \mathbb{C}^{C\times C}$ that mixes channels *at that mode*; $\mathcal{W}$ is a position-wise linear map (skip) and σ a pointwise non-linearity. The practitioners' guide of Duruisseaux, Kossaifi and Anandkumar [32] identifies the number of retained modes K as a hyper-parameter that is frequently misconfigured (too small, the operator is under-parametrised; too large, it fits absent or aliased high frequencies). In this section C denotes the number of channels of the field and C_{cl} the number of classes.

I.1 Structure: a spectral block is a bank of K independent $C \times C$ problems

Write the spectral convolution $\mathcal{K} = \mathcal{F}^{-1}\,\mathrm{diag}_k(R(k))\mathcal{F}$ on a C-channel field. Three elementary facts fix the framework. (i) **Mode-wise idempotence.** $\mathcal{K}^2 = \mathcal{K}$ if and only if $R(k)^2 = R(k)$ for every k. The oblique projectors of this form are exactly the families $\{P(k) = U_k(W_k^H U_k)^{-1}W_k^H\}_k$ of ranks $r_k \le C$, total rank $\sum_k r_k$. With a single channel, $R(k)$ is a scalar and idempotence forces $R(k) \in \{0, 1\}$: a circulant block (one complex gain per frequency) is in general a *filter*; it is a projector only as a 0/1 spectral mask, which is an orthogonal projector (circulant matrices are normal), and it can never be oblique. Oblique geometry enters a Fourier operator only *inside each mode*, between channels. (ii) **Closed form**

per mode. The K matrices $R(k)$ are decoupled: once the data are in Fourier space, each mode is an ordinary $C \times C$ linear problem (the Fisher GEVD on the mode's coefficients for a classifier, with the $C_{\mathrm{cl}} \times C_{\mathrm{cl}}$ reduction of Proposition 4 and batched permutations; complex least squares for an operator). The K problems are independent: parallelism with no communication. (iii) **The number of modes can become a rank.** A permutation test per mode, or per band of modes, can assign $r_k = 0$ to modes carrying no information; K is then the number of retained modes rather than a hyper-parameter. This is demonstrated on frequency bands in experiment E3 below; in E1 and E2 the retained modes are simply those with non-zero energy in the simulated data.

Operator version (regression). To predict $u_{t+\Delta t}$ from u_t on a periodic grid, the learned lifting $\mathcal{P}$ is replaced by a *fixed* dictionary $z_k(u) = (\hat{u}_k, \widehat{u^2}_k, \ldots, \widehat{u^m}_k) \in \mathbb{C}^m$, powers computed on a grid twice as fine and truncated back (exact de-aliasing). Per mode, $\hat{y}_k \approx w_k^H z_k$ by ridge least squares; the map $z_k \mapsto e_1 w_k^H z_k$ has rank one, representation direction e_1 (the state) and measurement direction w_k (the powers): a mode-wise oblique Petrov–Galerkin reduction. This is extended dynamic mode decomposition [33] written in a Fourier basis with a polynomial dictionary; nothing is new in the method itself; what is added is its reading as a analytic oblique block. Roll-outs re-apply the dictionary at each step: as in Appendix H, the non-linearity is supplied by a fixed transformation (powers here, the matrix logarithm there), not by a network. Theorem 1 does not apply: the propagator is not idempotent, and the recursion is the very object (the time step). For a linear PDE the gain per mode is $e^{\lambda_k \Delta t}$ and T steps give $e^{\lambda_k T \Delta t}$: powers of a diagonal operator, exact and of no architectural interest; with a non-linear re-lift at each step no such closed form is available.

I.2 Experiments

The FNO used for comparison is a `numpy` implementation with hand-written gradients (real DFT by explicit $K \times N$ matrices, gradients verified by finite differences to 4.9×10^{-7}): lifting $2 \to C$, L layers [spectral convolution with K modes and complex $C \times C$ weights + $C \times C$ skip + bias, GELU], projection $C \to 1$, residual output; Adam, 2000 steps, cosine schedule, mini-batches of 200. It is *small* ($C = 16$, $L = 2$, about a minute of CPU) and trained briefly relative to common practice, with no hyper-parameter search: the figures compare a closed form with a modest FNO baseline, not with a well-tuned FNO, and no conclusion about FNOs in general should be drawn from them.

E1: linear PDE (advection–diffusion, $c = 1$, $\nu = 0.02$, $\Delta t = 0.1$, $N = 64$; ten trajectories of 20 steps for training). The analytic spectral block ($m = 1$) retains the six modes that carry energy (indices 1–6: the initial conditions are band-limited, the modes with zero energy are removed by an energy guard, and the permutation test rejects none of the remaining ones), i.e. 12 real parameters, in a few tens of milliseconds, and recovers the exact propagator $e^{(-\nu k^2 - ick)\Delta t}$ to 10^{-6}: relative one-step error 1.0×10^{-6}, 20-step roll-out 2.0×10^{-5}. The FNO (12 897 parameters, about one minute): 4.9×10^{-3} at one step, 8.1×10^{-2} after 20 steps. The MLP (33 088 parameters): 0.25 and 0.61. On a linear PDE with periodic boundary conditions the model class of the closed form contains the exact operator, which it identifies from these noise-free data; the FNO approximates it.

E2: Burgers ($\nu = 0.05$, $\Delta t = 0.1$, $N = 64$). Relative L^2 error at one step, then mean and final value over a 20-step roll-out (20 test trajectories, smooth Gaussian initial conditions). In this experiment the analytic cubic dictionary (126 parameters, a fraction of a second) is more accurate than the small FNO at $K = 12$ (12 897 parameters, about one minute) at one step and in roll-out, at 200 and at 1000 pairs, with a hundred times fewer parameters. The quadratic

Table 22: Burgers: analytic spectral blocks with dictionaries of degree m (on the 21 modes that carry energy after the 2/3 de-aliasing of the solver; the permutation test rejects none of them), numpy FNO, MLP. Relative L^2 errors; one run; times on a sandbox CPU.

	params	time	1 step	roll-out mean	roll-out final
200 training pairs (10 trajectories)					
analytic spectral $m = 1$ (21 modes)	42	0.2 s	5.9×10^{-2}	0.48	0.69
analytic spectral $m = 2$ (21 modes)	84	0.2 s	7.2×10^{-3}	48.5	940
analytic spectral $m = 3$ (21 modes)	126	0.2 s	$\mathbf{2.8 \times 10^{-3}}$	$\mathbf{1.3 \times 10^{-2}}$	$\mathbf{1.5 \times 10^{-2}}$
FNO $C = 16$, $L = 2$, $K = 4$	4 705	41 s	3.1×10^{-2}	0.15	0.17
FNO $C = 16$, $L = 2$, $K = 12$	12 897	48 s	5.1×10^{-3}	3.8×10^{-2}	5.9×10^{-2}
FNO $C = 16$, $L = 2$, $K = 24$	25 185	56 s	4.9×10^{-3}	3.9×10^{-2}	6.8×10^{-2}
MLP (256)	33 088	0.1 s	0.42	0.66	0.85
1000 training pairs (50 trajectories)					
analytic spectral $m = 3$ (21 modes)	126	0.2 s	$\mathbf{2.5 \times 10^{-3}}$	$\mathbf{1.1 \times 10^{-2}}$	$\mathbf{1.3 \times 10^{-2}}$
analytic spectral $m = 2$ (21 modes)	84	0.2 s	7.0×10^{-3}	0.10	0.44
FNO $C = 16$, $L = 2$, $K = 12$	12 897	48 s	3.8×10^{-3}	2.3×10^{-2}	3.2×10^{-2}
FNO $C = 16$, $L = 2$, $K = 24$	25 185	54 s	3.4×10^{-3}	2.2×10^{-2}	3.5×10^{-2}
MLP (256)	33 088	0.3 s	0.20	0.45	0.57

dictionary, although the quadratic term is the non-linearity of Burgers' equation in semi-discrete form, diverges in roll-out (940 at 200 pairs): the finite-Δt map contains higher-order terms, and a slightly wrong per-mode fit is amplified over 20 steps. The closed form does *not* guarantee roll-out stability; the choice of dictionary is its hyper-parameter, and a poor choice fails badly. The FNO's number of modes behaves as expected: $K = 4$ is clearly insufficient, $K = 24$ brings nothing over $K = 12$ and slightly degrades the roll-out; the closed form used the 21 modes that carry energy, with no tuning. The regime is favourable to the dictionary (1-D, viscous, smooth initial conditions, noise-free data, small FNO); nothing here indicates that a polynomial closed form would keep up with a well-trained FNO on 2-D Navier–Stokes or on shocks, the setting for which a *learned* lifting is designed.

E3: classification, automatic band selection. Trials with $p = 8$ channels, $T = 64$ samples, $C = 3$ classes: spatially correlated noise whose spectral gain varies per trial and per band (nuisance, $e^{0.7\mathcal{N}(0,1)}$), plus a class component (rank-2 spatial pattern, separation 0.5) carried *only* by temporal modes 8–11. Eight bands of four modes; per band, cross-spectral matrix $\to$ log-Euclidean $\to$ fast block; concatenated codes, ridge read-out (used for every variant, including the broadband one, because the centroid rule has the meaning of LDA only on the code of a single block); five independent draws, unstratified 50/50 split (the numbers of trials per class are averages), test accuracy in %.

	10 trials/class	30 trials/class
broadband (one covariance, rank $C - 1$)	84.7 ± 4.5	87.6 ± 1.6
8 bands, rank $C - 1$ each	66.7 ± 7.6	92.0 ± 3.1
8 bands, rank by total-scatter permutation test, per-band level 5%	94.7 ± 4.5	96.9 ± 2.4
8 bands, same test with family-wise min-p control at 5%	$\mathbf{98.0} \pm 1.6$	$\mathbf{97.6} \pm 2.4$
MLP(64) on the 8 bands	58.0 ± 12.8	69.8 ± 5.2

Ranks retained per band for the five seeds at 10 per class, with the per-band level: $[0, 0, 2, 0, 0, 1, 0, 0]$, $[0, 0, 2, 0, 0, 0, 0, 1]$, $[0, 0, 2, 0, 0, 0, 0, 0]$, $[0, 0, 2, 0, 0, 0, 0, 1]$, $[0, 0, 2, 0, 0, 0, 0, 0]$: band 2 (modes 8–11) is found every time with its true rank 2; three of the 35 null bands receive

a rank of one (and three of 35 again at 30 per class), a rate compatible with the nominal per-band level given so small a count. With the family-wise control of Section 8.5 the ranks are $[0, 0, 2, 0, 0, 0, 0, 0]$ in the ten draws: the informative band is always retained and no null band is. The accuracy is then 98.0 and 97.6% instead of 94.7 and 96.9% (five draws, no interval computed): here the two controls differ in what they claim, a rate per band or a statement about the whole family, more than in accuracy. Keeping all bands at rank $C - 1$ is *worse* than broadband at 10 per class (66.7 vs 84.7): the codes of the noise bands swamp the read-out; rank selection brings the accuracy to 94.7. In the vocabulary of brain–computer interfaces this is close to a filter-bank CSP [34] whose band selection is made by the rank test instead of a mutual-information criterion; it is the classification analogue of choosing the number of modes K. It is this experiment that motivated the calibration study of Section 8: with the shared-S_W test, whose false-positive rate exceeds 30% at $n = 30$, $d = 36$ (Table 5), a large fraction of null bands would be retained.

I.3 Interest and contribution

A Fourier operator is a bank of K decoupled small problems, a favourable case for the closed form, which solves each of them directly and in parallel. Oblique geometry is confined inside the modes, between channels: a single-channel circulant block cannot be oblique, and obliquity is available between channels at a fixed basis. What was measured, on the favourable cases described above: recovery of the exact propagator of a linear periodic PDE from noise-free data; on 1-D viscous Burgers, an accuracy above that of a small, briefly trained FNO, with a hundred times fewer parameters; in classification, a band selection by rank test that gains 10 points over broadband at 10 trials per class; and, along the way, the calibration correction of the permutation test. What the closed form lacks: a learned lifting (it requires a chosen dictionary, and a poor choice diverges), mode couplings beyond the dictionary, and any roll-out stability guarantee. Depth here is neither a repetition of one projector nor learned: it is physical time.

J Python code

The numerical results of this report are produced by the scripts reproduced below (`numpy`, `scipy`, `scikit-learn`; no deep-learning framework; all gradients hand-written and verified by finite differences); the logs of the run from which the tables were filled accompany the report. Accuracies are reproducible given the seeds, up to the numerical differences between library versions; timings are not. The scripts are ASCII-only and self-contained within the set; they import one another by the file names given here, and `ropn_auto_charlm.py` reads the frozen corpus `ropn_corpus.txt`. The benchmark script attempts an optional MNIST download, which is skipped without network access, as was the case for this report. Inline comments are in French.

Contents of the code appendix

1. `ropn_auto_nobp.py` — the analytic block (Fisher GEVP, convention (8), re-whitening, permutation rank, analytic gate), the synthetic Gaussian generator, and the recursion primitives used to verify Theorem 1 and Proposition 1.

2. `ropn_auto_nobp_diag.py` — small-sample diagnosis: ablation of the baseline block (shrinkage, code read-out, structural rank, T), principal angles to the true subspaces at the tested rank and at rank $C - 1$, Bayes oracle, extreme regime $n < d$ (Sections 12 and 14); the least-squares and centroid read-outs (Proposition 3).

3. `ropn_auto_finetune.py` — gradient fine-tuning from the corrected block: parametrisation with the image frozen and trainable-group masks (read-out only, gate and read-out,

M and read-out, M, gate and read-out); model with both subspaces free, with exact idempotence through $(W^\top U)^{-1}$ or without constraint, in a $2 \times 2 \times 2$ factorial design; manual gradients with gradient checks, heteroscedastic generator and QDA oracle, paired differences (Section 12.3).

4. `ropn_auto_nobp_sweep.py` — anisotropy sweep $\kappa \times$seeds, baseline vs corrected vs orthogonal vs MLP, with paired differences (Tables 13 and 14).

5. `ropn_auto_benchmark.py` — benchmark on synthetic and real datasets with accuracy, parameters, convergence and time; CSV export (Appendix B).

6. `ropn_auto_fast.py` — the fast block: exact $C \times C$ factorisation (Proposition 4), Ledoit–Wolf coefficient from Gram identities, primal/dual solver, batched permutation test with label-free total-scatter metric and exact p-value, sequential rule with deflation, Bartlett option, structural, tested and used ranks, min-p family-wise selection, online rank-one update (Sections 6 and 8).

7. `ropn_auto_fast_test.py` — exactness and timing of the factorisation against the full GEVD reference (Table 3) and against scikit-learn's LDA solvers (Table 4), and test of the online update.

8. `ropn_auto_lattice.py` — the back-propagation-free lattice (transversal stage, fixed quadratic non-linearities, dual second stage, configuration chosen by cross-validation) and its comparison (Appendix C).

9. `ropn_auto_bioblique.py` — doubly oblique geometry for matrix inputs: Kronecker projector (not retained) and Kronecker metric with alternating estimation (Table 17).

10. `ropn_auto_kernel.py` — the kernel oblique block (dual form with a kernel Gram matrix) and the comparison with LDA, regularised QDA and SVM (fixed and cross-validated), boosting, k-NN and MLP (Appendix E).

11. `ropn_auto_charlm.py` — next-character prediction on the frozen corpus: n-gram, analytic blocks, logistic regression, lattice, MLP (Appendix F).

12. `ropn_auto_crossgram.py` — the shrinkage path from the Fisher block to the Euclidean cross-Gram, with fixed and cross-validated λ, and PLS-DA (Appendix G).

13. `ropn_auto_triceptron.py` — gated and ReLU layers with or without a linear skip, trained by the same hand-written Adam (manual gradients, gradient checks), at nearly equal parameter counts; constructed weights of Theorem 4 against learned weights; analytic-feature triceptron TRI-CF (Section 4.2).

14. `ropn_auto_depth_theory.py` — numerical verification of Theorems 3 and 4 (invariances, closed forms, quadric constructions, the disc experiment with ReLU networks and the polygon bound), of Proposition 1, of the determinants of Theorem 2 and Proposition 2, and of the monotonicity condition of Corollary 2.

15. `ropn_auto_examples.py` — the worked examples of Sections 3 and 4 with explicit numbers.

16. `ropn_auto_riemann.py` — SPD utilities (matrix functions, Karcher mean, tangent space, MDM), congruence-invariance check, covariance vs mean regimes, session change (Appendix H).

17. `ropn_auto_fourier.py` — spectral PDE solvers, analytic spectral blocks with polynomial dictionaries and mode selection, numpy FNO with manual gradients, band-selection classification with per-band or family-wise control, permutation-test calibration (Appendix I).

18. `ropn_auto_risques.py` — the risk audit of Section 13 (norms, robustness, tested ranks, global Jacobians of the linear, un-normalised gated and residual-normalised gated recursions, control variants, effect of shrinkage).

19. `ropn_auto_exemple_complet.py` — the worked example of Section 7 with every matrix printed, and the generation of the figures of this report (geometry, worked example, recursion, norm versus sample size, triceptron constructions, interpolation threshold).

20. `ropn_auto_stabilite.py` — numerical verification of the stability limits and their control (Section 9): norm–angle identity, metric bound (with its rank-one equality case) and shrinkage control, best approximation in the Fisher metric, kernel invisibility, residual inverse, cross-Gram versus metric regularisation, depth bounds and Jacobian conditioning, singular spectrum $1/\cos\theta_i$ and two-step reading, interpolation threshold.

21. `ropn_auto_stats.py` — paired differences, Student intervals and verdicts relative to the equivalence margin, used by the other scripts.

22. `ropn_auto_rang.py` — the rank study of Section 8: calibration under the global null hypothesis, rank recovery and predictive risk under alternatives of every rank, per-block against family-wise control.

23. `ropn_auto_central.py` — the central benchmark of Section 11: default and selected configurations on common folds, paired differences, timings by phase (preprocessing, selection, final fit, inference), exact equivalence with LDA, tested rank on digits.

Listing 1: ropn_auto_nobp.py — analytic block, collapse, gate, recursion

```
"""
ROPN-auto sans back-propagation : bloc unique a rang selectionne, applique par recursion.

Ce script :
  1) construit un projecteur oblique idempotent P = U (W^T U)^{-1} W^T en forme fermee
     (GEVP de Fisher multi-classe S_B v = lambda S_W v, sans descente de gradient) ;
  2) verifie numeriquement le theoreme de collapse : l'application recursive de L copies
     du MEME bloc lineaire idempotent est EXACTEMENT egale a une seule application du bloc
     avec un coefficient scalaire c_L = (1+1/L)^L - 1 ;
  3) etend le bloc avec une porte oblique non-lineaire (elle aussi fixee en forme fermee) ;
     NB : le code W^T h_T est invariant par la recursion, et avec s dans span(W) la
     porte collapse aussi : h_T = P x + (1-a(x))^T (I-P) x  (proposition d'invariance du code) ;
  4) selectionne le rang automatiquement (borne structurelle C-1 + test de permutation) ;
  5) compare accuracy / #parametres a un MLP entraine par retropropagation (sklearn).
Ce script produit la demonstration de base (collapse, bloc de reference, comparaison a un MLP).
"""
import numpy as np
from numpy.linalg import qr, eigh, norm, pinv
from scipy.linalg import eigh as geigh
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.covariance import LedoitWolf

rng = np.random.default_rng(0)

# ---------------------------------------------------------------------------
# 0. Donnees synthetiques anisotropes multi-classes (kappa(S_W) controle)
# ---------------------------------------------------------------------------
def make_multiclass(n_classes=4, d=20, n_per=60, kappa=200.0, sep=2.0, seed=0):
    rng = np.random.default_rng(seed)
    eig = np.logspace(0, np.log10(kappa), d); eig = eig / eig.mean()
    Q, _ = qr(rng.standard_normal((d, d)))
```

```
    Sigma = Q @ np.diag(eig) @ Q.T
    L = np.linalg.cholesky(Sigma + 1e-9 * np.eye(d))
    mus = sep * rng.standard_normal((n_classes, d))
    mus -= mus.mean(0, keepdims=True)  # centrer les moyennes de classe
    X = np.zeros((n_classes * n_per, d))
    y = np.zeros(n_classes * n_per, dtype=int)
    for c in range(n_classes):
        Z = rng.standard_normal((n_per, d)) @ L.T
        X[c*n_per:(c+1)*n_per] = Z + mus[c]
        y[c*n_per:(c+1)*n_per] = c
    return X, y, Sigma, mus


# ---------------------------------------------------------------------------
# 1. Construction fermee du bloc oblique idempotent (Fisher multi-classe, M-orthogonal)
# ---------------------------------------------------------------------------
def fit_oblique_block_closed_form(X, y, r=None, shrink="auto", perm_test=True, n_perm=200, seed=0):
    """Construit P = U (W^T U)^{-1} W^T, idempotent, W^T U = I_r, EN FORME FERMEE.

    Methode : GEVP de Fisher S_B v = lambda S_W v (LDA multi-classe generalisee a l'oblique).
    W = les r premiers vecteurs propres generalises (directions de MESURE, w_i = S_W^{-1} u_i),
    U = S_W W (directions de REPRESENTATION, dans span des moyennes de classe), puis
    reblanchiment pour imposer W^T U = I_r exactement (idempotence, kappa(G)=1 par construction).
    Les vecteurs propres generalises de scipy etant deja S_W-orthonormes, le reblanchiment ne
    corrige que le petit terme de regularisation et les arrondis.
    Aucune retropropagation, aucun optimiseur : uniquement algebre lineaire (eigh, solve, QR).
    """
    rng = np.random.default_rng(seed)
    classes = np.unique(y)
    C, n, d = len(classes), *X.shape
    mu = X.mean(0)
    mus = np.stack([X[y == c].mean(0) for c in classes])

    # Scatter intra-classe (poolee) et inter-classe
    Xc = X - mus[np.searchsorted(classes, y)]
    S_W = (Xc.T @ Xc) / max(n - C, 1)
    ns = np.array([np.sum(y == c) for c in classes])
    S_B = ((mus - mu).T * ns) @ (mus - mu) / max(C - 1, 1)

    # Shrinkage Ledoit-Wolf si regime pauvre en donnees (n <= d), sinon S_W brut
    # (variante de reference ; le bloc corrige applique le retrecissement systematiquement)
    used_shrink = False
    if shrink == "auto":
        shrink = (n <= d)
    if shrink:
        S_W = LedoitWolf().fit(Xc).covariance_
        used_shrink = True

    # GEVP S_B v = lambda S_W v  (borne structurelle : rang(S_B) <= C-1)
    evals, evecs = geigh(S_B, S_W + 1e-10 * np.trace(S_W) / d * np.eye(d))
    order = np.argsort(evals)[::-1]
    evals, evecs = evals[order], evecs[:, order]

    # ---- selection automatique du rang (sans backprop) --------------------
    r_struct = min(C - 1, d)
    if r is not None:
        r_star = r
    elif perm_test:
        null_max = np.empty(n_perm)
        for i in range(n_perm):
            yp = rng.permutation(y)
            musp = np.stack([X[yp == c].mean(0) for c in classes])
            S_Bp = ((musp - mu).T * ns) @ (musp - mu) / max(C - 1, 1)
            ev = geigh(S_Bp, S_W + 1e-10 * np.trace(S_W) / d * np.eye(d), eigvals_only=True)
            null_max[i] = ev.max()
        thresh = np.quantile(null_max, 0.95)
        r_star = int(min(r_struct, np.sum(evals[:r_struct] > thresh)))
        r_star = max(r_star, 1)
    else:
        r_star = r_struct

    # Convention de Fisher (rang 1 : u = mu_1 - mu_0, w* = S_W^{-1} u), generalisee :
    #   les vecteurs propres generalises v_i sont les directions de MESURE de Fisher
```

```
    #   (v_i = S_W^{-1} u_i), et u_i = S_W v_i = S_B v_i / lambda_i appartient a
    #   span(mu_c - mu) : ce sont les directions de REPRESENTATION (moyennes de classe).
    #   Le code c = G^{-1} W^T h est alors la statistique suffisante de la LDA.
    V_r = evecs[:, :r_star]
    W_raw = V_r                # mesure : w_i = v_i = S_W^{-1} u_i
    U_raw = S_W @ V_r          # representation : u_i = S_W v_i  (dans span des moyennes)

    # Reblanchiment pour imposer EXACTEMENT W^T U = I_r (forme idempotente)
    G = W_raw.T @ U_raw                       # (r,r) SPD
    Gvals, Gvecs = eigh(G)
    Gvals = np.clip(Gvals, 1e-12, None)
    G_mhalf = Gvecs @ np.diag(Gvals ** -0.5) @ Gvecs.T
    U = U_raw @ G_mhalf
    W = W_raw @ G_mhalf

    diagnostics = dict(r_star=r_star, r_struct=r_struct, eigenvalues=evals,
                       shrink=used_shrink, WtU_err=norm(W.T @ U - np.eye(r_star)))
    return U, W, diagnostics


def apply_P(U, W, h):
    return (h @ W) @ U.T


# ---------------------------------------------------------------------------
# 2. Theoreme de collapse : (I + P/L)^L = I + [(1+1/L)^L - 1] P  (P idempotent)
# ---------------------------------------------------------------------------
def recurse_linear(U, W, h, L):
    for _ in range(L):
        h = h + apply_P(U, W, h) / L
    return h


def collapse_closed_form(U, W, h, L):
    c_L = (1.0 + 1.0 / L) ** L - 1.0
    return h + c_L * apply_P(U, W, h)


# ---------------------------------------------------------------------------
# 3. Porte oblique non-lineaire fixee en forme fermee (pas de backprop)
# ---------------------------------------------------------------------------
def make_closed_form_gate(U, W, X):
    """s = direction de Fisher dominante (colonne de W), b centre les deux classes
    de score sur cette direction. Purement algebrique, aucune optimisation."""
    s = W[:, 0] / norm(W[:, 0])
    proj = X @ s
    b = -proj.mean()
    return s, b


def sigmoid(z):
    z = np.clip(z, -60.0, 60.0)  # evite l'overflow de exp pour |z| grand
    return 1.0 / (1.0 + np.exp(-z))


def recurse_gated(U, W, s, b, h, T):
    """Bloc PARTAGE applique T fois (poids identiques a chaque pas, comme un
    reseau recurrent / DEQ) : h <- (1-a(h))h + a(h) P h,  a(h)=sigma(s^T h + b)."""
    for _ in range(T):
        a = sigmoid(h @ s + b)[:, None]
        h = (1 - a) * h + a * apply_P(U, W, h)
    return h


if __name__ == "__main__":
    # ---- Donnees ----
    X, y, Sigma, mus = make_multiclass(n_classes=4, d=20, n_per=60, kappa=200.0, sep=2.0, seed=0)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

    # ---- Bloc ferme ----
    U, W, diag = fit_oblique_block_closed_form(Xtr, ytr, seed=0)
    print("=== Bloc auto-ROPN ferme (sans backprop) ===")
    print(f"rang structurel C-1 = {diag['r_struct']}, rang auto-selectionne r* = {diag['r_star']}")
```

```
    print(f"valeurs propres GEVP (top 6) = {np.round(diag['eigenvalues'][:6], 3)}")
    print(f"||W^T U - I_r|| = {diag['WtU_err']:.2e}  (idempotence exacte attendue)")
    P = U @ W.T
    print(f"||P^2 - P||_F = {norm(P @ P - P):.2e}")

    # ---- Theoreme de collapse (verification numerique) ----
    print("\n=== Verification du theoreme de collapse ===")
    h0 = rng.standard_normal((5, X.shape[1]))
    for L in [1, 2, 4, 8, 16, 64, 256]:
        h_iter = recurse_linear(U, W, h0.copy(), L)
        h_closed = collapse_closed_form(U, W, h0.copy(), L)
        err = norm(h_iter - h_closed) / norm(h_closed)
        c_L = (1 + 1 / L) ** L - 1
        print(f"L={L:4d}  c_L={c_L:.6f}  erreur relative iteratif-vs-ferme = {err:.2e}")

    # ---- Recursion non-lineaire : verifier que ca NE collapse PAS trivialement ----
    print("\n=== Porte non-lineaire : evolution de h_T avec T (forme close : (1-a)^T) ===")
    s, b = make_closed_form_gate(U, W, Xtr)
    for T in [1, 2, 4, 8, 16]:
        hT = recurse_gated(U, W, s, b, h0.copy(), T)
        print(f"T={T:2d}  ||h_T - h_0|| = {norm(hT - h0):.4f}   ||h_T|| = {norm(hT):.4f}")

    # ---- Comparaison accuracy / parametres ----
    print("\n=== Comparaison accuracy / parametres ===")

    def closed_form_predict(U, W, s, b, Xtr, ytr, Xte, T):
        Htr = recurse_gated(U, W, s, b, Xtr, T)
        Hte = recurse_gated(U, W, s, b, Xte, T)
        # tete de lecture fermee (moindres carres / ridge, PAS de backprop)
        classes = np.unique(ytr)
        Ytr = np.eye(len(classes))[ytr]
        Wro, *_ = np.linalg.lstsq(np.c_[Htr, np.ones(len(Htr))],
                                  Ytr, rcond=None)
        pred = np.c_[Hte, np.ones(len(Hte))] @ Wro
        return classes[pred.argmax(1)], Wro

    results = {}
    for T in [1, 4, 8]:
        pred, Wro = closed_form_predict(U, W, s, b, Xtr, ytr, Xte, T)
        acc = (pred == yte).mean()
        n_params_block = U.size  # W, s, b derives de U (pas des parametres libres supplementaires)
        n_params_head = Wro.size
        results[f"auto-ROPN-1bloc (T={T})"] = (acc, n_params_block + n_params_head, 0)

    mlp = MLPClassifier(hidden_layer_sizes=(32,), max_iter=2000, random_state=0)
    mlp.fit(Xtr, ytr)
    acc_mlp = mlp.score(Xte, yte)
    n_params_mlp = sum(w.size for w in mlp.coefs_) + sum(b.size for b in mlp.intercepts_)
    results["MLP (backprop, 1 couche cachee=32)"] = (acc_mlp, n_params_mlp, n_params_mlp)

    print(f"{'modele':38s} {'acc':>8s} {'#param (tot)':>13s} {'#param (backprop)':>18s}")
    for k, (acc, npar, nbp) in results.items():
        print(f"{k:38s} {acc*100:7.1f}% {npar:13d} {nbp:18d}")
```

Listing 2: ropn_auto_nobp_diag.py — small-sample ablation and subspace angles

```
"""
Diagnostic du regime petits echantillons : ou le bloc ferme perd-il de l'information ?

Decomposition (par graine, moyenne sur 20 graines) :
  (0) Bayes oracle  : regle LDA avec la VRAIE covariance et les VRAIES moyennes (borne sup.)
  (1) LDA-shrinkage : reference classique (sklearn, shrinkage='auto')
  (2) bloc ferme, variantes d'ablation :
        base        : S_W brut (n>d), rang auto (permutation), lecture LS sur h_T (dim d), T=4
        +shrink     : Ledoit-Wolf systematique sur S_W
        +code       : lecture sur le code oblique c = W^T h_T (dim r) au lieu de h_T (dim d)
        +rankC1     : rang structurel r = C-1 (pas de test de permutation)
        +T1         : une seule application (pas de recursion)
        +code+rankC1 (sans retrecissement) : isole l'effet propre du retrecissement
      les lignes "+code" gardent la lecture par moindres carres du pipeline de base (un facteur a la fois
     ) ;
```

```
        corrige, lecture centroides : retrecissement + code + C-1 avec la lecture par centroides (regle LDA),
        c'est le bloc corrige utilise dans le reste du rapport
  Mesures geometriques : angle principal max entre span(U_hat) et span(mu_c - mu)
  (representation vraie), et entre span(W_hat) et span(Sigma^{-1}(mu_c - mu)) (mesure vraie),
  au rang du test de permutation (rang variable) ET au rang C-1 (comparaison a rang egal).
  Regime extreme (4 exemples par classe, n_train < d) en fin de script.
"""
import numpy as np
from numpy.linalg import norm, pinv, eigh, svd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.covariance import LedoitWolf
from scipy.linalg import eigh as geigh, subspace_angles
from ropn_auto_nobp import make_multiclass, make_closed_form_gate, recurse_gated, apply_P


def fit_block(X, y, shrink=False, rank="perm", n_perm=200, seed=0):
    rng = np.random.default_rng(seed)
    classes = np.unique(y); C = len(classes); n, d = X.shape
    mu = X.mean(0)
    mus = np.stack([X[y == c].mean(0) for c in classes])
    Xc = X - mus[np.searchsorted(classes, y)]
    ns = np.array([np.sum(y == c) for c in classes])
    S_B = ((mus - mu).T * ns) @ (mus - mu) / max(C - 1, 1)
    S_W = LedoitWolf().fit(Xc).covariance_ if shrink else (Xc.T @ Xc) / max(n - C, 1)
    reg = 1e-10 * np.trace(S_W) / d * np.eye(d)
    evals, evecs = geigh(S_B, S_W + reg)
    order = np.argsort(evals)[::-1]; evals, evecs = evals[order], evecs[:, order]
    r_struct = min(C - 1, d)
    if rank == "perm":
        null_max = np.empty(n_perm)
        for i in range(n_perm):
            yp = rng.permutation(y)
            musp = np.stack([X[yp == c].mean(0) for c in classes])
            S_Bp = ((musp - mu).T * ns) @ (musp - mu) / max(C - 1, 1)
            null_max[i] = geigh(S_Bp, S_W + reg, eigvals_only=True).max()
        r_star = max(1, int(min(r_struct, np.sum(evals[:r_struct] > np.quantile(null_max, 0.95)))))
    else:
        r_star = r_struct
    V_r = evecs[:, :r_star]; W_raw = V_r; U_raw = S_W @ V_r   # convention : w = S_W^{-1} u
    G = W_raw.T @ U_raw; gv, gV = eigh(G); gv = np.clip(gv, 1e-12, None)
    Gmh = gV @ np.diag(gv ** -0.5) @ gV.T
    return U_raw @ Gmh, W_raw @ Gmh, r_star


def readout_fit_predict(Ftr, ytr, Fte, ridge=0.0):
    classes = np.unique(ytr); Y = np.eye(len(classes))[ytr]
    A = np.c_[Ftr, np.ones(len(Ftr))]
    if ridge > 0:
        R = ridge * np.eye(A.shape[1]); R[-1, -1] = 0
        Wro = np.linalg.solve(A.T @ A + R, A.T @ Y)
    else:
        Wro, *_ = np.linalg.lstsq(A, Y, rcond=None)
    return classes[(np.c_[Fte, np.ones(len(Fte))] @ Wro).argmax(1)]


def centroid_readout_weights(Ctr, ytr, priors=True):
    """Lecture par centroides ecrite comme une application AFFINE du code : scores = [c, 1] @ Wro, avec
    Wro = [M^T ; -||m_k||^2 / 2 + log pi_k] de forme (r+1, C) (M : centroides de classe dans l'espace du code).
    Meme nombre de coefficients que la lecture par moindres carres, mais aucun ajustement.
    Retourne (classes, Wro)."""
    classes, yi = np.unique(ytr, return_inverse=True)
    M = np.stack([Ctr[yi == k].mean(0) for k in range(len(classes))])
    bias = -0.5 * np.sum(M ** 2, 1)
    if priors: bias = bias + np.log(np.bincount(yi) / len(yi))
    return classes, np.vstack([M.T, bias[None, :]])


def readout_centroid_fit_predict(Ctr, ytr, Cte, priors=True):
    """Lecture analytique par centroides dans l'espace du code : score_k = -||c - m_k||^2 / 2 + log pi_k.
```

```
    Pour le bloc (W = S^{-1} D Q Lambda^{-1/2}) on a W^T S W = I_r : le code est blanchi par rapport a la
    covariance intra-classe S, et cette regle est EXACTEMENT la regle de la LDA de covariance S (rang C
     -1).
    Aucun ajustement : r C nombres (les centroides) et C a priori. C'est la lecture de reference du bloc.
     """
    classes, Wro = centroid_readout_weights(Ctr, ytr, priors)
    return classes[(np.c_[Cte, np.ones(len(Cte))] @ Wro).argmax(1)]


def block_predict(U, W, Xtr, ytr, Xte, T=4, feat="h", ridge=0.0, readout="ls"):
    """readout="ls" : moindres carres (lecture du pipeline de base, objet du diagnostic) ;
    readout="centroid" : lecture par centroides sur le code (regle LDA, lecture de reference)."""
    s, b = make_closed_form_gate(U, W, Xtr)
    Htr = recurse_gated(U, W, s, b, Xtr, T); Hte = recurse_gated(U, W, s, b, Xte, T)
    if feat == "code":
        Htr, Hte = Htr @ W, Hte @ W          # code oblique c = W^T h (dim r)
    if readout == "centroid":
        assert feat == "code", "la lecture par centroides n'a le sens de la regle LDA que sur le code"
        return readout_centroid_fit_predict(Htr, ytr, Hte)
    return readout_fit_predict(Htr, ytr, Hte, ridge)


def oracle_bayes(Sigma, mus, Xte):
    Sinv = pinv(Sigma)
    scores = np.stack([Xte @ Sinv @ m - 0.5 * m @ Sinv @ m for m in mus], 1)
    return scores.argmax(1)


def true_subspaces(Sigma, mus):
    mu = mus.mean(0)
    D = (mus - mu).T                         # (d, C) directions de moyennes (rang C-1)
    Uq, s, _ = svd(D, full_matrices=False); U_true = Uq[:, :mus.shape[0] - 1]    # span des moyennes
    W_true = pinv(Sigma) @ U_true            # directions de mesure de Fisher vraies : Sigma^{-1} u
    return U_true, W_true


def run(kappa, n_per, seeds=20, d=20, C=4, sep=0.8):
    keys = ["oracle", "LDA-shrink", "MLP", "base", "+shrink", "+code", "+rankC1", "+T1",
            "+shrink+code", "+code+rankC1", "+shrink+code+rankC1", "+shrink+code+rankC1+T1",
            "corrige, lecture centroides"]
    geo = {"S_W brut, rang perm.": (False, "perm"), "Ledoit-Wolf, rang perm.": (True, "perm"),
           "S_W brut, rang C-1": (False, "C1"), "Ledoit-Wolf, rang C-1": (True, "C1")}
    acc = {k: [] for k in keys}; angU = {k: [] for k in geo}; angW = {k: [] for k in geo}
    ranks = {k: [] for k in geo}
    for s in range(seeds):
        X, y, Sigma, mus = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=kappa, sep=sep, seed=s)
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=s)
        U_true, W_true = true_subspaces(Sigma, mus)
        acc["oracle"].append((oracle_bayes(Sigma, mus, Xte) == yte).mean())
        acc["LDA-shrink"].append(LDA(solver="lsqr", shrinkage="auto").fit(Xtr, ytr).score(Xte, yte))
        acc["MLP"].append(MLPClassifier(hidden_layer_sizes=(32,), max_iter=3000, random_state=s).fit(Xtr,
     ytr).score(Xte, yte))

        cfg = {
            "base":       dict(shrink=False, rank="perm", T=4, feat="h"),
            "+shrink":    dict(shrink=True,  rank="perm", T=4, feat="h"),
            "+code":      dict(shrink=False, rank="perm", T=4, feat="code"),
            "+rankC1":    dict(shrink=False, rank="C1",   T=4, feat="h"),
            "+T1":        dict(shrink=False, rank="perm", T=1, feat="h"),
            "+shrink+code":           dict(shrink=True, rank="perm", T=4, feat="code"),
            "+code+rankC1":           dict(shrink=False, rank="C1",  T=4, feat="code"),
            "+shrink+code+rankC1":    dict(shrink=True, rank="C1",   T=4, feat="code"),
            "+shrink+code+rankC1+T1": dict(shrink=True, rank="C1",   T=1, feat="code"),
            # bloc corrige avec la lecture de reference (regle LDA) ; T sans effet : le code est
     invariant
            "corrige, lecture centroides": dict(shrink=True, rank="C1", T=4, feat="code", readout="
     centroid"),
        }
        cache = {}
        for k, c in cfg.items():
            key = (c["shrink"], c["rank"])
            if key not in cache:
                cache[key] = fit_block(Xtr, ytr, shrink=c["shrink"], rank=c["rank"], seed=s)
```

```
            U, W, r = cache[key]
            acc[k].append((block_predict(U, W, Xtr, ytr, Xte, T=c["T"], feat=c["feat"], readout=c.get("readout", "ls")) == yte).mean())
        for k, key in geo.items():
            U, W, r = cache[key]
            angU[k].append(np.degrees(subspace_angles(U, U_true).max()))
            angW[k].append(np.degrees(subspace_angles(W, W_true).max()))
            ranks[k].append(r)
    print(f"\n=== kappa={kappa:.0f}, n={n_per}/classe (n_train={int(0.7*n_per*C)}), d={d}, C={C}, {seeds} graines ===")
    for k in keys:
        print(f"  {k:26s} acc = {np.mean(acc[k])*100:5.1f} +/- {np.std(acc[k])*100:4.1f}")
    for k in angU:
        print(f"  geometrie {k:24s}: angle max span(U_hat)/span(U_vrai) = {np.mean(angU[k]):5.1f} deg | "
              f"span(W_hat)/span(W_vrai) = {np.mean(angW[k]):5.1f} deg | r* = {np.mean(ranks[k]):.2f}")
    from ropn_auto_stats import fmt_paired
    ref = acc["corrige, lecture centroides"]
    print("  differences appariees au bloc corrige, lecture par centroides [IC 95 %] :")
    for k in ("oracle", "LDA-shrink", "MLP", "+shrink+code+rankC1"):
        print(f"    {k:26s} - bloc corrige (centroides) : {fmt_paired(acc[k], ref)}")
    return acc


if __name__ == "__main__":
    for kappa in [10.0, 200.0]:
        run(kappa, 15)
    print("\n--- controle : regime abondant (n=200/classe), kappa=200 ---")
    run(200.0, 200, seeds=10)

    # regime extreme : 4 exemples par classe, n_train = 11 < d = 20 (apres decoupage 70/30 : 5 points de test par graine)
    from sklearn.neighbors import NearestCentroid
    print("\n--- regime extreme : 4 exemples/classe (n_train = 11 < d = 20), 20 graines, 5 points de test par graine ---")
    for kappa in (10.0, 200.0):
        a = {k: [] for k in ("bloc corrige", "bloc corrige (MC)", "MLP", "LDA-shrink", "centroide")}
        for s in range(20):
            X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=4, kappa=kappa, sep=0.8, seed=s)
            Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=s)
            U, W, r = fit_block(Xtr, ytr, shrink=True, rank="C1", seed=s)
            a["bloc corrige"].append((readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W) == yte).mean())
            a["bloc corrige (MC)"].append((readout_fit_predict(Xtr @ W, ytr, Xte @ W) == yte).mean())
            a["MLP"].append(MLPClassifier(hidden_layer_sizes=(32,), max_iter=3000, random_state=s).fit(Xtr, ytr).score(Xte, yte))
            a["LDA-shrink"].append(LDA(solver="lsqr", shrinkage="auto").fit(Xtr, ytr).score(Xte, yte))
            a["centroide"].append(NearestCentroid().fit(Xtr, ytr).score(Xte, yte))
        print(f"  kappa={kappa:5.0f} (n_train={len(ytr)}, n_test={len(yte)}) : " + "  ".join(f"{k} {100*np.mean(v):.1f} +/- {100*np.std(v):.1f}" for k, v in a.items()))
```

Listing 3: ropn_auto_finetune.py — gradient fine-tuning of the block (manual gradients)

```
"""
Reinjection par retropropagation : fine-tuning du bloc unique a partir de son
initialisation fermee (warm start de Fisher), en numpy avec gradients manuels
(verifies par differences finies).

Modele fine-tune (T applications du meme bloc, poids partages) :
    Uo  : base orthonormee de span(U_ferme)  -- FIXEE (representation)
    W   = Uo + (I - Uo Uo^T) M               -- M appris : W^T Uo = I_r reste EXACT
    a_t = sigma(h_t s + b)                   -- porte, (s, b) appris
    h_{t+1} = (1 - a_t) h_t + a_t (h_t W) Uo^T
    logits = h_T Wo + bo                     -- lecture apprise sur l'etat complet (dim d)
    perte = entropie croisee + wd * (||M||^2 + ||Wo||^2)

Initialisation fermee : le bloc CORRIGE (retrecissement, rang C-1, lecture par centroides sur le code,
c'est-a-dire la regle LDA), ecrit comme une lecture affine sur h_T dont les poids sont dans span(W)
(Wo = W M^T, bo = -||m_k||^2/2 + log pi_k) : le code W^T h_T etant invariant par la recursion, avant tout
pas de gradient le modele fine-tune predit exactement comme le bloc corrige, et ses logits sont les
scores discriminants de la LDA.
Variantes du modele contraint (Uo FIXE dans tous les cas : seul le sous-espace de mesure peut bouger) :
(M, porte, lecture) avec deux reglages wd / nombre de pas, lecture seule, porte + lecture (M fixe),
```

```
M + lecture (porte fixe), et initialisation aleatoire du meme modele aux memes reglages.

Modele LIBRE (les DEUX sous-espaces evoluent) : U et W tous deux appris,
    h_{t+1} = (1 - a_t) h_t + a_t (h_t W) U^T,
dans un plan factoriel 2 x 2 x 2 qui separe trois effets : initialisation (bloc ferme / aleatoire),
contrainte geometrique (P = U (W^T U)^{-1} W^T, idempotent exactement a chaque pas, le gradient traversant
l'inverse ; ou P = U W^T sans contrainte), regularisation (wd 1e-3, 300 pas / wd 1e-1, 100 pas). La penalite porte sur la
lecture et sur l'ecart de (U, W) a leur initialisation.

Chaque variante est comparee au bloc ferme par differences APPARIEES sur les memes tirages (intervalle
de confiance a 95 %, marge d'equivalence de 1 point : voir ropn_auto_stats.py).

Deux jeux de donnees :
    gauss  : covariance partagee (LDA = Bayes) -> la forme fermee est optimale en population
    hetero : covariances differentes par classe (LDA mal specifiee) -> information hors span(W)
"""
import numpy as np
from numpy.linalg import qr, norm, pinv, slogdet
from sklearn.neural_network import MLPClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import train_test_split
from ropn_auto_nobp import make_multiclass, make_closed_form_gate, recurse_gated, sigmoid
from ropn_auto_nobp_diag import fit_block, readout_centroid_fit_predict, centroid_readout_weights
from ropn_auto_stats import fmt_paired


# ------------------------------------------------------------------ donnees heteroscedastiques
def make_hetero(n_classes=4, d=20, n_per=60, kappa=50.0, sep=0.8, seed=0):
    rng = np.random.default_rng(seed)
    eig = np.logspace(0, np.log10(kappa), d); eig = eig / eig.mean()
    mus = sep * rng.standard_normal((n_classes, d)); mus -= mus.mean(0, keepdims=True)
    X = np.zeros((n_classes * n_per, d)); y = np.zeros(n_classes * n_per, dtype=int); Sigmas = []
    for c in range(n_classes):
        Q, _ = qr(rng.standard_normal((d, d)))           # orientation PROPRE a chaque classe
        Sigma_c = Q @ np.diag(eig) @ Q.T; Sigmas.append(Sigma_c)
        L = np.linalg.cholesky(Sigma_c + 1e-9 * np.eye(d))
        X[c*n_per:(c+1)*n_per] = rng.standard_normal((n_per, d)) @ L.T + mus[c]
        y[c*n_per:(c+1)*n_per] = c
    return X, y, Sigmas, mus


def oracle_qda(Sigmas, mus, X):
    sc = []
    for S, m in zip(Sigmas, mus):
        Si = pinv(S); Xc = X - m
        sc.append(-0.5 * np.einsum("ij,jk,ik->i", Xc, Si, Xc) - 0.5 * slogdet(S)[1])
    return np.stack(sc, 1).argmax(1)


# ------------------------------------------------------------------ modele + gradients manuels
def forward(params, Uo, X, T):
    M, s, b, Wo, bo = params
    d, r = Uo.shape
    Pperp = np.eye(d) - Uo @ Uo.T
    W = Uo + Pperp @ M
    hs, As, Ps = [X], [], []
    h = X
    for _ in range(T):
        a = sigmoid(h @ s + b)[:, None]
        p = (h @ W) @ Uo.T
        h = (1 - a) * h + a * p
        As.append(a); Ps.append(p); hs.append(h)
    logits = h @ Wo + bo
    return logits, (W, Pperp, hs, As, Ps)


def loss_and_grads(params, Uo, X, y, T, wd):
    M, s, b, Wo, bo = params
    n = len(y)
    logits, (W, Pperp, hs, As, Ps) = forward(params, Uo, X, T)
    z = logits - logits.max(1, keepdims=True); pr = np.exp(z); pr /= pr.sum(1, keepdims=True)
```

```
    loss = -np.log(pr[np.arange(n), y] + 1e-12).mean() + wd * (np.sum(M**2) + np.sum(Wo**2))
    dlog = pr.copy(); dlog[np.arange(n), y] -= 1.0; dlog /= n
    hT = hs[-1]
    dWo = hT.T @ dlog + 2 * wd * Wo; dbo = dlog.sum(0)
    dh = dlog @ Wo.T
    dW = np.zeros_like(W); ds = np.zeros_like(s); db = 0.0
    for t in reversed(range(T)):
        h, a, p = hs[t], As[t], Ps[t]
        da = np.sum(dh * (p - h), 1, keepdims=True)
        dz = da * a * (1 - a)
        ds += h.T @ dz[:, 0]; db += dz.sum()
        dp = a * dh
        dW += h.T @ (dp @ Uo)
        dh_new = (1 - a) * dh + (dp @ Uo) @ W.T + dz * s[None, :]
        dh = dh_new
    dM = Pperp @ dW + 2 * wd * M
    return loss, (dM, ds, db, dWo, dbo)


def gradcheck(seed=0):
    rng = np.random.default_rng(seed)
    d, r, C, n, T = 6, 2, 3, 7, 3
    Uo, _ = qr(rng.standard_normal((d, r)))
    params = [rng.standard_normal((d, r)) * 0.3, rng.standard_normal(d) * 0.3, 0.1,
              rng.standard_normal((d, C)) * 0.3, rng.standard_normal(C) * 0.1]
    X = rng.standard_normal((n, d)); y = rng.integers(0, C, n)
    loss, grads = loss_and_grads(params, Uo, X, y, T, wd=1e-3)
    eps = 1e-6; worst = 0.0
    for i, p in enumerate(params):
        g = grads[i]
        if np.ndim(p) == 0:
            pp = list(params); pp[i] = p + eps; lp = loss_and_grads(pp, Uo, X, y, T, 1e-3)[0]
            pp[i] = p - eps; lm = loss_and_grads(pp, Uo, X, y, T, 1e-3)[0]
            worst = max(worst, abs((lp - lm) / (2 * eps) - g) / (abs(g) + 1e-8))
        else:
            it = np.nditer(p, flags=["multi_index"])
            for _ in it:
                idx = it.multi_index
                pp = [q.copy() if np.ndim(q) else q for q in params]
                pp[i][idx] += eps; lp = loss_and_grads(pp, Uo, X, y, T, 1e-3)[0]
                pp[i][idx] -= 2 * eps; lm = loss_and_grads(pp, Uo, X, y, T, 1e-3)[0]
                worst = max(worst, abs((lp - lm) / (2 * eps) - g[idx]) / (abs(g[idx]) + 1e-8))
    return worst


def adam_train(params, Uo, X, y, T, steps=300, lr=1e-2, wd=1e-3, train=(1, 1, 1, 1, 1)):
    """train : masque (M, s, b, Wo, bo) des groupes de parametres appris ; les autres restent figes."""
    params = [p.copy() if np.ndim(p) else float(p) for p in params]
    m = [np.zeros_like(p) if np.ndim(p) else 0.0 for p in params]
    v = [np.zeros_like(p) if np.ndim(p) else 0.0 for p in params]
    b1, b2, eps = 0.9, 0.999, 1e-8
    for k in range(1, steps + 1):
        _, g = loss_and_grads(params, Uo, X, y, T, wd)
        for i in range(len(params)):
            if not train[i]: continue
            m[i] = b1 * m[i] + (1 - b1) * g[i]; v[i] = b2 * v[i] + (1 - b2) * (g[i] ** 2)
            mh = m[i] / (1 - b1 ** k); vh = v[i] / (1 - b2 ** k)
            params[i] = params[i] - lr * mh / (np.sqrt(vh) + eps)
    return params


def predict(params, Uo, X, T):
    return forward(params, Uo, X, T)[0].argmax(1)


# ------------------------------------------------------------------ modele libre : U et W appris tous les deux
def forward_free(params, X, T, idem):
    """idem=True : p = h W (U^T W)^{-1} U^T, projecteur oblique EXACTEMENT idempotent pour tous (U, W) tels que
    U^T W soit inversible ; idem=False : p = h W U^T, application de rang r sans contrainte."""
    U, W, s, b, Wo, bo = params
    Gi = np.linalg.inv(U.T @ W) if idem else None
```

```
    hs, As, Ps, Bs = [X], [], [], []; h = X
    for _ in range(T):
        a = sigmoid(h @ s + b)[:, None]; B = (h @ W) @ Gi if idem else h @ W; p = B @ U.T
        h = (1 - a) * h + a * p
        As.append(a); Ps.append(p); Bs.append(B); hs.append(h)
    return h @ Wo + bo, (hs, As, Ps, Bs, Gi)


def loss_and_grads_free(params, ref, X, y, T, wd, idem):
    """perte = entropie croisee + wd (||Wo||^2 + ||U - U_ref||^2 + ||W - W_ref||^2), ref = (U_ref, W_ref)
    ."""
    U, W, s, b, Wo, bo = params; n = len(y)
    logits, (hs, As, Ps, Bs, Gi) = forward_free(params, X, T, idem)
    z = logits - logits.max(1, keepdims=True); pr = np.exp(z); pr /= pr.sum(1, keepdims=True)
    loss = -np.log(pr[np.arange(n), y] + 1e-12).mean() + wd * (np.sum(Wo**2) + np.sum((U - ref[0])**2) +
     np.sum((W - ref[1])**2))
    dlog = pr.copy(); dlog[np.arange(n), y] -= 1.0; dlog /= n
    dWo = hs[-1].T @ dlog + 2 * wd * Wo; dbo = dlog.sum(0); dh = dlog @ Wo.T
    dU = np.zeros_like(U); dW = np.zeros_like(W); ds = np.zeros_like(s); db = 0.0
    for t in reversed(range(T)):
        h, a, p, B = hs[t], As[t], Ps[t], Bs[t]
        dz = np.sum(dh * (p - h), 1, keepdims=True) * a * (1 - a)
        ds += h.T @ dz[:, 0]; db += dz.sum()
        dp = a * dh; dB = dp @ U; dU += dp.T @ B
        if idem:                                        # B = (h W) G^{-1}, G = U^T W
            dA = dB @ Gi.T; dG = -(B.T @ dB) @ Gi.T
            dU += W @ dG.T; dW += U @ dG
        else:
            dA = dB
        dW += h.T @ dA
        dh = (1 - a) * dh + dA @ W.T + dz * s[None, :]
    return loss, (dU + 2 * wd * (U - ref[0]), dW + 2 * wd * (W - ref[1]), ds, db, dWo, dbo)


def gradcheck_free(idem, seed=0):
    rng = np.random.default_rng(seed); d, r, C, n, T = 6, 2, 3, 7, 3
    U0 = rng.standard_normal((d, r)) * 0.5
    params = [U0, U0 + 0.3 * rng.standard_normal((d, r)), rng.standard_normal(d) * 0.3, 0.1,
              rng.standard_normal((d, C)) * 0.3, rng.standard_normal(C) * 0.1]
    ref = (rng.standard_normal((d, r)) * 0.5, rng.standard_normal((d, r)) * 0.5)
    X = rng.standard_normal((n, d)); y = rng.integers(0, C, n)
    _, grads = loss_and_grads_free(params, ref, X, y, T, 1e-3, idem); eps = 1e-6; worst = 0.0
    for i, p in enumerate(params):
        g = grads[i]
        if np.ndim(p) == 0:
            pp = list(params); pp[i] = p + eps; lp = loss_and_grads_free(pp, ref, X, y, T, 1e-3, idem)[0]
            pp[i] = p - eps; lm = loss_and_grads_free(pp, ref, X, y, T, 1e-3, idem)[0]
            worst = max(worst, abs((lp - lm) / (2 * eps) - g) / (abs(g) + 1e-8))
        else:
            it = np.nditer(p, flags=["multi_index"])
            for _ in it:
                idx = it.multi_index; pp = [q.copy() if np.ndim(q) else q for q in params]
                pp[i][idx] += eps; lp = loss_and_grads_free(pp, ref, X, y, T, 1e-3, idem)[0]
                pp[i][idx] -= 2 * eps; lm = loss_and_grads_free(pp, ref, X, y, T, 1e-3, idem)[0]
                worst = max(worst, abs((lp - lm) / (2 * eps) - g[idx]) / (abs(g[idx]) + 1e-8))
    return worst


def adam_train_free(params, X, y, T, steps=300, lr=1e-2, wd=1e-3, idem=True):
    """Adam sur (U, W, s, b, Wo, bo) ; les deux sous-espaces evoluent. idem=True : l'idempotence est
     exacte a
    chaque pas par construction (le gradient traverse (U^T W)^{-1}) ; idem=False : aucune contrainte."""
    params = [p.copy() if np.ndim(p) else float(p) for p in params]; ref = (params[0].copy(), params[1].
     copy())
    m = [np.zeros_like(p) if np.ndim(p) else 0.0 for p in params]; v = [np.zeros_like(p) if np.ndim(p)
     else 0.0 for p in params]
    b1, b2, eps = 0.9, 0.999, 1e-8
    for k in range(1, steps + 1):
        _, g = loss_and_grads_free(params, ref, X, y, T, wd, idem)
        for i in range(len(params)):
            m[i] = b1 * m[i] + (1 - b1) * g[i]; v[i] = b2 * v[i] + (1 - b2) * (g[i] ** 2)
            params[i] = params[i] - lr * (m[i] / (1 - b1 ** k)) / (np.sqrt(v[i] / (1 - b2 ** k)) + eps)
    return params
```

```python


def init_free_closed_form(U, W, Xtr, ytr):
    """bloc corrige ecrit dans le modele libre : (U, W) biorthogonaux, porte fermee, lecture par centroides
    remontee sur h_T (Wo = W M^T) : h W est invariant par la recursion, donc predictions initiales = bloc."""
    s0, b0 = make_closed_form_gate(U, W, Xtr)
    _, V = centroid_readout_weights(Xtr @ W, ytr)
    return [U.copy(), W.copy(), s0.copy(), float(b0), W @ V[:-1], V[-1].copy()]


def init_free_random(d, r, C, rng):
    U, _ = qr(rng.standard_normal((d, r)))
    return [U.copy(), U + 0.1 * rng.standard_normal((d, r)), rng.standard_normal(d) * 0.1, 0.0, rng.standard_normal((d, C)) * 0.1, np.zeros(C)]


# ---------------------------------------------------------------- inits
def init_closed_form(U, W, Xtr, ytr, T):
    """Warm start de Fisher = bloc corrige : Uo = QR(U), M tel que W_param = W R^T exactement,
    porte fermee, lecture par centroides sur le code W^T x remontee sur h_T (Wo = W M^T) : le projecteur
    Uo W_param^T est egal a U W^T, donc W^T h_T = W^T x et les predictions initiales sont celles du bloc.
    """
    Uo, R = qr(U)
    Wp = W @ R.T                                    # W_p^T Uo = I_r
    M0 = Wp - Uo                                    # (I - Uo Uo^T) M0 = Wp - Uo exactement
    s0, b0 = make_closed_form_gate(U, W, Xtr)
    _, V = centroid_readout_weights(Xtr @ W, ytr)
    return Uo, [M0, s0.copy(), float(b0), W @ V[:-1], V[-1].copy()]


def init_random(d, r, C, rng):
    Uo, _ = qr(rng.standard_normal((d, r)))
    return Uo, [rng.standard_normal((d, r)) * 0.1, rng.standard_normal(d) * 0.1, 0.0,
                rng.standard_normal((d, C)) * 0.1, np.zeros(C)]


# ---------------------------------------------------------------- experience
VARIANTS = [  # modele contraint, Uo fixe : (nom, init, masque (M, s, b, Wo, bo), wd, pas)
    ("Uo fixe | ferme -> M, porte, lecture, wd 1e-3, 300 pas", "ferme", (1, 1, 1, 1, 1), 1e-3, 300),
    ("Uo fixe | ferme -> M, porte, lecture, wd 1e-1, 100 pas", "ferme", (1, 1, 1, 1, 1), 1e-1, 100),
    ("Uo fixe | ferme -> lecture seule, wd 1e-3, 300 pas",     "ferme", (0, 0, 0, 1, 1), 1e-3, 300),
    ("Uo fixe | ferme -> porte, lecture, wd 1e-3, 300 pas",    "ferme", (0, 1, 1, 1, 1), 1e-3, 300),
    ("Uo fixe | ferme -> M, lecture, wd 1e-3, 300 pas",        "ferme", (1, 0, 0, 1, 1), 1e-3, 300),
    ("Uo fixe | alea  -> M, porte, lecture, wd 1e-3, 300 pas", "alea",  (1, 1, 1, 1, 1), 1e-3, 300),
    ("Uo fixe | alea  -> M, porte, lecture, wd 1e-1, 100 pas", "alea",  (1, 1, 1, 1, 1), 1e-1, 100),
]
FREE_VARIANTS = [  # modele libre, U et W appris : (nom, init, retraction, wd, pas) -- plan factoriel 2 x 2 x 2
    (f"U, W libres | {ini:5s} | {'idempotent     ' if rt else 'sans contrainte'} | wd {wd:g}, {st} pas", ini, rt, wd, st)
    for ini in ("ferme", "alea") for rt in (True, False) for wd, st in ((1e-3, 300), (1e-1, 100))
]


def run(dataset, kappa, n_per, seeds=20, T=4, lr=1e-2, d=20, C=4):
    keys = ["oracle", "LDA-shrink", "MLP", "ferme corrige (shrink+code+C-1)", "ferme, lecture h_T (init. du fine-tuning)"] + [v[0] for v in VARIANTS] + [v[0] for v in FREE_VARIANTS]
    acc = {k: [] for k in keys}
    for sd in range(seeds):
        rng = np.random.default_rng(1000 + sd)
        if dataset == "gauss":
            X, y, Sigma, mus = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=kappa, sep=0.8, seed=sd)
            Sigmas = [Sigma] * C
        else:
            X, y, Sigmas, mus = make_hetero(n_classes=C, d=d, n_per=n_per, kappa=kappa, sep=0.8, seed=sd)
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
        acc["oracle"].append((oracle_qda(Sigmas, mus, Xte) == yte).mean())
        acc["LDA-shrink"].append(LDA(solver="lsqr", shrinkage="auto").fit(Xtr, ytr).score(Xte, yte))
        acc["MLP"].append(MLPClassifier(hidden_layer_sizes=(32,), max_iter=3000, random_state=sd).fit(Xtr, ytr).score(Xte, yte))
```

```
295
296         Ub, Wb, _ = fit_block(Xtr, ytr, shrink=True, rank="C1", seed=sd)
297         acc["ferme corrige (shrink+code+C-1)"].append((readout_centroid_fit_predict(Xtr @ Wb, ytr, Xte @
      Wb) == yte).mean())
298
299         Uo, p0 = init_closed_form(Ub, Wb, Xtr, ytr, T)
300         acc["ferme, lecture h_T (init. du fine-tuning)"].append((predict(p0, Uo, Xte, T) == yte).mean())
301         Uo_r, pr0 = init_random(d, Ub.shape[1], C, rng)
302         for name, init, mask, wd, steps in VARIANTS:
303             U_, p_ = (Uo, p0) if init == "ferme" else (Uo_r, pr0)
304             p1 = adam_train(p_, U_, Xtr, ytr, T, steps, lr, wd, train=mask)
305             acc[name].append((predict(p1, U_, Xte, T) == yte).mean())
306         f0 = init_free_closed_form(Ub, Wb, Xtr, ytr); fr0 = init_free_random(d, Ub.shape[1], C, rng)
307         for name, init, retract, wd, steps in FREE_VARIANTS:
308             pf = adam_train_free(f0 if init == "ferme" else fr0, Xtr, ytr, T, steps, lr, wd, idem=retract
      )
309             acc[name].append((forward_free(pf, Xte, T, retract)[0].argmax(1) == yte).mean())
310     print(f"\n=== {dataset}, kappa={kappa:.0f}, n={n_per}/classe, T={T}, {seeds} graines ===")
311     ref = acc["ferme corrige (shrink+code+C-1)"]
312     print(f"  {'methode':66s} {'precision':>14s}   difference appariee au bloc ferme : moyenne [IC 95 %]
       verdict (marge 1 point)")
313     for k in keys:
314         print(f"  {k:66s} {np.mean(acc[k])*100:5.1f} +/- {np.std(acc[k])*100:4.1f}   {fmt_paired(acc[k],
       ref) if k != 'ferme corrige (shrink+code+C-1)' else ''}")
315     # effets principaux du plan factoriel 2 x 2 x 2 du modele libre : pour chaque graine, moyenne des
       variantes d'un
316     # niveau, puis difference appariee entre les deux niveaux (memes graines, memes decoupages)
317     def level(pred):
318         return np.mean([acc[v[0]] for v in FREE_VARIANTS if pred(v)], axis=0)
319     print("  effets du plan factoriel (U, W libres), differences appariees :")
320     print(f"    initialisation : ferme - alea (moyenne sur contrainte et regularisation)        {
       fmt_paired(level(lambda v: v[1] == 'ferme'), level(lambda v: v[1] == 'alea'))}")
321     print(f"    contrainte : idempotent - sans contrainte, depuis l'initialisation fermee        {
       fmt_paired(level(lambda v: v[1] == 'ferme' and v[2]), level(lambda v: v[1] == 'ferme' and not v[2]))}
       ")
322     print(f"    contrainte : idempotent - sans contrainte, depuis l'initialisation aleatoire     {
       fmt_paired(level(lambda v: v[1] == 'alea' and v[2]), level(lambda v: v[1] == 'alea' and not v[2]))}")
323     print(f"    regularisation : (wd 1e-3, 300 pas) - (wd 1e-1, 100 pas), initialisation fermee  {
       fmt_paired(level(lambda v: v[1] == 'ferme' and v[3] == 1e-3), level(lambda v: v[1] == 'ferme' and v
       [3] == 1e-1))}")
324     print(f"    regularisation : (wd 1e-3, 300 pas) - (wd 1e-1, 100 pas), initialisation aleatoire {
       fmt_paired(level(lambda v: v[1] == 'alea' and v[3] == 1e-3), level(lambda v: v[1] == 'alea' and v[3]
       == 1e-1))}")
325     return acc
326
327
328 if __name__ == "__main__":
329     print(f"gradcheck (erreur relative max) = {gradcheck():.2e} ; modele libre idempotent : {
       gradcheck_free(True):.2e}, sans contrainte : {gradcheck_free(False):.2e}")
330     run("gauss", 10.0, 15)
331     run("gauss", 200.0, 15)
332     run("hetero", 50.0, 15)
333     run("hetero", 50.0, 60)
```

Listing 4: ropn_auto_nobp_sweep.py — anisotropy sweep

```
1  """Balayage kappa x graines : accuracy moyenne +/- std, comparaison
2  bloc unique auto-ROPN (sans backprop) vs MLP (backprop), et regime n<=d (shrinkage)."""
3  import numpy as np
4  from sklearn.neural_network import MLPClassifier
5  from sklearn.neighbors import NearestCentroid
6  from sklearn.model_selection import train_test_split
7  from ropn_auto_nobp import (make_multiclass, fit_oblique_block_closed_form,
8                              make_closed_form_gate, recurse_gated)
9  from ropn_auto_nobp_diag import fit_block, readout_fit_predict, readout_centroid_fit_predict
10
11 def run_once(kappa, n_per, d=20, n_classes=4, T=4, seed=0, sep=0.8):
12     X, y, *_ = make_multiclass(n_classes=n_classes, d=d, n_per=n_per, kappa=kappa, sep=sep, seed=seed)
13     Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=seed)
14     nc = NearestCentroid().fit(Xtr, ytr)
15     acc_ortho = nc.score(Xte, yte)
```

```
    U, W, diag = fit_oblique_block_closed_form(Xtr, ytr, seed=seed)
    s, b = make_closed_form_gate(U, W, Xtr)
    Htr = recurse_gated(U, W, s, b, Xtr, T)
    Hte = recurse_gated(U, W, s, b, Xte, T)
    classes = np.unique(ytr)
    Ytr = np.eye(len(classes))[ytr]
    Wro, *_ = np.linalg.lstsq(np.c_[Htr, np.ones(len(Htr))], Ytr, rcond=None)
    pred = classes[(np.c_[Hte, np.ones(len(Hte))] @ Wro).argmax(1)]
    acc_auto = (pred == yte).mean()
    n_par_auto = U.size + Wro.size

    # pipeline corrige (section petits echantillons) : shrinkage systematique, rang structurel C-1,
    # lecture par centroides sur le code oblique c = W^T h_T (dim r) = regle LDA ; la lecture par
    # moindres carres sur le meme code est conservee pour comparaison
    Ub, Wb, _ = fit_block(Xtr, ytr, shrink=True, rank="C1", seed=seed)
    sb, bb = make_closed_form_gate(Ub, Wb, Xtr)
    Ctr = recurse_gated(Ub, Wb, sb, bb, Xtr, T) @ Wb; Cte = recurse_gated(Ub, Wb, sb, bb, Xte, T) @ Wb
    acc_best = (readout_centroid_fit_predict(Ctr, ytr, Cte) == yte).mean()
    acc_ls = (readout_fit_predict(Ctr, ytr, Cte) == yte).mean()
    n_par_best = Ub.size + (Wb.shape[1] + 1) * len(classes)       # centroides r C + a priori C = (r+1) C

    mlp = MLPClassifier(hidden_layer_sizes=(32,), max_iter=3000, random_state=seed)
    mlp.fit(Xtr, ytr)
    acc_mlp = mlp.score(Xte, yte)
    n_par_mlp = sum(w.size for w in mlp.coefs_) + sum(bb.size for bb in mlp.intercepts_)

    return acc_auto, n_par_auto, acc_mlp, n_par_mlp, diag['r_star'], diag['shrink'], acc_ortho, acc_best,
      n_par_best, acc_ls


if __name__ == "__main__":
    print(f"{'kappa':>7} {'n/classe':>9} {'acc ortho':>9} {'acc base':>9} {'acc corr':>9} {'acc MLP':>8}
     "
          f"{'#par base':>10} {'#par corr':>10} {'#par MLP':>9} {'r*':>3} {'shrink':>7}")
    SEEDS = 20
    from ropn_auto_stats import fmt_paired
    for kappa in [1.0, 10.0, 50.0, 200.0, 1000.0]:
        for n_per in [200, 15]:
            accs_a, accs_m, accs_o, accs_b, npa, npm, npb, rst, accs_l = [], [], [], [], [], [], [], [], [],
     []
            shrink_used = False
            for s in range(SEEDS):
                aa, na, am, nm, r, sh, ao, ab, nb, al = run_once(kappa, n_per, seed=s, T=4)
                accs_a.append(aa); accs_m.append(am); accs_o.append(ao); accs_b.append(ab); accs_l.append
     (al)
                npa.append(na); npm.append(nm); npb.append(nb); rst.append(r)
                shrink_used = shrink_used or sh
            print(f"{kappa:7.0f} {n_per:9d} "
                  f"{np.mean(accs_o)*100:8.1f}% {np.mean(accs_a)*100:8.1f}% {np.mean(accs_b)*100:8.1f}% {
     np.mean(accs_m)*100:7.1f}% "
                  f"{int(np.mean(npa)):10d} {int(np.mean(npb)):10d} {int(np.mean(npm)):9d} "
                  f"{np.mean(rst):3.1f} {'oui' if shrink_used else 'non':>7}"
                  f"   (std base={np.std(accs_a)*100:.1f}, std corr={np.std(accs_b)*100:.1f}, std MLP={np
     .std(accs_m)*100:.1f})")
            print(f"        differences appariees [IC 95 %] : corrige - MLP {fmt_paired(accs_b, accs_m)}
     | corrige - orthogonal {fmt_paired(accs_b, accs_o)}"
                  f" | corrige (centroides) - corrige (MC) {fmt_paired(accs_b, accs_l)} [MC : {np.mean(
     accs_l)*100:.1f}%]")
```

Listing 5: ropn_auto_benchmark.py — benchmark on synthetic and real datasets

```
"""
Banc d'essai : orthogonal vs bloc ferme (base) vs bloc ferme (corrige) vs MLP (backprop)
sur des jeux synthetiques et reels.

Pour chaque (jeu, methode) on rapporte :
  - precision test (et balanced accuracy pour les jeux desequilibres)
  - convergence : nb d'iterations (MLP : n_iter_ et perte finale ; forme fermee : 1 passe,
    ou 1 + n_perm resolutions GEVP pour le test de permutation de la variante 'base')
  - temps de calcul mesure (fit + predict, wall-clock) et cout asymptotique
```

```
  - nombre de parametres stockes, et parmi eux ceux optimises par gradient

Methodes (definitions strictement identiques a celles des autres scripts) :
  orthogonal      : plus proche centroide (n'exploite pas S_W)                 [sklearn]
  bloc ferme base : S_W brut (shrink si n<=d), rang par permutation,
                    lecture LS sur h_T (dim d), T=4                            [ropn_auto_nobp]
  bloc ferme corr.: shrinkage Ledoit-Wolf, rang structurel C-1,
                    lecture par centroides sur le code c = W^T h_T (regle LDA)  [ropn_auto_nobp_diag]
  MLP backprop    : 1 couche cachee (largeur adaptee a d), Adam                [sklearn]

Jeux : synthetiques (gauss kappa=10 n=15/cl ; gauss kappa=200 n=200/cl ; heteroscedastique),
       reels sklearn (digits 8x8 ~ MNIST reduit ; breast cancer Wisconsin = medical ; wine),
       MNIST 784 via OpenML si le reseau le permet (sinon ignore avec un message).
"""
import time
import warnings
import numpy as np
from sklearn.datasets import load_digits, load_breast_cancer, load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestCentroid
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import balanced_accuracy_score
from sklearn.exceptions import ConvergenceWarning

from ropn_auto_nobp import (make_multiclass, fit_oblique_block_closed_form,
                            make_closed_form_gate, recurse_gated)
from ropn_auto_nobp_diag import fit_block, readout_fit_predict, readout_centroid_fit_predict
from ropn_auto_finetune import make_hetero

warnings.filterwarnings("ignore", category=ConvergenceWarning)
T_REC = 4


# ---------------------------------------------------------------- jeux de donnees
def dataset_registry(seed=0, mnist_n_train=10000, mnist_n_test=2000):
    reg = {}
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=10.0, sep=0.8, seed=seed)
    reg["synth gauss k=10, n=15/cl"] = (X, y)
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=200, kappa=200.0, sep=0.8, seed=seed)
    reg["synth gauss k=200, n=200/cl"] = (X, y)
    X, y, *_ = make_hetero(n_classes=4, d=20, n_per=60, kappa=50.0, sep=0.8, seed=seed)
    reg["synth hetero k=50, n=60/cl"] = (X, y)
    d = load_digits();        reg["digits 8x8 (10 cl.)"] = (d.data, d.target)
    b = load_breast_cancer(); reg["breast cancer (medical, 2 cl.)"] = (b.data, b.target)
    w = load_wine();          reg["wine (3 cl.)"] = (w.data, w.target)
    try:
        from sklearn.datasets import fetch_openml
        Xm, ym = fetch_openml("mnist_784", version=1, return_X_y=True, as_frame=False, cache=True)
        rng = np.random.default_rng(seed); idx = rng.permutation(len(ym))
        Xm = Xm[idx[:mnist_n_train + mnist_n_test]].astype(np.float64)
        ym = ym[idx[:mnist_n_train + mnist_n_test]].astype(int)
        reg[f"MNIST 784 (sous-ech. {mnist_n_train}+{mnist_n_test})"] = (Xm, ym)
    except Exception as e:  # reseau indisponible : on l'indique, on ne fabrique rien
        print(f"[info] MNIST (OpenML) indisponible ici : {type(e).__name__}. "
              f"Lancer ce script avec acces reseau pour l'inclure.")
    return reg


def preprocess(X, y, seed, test_size=0.3):
    """Standardisation + suppression des colonnes constantes (pixels toujours nuls),
    ajustees sur le train uniquement."""
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=test_size, stratify=y, random_state=seed)
    keep = Xtr.std(0) > 1e-8
    sc = StandardScaler().fit(Xtr[:, keep])
    return sc.transform(Xtr[:, keep]), sc.transform(Xte[:, keep]), ytr, yte


# ---------------------------------------------------------------- methodes
def run_orthogonal(Xtr, ytr, Xte):
    m = NearestCentroid().fit(Xtr, ytr)
    C, d = len(np.unique(ytr)), Xtr.shape[1]
    return m.predict(Xte), dict(params=C * d, params_grad=0, iters="1 passe (moyennes)",
```

```
                                cost="O(n d)")


def run_block_base(Xtr, ytr, Xte, seed):
    U, W, diag = fit_oblique_block_closed_form(Xtr, ytr, seed=seed)
    s, b = make_closed_form_gate(U, W, Xtr)
    Htr = recurse_gated(U, W, s, b, Xtr, T_REC); Hte = recurse_gated(U, W, s, b, Xte, T_REC)
    C = len(np.unique(ytr))
    pred = readout_fit_predict(Htr, ytr, Hte)
    return pred, dict(params=U.size + (Xtr.shape[1] + 1) * C, params_grad=0,
                      iters=f"1 GEVP + 200 GEVP (permutation), r*={diag['r_star']}",
                      cost="O(n d^2 + d^3) x (1+n_perm)")


def run_block_corr(Xtr, ytr, Xte, seed):
    U, W, r = fit_block(Xtr, ytr, shrink=True, rank="C1", seed=seed)
    s, b = make_closed_form_gate(U, W, Xtr)
    Ctr = recurse_gated(U, W, s, b, Xtr, T_REC) @ W; Cte = recurse_gated(U, W, s, b, Xte, T_REC) @ W
    C = len(np.unique(ytr))
    pred = readout_centroid_fit_predict(Ctr, ytr, Cte)              # lecture par centroides = regle LDA
    return pred, dict(params=U.size + (r + 1) * C, params_grad=0,
                      iters=f"1 GEVP (Ledoit-Wolf), r*={r}", cost="O(n d^2 + d^3)")


def run_mlp(Xtr, ytr, Xte, seed):
    d = Xtr.shape[1]; h = int(np.clip(2 * d, 32, 256))
    m = MLPClassifier(hidden_layer_sizes=(h,), max_iter=500, random_state=seed,
                      early_stopping=False, n_iter_no_change=20, tol=1e-4)
    m.fit(Xtr, ytr)
    n_par = sum(w.size for w in m.coefs_) + sum(b.size for b in m.intercepts_)
    return m.predict(Xte), dict(params=n_par, params_grad=n_par,
                                iters=f"{m.n_iter_} epoques Adam (perte finale {m.loss_:.3f})"
                                      + (" [max_iter atteint]" if m.n_iter_ >= 500 else ""),
                                cost=f"O(epoques x n x d x h), h={h}")


METHODS = {
    "orthogonal (centroides)": lambda Xtr, ytr, Xte, s: run_orthogonal(Xtr, ytr, Xte),
    "bloc ferme base":         run_block_base,
    "bloc ferme corrige":      run_block_corr,
    "MLP (backprop)":          run_mlp,
}


# ------------------------------------------------------------------ boucle principale
def benchmark(seeds=(0, 1, 2, 3, 4), verbose=True, csv_path="ropn_auto_benchmark_results.csv"):
    results = {}
    reg = dataset_registry(seed=0)
    for dname, (X, y) in reg.items():
        results[dname] = {}
        for mname, fn in METHODS.items():
            accs, baccs, times, info = [], [], [], None
            for sd in seeds:
                Xtr, Xte, ytr, yte = preprocess(X, y, sd)
                t0 = time.perf_counter()
                try:
                    pred, info = fn(Xtr, ytr, Xte, sd)
                    ok = True
                except Exception as e:
                    ok = False; info = dict(params=-1, params_grad=-1, iters=f"ECHEC: {type(e).__name__}"
    , cost="-")
                times.append(time.perf_counter() - t0)
                if ok:
                    accs.append((pred == yte).mean()); baccs.append(balanced_accuracy_score(yte, pred))
            results[dname][mname] = dict(acc=np.mean(accs) if accs else np.nan,
                                         acc_std=np.std(accs) if accs else np.nan,
                                         bacc=np.mean(baccs) if baccs else np.nan,
                                         time=np.mean(times), **info)
        if verbose:
            n, d, C = X.shape[0], X.shape[1], len(np.unique(y))
            print(f"\n=== {dname}  (n={n}, d={d}, C={C}, {len(seeds)} graines, split 70/30) ===")
            print(f"  {'methode':26s} {'acc':>13s} {'bal.acc':>8s} {'temps(s)':>9s} {'#param':>8s} {'#
    grad':>7s}  convergence | cout")
```

```
            for mname, r in results[dname].items():
                print(f"  {mname:26s} {r['acc']*100:6.1f} +/-{r['acc_std']*100:4.1f} {r['bacc']*100:7.1f}
 "
                      f"{r['time']:9.3f} {r['params']:8d} {r['params_grad']:7d}  {r['iters']} | {r['cost
']}")
    # ---- export CSV + synthese ----
    with open(csv_path, "w") as f:
        f.write("dataset,methode,acc,acc_std,bal_acc,temps_s,n_params,n_params_grad,convergence,cout\n")
        for dname, per in results.items():
            for mname, r in per.items():
                f.write(f'"{dname}","{mname}",{r["acc"]:.4f},{r["acc_std"]:.4f},{r["bacc"]:.4f},'
                        f'{r["time"]:.4f},{r["params"]},{r["params_grad"]},"{r["iters"]}","{r["cost"]}"\n
')
    if verbose:
        print(f"\n[resultats ecrits dans {csv_path}]")
        print("\n=== Synthese : meilleure methode par jeu (precision moyenne) ===")
        for dname, per in results.items():
            best = max(per.items(), key=lambda kv: kv[1]["acc"])
            corr, mlp = per["bloc ferme corrige"], per["MLP (backprop)"]
            print(f"  {dname:36s} -> {best[0]:24s} ({best[1]['acc']*100:.1f}%) | "
                  f"corrige vs MLP : {corr['acc']*100:.1f} vs {mlp['acc']*100:.1f}, "
                  f"params {corr['params']} vs {mlp['params']}, temps x{mlp['time']/max(corr['time'],1e
-6):.0f}")
    return results


if __name__ == "__main__":
    benchmark()
```

Listing 6: ropn_auto_fast.py — exact $C \times C$ factorisation, dual form, batched permutations, Bartlett, online update

```
"""
Factorisation a cout reduit de la GEVD des blocs fermes (base et corrige).

Idee centrale (exacte, sans perte) : S_B = D D^T est de rang <= C-1, avec
    D = [sqrt(n_c) (mu_c - mu)]_c / sqrt(C-1)   (d x C).
Les paires propres NON NULLES de S_B v = lambda S_W v se lisent sur le petit probleme C x C
    M = D^T S_W^{-1} D = Q Lambda Q^T,
    W = S_W^{-1} D Q_r Lambda_r^{-1/2}    (mesure),
    U = D Q_r Lambda_r^{-1/2}             (representation),
et l'on a W^T U = I_r AUTOMATIQUEMENT : le reblanchiment devient inutile.
Cout : un seul systeme S_W Z = D (Cholesky d^3/3 + C solves) + eigh C x C, au lieu d'une GEVD
pleine d x d (~10 d^3). Aucune approximation.

Forme duale (n < d) : avec le shrinkage de Ledoit-Wolf, S = (1-a) Xc^T Xc / n + a mu I est
"identite + bas rang", donc S^{-1} D s'obtient par Woodbury avec un systeme n x n
(cout O(n^2 d + n^3)) sans jamais former de matrice d x d. Le coefficient de Ledoit-Wolf
lui-meme se calcule a partir de la matrice de Gram n x n et des normes ||x_i||^4 (identites
exactes ||X^T X||_F^2 = ||X X^T||_F^2 et sum(X2^T X2) = sum_i ||x_i||^4).

Test de permutation : D_p = Xbar^T Y_p N^{-1/2} (sommes par groupe permute), UNE factorisation
de la metrique partagee par toutes les permutations, et les n_perm problemes C x C resolus en
lot (eigvalsh sur une pile) : O(n_perm (n d + d^2 C)) au lieu de O(n_perm d^3), en BLAS-3.
Metrique du test : par defaut le scatter TOTAL S_T (independant des etiquettes, donc
echangeable sous permutation) ; perm_metric="within" donne le test a S_W partage
(calcule avec les vraies etiquettes), anti-conservateur des que n n'est pas grand devant d.
Le rang retourne est borne inferieurement par 1 ; info["r_tested"] donne le rang teste (0 possible).
Chaque valeur propre est comparee au quantile 95 % du MAXIMUM sous permutation : le niveau n'est
garanti que pour l'hypothese nulle globale (aucune structure), c'est une regle conservatrice au-dela.

Option supplementaire : test de Bartlett/Wilks (analytique, sans permutation) pour le rang.
"""
import numpy as np
from numpy.linalg import cholesky, eigh, norm
from scipy.linalg import cho_factor, cho_solve
from scipy.stats import chi2


# ------------------------------------------------------------------ Ledoit-Wolf sans matrice d x d
def ledoit_wolf_coef(Xc, gram=None):
```

```
    """Coefficient de Ledoit-Wolf (formule sklearn) calcule sans former Xc^T Xc.
    Xc : residus centres (n, d). gram : Xc Xc^T optionnel (n, n)."""
    n, d = Xc.shape
    sq = np.sum(Xc ** 2, axis=1)                        # ||x_i||^2
    emp_trace = np.sum(Xc ** 2, axis=0) / n
    mu = emp_trace.sum() / d
    if gram is not None:
        delta_ = np.sum(gram ** 2) / n ** 2             # ||Xc Xc^T||_F^2 = ||Xc^T Xc||_F^2
    elif n < d:
        delta_ = np.sum((Xc @ Xc.T) ** 2) / n ** 2      # dual : Gram n x n
    else:
        delta_ = np.sum((Xc.T @ Xc) ** 2) / n ** 2      # primal : d x d
    beta_ = np.sum(sq ** 2)                             # = sum(X2^T X2)
    beta = (beta_ / n - delta_) / (d * n)
    delta = (delta_ - 2.0 * mu * emp_trace.sum() + d * mu ** 2) / d
    beta = min(beta, delta)
    return (0.0 if beta == 0 else beta / delta), mu


# ------------------------------------------------------------------ solveur S^{-1} B (primal / dual)
class ShrunkSolver:
    """Applique S^{-1} avec S = (1-a) Xc^T Xc / n + a mu I  (a = 0 : S_W brut / m).
    Primal (n >= d) : Cholesky de S (d x d).  Dual (n < d) : Woodbury sur n x n."""

    def __init__(self, Xc, alpha, mu, denom=None, ridge_rel=1e-10):
        n, d = Xc.shape
        self.Xc, self.alpha, self.mu = Xc, alpha, mu
        denom = n if denom is None else denom
        self.scale = (1.0 - alpha) / denom              # S = scale * Xc^T Xc + alpha mu I
        self.diag = alpha * mu
        self.dual = n < d
        if self.dual:
            if self.diag <= 0:                          # S_W brut singulier en n < d : petit ridge
                self.diag = ridge_rel * self.scale * np.sum(Xc ** 2) / d
            K = self.scale * (Xc @ Xc.T) + self.diag * np.eye(n)
            self.cf = cho_factor(K, lower=True)
        else:
            S = self.scale * (Xc.T @ Xc) + self.diag * np.eye(d)
            if self.diag <= 0:
                S += ridge_rel * np.trace(S) / d * np.eye(d)
            self.cf = cho_factor(S, lower=True)

    def solve(self, B):
        if not self.dual:
            return cho_solve(self.cf, B)
        # Woodbury : (c I + s X^T X)^{-1} B = B/c - (s/c) X^T (c I + s X X^T)^{-1} X B
        c, s, X = self.diag, self.scale, self.Xc
        return B / c - (s / c) * (X.T @ cho_solve(self.cf, X @ B))


# ------------------------------------------------------------------ bloc ferme rapide
def fit_block_fast(X, y, shrink=True, rank="C1", n_perm=200, seed=0, alpha_bartlett=0.05, perm_metric="
    total",
                   min_rank=1, alpha=0.05, threshold="exact"):
    """Meme bloc que fit_block (ropn_auto_nobp_diag) / fit_oblique_block_closed_form,
    via la factorisation bas rang. Retourne U, W, r, info.

    Trois rangs sont distingues (tous dans info) :
      r_struct : borne structurelle min(C-1, d) ;
      r_tested : rang donne par la regle de test (peut valoir 0) ;
      r_used   : rang effectivement utilise = max(min_rank, r_tested) (= r retourne).
    Regles (parametre rank) :
      "C1"       : r_tested = r_struct (aucun test) ;
      "perm"     : chaque valeur propre est comparee au seuil exact de niveau alpha du MAXIMUM sous
    permutation
                   (niveau garanti pour l'hypothese nulle globale seulement) ;
      "seq"      : test de permutation sequentiel avec deflation : a l'etape k, la k+1-ieme valeur propre
     est
                   comparee a la plus grande valeur propre obtenue en permutant les etiquettes des
    donnees
                   dont les k premieres directions ont ete retirees ; arret au premier non-rejet ;
      "bartlett" : test sequentiel de Bartlett sur le Lambda de Wilks (asymptotique)."""
    rng = np.random.default_rng(seed)
```

```
    classes, yi = np.unique(y, return_inverse=True)
    C = len(classes); n, d = X.shape
    ns = np.bincount(yi, minlength=C).astype(float)
    mu = X.mean(0)
    Y = np.zeros((n, C)); Y[np.arange(n), yi] = 1.0
    mus = (Y.T @ X) / ns[:, None]
    Xbar = X - mu
    D = ((mus - mu).T * np.sqrt(ns)) / np.sqrt(max(C - 1, 1))       # S_B = D D^T
    Xc = X - mus[yi]

    if shrink:
        a, m = ledoit_wolf_coef(Xc)
        solver = ShrunkSolver(Xc, a, m, denom=n)                     # convention sklearn LedoitWolf
    else:
        solver = ShrunkSolver(Xc, 0.0, 0.0, denom=max(n - C, 1))     # S_W brut / (n-C)

    Z = solver.solve(D)                                              # S^{-1} D   (d x C)
    M = D.T @ Z; M = 0.5 * (M + M.T)                                 # C x C
    lam, Q = eigh(M); order = np.argsort(lam)[::-1]; lam, Q = lam[order], Q[:, order]
    r_struct = min(C - 1, d)

    if rank == "C1":
        r = r_struct
    elif rank in ("perm", "seq"):
        # D_p = Xbar^T Y_p N^{-1/2} / sqrt(C-1) pour chaque permutation, en lot.
        # Metrique du test : perm_metric="total" (defaut) utilise S_T = scatter TOTAL (retreci), qui ne
        # depend pas des etiquettes : sans retrecissement, S_T = S_W + (C-1)/n S_B (normalisations 1/n,
    1/n, 1/(C-1)),
        # donc memes vecteurs propres et lam_T = lam_W / (1 + (C-1) lam_W / n), fonction croissante de
    lam_W ;
        # avec deux retrecissements separes l'egalite n'est qu'approchee. Le test est echangeable avec
    une
        # seule factorisation. "within" (variante de comparaison) partage S_W calcule avec les VRAIES
    etiquettes : S_W exclut
        # la direction inter-classes vraie et non les directions permutees -> anti-conservateur si n <~ d
    .
        Yall = np.zeros((n, n_perm * C))
        for p in range(n_perm):
            yp = rng.permutation(yi)
            Yall[np.arange(n), p * C + yp] = 1.0
        Dall = (Xbar.T @ Yall) / np.sqrt(np.tile(ns, n_perm)) / np.sqrt(max(C - 1, 1))
        if perm_metric == "total":
            aT, mT = ledoit_wolf_coef(Xbar) if shrink else (0.0, 0.0)
            solver_T = ShrunkSolver(Xbar, aT, mT, denom=n if shrink else max(n - 1, 1))
            Z_test = solver_T.solve(D); Zall = solver_T.solve(Dall)
        else:
            Z_test = Z; Zall = solver.solve(Dall)                    # une seule factorisation (S_W)
        MT = D.T @ Z_test; MT = 0.5 * (MT + MT.T); lam_test, Q_test = eigh(MT)
        o_t = np.argsort(lam_test)[::-1]; lam_test, Q_test = lam_test[o_t], Q_test[:, o_t]
        Dp = Dall.T.reshape(n_perm, C, d); Zp = Zall.T.reshape(n_perm, C, d)
        Mp = Dp @ Zp.transpose(0, 2, 1); Mp = 0.5 * (Mp + Mp.transpose(0, 2, 1))
        null_max = np.linalg.eigvalsh(Mp)[:, -1]                     # lot de problemes C x C
        # seuil EXACT du test de permutation : avec m = floor(alpha (n_perm + 1)), on rejette si moins de
     m valeurs
        # permutees atteignent la valeur observee, c'est-a-dire si p = (1 + #{nul >= obs}) / (n_perm + 1)
     <= alpha.
        # (Le quantile empirique interpole de numpy serait decale d'un rang et donnerait un niveau reel d
    'environ 6 %.)
        # threshold="quantile" utilise le quantile empirique, convention de l'implementation de reference
     du bloc de base.
        m_rej = int(np.floor(alpha * (n_perm + 1)))
        if threshold == "quantile": null_q95 = float(np.quantile(null_max, 1.0 - alpha))
        else: null_q95 = float(np.sort(null_max)[::-1][m_rej - 1]) if m_rej >= 1 else np.inf
        p_value = float((1 + np.sum(null_max >= lam_test[0])) / (n_perm + 1))   # hypothese nulle globale
        if rank == "perm":
            r0 = int(min(r_struct, np.sum(lam_test[:r_struct] > null_q95)))  # rang teste (peut valoir 0)
        else:
            # test sequentiel avec deflation. Avec (U_k, W_k) la paire biorthogonale des k premieres
    directions
            # dans la metrique du test, (I-P_k)^T S^{-1} (I-P_k) = S^{-1} - W_k W_k^T : la statistique
    nulle de
            # l'etape k est la plus grande valeur propre de M_p - B_p^T B_p, B_p = W_k^T D_p (k x C),
    sans
```

```
            # nouvelle factorisation. La valeur observee est lam_test[k].
            Ap = Z_test.T @ Dall                                        # (C, n_perm C) = D^T S^{-1} D_p
            Ap = Ap.reshape(C, n_perm, C).transpose(1, 0, 2)            # (n_perm, C, C)
            r0, p_seq = 0, []
            for k in range(r_struct):
                if k == 0:
                    null_k = null_max
                else:
                    Bk = (Q_test[:, :k] / np.sqrt(np.clip(lam_test[:k], 1e-12, None))).T @ Ap   # (n_perm, k, C)
                    Mk = Mp - Bk.transpose(0, 2, 1) @ Bk
                    null_k = np.linalg.eigvalsh(0.5 * (Mk + Mk.transpose(0, 2, 1)))[:, -1]
                pk = float((1 + np.sum(null_k >= lam_test[k])) / (n_perm + 1)); p_seq.append(pk)
                if pk > alpha: break
                r0 = k + 1
        r = max(min_rank, r0)
    elif rank == "bartlett":
        # test sequentiel de Bartlett sur Wilks' Lambda (valeurs propres de W^{-1} B, echelle scatter)
        lam_sc = np.clip(lam[:r_struct], 0, None) * (C - 1) / max(n - C, 1)
        r = r_struct
        for k in range(r_struct):
            stat = (n - 1 - (d + C) / 2.0) * np.sum(np.log1p(lam_sc[k:]))
            df = (d - k) * (C - 1 - k)
            if chi2.sf(stat, df) > alpha_bartlett:
                r = k; break
        r0 = r; r = max(min_rank, r0)
    else:
        raise ValueError(rank)

    lam_r = np.clip(lam[:r], 1e-12, None)
    Qr = Q[:, :r]
    W = Z @ Qr / np.sqrt(lam_r)                                         # mesure : S^{-1} u
    U = D @ Qr / np.sqrt(lam_r)                                         # representation
    info = dict(r=r, eigenvalues=lam, dual=solver.dual, WtU_err=norm(W.T @ U - np.eye(r)) if r > 0 else 0.0,
                r_struct=r_struct, r_tested=r_struct if rank == "C1" else r0, r_used=r)
    if rank in ("perm", "seq"):
        info["null_q95"] = null_q95; info["eigenvalues_test"] = lam_test; info["null_max"] = null_max; info["p_value"] = p_value
        if rank == "seq": info["p_seq"] = p_seq
    return U, W, r, info


# ------------------------------------------------------------------ controle global d'une famille de tests
def fwer_select(infos, alpha=0.05):
    """Controle GLOBAL (family-wise) d'une famille de B tests de permutation menes avec les MEMES permutations
    (meme graine, memes etiquettes, par exemple un bloc par bande de frequence) : procedure min-p a un pas.
    Chaque statistique (observee ou permutee) est convertie en p-valeur dans sa propre famille, ce qui rend les
    blocs comparables ; la p-valeur ajustee d'un bloc est la proportion des tirages (observe inclus) dont la plus
    petite p-valeur sur les B blocs est <= la sienne. Sous l'hypothese nulle globale, la probabilite de retenir
    au moins un bloc est <= alpha. infos : liste des dictionnaires info de fit_block_fast(rank="perm" ou "seq").
    Retourne (masque des blocs retenus, p-valeurs ajustees)."""
    N = np.stack([i["null_max"] for i in infos]); obs = np.array([i["eigenvalues_test"][0] for i in infos])
    A = np.concatenate([obs[:, None], N], axis=1)                       # (B, n_perm + 1), colonne 0 = observe
    P = (A[:, None, :] >= A[:, :, None]).sum(2) / A.shape[1]            # P[b, j] = p-valeur du tirage j dans le bloc b
    minP = P.min(0)
    p_adj = np.array([(minP <= P[b, 0]).mean() for b in range(len(infos))])
    return p_adj <= alpha, p_adj


# ------------------------------------------------------------------ mise a jour en ligne (BCI)
def chol_rank1_update(L, x):
    """L L^T <- L L^T + x x^T en O(d^2) (L triangulaire inferieure, modifiee en place)."""
    x = x.copy(); d = len(x)
```

```
    for k in range(d):
        r = np.hypot(L[k, k], x[k]); c = r / L[k, k]; s = x[k] / L[k, k]
        L[k, k] = r
        if k + 1 < d:
            L[k+1:, k] = (L[k+1:, k] + s * x[k+1:]) / c
            x[k+1:] = c * x[k+1:] - s * L[k+1:, k]
    return L


class OnlineBlock:
    """Bloc ferme corrige mis a jour echantillon par echantillon, a shrinkage FIXE (a, mu) :
    S = (1-a)/n * sum_i (x_i - mu_{y_i})(...)^T + a mu I. Chaque nouvel exemple coute
    O(d^2) (mise a jour de Cholesky) + O(d^2 C) (C solves) + O(C^3) : pas de GEVD.
    La mise a jour rang-1 du scatter intra-classe est EXACTE : si la classe c a n_c exemples de
    moyenne m_c, ajouter x donne S_raw <- S_raw + n_c/(n_c+1) (x - m_c)(x - m_c)^T. La seule
    approximation par rapport a un reajustement complet est le gel de (a, mu) et du n de la
    normalisation entre deux reajustements."""

    def __init__(self, X, y, alpha, mu, n_classes=None):
        self.C = int(y.max()) + 1 if n_classes is None else n_classes; n, d = X.shape
        self.alpha, self.mu, self.n = alpha, mu, n
        self.sum_x = np.zeros((self.C, d)); self.cnt = np.zeros(self.C)
        self.S_raw = np.zeros((d, d))                  # sum_i (x_i - mu_{y_i})(x_i - mu_{y_i})^T
        for xi, yi in zip(X, y):
            self.sum_x[yi] += xi; self.cnt[yi] += 1
        mus = self.sum_x / self.cnt[:, None]
        Xc = X - mus[y]; self.S_raw = Xc.T @ Xc
        self._factor()

    def _S(self):
        d = self.S_raw.shape[0]
        return (1 - self.alpha) / self.n * self.S_raw + self.alpha * self.mu * np.eye(d)

    def _factor(self):
        self.L = cholesky(self._S())

    def add(self, x, c):
        """Ajoute un exemple (x, classe c) : rang-1 sur S_raw + rang-1 sur le Cholesky (echelle)."""
        mus_c = self.sum_x[c] / max(self.cnt[c], 1)
        r = np.sqrt(self.cnt[c] / (self.cnt[c] + 1.0)) * (x - mus_c)      # facteur n_c/(n_c+1) : mise a
     jour exacte
        self.S_raw += np.outer(r, r); self.sum_x[c] += x; self.cnt[c] += 1
        # S = k * S_raw + a mu I avec k = (1-a)/n ; on ne remet pas n a jour entre deux refit
        chol_rank1_update(self.L, np.sqrt((1 - self.alpha) / self.n) * r)

    def block(self):
        mu = self.sum_x.sum(0) / self.cnt.sum()
        mus = self.sum_x / self.cnt[:, None]
        D = ((mus - mu).T * np.sqrt(self.cnt)) / np.sqrt(max(self.C - 1, 1))
        Z = cho_solve((self.L, True), D)
        M = D.T @ Z; M = 0.5 * (M + M.T)
        lam, Q = eigh(M); o = np.argsort(lam)[::-1]; lam, Q = lam[o], Q[:, o]
        r = self.C - 1; lr = np.clip(lam[:r], 1e-12, None)
        return D @ Q[:, :r] / np.sqrt(lr), Z @ Q[:, :r] / np.sqrt(lr)
```

Listing 7: ropn_auto_fast_test.py — exactness and timing of the factorisation

```
"""Verification d'exactitude et mesure des gains de la factorisation (ropn_auto_fast)."""
import time
import numpy as np
from numpy.linalg import norm
from sklearn.covariance import ledoit_wolf_shrinkage
from sklearn.model_selection import train_test_split
from ropn_auto_nobp import make_multiclass, fit_oblique_block_closed_form, make_closed_form_gate,
    recurse_gated
from ropn_auto_nobp_diag import fit_block, readout_fit_predict, readout_centroid_fit_predict
from ropn_auto_fast import fit_block_fast, ledoit_wolf_coef, OnlineBlock


def proj_err(U1, W1, U2, W2):
    P1, P2 = U1 @ W1.T, U2 @ W2.T
```

```
    return norm(P1 - P2) / norm(P2)


print("=== 1. Exactitude (d=20, C=4, n=60 puis n=800) ===")
for n_per in [15, 200]:
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=n_per, kappa=50.0, sep=0.8, seed=0)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
    classes = np.unique(ytr); mus = np.stack([Xtr[ytr == c].mean(0) for c in classes])
    Xc = Xtr - mus[np.searchsorted(classes, ytr)]
    a_fast, _ = ledoit_wolf_coef(Xc); a_sk = ledoit_wolf_shrinkage(Xc, assume_centered=True)
    print(f"  n={len(ytr):3d} | coef Ledoit-Wolf : fast={a_fast:.6f} sklearn={a_sk:.6f} | ecart {abs(
     a_fast-a_sk):.1e}")
    # corrige : reference fit_block(shrink=True, rank=C1) vs fast
    U1, W1, r1 = fit_block(Xtr, ytr, shrink=True, rank="C1")
    U2, W2, r2, info = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")
    print(f"        corrige : ||P_fast - P_ref||/||P_ref|| = {proj_err(U2, W2, U1, W1):.1e} | "
          f"W^T U - I : {info['WtU_err']:.1e} (sans reblanchiment) | r={r2}")
    # base : reference fit_oblique_block_closed_form (S_W brut si n>d, permutation) vs fast
    U1, W1, dg = fit_oblique_block_closed_form(Xtr, ytr, seed=0)
    U2, W2, r2, info = fit_block_fast(Xtr, ytr, shrink=False, rank="perm", seed=0, perm_metric="within",
     threshold="quantile")   # meme test que la reference (S_W partage) ; le defaut est "total"
    print(f"        base    : ||P_fast - P_ref||/||P_ref|| = {proj_err(U2, W2, U1, W1):.1e} | "
          f"r*: ref={dg['r_star']} fast={r2} | bartlett r={fit_block_fast(Xtr, ytr, shrink=True, rank='
     bartlett')[2]}")

print("\n=== 2. Forme duale (n < d) : coherence primal/dual et precision (d=200, n=15/classe) ===")
X, y, *_ = make_multiclass(n_classes=4, d=200, n_per=15, kappa=50.0, sep=1.5, seed=1)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=1)
U1, W1, r1 = fit_block(Xtr, ytr, shrink=True, rank="C1")              # reference (matrice 200x200)
U2, W2, r2, info = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")  # dual (n_train=42 < 200)
print(f"  dual={info['dual']} | ||P_fast - P_ref||/||P_ref|| = {proj_err(U2, W2, U1, W1):.1e}")
for name, (U, W) in {"ref": (U1, W1), "fast": (U2, W2)}.items():
    s, b = make_closed_form_gate(U, W, Xtr)
    acc = (readout_centroid_fit_predict(recurse_gated(U, W, s, b, Xtr, 4) @ W, ytr,
                                        recurse_gated(U, W, s, b, Xte, 4) @ W) == yte).mean()
    print(f"  acc {name:4s} = {acc*100:.1f}%")

print("\n=== 3. Temps (fit du bloc seul ; corrige : min de 3 executions apres chauffe ; base : 1
     execution apres chauffe ; base ref non mesuree a d=2080) ===")
print(f"  {'d':>5} {'n_train':>8} {'C':>3} | {'corr. ref (GEVD pleine)':>24} {'corr. fast':>11} {'gain
     ':>6} | "
      f"{'base ref (+200 perm)':>21} {'base fast':>10} {'gain':>6}")
for d, n_per, C in [(20, 200, 4), (64, 120, 10), (200, 100, 4), (784, 100, 10), (2080, 30, 2), (2080,
     150, 4)]:
    X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=50.0, sep=1.0, seed=0)
    def tm(fn, k=3):
        fn()                                    # chauffe (threads BLAS)
        ts = []
        for _ in range(k):
            t0 = time.perf_counter(); fn(); ts.append(time.perf_counter() - t0)
        return np.min(ts)
    t_ref_c = tm(lambda: fit_block(X, y, shrink=True, rank="C1"))
    t_fast_c = tm(lambda: fit_block_fast(X, y, shrink=True, rank="C1"))
    if d <= 784:
        t_ref_b = tm(lambda: fit_oblique_block_closed_form(X, y, seed=0), k=1)
    else:
        t_ref_b = float("nan")      # ~200 GEVD 2080x2080 : plusieurs minutes, non mesure
    t_fast_b = tm(lambda: fit_block_fast(X, y, shrink=False, rank="perm", seed=0, perm_metric="within",
     threshold="quantile"), k=1)
    print(f"  {d:5d} {len(y):8d} {C:3d} | {t_ref_c:24.3f} {t_fast_c:11.4f} {t_ref_c/t_fast_c:6.0f}x | "
          f"{t_ref_b:21.2f} {t_fast_b:10.3f} {t_ref_b/t_fast_b:6.0f}x")


print("\n=== 4. Mise a jour en ligne (OnlineBlock) : exactitude et cout ===")
def P_of(U, W): return U @ W.T
X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=100, kappa=50.0, sep=0.8, seed=0)
perm = np.random.default_rng(0).permutation(len(y)); X, y = X[perm], y[perm]
n0, m_add = 200, 100
classes = np.unique(y[:n0]); mus = np.stack([X[:n0][y[:n0] == c].mean(0) for c in classes])
a0, mu0 = ledoit_wolf_coef(X[:n0] - mus[y[:n0]])
ob = OnlineBlock(X[:n0], y[:n0], a0, mu0, n_classes=4)
for xi, yi in zip(X[n0:n0 + m_add], y[n0:n0 + m_add]): ob.add(xi, int(yi))
Uo, Wo = ob.block()
```

```
# (a) meme (a, mu, n) geles, scatter recalcule en bloc : verifie l'exactitude des mises a jour rang-1
ob_ref = OnlineBlock(X[:n0 + m_add], y[:n0 + m_add], a0, mu0, n_classes=4); ob_ref.n = n0; ob_ref._factor(); Ur, Wr = ob_ref.block()
print(f"  d=20, {n0} exemples puis {m_add} ajouts : ecart au recalcul en bloc a (a, mu, n) geles = {norm(P_of(Uo, Wo) - P_of(Ur, Wr)) / norm(P_of(Ur, Wr)):.1e}")
# (b) ecart au reajustement complet (a, mu, n remis a jour)
Uf, Wf, _, _ = fit_block_fast(X[:n0 + m_add], y[:n0 + m_add], shrink=True, rank="C1")
print(f"  ecart au reajustement complet (Ledoit-Wolf et n remis a jour)                 = {norm(P_of(Uo, Wo) - P_of(Uf, Wf)) / norm(P_of(Uf, Wf)):.1e}")
# (c) cout par exemple a d = 2080 (n = 600, C = 4) contre un reajustement complet rapide
X, y, *_ = make_multiclass(n_classes=4, d=2080, n_per=150, kappa=50.0, sep=1.0, seed=0)
classes = np.unique(y); mus = np.stack([X[y == c].mean(0) for c in classes]); a0, mu0 = ledoit_wolf_coef(X - mus[y])
ob = OnlineBlock(X[:-5], y[:-5], a0, mu0, n_classes=4); ts = []
for xi, yi in zip(X[-5:], y[-5:]):
    t0 = time.perf_counter(); ob.add(xi, int(yi)); ob.block(); ts.append(time.perf_counter() - t0)
t0 = time.perf_counter(); fit_block_fast(X, y, shrink=True, rank="C1"); t_full = time.perf_counter() - t0
print(f"  d=2080, n=600 : ajout d'un exemple + bloc = {1000*np.median(ts):.0f} ms (boucle Python rang-1) ; reajustement complet rapide = {1000*t_full:.0f} ms")


print("\n=== 5. Contre une LDA specialisee (scikit-learn) : ajustement seul, min de 3 executions apres chauffe (s) ===")
print("    le bloc rapide et LDA(lsqr, shrinkage='auto') estiment le meme type de modele (LDA a covariance retrecie) ; LDA(svd) est sans retrecissement")
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as SkLDA
print(f"  {'d':>5} {'n_train':>8} {'C':>3} | {'bloc rapide':>12} {'LDA lsqr+LW':>12} {'LDA eigen+LW':>13} {'LDA svd':>9} | {'lsqr / bloc':>12}")
for d, n_per, C in [(20, 200, 4), (64, 120, 10), (200, 100, 4), (784, 100, 10), (2080, 30, 2), (2080, 150, 4)]:
    X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=50.0, sep=1.0, seed=0)
    def tm(fn, k=3):
        fn(); ts = []
        for _ in range(k):
            t0 = time.perf_counter(); fn(); ts.append(time.perf_counter() - t0)
        return np.min(ts)
    t_b = tm(lambda: fit_block_fast(X, y, shrink=True, rank="C1"))
    t_l = tm(lambda: SkLDA(solver="lsqr", shrinkage="auto").fit(X, y))
    t_e = tm(lambda: SkLDA(solver="eigen", shrinkage="auto").fit(X, y)) if d <= 784 else float("nan")
    t_s = tm(lambda: SkLDA(solver="svd").fit(X, y))
    print(f"  {d:5d} {len(y):8d} {C:3d} | {t_b:12.4f} {t_l:12.4f} {t_e:13.4f} {t_s:9.4f} | {t_l/t_b:11.1f}x")
# accord des predictions entre le bloc (lecture par centroides, puis par moindres carres) et la LDA retrecie de
# scikit-learn (qui retrecit chaque covariance de classe separement : estimateur de covariance different)
agree, agree_ls = [], []
for sd in range(20):
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=215, kappa=50.0, sep=0.8, seed=sd)
    tr = np.concatenate([np.where(y == c)[0][:15] for c in range(4)]); te = np.setdiff1d(np.arange(len(y)), tr)
    U, W, r, _ = fit_block_fast(X[tr], y[tr], shrink=True, rank="C1")
    p_sk = SkLDA(solver="lsqr", shrinkage="auto").fit(X[tr], y[tr]).predict(X[te])
    agree.append((readout_centroid_fit_predict(X[tr] @ W, y[tr], X[te] @ W) == p_sk).mean())
    agree_ls.append((readout_fit_predict(X[tr] @ W, y[tr], X[te] @ W) == p_sk).mean())
print(f"  accord des predictions bloc / LDA(lsqr, shrinkage='auto') sur 800 points de test, d=20, n=15/classe, 20 tirages : "
      f"lecture par centroides {100*np.mean(agree):.1f} % (min {100*np.min(agree):.1f} %) ; lecture par moindres carres {100*np.mean(agree_ls):.1f} % (min {100*np.min(agree_ls):.1f} %)")
```

Listing 8: ropn_auto_lattice.py — back-propagation-free lattice

```
"""
Reseau en treillis de blocs ROPN-auto fast (entierement ferme, sans retropropagation).

Principe. Un bloc ferme = LDA regularisee de rang <= C-1 : empiler des blocs lineaires n'ajoute
aucune expressivite (leur composition est lineaire), et le residu (I-P)x d'un bloc a des moyennes de classe
IDENTIQUES (U engendre span(mu_c - mu)), donc un second bloc lineaire sur ce residu ne trouve rien.
La profondeur ne peut apporter quelque chose que si une NON-LINEARITE FIXE separe deux etages, et
la largeur que si les blocs d'un meme etage voient des ENTREES DIFFERENTES (transversal).

```

```
Etage 1 (transversal, parallele) : G groupes de variables (patchs / bandes / groupes aleatoires),
    un bloc fast par groupe -> codes c_g in R^{C-1}, plus un bloc global -> code c_0 et residu r_0.
Non-linearite fixe : phi(c) = [c, c^2] (expose les differences de variance) ; sur le residu,
    monomes de degre 2 des k premieres composantes principales du residu (information de
    second ordre = covariances propres a chaque classe, la ou la LDA est aveugle).
Etage 2 : un bloc fast sur la concatenation (souvent n < d : forme duale), code final -> lecture.

Tout est ferme (Cholesky/Woodbury + problemes C x C), les blocs d'un etage sont independants.
"""
import numpy as np
from ropn_auto_nobp_diag import centroid_readout_weights
from itertools import combinations_with_replacement
from ropn_auto_fast import fit_block_fast


def _quad(R):
    """Monomes de degre 2 (avec carres) des colonnes de R : d(d+1)/2 colonnes."""
    idx = list(combinations_with_replacement(range(R.shape[1]), 2))
    return np.stack([R[:, i] * R[:, j] for i, j in idx], 1)


class LatticeROPN:
    def __init__(self, n_groups=4, group_mode="contiguous", k_quad=8, use_codes_sq=True,
                 use_resid_quad=True, seed=0):
        self.G, self.mode, self.k_quad = n_groups, group_mode, k_quad
        self.use_codes_sq, self.use_resid_quad, self.seed = use_codes_sq, use_resid_quad, seed

    # ---- etage 1 ----
    def _features1(self, X, fit=False, y=None):
        d = X.shape[1]
        if fit:
            rng = np.random.default_rng(self.seed)
            idx = np.arange(d) if self.mode == "contiguous" else rng.permutation(d)
            self.groups = np.array_split(idx, self.G)
            self.blocks = [fit_block_fast(X[:, g], y, shrink=True, rank="C1")[:2] for g in self.groups]
            self.U0, self.W0, _, _ = fit_block_fast(X, y, shrink=True, rank="C1")
        codes = [X[:, g] @ W for g, (U, W) in zip(self.groups, self.blocks)]          # transversal
        c0 = X @ self.W0                                                              # global
        feats = codes + [c0]
        if self.use_codes_sq:
            feats += [c ** 2 for c in feats]
        if self.use_resid_quad:
            R = X - (X @ self.W0) @ self.U0.T                                         # residu (I-P)x
            if fit:                                                                   # ACP du residu
                Rc = R - R.mean(0); _, _, Vt = np.linalg.svd(Rc, full_matrices=False)
                self.r_mean, self.Vk = R.mean(0), Vt[:self.k_quad].T
                Rk = (R - self.r_mean) @ self.Vk; self.r_scale = Rk.std(0) + 1e-12
            Rk = ((R - self.r_mean) @ self.Vk) / self.r_scale
            feats += [_quad(Rk)]
        Z = np.concatenate(feats, 1)
        if fit:
            self.z_mean, self.z_std = Z.mean(0), Z.std(0) + 1e-12
        return (Z - self.z_mean) / self.z_std

    # ---- etage 2 + lecture ----
    def fit(self, X, y):
        Z = self._features1(X, fit=True, y=y)
        self.U2, self.W2, self.r2, self.info2 = fit_block_fast(Z, y, shrink=True, rank="C1")
        F = Z @ self.W2
        # lecture par centroides sur le code de l'etage 2 = regle LDA sur les variables de l'etage 2
        self.classes, self.Wro = centroid_readout_weights(F, y)
        self.n_features2 = Z.shape[1]
        return self

    def predict(self, X):
        F = self._features1(X) @ self.W2
        return self.classes[(np.c_[F, np.ones(len(F))] @ self.Wro).argmax(1)]

    def n_params(self):
        n = self.U0.size + sum(U.size for U, _ in self.blocks) + self.U2.size + self.Wro.size
        if self.use_resid_quad:
            n += self.Vk.size
        return int(n)
```

```python


# ------------------------------------------------------------- selection automatique de la configuration du treillis
def fit_lattice_auto(Xtr, ytr, n_groups=4, k_grid=(0, 4, 8, 16), sq_grid=(False, True), n_folds=3, seed=0):
    """Choisit (k_quad, codes^2) par validation croisee interne (chaque ajustement est ferme et coute quelques ms) : c'est la boucle grow/prune de l'auto-ROPN, sans gradient.
    k_quad = 0 desactive l'etage quadratique."""
    from sklearn.model_selection import StratifiedKFold
    best, best_cfg = -1.0, None
    skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=seed)
    for sq in sq_grid:
        for k in k_grid:
            cfg = dict(n_groups=n_groups, use_codes_sq=sq, use_resid_quad=(k > 0), k_quad=max(k, 1), seed=seed)
            accs = []
            for tr, va in skf.split(Xtr, ytr):
                m = LatticeROPN(**cfg).fit(Xtr[tr], ytr[tr]); accs.append((m.predict(Xtr[va]) == ytr[va]).mean())
            if np.mean(accs) > best + 1e-9:
                best, best_cfg = np.mean(accs), cfg
    return LatticeROPN(**best_cfg).fit(Xtr, ytr), best_cfg


if __name__ == "__main__":
    import time
    from sklearn.model_selection import train_test_split
    from sklearn.neural_network import MLPClassifier
    from sklearn.datasets import load_digits, load_breast_cancer, load_wine
    from sklearn.preprocessing import StandardScaler
    from ropn_auto_nobp import make_multiclass
    from ropn_auto_finetune import make_hetero
    from ropn_auto_nobp_diag import readout_centroid_fit_predict

    def prep(X, y, sd):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
        keep = Xtr.std(0) > 1e-8; sc = StandardScaler().fit(Xtr[:, keep])
        return sc.transform(Xtr[:, keep]), sc.transform(Xte[:, keep]), ytr, yte

    datasets = {}
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=10.0, sep=0.8, seed=0); datasets["gauss k=10 n=15/cl"] = (X, y)
    X, y, *_ = make_hetero(n_classes=4, d=20, n_per=60, kappa=50.0, sep=0.8, seed=0);       datasets["hetero k=50 n=60/cl"] = (X, y)
    X, y, *_ = make_hetero(n_classes=4, d=20, n_per=300, kappa=50.0, sep=0.8, seed=0);      datasets["hetero k=50 n=300/cl"] = (X, y)
    d_ = load_digits(); datasets["digits 8x8"] = (d_.data, d_.target)
    b_ = load_breast_cancer(); datasets["breast cancer"] = (b_.data, b_.target)
    w_ = load_wine(); datasets["wine"] = (w_.data, w_.target)

    configs = {
        "bloc unique (ref.)":            None,
        "treillis: transversal seul":    dict(n_groups=4, use_codes_sq=False, use_resid_quad=False),
        "treillis: transv. + [c,c^2]":   dict(n_groups=4, use_codes_sq=True,  use_resid_quad=False),
        "treillis: + quad(residu, k=8)": dict(n_groups=4, use_codes_sq=True,  use_resid_quad=True, k_quad=8),
        "treillis: + quad(residu,k=16)": dict(n_groups=4, use_codes_sq=True,  use_resid_quad=True, k_quad=16),
        "treillis AUTO (CV interne)":    "auto",
    }
    SEEDS = 5
    for dname, (X, y) in datasets.items():
        n, d, C = X.shape[0], X.shape[1], len(np.unique(y))
        print(f"\n=== {dname} (n={n}, d={d}, C={C}, {SEEDS} decoupages 70/30 d'un meme tirage) ===")
        res = {k: [] for k in list(configs) + ["MLP (backprop)"]}; times = {k: [] for k in res}; npar = {}
        for sd in range(SEEDS):
            Xtr, Xte, ytr, yte = prep(X, y, sd)
            for k, cfg in configs.items():
                t0 = time.perf_counter()
                if cfg is None:
                    U, W, r, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")
```

```
                    pred = readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W); npar.setdefault(k, []).append(U.size + (r + 1) * C)
                elif cfg == "auto":
                    m, _ = fit_lattice_auto(Xtr, ytr, seed=sd); pred = m.predict(Xte); npar.setdefault(k, []).append(m.n_params())
                else:
                    m = LatticeROPN(seed=sd, **cfg).fit(Xtr, ytr); pred = m.predict(Xte); npar.setdefault(k, []).append(m.n_params())
                times[k].append(time.perf_counter() - t0); res[k].append((pred == yte).mean())
            t0 = time.perf_counter()
            h = int(np.clip(2 * Xtr.shape[1], 32, 256))
            mlp = MLPClassifier(hidden_layer_sizes=(h,), max_iter=500, random_state=sd).fit(Xtr, ytr)
            times["MLP (backprop)"].append(time.perf_counter() - t0)
            res["MLP (backprop)"].append(mlp.score(Xte, yte))
            npar.setdefault("MLP (backprop)", []).append(sum(w.size for w in mlp.coefs_) + sum(b.size for b in mlp.intercepts_))
        for k in res:
            print(f"  {k:32s} acc = {np.mean(res[k])*100:5.1f} +/- {np.std(res[k])*100:4.1f} | "
                  f"{np.mean(times[k])*1000:7.1f} ms | {np.mean(npar[k]):7.0f} param. (moyenne sur les decoupages)")
```

Listing 9: ropn_auto_bioblique.py — doubly oblique geometry (Kronecker projector vs Kronecker metric)

```
"""
Geometrie DOUBLEMENT oblique : projecteur de Kronecker P = P_R (x) P_L pour des entrees
matricielles X (canaux x temps), avec un projecteur oblique ferme a gauche (espace des
canaux, W_L = S_L^{-1} U_L) et un a droite (espace temporel, W_R = S_R^{-1} U_R).

    code  C = W_L^T X W_R   (r_L x r_R),   P vec(X) = vec(U_L C U_R^T)
    (P_R (x) P_L)^2 = P_R^2 (x) P_L^2 = P_R (x) P_L   -> idempotent, rang r_L r_R,
    det(I + P/L) = (1+1/L)^{r_L r_R}.

Ajustement ferme par moindres carres alternes (2-3 passes), chaque passe = deux blocs fast
(GEVD C x C) sur c et t dimensions au lieu d'une sur c*t : O(n(c^3 + t^3)) vs O(n (ct)^2 + (ct)^3),
et surtout O(c^2 + t^2) statistiques a estimer au lieu de O(c^2 t^2).
Deuxieme variante (KroneckerMetricBlock) : la METRIQUE est de Kronecker, la representation reste pleine.
Comparaison : bloc vectorise (d = c t, forme duale), les deux variantes de Kronecker et un MLP.
"""
import time
import numpy as np
from numpy.linalg import qr, cholesky
from ropn_auto_fast import fit_block_fast


def make_matrix_data(C=3, c=16, t=32, n_per=20, kappa_s=30.0, kappa_t=30.0, sep=1.0, seed=0):
    rng = np.random.default_rng(seed)
    def cov(k, d):
        eig = np.logspace(0, np.log10(k), d); eig /= eig.mean()
        Q, _ = qr(rng.standard_normal((d, d))); return Q @ np.diag(eig) @ Q.T
    Sig_s, Sig_t = cov(kappa_s, c), cov(kappa_t, t)
    Ls, Lt = cholesky(Sig_s), cholesky(Sig_t)
    # motifs de classe rang-2 spatio-temporels (moyennes matricielles)
    Ms = [sep * rng.standard_normal((c, 2)) @ rng.standard_normal((2, t)) / np.sqrt(2) for _ in range(C)]
    X = np.zeros((C * n_per, c, t)); y = np.zeros(C * n_per, dtype=int)
    for k in range(C):
        for i in range(n_per):
            X[k * n_per + i] = Ms[k] + Ls @ rng.standard_normal((c, t)) @ Lt.T   # bruit Sig_s (x) Sig_t
            y[k * n_per + i] = k
    return X, y


class BiObliqueBlock:
    """Bloc doublement oblique ferme (alternance gauche/droite)."""
    def __init__(self, n_iter=3): self.n_iter = n_iter

    def fit(self, X, y):
        n, c, t = X.shape
        Cn = len(np.unique(y))
        # init droite : bloc fast sur les moyennes temporelles (t-dim) de chaque essai
        self.W_R = np.eye(t)[:, :min(t, Cn - 1)]          # depart : premieres colonnes canoniques
```

```
        for _ in range(self.n_iter):
            # gauche : echantillons = colonnes de X_i W_R (c-dim), etiquette repetee
            XR = np.einsum("ncT,Tk->nck", X, self.W_R)      # (n, c, r_R)
            Zl = XR.transpose(0, 2, 1).reshape(-1, c); yl = np.repeat(y, XR.shape[2])
            self.U_L, self.W_L, _, _ = fit_block_fast(Zl, yl, shrink=True, rank="C1")
            # droite : echantillons = lignes de W_L^T X_i (t-dim)
            LX = np.einsum("ck,nct->nkt", self.W_L, X)      # (n, r_L, t)
            Zr = LX.reshape(-1, t); yr = np.repeat(y, LX.shape[1])
            self.U_R, self.W_R, _, _ = fit_block_fast(Zr, yr, shrink=True, rank="C1")
        F = self.code(X)
        from ropn_auto_nobp_diag import centroid_readout_weights
        self.classes, self.Wro = centroid_readout_weights(F, y)        # lecture par centroides sur le code
        return self

    def code(self, X):
        return np.einsum("ck,nct,tl->nkl", self.W_L, X, self.W_R).reshape(len(X), -1)

    def predict(self, X):
        F = self.code(X); return self.classes[(np.c_[F, np.ones(len(F))] @ self.Wro).argmax(1)]

    def n_params(self): return int(self.U_L.size + self.U_R.size + self.Wro.size)

    def check_idempotent(self):
        P_L = self.U_L @ self.W_L.T; P_R = self.U_R @ self.W_R.T
        P = np.kron(P_R, P_L)
        return np.linalg.norm(P @ P - P) / np.linalg.norm(P), np.linalg.matrix_rank(P)


# ------------------------------------------------------------- variante 2 : METRIQUE doublement oblique
class KroneckerMetricBlock:
    """Bloc oblique ferme dont la METRIQUE de mesure est de Kronecker : M = Sig_T^{-1} (x) Sig_S^{-1}.
    Representation : U = vec(M_c - M_bar) (rang C-1 complet, aucune restriction de Kronecker) ;
    mesure : w_c = vec(Sig_S^{-1} (M_c - M_bar) Sig_T^{-1}), c'est-a-dire oblique en espace
    (canaux) ET en temps. Sig_S, Sig_T estimees par flip-flop (Dutilleul) sur les residus,
    avec ridge : O(c^2 + t^2) statistiques au lieu de O((ct)^2). Jamais de matrice ct x ct."""
    def __init__(self, n_ff=5, ridge=0.05): self.n_ff, self.ridge = n_ff, ridge

    def fit(self, X, y):
        n, c, t = X.shape
        self.classes, yi = np.unique(y, return_inverse=True); Cn = len(self.classes)
        ns = np.bincount(yi, minlength=Cn).astype(float)
        Ms = np.stack([X[yi == k].mean(0) for k in range(Cn)]); Mbar = X.mean(0)
        R = X - Ms[yi]
        # flip-flop
        Sig_T_inv = np.eye(t)
        for _ in range(self.n_ff):
            Sig_S = np.einsum("nct,tu,nvu->cv", R, Sig_T_inv, R) / (n * t)
            Sig_S += self.ridge * np.trace(Sig_S) / c * np.eye(c); Sig_S_inv = np.linalg.inv(Sig_S)
            Sig_T = np.einsum("nct,cu,nuv->tv", R, Sig_S_inv, R) / (n * c)
            Sig_T += self.ridge * np.trace(Sig_T) / t * np.eye(t); Sig_T_inv = np.linalg.inv(Sig_T)
        self.Sig_S_inv, self.Sig_T_inv = Sig_S_inv, Sig_T_inv
        # bloc fast avec solveur de Kronecker : S^{-1} vec(A) = vec(Sig_S^{-1} A Sig_T^{-1})
        Dm = np.stack([(Ms[k] - Mbar) * np.sqrt(ns[k]) for k in range(Cn)]) / np.sqrt(max(Cn - 1, 1))  # (C, c, t)
        Zm = np.einsum("cu,kut,tv->kcv", Sig_S_inv, Dm, Sig_T_inv)
        # (C, c, t)
        D = Dm.reshape(Cn, -1).T; Z = Zm.reshape(Cn, -1).T
        M = D.T @ Z; M = 0.5 * (M + M.T)
        lam, Q = np.linalg.eigh(M); o = np.argsort(lam)[::-1]; lam, Q = lam[o], Q[:, o]
        r = Cn - 1; lr = np.clip(lam[:r], 1e-12, None)
        self.W = Z @ Q[:, :r] / np.sqrt(lr); self.U = D @ Q[:, :r] / np.sqrt(lr)
        F = X.reshape(n, -1) @ self.W
        from ropn_auto_nobp_diag import centroid_readout_weights
        _, self.Wro = centroid_readout_weights(F, yi)                  # regle LDA de covariance de Kronecker
        return self

    def predict(self, X):
        F = X.reshape(len(X), -1) @ self.W
        return self.classes[(np.c_[F, np.ones(len(F))] @ self.Wro).argmax(1)]

```

```
    def n_params(self): return int(self.U.size + self.Wro.size)


if __name__ == "__main__":
    from sklearn.model_selection import train_test_split
    from sklearn.neural_network import MLPClassifier
    from ropn_auto_nobp_diag import readout_centroid_fit_predict
    SEEDS = 8
    print(f"=== donnees matricielles 16 x 32 (d = 512), C = 3, motifs de classe de rang 2, bruit Sig_S (x ) Sig_T (kappa = 30 de chaque cote), {SEEDS} graines, 70/30 stratifie ===")
    print(f"{'separation':>10s} {'n/classe':>8s} {'n_train':>7s} | {'bloc vectorise':>15s} | {'metrique Kronecker':>18s} | {'projecteur Kronecker':>20s} | {'MLP (64)':>12s}")
    for sep, n_per in [(0.25, 20), (0.25, 60), (0.15, 60), (0.15, 200), (0.10, 200)]:
        res = {k: [] for k in ("vec", "kmet", "kproj", "mlp")}
        for sd in range(SEEDS):
            X, y = make_matrix_data(n_per=n_per, sep=sep, seed=sd)
            Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
            n = len(Xtr); Vtr, Vte = Xtr.reshape(n, -1), Xte.reshape(len(Xte), -1)
            U, W, r, _ = fit_block_fast(Vtr, ytr, shrink=True, rank="C1")          # bloc vectorise (d = c t, forme duale)
            res["vec"].append((readout_centroid_fit_predict(Vtr @ W, ytr, Vte @ W) == yte).mean())
            res["kmet"].append((KroneckerMetricBlock().fit(Xtr, ytr).predict(Xte) == yte).mean())
            m = BiObliqueBlock().fit(Xtr, ytr); res["kproj"].append((m.predict(Xte) == yte).mean())
            if sd == 0 and (sep, n_per) == (0.25, 20):
                err, rk = m.check_idempotent(); idem = (err, rk, m.W_L.shape[1] * m.W_R.shape[1])
            res["mlp"].append(MLPClassifier(hidden_layer_sizes=(64,), max_iter=500, random_state=sd).fit(Vtr, ytr).score(Vte, yte))
        f = lambda v: f"{100*np.mean(v):5.1f} +/- {100*np.std(v):4.1f}"
        print(f"{sep:10.2f} {n_per:8d} {n:7d} | {f(res['vec']):>15s} | {f(res['kmet']):>18s} | {f(res['kproj']):>20s} | {f(res['mlp']):>12s}")
    print(f"idempotence du projecteur de Kronecker P_R (x) P_L : ||P^2-P||/||P|| = {idem[0]:.1e}, rang = {idem[1]} = r_L r_R = {idem[2]}")
```

Listing 10: ropn_auto_kernel.py — kernel oblique block and classical alternatives

```

"""
Bloc oblique A NOYAU (piste "cachee" dans la forme duale) et comparaison aux alternatives
classiques du MLP (SVM-RBF, gradient boosting, k-NN, LDA-shrinkage).

La forme duale de ropn_auto_fast (Woodbury sur la Gram Xc Xc^T) ne depend des donnees que
par des produits scalaires : remplacer la Gram par un noyau K = k(x_i, x_j) donne, en forme
fermee, le meme bloc oblique dans l'espace de caracteristiques phi :
    representation U = Phi^T B          (moyennes de classe dans l'espace phi, rang C-1)
    mesure         W = S^{-1} U = Phi^T A,  S = s Phi_c^T Phi_c + c I   (Woodbury -> n x n)
    code           W^T phi(x) = alpha^T k(X, x)
C'est l'analyse discriminante de Fisher a noyau (Mika et al. 1999) relue comme projecteur
oblique : W^T U = I_r automatiquement, idempotence exacte, rang C-1, capacite gouvernee par n
(pas par la profondeur), zero gradient. Hyperparametres : gamma (heuristique de la mediane
x facteur) et ridge, choisis par validation croisee interne.
"""
import numpy as np
from numpy.linalg import eigh
from scipy.linalg import cho_factor, cho_solve
from scipy.spatial.distance import cdist


def rbf(A, B, gamma):
    return np.exp(-gamma * cdist(A, B, "sqeuclidean"))


class KernelObliqueBlock:
    def __init__(self, gamma_scale=1.0, ridge=1e-2):
        self.gamma_scale, self.ridge = gamma_scale, ridge

    def fit(self, X, y):
        n = len(X)
        self.X = X
        self.classes, yi = np.unique(y, return_inverse=True); C = len(self.classes)
        ns = np.bincount(yi, minlength=C).astype(float)
        d2 = cdist(X, X, "sqeuclidean")
```

```
        self.gamma = self.gamma_scale / np.median(d2[d2 > 0])
        K = np.exp(-self.gamma * d2)
        Y = np.zeros((n, C)); Y[np.arange(n), yi] = 1.0
        H = Y @ np.diag(1.0 / ns) @ Y.T                      # projecteur sur les moyennes de classe
        I = np.eye(n); Hc = I - H
        Kc = Hc @ K @ Hc                                     # Gram des residus intra-classe (Phi_c Phi_c^T)
        B = (I - np.ones((n, n)) / n) @ Y / np.sqrt(ns) / np.sqrt(max(C - 1, 1))   # D = Phi^T B
        s = 1.0 / n; c = self.ridge * np.trace(Kc) / n       # S = s Phi_c^T Phi_c + c I
        cf = cho_factor(c * I + s * Kc, lower=True)
        # S^{-1} D = Phi^T [ B/c - (s/c) Hc (cI + sKc)^{-1} Hc K B ]
        A = B / c - (s / c) * (Hc @ cho_solve(cf, Hc @ (K @ B)))
        M = B.T @ K @ A; M = 0.5 * (M + M.T)                 # D^T S^{-1} D  (C x C)
        lam, Q = eigh(M); o = np.argsort(lam)[::-1]; lam, Q = lam[o], Q[:, o]
        r = C - 1; lr = np.clip(lam[:r], 1e-12, None)
        self.alpha = A @ Q[:, :r] / np.sqrt(lr)              # W = Phi^T alpha
        self.beta = B @ Q[:, :r] / np.sqrt(lr)               # U = Phi^T beta
        self.WtU = self.alpha.T @ K @ self.beta              # doit valoir I_r
        F = K @ self.alpha
        from ropn_auto_nobp_diag import centroid_readout_weights
        _, self.Wro = centroid_readout_weights(F, yi)        # lecture par centroides sur le code noyau
        return self

    def predict(self, X):
        F = rbf(X, self.X, self.gamma) @ self.alpha
        return self.classes[(np.c_[F, np.ones(len(F))] @ self.Wro).argmax(1)]


def fit_kernel_auto(Xtr, ytr, gammas=(0.25, 1.0, 4.0), ridges=(1e-3, 1e-2, 1e-1), n_folds=3, seed=0):
    from sklearn.model_selection import StratifiedKFold
    skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=seed)
    best, cfg = -1, None
    for g in gammas:
        for r in ridges:
            accs = [(KernelObliqueBlock(g, r).fit(Xtr[tr], ytr[tr]).predict(Xtr[va]) == ytr[va]).mean()
                    for tr, va in skf.split(Xtr, ytr)]
            if np.mean(accs) > best + 1e-9: best, cfg = np.mean(accs), (g, r)
    return KernelObliqueBlock(*cfg).fit(Xtr, ytr), cfg


if __name__ == "__main__":
    import time, warnings; warnings.filterwarnings("ignore")
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.datasets import load_digits, load_breast_cancer, load_wine
    from sklearn.neural_network import MLPClassifier
    from sklearn.svm import SVC
    from sklearn.ensemble import HistGradientBoostingClassifier
    from sklearn.neighbors import KNeighborsClassifier
    from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
    from ropn_auto_nobp import make_multiclass
    from ropn_auto_finetune import make_hetero
    from ropn_auto_fast import fit_block_fast
    from ropn_auto_nobp_diag import readout_fit_predict, readout_centroid_fit_predict
    from ropn_auto_lattice import fit_lattice_auto

    def prep(X, y, sd):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
        keep = Xtr.std(0) > 1e-8; sc = StandardScaler().fit(Xtr[:, keep])
        return sc.transform(Xtr[:, keep]), sc.transform(Xte[:, keep]), ytr, yte

    ds = {}
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=10.0, sep=0.8, seed=0); ds["gauss k=10 n=15/cl"] = (X, y)
    X, y, *_ = make_hetero(n_classes=4, d=20, n_per=60, kappa=50.0, sep=0.8, seed=0);       ds["hetero k=50 n=60/cl"] = (X, y)
    d_ = load_digits(); ds["digits 8x8"] = (d_.data, d_.target)
    b_ = load_breast_cancer(); ds["breast cancer"] = (b_.data, b_.target)
    w_ = load_wine(); ds["wine"] = (w_.data, w_.target)

    from sklearn.model_selection import GridSearchCV, StratifiedKFold
    from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis as QDA
```

```
106
107     def svm_cv(Xtr, ytr, Xte, sd):
108         """SVM RBF, (C, gamma) par validation croisee interne (3 plis)."""
109         d = Xtr.shape[1]; grid = {"C": [0.1, 1.0, 10.0, 100.0], "gamma": [0.3 / d, 1.0 / d, 3.0 / d]}
110         cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=sd)
111         return GridSearchCV(SVC(kernel="rbf"), grid, cv=cv).fit(Xtr, ytr).predict(Xte)
112
113     def qda_cv(Xtr, ytr, Xte, sd):
114         """QDA regularisee (reg_param de sklearn), choisie par validation croisee interne (3 plis)."""
115         cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=sd)
116         return GridSearchCV(QDA(), {"reg_param": [0.01, 0.03, 0.1, 0.3, 0.6, 0.9]}, cv=cv).fit(Xtr, ytr).
    predict(Xte)
117
118     methods = {
119         "bloc fast (lineaire)": lambda Xtr, ytr, Xte, sd: (lambda U, W, r, _: readout_fit_predict(Xtr @ W
    , ytr, Xte @ W))(*fit_block_fast(Xtr, ytr, shrink=True, rank="C1")),
120         "bloc fast, lecture centroide": lambda Xtr, ytr, Xte, sd: (lambda U, W, r, _:
    readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W))(*fit_block_fast(Xtr, ytr, shrink=True, rank="C1"
    )),
121         "treillis AUTO":        lambda Xtr, ytr, Xte, sd: fit_lattice_auto(Xtr, ytr, seed=sd)[0].predict(
    Xte),
122         "bloc oblique a NOYAU (CV)": lambda Xtr, ytr, Xte, sd: fit_kernel_auto(Xtr, ytr, seed=sd)[0].
    predict(Xte),
123         "LDA-shrinkage":        lambda Xtr, ytr, Xte, sd: LDA(solver="lsqr", shrinkage="auto").fit(Xtr,
    ytr).predict(Xte),
124         "SVM RBF (C=1)":        lambda Xtr, ytr, Xte, sd: SVC(kernel="rbf", C=1.0, gamma="scale").fit(Xtr
    , ytr).predict(Xte),
125         "SVM RBF (C=10)":       lambda Xtr, ytr, Xte, sd: SVC(kernel="rbf", C=10.0, gamma="scale").fit(
    Xtr, ytr).predict(Xte),
126         "SVM RBF (CV interne)": svm_cv,
127         "QDA regularisee (CV interne)": qda_cv,
128         "HistGradientBoosting": lambda Xtr, ytr, Xte, sd: HistGradientBoostingClassifier(random_state=sd)
    .fit(Xtr, ytr).predict(Xte),
129         "k-NN (k=5)":           lambda Xtr, ytr, Xte, sd: KNeighborsClassifier(5).fit(Xtr, ytr).predict(
    Xte),
130         "MLP (backprop)":       lambda Xtr, ytr, Xte, sd: MLPClassifier(hidden_layer_sizes=(int(np.clip(2
     * Xtr.shape[1], 32, 256)),), max_iter=500, random_state=sd).fit(Xtr, ytr).predict(Xte),
131     }
132     SEEDS = 5
133     for dname, (X, y) in ds.items():
134         print(f"\n=== {dname} (n={X.shape[0]}, d={X.shape[1]}, C={len(np.unique(y))}, {SEEDS} decoupages
    70/30 d'un meme tirage) ===")
135         for mname, fn in methods.items():
136             accs, ts = [], []
137             try:
138                 for sd in range(SEEDS):
139                     Xtr, Xte, ytr, yte = prep(X, y, sd)
140                     t0 = time.perf_counter(); pred = fn(Xtr, ytr, Xte, sd); ts.append(time.perf_counter()
     - t0)
141                     accs.append((pred == yte).mean())
142             except Exception as e:                      # p. ex. QDA quand une classe a moins d'exemples
    que de variables
143                 print(f"  {mname:27s} echec : {type(e).__name__}"); continue
144             print(f"  {mname:30s} acc = {np.mean(accs)*100:5.1f} +/- {np.std(accs)*100:4.1f} | {np.mean(
    ts)*1000:7.0f} ms")
145     # sanity : biorthogonalite dans l'espace de caracteristiques
146     Xtr, Xte, ytr, yte = prep(*ds["digits 8x8"], 0)
147     m = KernelObliqueBlock().fit(Xtr, ytr)
148     print(f"\n[noyau] ||W^T U - I_r|| dans l'espace phi = {np.linalg.norm(m.WtU - np.eye(m.WtU.shape[0]))
    :.1e}")
```

Listing 11: ropn_auto_charlm.py — next-character prediction

```
1  """
2  Mini-experience "modele de langage" : prediction du caractere suivant a partir des k precedents,
3  pour situer les blocs fermes (LDA oblique = bloc fast, treillis) face a un MLP entraine par
4  retropropagation, sur une tache de type LM (C = taille de l'alphabet, entree = contexte one-hot).
5
6  Corpus : fichier fige ropn_corpus.txt (environ 1,5 x 10^5 caracteres : code source Python et commentaires
7  en francais, alphabet reduit a 35 symboles : lettres minuscules, espace, ponctuation courante). Le corpus
8  est fige pour que les resultats ne dependent pas de l'etat des scripts ; build_corpus() le reconstruit a
```

```
partir de fichiers sources. Metriques : precision top-1, perplexite (exp de la log-perte) quand le modele
fournit des probabilites.
"""
import re, time
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from ropn_auto_fast import fit_block_fast
from ropn_auto_lattice import fit_lattice_auto


def load_corpus(path="ropn_corpus.txt"):
    """corpus fige (deja normalise)."""
    return open(path, encoding="utf-8").read()


def build_corpus(paths):
    """normalisation utilisee pour construire ropn_corpus.txt a partir d'une liste de fichiers texte."""
    txt = ""
    for pth in paths:
        txt += open(pth, encoding="utf-8").read().lower() + " "
    txt = re.sub(r"\\[a-z]+", " ", txt)                    # commandes LaTeX -> espace
    txt = re.sub("[^a-z\u00e0\u00e2\u00e7\u00e9\u00e8\u00ea\u00eb\u00ee\u00ef\u00f4\u00fb\u00f9\u00fc\
     u00ff\u0153 ,.;:'()-]+", " ", txt)  # accents en echappements unicode (source ASCII)
    txt = re.sub(r"\s+", " ", txt)
    return txt


def make_dataset(txt, k=4, n_max=30000, seed=0):
    chars = sorted(set(txt)); idx = {c: i for i, c in enumerate(chars)}; V = len(chars)
    ids = np.array([idx[c] for c in txt])
    n = len(ids) - k
    rng = np.random.default_rng(seed); pos = rng.choice(n, size=min(n, n_max), replace=False)
    X = np.zeros((len(pos), k * V), dtype=np.float32)
    for j in range(k):
        X[np.arange(len(pos)), j * V + ids[pos + j]] = 1.0
    y = ids[pos + k]
    keep = np.isin(y, np.where(np.bincount(y, minlength=V) >= 30)[0])    # classes rares retirees
    X, y = X[keep], y[keep]
    classes, y = np.unique(y, return_inverse=True)                        # etiquettes contigues
    return X, y, len(classes), V


def perplexity(P, y):
    p = np.clip(P[np.arange(len(y)), y], 1e-12, None)
    return float(np.exp(-np.mean(np.log(p))))


if __name__ == "__main__":
    txt = load_corpus(); print(f"corpus : {len(txt)} caracteres, alphabet de {len(set(txt))} symboles")
    for k in [2, 4]:
        X, y, V, Va = make_dataset(txt, k=k)    # V = classes de sortie, Va = taille de l'alphabet (one-
     hot)
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=0, stratify=y)
        # aligner les classes (certaines lettres rares peuvent manquer du train)
        print(f"\n=== contexte k={k} : d = {X.shape[1]} (one-hot), C = {V} classes, n_train = {len(ytr)}
     ===")
        # 1. n-gramme (comptage) : baseline classique
        t0 = time.perf_counter()
        ctx_tr = Xtr.reshape(len(Xtr), k, Va).argmax(2); ctx_te = Xte.reshape(len(Xte), k, Va).argmax(2)
        table = {}
        for c, t in zip(map(tuple, ctx_tr), ytr): table.setdefault(c, np.zeros(V))[t] += 1
        uni = np.bincount(ytr, minlength=V) + 1.0
        P = np.stack([(table.get(tuple(c), np.zeros(V)) + 0.5 * uni / uni.sum()) for c in ctx_te]); P /=
     P.sum(1, keepdims=True)
        print(f"  {'n-gramme (comptage, lissage)':34s} acc = {(P.argmax(1) == yte).mean()*100:5.1f}  ppl
     = {perplexity(P, yte):6.2f}  | {time.perf_counter()-t0:5.1f} s")
        # 2. bloc fast = LDA oblique (lecture par centroides sur le code)
        t0 = time.perf_counter()
        U, W, r, info = fit_block_fast(Xtr.astype(float), ytr, shrink=True, rank="C1")
        from ropn_auto_nobp_diag import readout_centroid_fit_predict, readout_fit_predict
```

```
        pred = readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W)
        print(f"  {'bloc fast (LDA oblique, rang C-1)':34s} acc = {(pred == yte).mean()*100:5.1f}  ppl =
        --   | {time.perf_counter()-t0:5.1f} s  (r={r}, dual={info['dual']})")
        pred_ls = readout_fit_predict(Xtr @ W, ytr, Xte @ W)            # meme code, lecture par moindres
     carres (comparaison)
        print(f"  {'bloc fast, lecture MC (comparaison)':34s} acc = {(pred_ls == yte).mean()*100:5.1f}
    ppl =    --   |")
        # 3. LDA-shrinkage sklearn (probabilites -> perplexite)
        t0 = time.perf_counter()
        lda = LDA(solver="lsqr", shrinkage="auto").fit(Xtr, ytr)
        P = np.zeros((len(Xte), V)); P[:, lda.classes_] = lda.predict_proba(Xte)
        print(f"  {'LDA-shrinkage (proba)':34s} acc = {(P.argmax(1) == yte).mean()*100:5.1f}  ppl = {
    perplexity(P, yte):6.2f}  | {time.perf_counter()-t0:5.1f} s")
        # 4. regression logistique (lineaire, gradient convexe)
        t0 = time.perf_counter()
        lr = LogisticRegression(max_iter=300, C=1.0).fit(Xtr, ytr)
        P = np.zeros((len(Xte), V)); P[:, lr.classes_] = lr.predict_proba(Xte)
        print(f"  {'regression logistique (lineaire)':34s} acc = {(P.argmax(1) == yte).mean()*100:5.1f}
    ppl = {perplexity(P, yte):6.2f}  | {time.perf_counter()-t0:5.1f} s")
        # 5. treillis AUTO (ferme, non lineaire fixe)
        if k == 2:
            t0 = time.perf_counter()
            m, cfg = fit_lattice_auto(Xtr.astype(float), ytr, n_groups=k, seed=0)
            print(f"  {'treillis AUTO (ferme)':34s} acc = {(m.predict(Xte.astype(float)) == yte).mean()
    *100:5.1f}  ppl =    --   | {time.perf_counter()-t0:5.1f} s  (cfg={cfg['k_quad'] if cfg['
    use_resid_quad'] else 0}, c^2={cfg['use_codes_sq']})")
        # 6. MLP (backprop)
        t0 = time.perf_counter()
        mlp = MLPClassifier(hidden_layer_sizes=(256,), max_iter=60, random_state=0).fit(Xtr, ytr)   # 60
     epoques au plus (budget fixe, non converge)
        P = np.zeros((len(Xte), V)); P[:, mlp.classes_] = mlp.predict_proba(Xte)
        print(f"  {'MLP 256 (backprop)':34s} acc = {(P.argmax(1) == yte).mean()*100:5.1f}  ppl = {
    perplexity(P, yte):6.2f}  | {time.perf_counter()-t0:5.1f} s")
```

Listing 12: ropn_auto_crossgram.py — shrinkage path to the Euclidean cross-Gram

```
"""
Cross-Gram euclidien vs cross-Gram de Fisher.

Deux objets portent le nom de "cross-Gram" :
  (a) le couplage G = W^T U (r x r) du projecteur P = U G^{-1} W^T. Le projecteur ne depend que des
      sous-espaces (im U, ker W^T) : changer G ne change pas P (invariance de jauge). Remplacer G^{-1}
      par une matrice A quelconque donne U A W^T, carte de rang r NON idempotente (LoRA) : on perd
      les garanties sans gagner de sous-espace.
  (b) la covariance croisee entree/cible S_XY = X_c^T Y_c (d x C). Le bloc de Fisher = CCA(X, Y) :
      directions de mesure W = S_XX^{-1} S_XY ... (metrique intra-classe). Le cross-Gram EUCLIDIEN
      (sans metrique, W = U = vecteurs singuliers de S_XY) est la PLS-DA : projecteur ORTHOGONAL.
      Entre les deux : la metrique retrecie (S_W + lambda I)^{-1}, lambda -> infini donne PLS-DA.
Ce script mesure ou chaque extremite gagne (petits n, n << d, digits) et verifie la relation.
"""
import numpy as np
from numpy.linalg import svd
from scipy.linalg import cho_factor, cho_solve
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.cross_decomposition import PLSRegression
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_digits
from ropn_auto_nobp import make_multiclass
from ropn_auto_fast import fit_block_fast
from ropn_auto_nobp_diag import readout_centroid_fit_predict   # lecture de reference : centroides sur le
     code (regle LDA)


def fit_crossgram_block(X, y, lam, r=None):
    """Bloc oblique a metrique retrecie M = (S_W + lam * tr(S_W)/d I)^{-1} ; lam=inf -> PLS-DA (euclidien
    ).
    Retourne U, W avec W^T U = I_r (reduction C x C exacte)."""
    classes, yi = np.unique(y, return_inverse=True); C = len(classes); n, d = X.shape
    ns = np.bincount(yi, minlength=C).astype(float); mu = X.mean(0)
    mus = np.stack([X[yi == k].mean(0) for k in range(C)])
```

```
    D = ((mus - mu).T * np.sqrt(ns)) / np.sqrt(max(C - 1, 1))          # S_B = D D^T,  D ~ S_XY N^{-1/2}
    Xc = X - mus[yi]; S_W = Xc.T @ Xc / max(n - C, 1)
    if np.isinf(lam):
        Z = D.copy()                                                    # metrique euclidienne : W = U
    else:
        S = S_W + lam * np.trace(S_W) / d * np.eye(d)
        Z = cho_solve(cho_factor(S, lower=True), D)
    M = D.T @ Z; M = 0.5 * (M + M.T)
    lamb, Q = np.linalg.eigh(M); o = np.argsort(lamb)[::-1]; lamb, Q = lamb[o], Q[:, o]
    r = C - 1 if r is None else r
    lr = np.clip(lamb[:r], 1e-12, None)
    return D @ Q[:, :r] / np.sqrt(lr), Z @ Q[:, :r] / np.sqrt(lr)


def acc_block(Xtr, ytr, Xte, yte, lam):
    U, W = fit_crossgram_block(Xtr, ytr, lam)
    return (readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W) == yte).mean()


def acc_plsda(Xtr, ytr, Xte, yte, n_comp):
    classes = np.unique(ytr); Y = np.eye(len(classes))[np.searchsorted(classes, ytr)]
    pls = PLSRegression(n_components=n_comp, scale=False).fit(Xtr, Y)
    return (classes[pls.predict(Xte).argmax(1)] == yte).mean()


if __name__ == "__main__":
    def prep(X, y, sd):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
        keep = Xtr.std(0) > 1e-8; sc = StandardScaler().fit(Xtr[:, keep])
        return sc.transform(Xtr[:, keep]), sc.transform(Xte[:, keep]), ytr, yte

    cases = {}
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=10.0, sep=0.8, seed=0);   cases["gauss d=20, n=15/cl, kappa=10"] = (X, y)
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=200.0, sep=0.8, seed=0);  cases["gauss d=20, n=15/cl, kappa=200"] = (X, y)
    X, y, *_ = make_multiclass(n_classes=4, d=500, n_per=15, kappa=50.0, sep=0.6, seed=0);  cases["gauss d=500, n=15/cl (n<<d)"] = (X, y)
    d_ = load_digits(); cases["digits 8x8"] = (d_.data, d_.target)

    # verification : lam -> inf du bloc = PLS-DA (memes sous-espaces)
    Xtr, Xte, ytr, yte = prep(*cases["gauss d=20, n=15/cl, kappa=10"], 0)
    U, W = fit_crossgram_block(Xtr, ytr, np.inf)
    print(f"[verif] cross-Gram euclidien : W^T U - I = {np.linalg.norm(W.T @ U - np.eye(W.shape[1])):.1e}, "
          f"||U - W||/||U|| = {np.linalg.norm(U - W)/np.linalg.norm(U):.1e} (projecteur orthogonal)")

    lams = [("LW auto (bloc fast)", None), ("lam=0.01", 0.01), ("lam=0.1", 0.1), ("lam=1", 1.0), ("lam=10", 10.0), ("euclidien (lam=inf)", np.inf)]

    def lam_by_cv(Xtr, ytr, sd, grid=(0.01, 0.1, 1.0, 10.0), n_folds=3):
        """lambda choisi par validation croisee INTERNE a l'apprentissage (aucun acces au test)."""
        from sklearn.model_selection import StratifiedKFold
        skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=sd); best, bl = -1.0, grid[0]
        for lam in grid:
            a_ = np.mean([acc_block(Xtr[tr], ytr[tr], Xtr[va], ytr[va], lam) for tr, va in skf.split(Xtr, ytr)])
            if a_ > best + 1e-9: best, bl = a_, lam
        return bl

    print("\n(5 decoupages 70/30 stratifies d'un meme tirage ; les colonnes lam=... sont des lambda FIXES ; 'lam CV' est choisi par validation croisee interne)")
    print(f"\n{'cas':34s} " + " ".join(f"{n[:12]:>12s}" for n, _ in lams) + f" {'lam CV':>8s} {'PLS-DA(C-1)':>12s} {'MLP':>8s}")
    for cname, (X, y) in cases.items():
        res = {n: [] for n, _ in lams}; pls, mlp, cv = [], [], []
        for sd in range(5):
            Xtr, Xte, ytr, yte = prep(X, y, sd)
            for n_, lam in lams:
                if lam is None:
                    U, W, r, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")
                    res[n_].append((readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W) == yte).mean())
                else:
```

```
                    res[n_].append(acc_block(Xtr, ytr, Xte, yte, lam))
            cv.append(acc_block(Xtr, ytr, Xte, yte, lam_by_cv(Xtr, ytr, sd)))
            pls.append(acc_plsda(Xtr, ytr, Xte, yte, len(np.unique(ytr)) - 1))
            h = int(np.clip(2 * Xtr.shape[1], 32, 256))
            mlp.append(MLPClassifier(hidden_layer_sizes=(h,), max_iter=500, random_state=sd).fit(Xtr, ytr
    ).score(Xte, yte))
        print(f"{cname:34s} " + " ".join(f"{np.mean(res[n])*100:12.1f}" for n, _ in lams) + f" {np.mean(
    cv)*100:8.1f} {np.mean(pls)*100:12.1f} {np.mean(mlp)*100:8.1f}")
```

Listing 13: ropn_auto_triceptron.py — gated and ReLU layers under one trainer, constructed and learned weights

```
"""
Triceptron : unite a TROIS directions
    u (representation), w (mesure), s (porte lue HORS de span(W)).
    perceptron  : y = sigma(w^T x + b)                          (1 vecteur)
    oblique     : y = sigma(w^T x / w^T u + b)                  (2 vecteurs)
    triceptron  : y = sigma(s^T x + b_s) * (w^T x / w^T u)  -> lecture   (3 vecteurs)
Version rang r : code c = W^T x (r), portes a = sigma(S^T x + b) (k), sortie U (a (x) c) ou lecture.

Deux realisations :
  * TRI-CF (ferme, sans gradient) : (U, W) = bloc fast (rang C-1, W^T U = I) ; S = k premieres
    composantes principales du RESIDU (I-P)x (donc hors de span(W)) ; caracteristiques
    [c, c_i * a_j] ; lecture par moindres carres ; k choisi par validation croisee interne.
  * TRI-BP (retropropagation, gradients manuels numpy, verifies) : logits = V (a (x) c) + b0 avec
    c = W^T x + b_w, a = sigma(S^T x + b_s) : c'est une unite GLU (gated linear unit), i.e. la
    structure du FFN des LLM modernes (SwiGLU : W_up = W, W_gate = S, W_down = V).
    Comparee a un MLP ReLU a nombre de parametres voisin entraine par le MEME code (meme initialisation,
    meme Adam, meme nombre de pas), avec et sans saut lineaire pour les deux familles ; un MLP sklearn
    L-BFGS est rapporte en plus comme controle d'un autre optimiseur. Poids construits (theoreme) contre
    poids appris : section 2 du programme principal.
"""
import numpy as np
from numpy.linalg import svd
from ropn_auto_fast import fit_block_fast


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -60, 60)))


# ------------------------------------------------------------------ TRI-CF
class TriceptronCF:
    def __init__(self, k_gates=None, seed=0):
        self.k, self.seed = k_gates, seed

    def _feats(self, X, fit=False, y=None):
        if fit:
            self.U, self.W, self.r, _ = fit_block_fast(X, y, shrink=True, rank="C1")
            R = X - (X @ self.W) @ self.U.T                    # residu (I-P)x : hors de span(W)
            self.r_mean = R.mean(0)
            _, _, Vt = svd(R - self.r_mean, full_matrices=False)
            k = self.r if self.k is None else self.k
            self.S = Vt[:k].T                                  # directions de porte s_j (orthonormees)
            G = (R - self.r_mean) @ self.S; self.g_scale = G.std(0) + 1e-12
        c = X @ self.W                                         # code oblique (r)
        if self.S.shape[1] == 0:
            return c
        a = sigmoid(((X - (X @ self.W) @ self.U.T - self.r_mean) @ self.S) / self.g_scale)   # portes (k)
        return np.concatenate([c, (c[:, :, None] * a[:, None, :]).reshape(len(X), -1)], 1)

    def fit(self, X, y):
        Z = self._feats(X, fit=True, y=y)
        self.classes = np.unique(y); Y = np.eye(len(self.classes))[np.searchsorted(self.classes, y)]
        A = np.c_[Z, np.ones(len(Z))]; self.Wro, *_ = np.linalg.lstsq(A, Y, rcond=None)
        return self

    def predict(self, X):
        Z = self._feats(X); return self.classes[(np.c_[Z, np.ones(len(Z))] @ self.Wro).argmax(1)]

    def n_params(self):
```

```
        return int(self.U.size + self.W.size + self.S.size + self.Wro.size)


def fit_tricf_auto(Xtr, ytr, k_grid=(0, 1, 2, 4, 8), n_folds=3, seed=0):
    from sklearn.model_selection import StratifiedKFold
    skf = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=seed)
    best, bk = -1, 0
    for k in k_grid:
        accs = [(TriceptronCF(k).fit(Xtr[tr], ytr[tr]).predict(Xtr[va]) == ytr[va]).mean() for tr, va in skf.split(Xtr, ytr)]
        if np.mean(accs) > best + 1e-9: best, bk = np.mean(accs), k
    return TriceptronCF(bk).fit(Xtr, ytr), bk


# ------------------------------------------------------------- reseaux a une couche cachee, gradients manuels
class TriceptronBP:
    """Reseau a une couche cachee entraine par Adam (gradients manuels), deux familles et un saut lineaire optionnel :
        kind="glu"  (triceptron, TRI-BP) : h = sigmoid(X S + b_s) * (X W + b_w)
        kind="relu" (MLP)                : h = max(0, X W + b_w)
        logits = h V + b0 (+ X K si skip=True, K de taille d x C : le saut lineaire de la construction theorique).
    Meme initialisation, meme optimiseur, meme nombre de pas et meme penalite L2 pour les deux familles : la comparaison ne depend pas d'un optimiseur different. Entropie croisee + L2."""
    def __init__(self, r=16, steps=400, lr=1e-2, wd=1e-4, seed=0, init=None, kind="glu", skip=False):
        self.r, self.steps, self.lr, self.wd, self.seed, self.init, self.kind, self.skip = r, steps, lr, wd, seed, init, kind, skip

    def _fwd(self, p, X):
        W, bw, S, bs, V, b0, K = p
        c = X @ W + bw
        if self.kind == "glu": a = sigmoid(X @ S + bs); h = a * c
        else: a = (c > 0).astype(float); h = a * c
        out = h @ V + b0
        if self.skip: out = out + X @ K
        return out, (c, a, h)

    def _loss_grads(self, p, X, yi, C):
        W, bw, S, bs, V, b0, K = p; n = len(X)
        logits, (c, a, h) = self._fwd(p, X)
        z = logits - logits.max(1, keepdims=True); pr = np.exp(z); pr /= pr.sum(1, keepdims=True)
        loss = -np.log(pr[np.arange(n), yi] + 1e-12).mean() + self.wd * (np.sum(W**2) + np.sum(S**2) + np.sum(V**2) + np.sum(K**2))
        dl = pr.copy(); dl[np.arange(n), yi] -= 1; dl /= n
        dV = h.T @ dl + 2 * self.wd * V; db0 = dl.sum(0)
        dK = (X.T @ dl if self.skip else 0.0) + 2 * self.wd * K
        dh = dl @ V.T
        dc = dh * a
        dW = X.T @ dc + 2 * self.wd * W; dbw = dc.sum(0)
        if self.kind == "glu":
            dz = dh * c * a * (1 - a); dS = X.T @ dz + 2 * self.wd * S; dbs = dz.sum(0)
        else:
            dS = np.zeros_like(S); dbs = np.zeros_like(bs)
        return loss, [dW, dbw, dS, dbs, dV, db0, dK]

    def _init(self, d, C, rng):
        r = self.r
        p = [rng.standard_normal((d, r)) / np.sqrt(d), np.zeros(r), rng.standard_normal((d, r)) / np.sqrt(d), np.zeros(r),
             rng.standard_normal((r, C)) / np.sqrt(r), np.zeros(C), np.zeros((d, C))]
        if self.kind != "glu": p[2] = np.zeros((d, r))                # pas de porte pour le MLP
        return p

    def fit(self, X, y):
        rng = np.random.default_rng(self.seed)
        self.classes, yi = np.unique(y, return_inverse=True); C = len(self.classes); d = X.shape[1]
        p = self._init(d, C, rng) if self.init is None else [q.copy() for q in self.init]
        m = [np.zeros_like(q) for q in p]; v = [np.zeros_like(q) for q in p]
        for t in range(1, self.steps + 1):
            _, g = self._loss_grads(p, X, yi, C)
            for i in range(len(p)):
                m[i] = 0.9 * m[i] + 0.1 * g[i]; v[i] = 0.999 * v[i] + 0.001 * g[i] ** 2
```

```
                p[i] -= self.lr * (m[i] / (1 - 0.9 ** t)) / (np.sqrt(v[i] / (1 - 0.999 ** t)) + 1e-8)
        self.p = p
        return self

    def predict(self, X):
        return self.classes[self._fwd(self.p, X)[0].argmax(1)]

    def n_params(self):
        W, bw, S, bs, V, b0, K = self.p
        n = W.size + bw.size + V.size + b0.size
        if self.kind == "glu": n += S.size + bs.size
        if self.skip: n += K.size
        return int(n)

    def max_weight(self):
        return float(max(np.abs(q).max() for q in self.p))

    def gradcheck(self, d=5, r=3, C=3, n=7, seed=0):
        rng = np.random.default_rng(seed)
        X = rng.standard_normal((n, d)); yi = rng.integers(0, C, n)
        p = [rng.standard_normal((d, r)), rng.standard_normal(r), rng.standard_normal((d, r)), rng.standard_normal(r),
             rng.standard_normal((r, C)), rng.standard_normal(C), rng.standard_normal((d, C)) if self.skip else np.zeros((d, C))]
        if self.kind != "glu": p[2] = np.zeros((d, r)); p[3] = np.zeros(r)
        _, g = self._loss_grads(p, X, yi, C); worst = 0.0; eps = 1e-6
        active = [0, 1, 4, 5] + ([2, 3] if self.kind == "glu" else []) + ([6] if self.skip else [])
        for i in active:
            it = np.nditer(p[i], flags=["multi_index"])
            for _ in it:
                idx = it.multi_index; q = [a.copy() for a in p]
                q[i][idx] += eps; lp = self._loss_grads(q, X, yi, C)[0]
                q[i][idx] -= 2 * eps; lm = self._loss_grads(q, X, yi, C)[0]
                worst = max(worst, abs((lp - lm) / (2 * eps) - g[i][idx]) / (abs(g[i][idx]) + 1e-8))
        return worst


def width_for_params(d, C, n_target, skip):
    """largeur h d'un MLP (d->h->C, + saut lineaire d x C si skip) dont le nombre de parametres approche n_target."""
    return max(1, int(round((n_target - C - (d * C if skip else 0)) / (d + 1 + C))))


def mlp_width_for_params(d, C, n_target):
    """largeur h d'un MLP (d->h->C) dont le nombre de parametres approche n_target."""
    return max(2, int(round((n_target - C) / (d + 1 + C))))


if __name__ == "__main__":
    import warnings; warnings.filterwarnings("ignore")
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import StandardScaler
    from sklearn.neural_network import MLPClassifier
    from sklearn.datasets import load_digits, load_breast_cancer, load_wine, make_moons, make_circles
    from ropn_auto_nobp import make_multiclass
    from ropn_auto_finetune import make_hetero
    from ropn_auto_nobp_diag import readout_fit_predict, readout_centroid_fit_predict
    from ropn_auto_stats import fmt_paired

    print("gradcheck : " + " ; ".join(f"{k}{'+saut' if sk else ''} {TriceptronBP(kind=k, skip=sk).gradcheck():.1e}" for k in ("glu", "relu") for sk in (False, True)))

    def prep(X, y, sd):
        Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
        keep = Xtr.std(0) > 1e-8; sc = StandardScaler().fit(Xtr[:, keep])
        return sc.transform(Xtr[:, keep]), sc.transform(Xte[:, keep]), ytr, yte

    def gen2d(name, seed):
        """tirage independant d'un probleme 2-D (n = 400)."""
        rng = np.random.default_rng(seed)
        if name == "two moons": return make_moons(400, noise=0.15, random_state=seed)
        if name == "circles": return make_circles(400, noise=0.1, factor=0.5, random_state=seed)
        X = rng.standard_normal((400, 2)); return X, ((X[:, 0] > 0) ^ (X[:, 1] > 0)).astype(int)
```

```
195
196     STEPS, f = 2000, (lambda v: f"{np.mean(v)*100:5.1f}+/-{np.std(v)*100:4.1f}")
197     # ------------------------------------------------------------------ 1. comparaison symetrique sur
        les problemes 2-D
198     DRAWS = 10
199     print(f"\n=== 1. Problemes 2-D : {DRAWS} tirages INDEPENDANTS (n = 400, 70/30), meme entraineur pour
        les deux familles (Adam, {STEPS} pas, lr 1e-2, L2 1e-4) ===")
200     print("    GLU = triceptron (r unites), MLP = ReLU (h unites) a nombre de parametres voisin ; avec ou
          sans saut lineaire ; L-BFGS : MLP sklearn sans saut (autre optimiseur)")
201     print(f"    {'jeu':10s} {'r':>2s} | {'par':>4s} {'GLU':>11s} | {'h':>2s} {'par':>4s} {'MLP':>11s} {'
        MLP L-BFGS':>11s} | {'par':>4s} {'GLU+saut':>11s} | {'h':>2s} {'par':>4s} {'MLP+saut':>11s} | GLU -
        MLP [IC 95 %] | GLU+saut - MLP+saut [IC 95 %]")
202     for name in ("two moons", "circles", "XOR"):
203         for r_ in (16, 4, 2, 1):
204             res = {k: [] for k in ("glu", "relu", "lbfgs", "glu_s", "relu_s")}; meta = {}
205             for sd in range(DRAWS):
206                 X, y = gen2d(name, sd); Xtr, Xte, ytr, yte = prep(X, y, sd); d, C = 2, 2
207                 g = TriceptronBP(r=r_, steps=STEPS, seed=sd, kind="glu").fit(Xtr, ytr); res["glu"].append
        ((g.predict(Xte) == yte).mean())
208                 h = width_for_params(d, C, g.n_params(), False)
209                 m = TriceptronBP(r=h, steps=STEPS, seed=sd, kind="relu").fit(Xtr, ytr); res["relu"].
        append((m.predict(Xte) == yte).mean())
210                 res["lbfgs"].append(MLPClassifier(hidden_layer_sizes=(h,), solver="lbfgs", max_iter=2000,
          random_state=sd).fit(Xtr, ytr).score(Xte, yte))
211                 gs = TriceptronBP(r=r_, steps=STEPS, seed=sd, kind="glu", skip=True).fit(Xtr, ytr); res["
        glu_s"].append((gs.predict(Xte) == yte).mean())
212                 hs = width_for_params(d, C, gs.n_params(), True)
213                 ms = TriceptronBP(r=hs, steps=STEPS, seed=sd, kind="relu", skip=True).fit(Xtr, ytr); res[
        "relu_s"].append((ms.predict(Xte) == yte).mean())
214                 meta = dict(pg=g.n_params(), h=h, pm=m.n_params(), pgs=gs.n_params(), hs=hs, pms=ms.
        n_params())
215             print(f"    {name:10s} {r_:2d} | {meta['pg']:4d} {f(res['glu']):>11s} | {meta['h']:2d} {meta
        ['pm']:4d} {f(res['relu']):>11s} {f(res['lbfgs']):>11s} | {meta['pgs']:4d} {f(res['glu_s']):>11s} | {
        meta['hs']:2d} {meta['pms']:4d} {f(res['relu_s']):>11s} | "
216                   f"{fmt_paired(res['glu'], res['relu'])} | {fmt_paired(res['glu_s'], res['relu_s'])}")
217
218     # ------------------------------------------------------------------ 2. poids construits contre poids
          appris
219     print("\n=== 2. Poids CONSTRUITS (constructions du theoreme de capacite, aucun apprentissage) contre
        poids APPRIS, donnees brutes non standardisees, 10 tirages ===")
220     sig = sigmoid
221     for gam in (1.0, 0.5, 0.2, 0.05):
222         acc_c, acc_x = [], []
223         for sd in range(DRAWS):
224             X, y = gen2d("circles", sd); fval = (sig(gam * X) * X - 0.5 * X).sum(1) - gam / 4 * 0.75 ** 2
                # 2 unites + saut ; seuil au rayon 0.75
225             acc_c.append(((fval < 0).astype(int) == y).mean())
                # classe 1 = cercle interieur
226             X, y = gen2d("XOR", sd); fx = sig(gam * X[:, 0]) * X[:, 1] - 0.5 * X[:, 1]
                 # 1 unite + saut
227             acc_x.append(((fx < 0).astype(int) == y).mean())
228         print(f"    construit, gamma = {gam:4.2f} : cercles (2 unites + saut) {f(acc_c)} | XOR (1 unite +
          saut) {f(acc_x)} | plus grand poids sous forme normalisee (lecture 4/gamma) = {4/gam:5.1f} | ecart
        radial theorique gamma^2 R^3/24 = {gam**2*0.75**3/24:.1e}")
229     for name, r_ in (("circles", 2), ("XOR", 1)):
230         acc_l, wmax, sharp = [], [], []
231         for sd in range(DRAWS):
232             X, y = gen2d(name, sd); Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y
        , random_state=sd)
233             g = TriceptronBP(r=r_, steps=STEPS, seed=sd, kind="glu", skip=True).fit(Xtr, ytr)
234             acc_l.append((g.predict(Xte) == yte).mean()); wmax.append(g.max_weight()); sharp.append(np.
        linalg.norm(g.p[2], axis=0).max())
235         print(f"    appris, {name:8s} GLU+saut r={r_} ({g.n_params()} parametres) : precision test {f(
        acc_l)} | plus grand poids {np.mean(wmax):5.1f} | norme max d'une direction de porte ||s|| = {np.mean
        (sharp):4.1f}")
236     print("    (precision des poids construits evaluee sur les 400 points de chaque tirage ; les unites
        apprises sont loin du regime doux gamma -> 0 de la construction.)")
237
238     # ------------------------------------------------------------------ 3. jeux de dimension plus elevee
239     rng = np.random.default_rng(0)
240     ds = {}
241     Xm, ym = make_moons(400, noise=0.15, random_state=0); ds["two moons (2D)"] = (Xm, ym)
242     Xc, yc = make_circles(400, noise=0.1, factor=0.5, random_state=0); ds["circles (2D)"] = (Xc, yc)
```

```
    Xx = rng.standard_normal((400, 2)); yx = ((Xx[:, 0] > 0) ^ (Xx[:, 1] > 0)).astype(int); ds["XOR (2D)"] = (Xx, yx)
    X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=15, kappa=10.0, sep=0.8, seed=0); ds["gauss n=15/cl"] = (X, y)
    X, y, *_ = make_hetero(n_classes=4, d=20, n_per=60, kappa=50.0, sep=0.8, seed=0); ds["hetero n=60/cl"] = (X, y)
    d_ = load_digits(); ds["digits"] = (d_.data, d_.target)
    b_ = load_breast_cancer(); ds["breast"] = (b_.data, b_.target)
    w_ = load_wine(); ds["wine"] = (w_.data, w_.target)
    SEEDS = 5
    print(f"\n=== 3. Tous les jeux, r = 16, {SEEDS} decoupages 70/30 d'un jeu fixe : bloc analytique, TRI-CF, GLU et MLP (meme entraineur), MLP sklearn L-BFGS ===")
    print(f"    {'jeu':16s} | {'bloc fast':>11s} | {'TRI-CF auto':>11s} {'k':>2s} | {'par':>5s} {'GLU':>11s} | {'h':>3s} {'par':>5s} {'MLP':>11s} {'MLP L-BFGS':>11s} | GLU - MLP [IC 95 %, variabilite du decoupage]")
    for name, (X, y) in ds.items():
        fast, cf, ks, res = [], [], [], {k: [] for k in ("glu", "relu", "lbfgs")}
        for sd in range(SEEDS):
            Xtr, Xte, ytr, yte = prep(X, y, sd); d, C = Xtr.shape[1], len(np.unique(ytr))
            U, W, r, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")
            fast.append((readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W) == yte).mean())   # lecture de reference
            m_, k = fit_tricf_auto(Xtr, ytr, seed=sd); cf.append((m_.predict(Xte) == yte).mean()); ks.append(k)
            g = TriceptronBP(r=16, steps=STEPS, seed=sd, kind="glu").fit(Xtr, ytr); res["glu"].append((g.predict(Xte) == yte).mean())
            h = width_for_params(d, C, g.n_params(), False)
            m = TriceptronBP(r=h, steps=STEPS, seed=sd, kind="relu").fit(Xtr, ytr); res["relu"].append((m.predict(Xte) == yte).mean())
            res["lbfgs"].append(MLPClassifier(hidden_layer_sizes=(h,), solver="lbfgs", max_iter=2000, random_state=sd).fit(Xtr, ytr).score(Xte, yte))
        print(f"    {name:16s} | {f(fast):>11s} | {f(cf):>11s} {np.mean(ks):2.0f} | {g.n_params():5d} {f(res['glu']):>11s} | {h:3d} {m.n_params():5d} {f(res['relu']):>11s} {f(res['lbfgs']):>11s} | {fmt_paired(res['glu'], res['relu'])}")
```

Listing 14: ropn_auto_depth_theory.py — verification of the depth and capacity theorems

```
"""
Verifications numeriques des resultats sur la recursion a poids partages et sur la capacite du triceptron.

A. Reduction de la profondeur gatee a poids partages :
   bloc h <- h + (1/L) U diag(a(h)) W^T h, a = sigma(S^T h + b), P = U W^T idempotent.
   (i) le residu rho = (I-P) h est invariant ; (ii) le code c = W^T h suit c <- c * (1 + a/L) avec
   a = sigma(A c + beta + b), A = S^T U, beta = S^T rho ; (iii) le code c_T ne depend de l'entree qu'a travers
   (c_0, S^T rho_0) : deux entrees de meme (c_0, S^T rho_0) et residus differents donnent le meme c_T ;
   (iv) si S dans span(W) alors beta = 0 et c_T ne depend pas du residu ; (v) pour r = 1 la
   reparametrisation c_0 -> c_T est strictement croissante sous la condition du corollaire, et un
   contre-exemple montre que la condition est necessaire.
   Variante scalaire : rho_T = Lambda_T rho_0, direction du residu fixe.

B. Capacite quadrique du triceptron :
   (i) swish = triceptron avec s = w ; ReLU = limite uniforme (ecart sup 0.2785 / kappa) ;
   (ii) k unites en regime doux + saut lineaire approchent toute quadrique d'inertie max(p, n) <= k :
        cercle (2 unites, ecart radial kappa^2 R^3 / 24) et XOR (1 unite + saut, exact en signe) ;
   (iv) approximation du disque par un reseau ReLU a m unites : la frontiere a au plus N(m) = 1 + m + C(m,2)
        segments, d'ou une erreur d'encadrement >= R tan^2(pi / 2N) ; comparaison avec l'erreur mesuree.

C. Invariance du code, determinant exact du bloc gate scalaire et determinant r x r du bloc gate par
   coordonnee : comparaison aux differences finies.
"""
import numpy as np
from numpy.linalg import norm, qr
from ropn_auto_fast import fit_block_fast
from ropn_auto_nobp import make_multiclass


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -60, 60)))
```

```python


# ============================================================== A
def gated_block_steps(h, U, W, S, b, L, T):
    for _ in range(T):
        a = sigmoid(h @ S + b)                      # (n, r) portes par coordonnee de code
        h = h + (1.0 / L) * ((h @ W) * a) @ U.T     # h + U diag(a) W^T h / L
    return h


def verify_A(seed=0):
    rng = np.random.default_rng(seed)
    X, y, *_ = make_multiclass(n_classes=4, d=12, n_per=40, kappa=20.0, sep=1.0, seed=seed)
    U, W, r, _ = fit_block_fast(X, y, shrink=True, rank="C1"); d = X.shape[1]
    P = U @ W.T; Q = np.eye(d) - P
    S = rng.standard_normal((d, r)); b = rng.standard_normal(r); L, T = 4, 6
    h0 = rng.standard_normal((5, d))
    hT = gated_block_steps(h0, U, W, S, b, L, T)
    # (i) residu invariant
    e_rho = norm(hT @ Q.T - h0 @ Q.T) / norm(h0 @ Q.T)
    # (ii) recursion du code en dimension r
    c = h0 @ W; beta = (h0 @ Q.T) @ S; A = S.T @ U
    for _ in range(T):
        a = sigmoid(c @ A.T + beta + b); c = c * (1 + a / L)
    e_code = norm(hT @ W - c) / norm(c)
    # (iii) deux entrees de meme (c_0, S^T rho_0), residus differents -> meme c_T
    #  construire rho' = rho + delta avec delta dans ker(P) et S^T delta = 0
    rho = h0 @ Q.T
    Nk = qr(Q)[0][:, :d - r]                            # base de ker(P) (= im(I-P))
    M = S.T @ Nk                                        # (r, d-r) ; on cherche delta = Nk z, M z = 0
    _, _, Vt = np.linalg.svd(M); z = Vt[-1]; delta = Nk @ z
    assert norm(P @ delta) < 1e-10 and norm(S.T @ delta) < 1e-10
    h0b = h0 + 3.0 * delta[None, :]
    hTb = gated_block_steps(h0b, U, W, S, b, L, T)
    e_iii = norm(hTb @ W - hT @ W) / norm(hT @ W)
    # (iv) S dans span(W) -> c_T independant du residu
    S2 = W @ rng.standard_normal((r, r))
    hT1 = gated_block_steps(h0, U, W, S2, b, L, T)
    hT2 = gated_block_steps(h0 + 5.0 * (rng.standard_normal((5, d)) @ Q.T), U, W, S2, b, L, T)   # residu
      change
    e_iv = norm(hT1 @ W - hT2 @ W) / norm(hT1 @ W)
    # (v) r = 1 : monotonie de c_0 -> c_T a beta fixe (sous condition d'inversibilite)
    cs = np.linspace(-3, 3, 401); A1 = 0.4; beta1 = 0.3; b1 = 0.1; cT = cs.copy()
    for _ in range(T):
        cT = cT * (1 + sigmoid(A1 * cT + beta1 + b1) / L)
    mono = bool(np.all(np.diff(cT) > 0))
    # variante scalaire : direction du residu fixe
    s = rng.standard_normal(d); bs = 0.2; h = h0.copy()
    for _ in range(T):
        a = sigmoid(h @ s + bs)[:, None]; h = h + (a / L) * (h @ P.T - h)
    rhoT = h @ Q.T; cosang = np.sum(rhoT * rho, 1) / (norm(rhoT, axis=1) * norm(rho, axis=1))
    print("=== A. Reduction de la profondeur gatee ===")
    print(f"  (i)   residu invariant                 : {e_rho:.1e}")
    print(f"  (ii)  recursion du code (dim r={r})     : {e_code:.1e}")
    print(f"  (iii) meme (c0, S^T rho0) -> meme c_T   : {e_iii:.1e}  (residus differents de {3*norm(delta
     ):.2f})")
    print(f"  (iv)  S dans span(W) -> c_T sans residu : {e_iv:.1e}")
    print(f"  (v)   r=1, c0 -> c_T strictement croissant : {mono}")
    print(f"  scalaire : cos(angle(rho_T, rho_0)) = {cosang.min():.12f} (direction du residu fixe)")


# ============================================================== B
def tri_circle(G, eps):
    """deux triceptrons doux (s_i = w_i = e_i) + saut lineaire : f = sum_i (x_i/2) tanh(eps x_i / 2)."""
    return sigmoid(eps * G[:, 0]) * G[:, 0] + sigmoid(eps * G[:, 1]) * G[:, 1] - 0.5 * (G[:, 0] + G[:,
     1])


def radial_gap(eps, R=1.0, n_ang=721):
    """ecart radial max entre la courbe de niveau f = eps R^2 / 4 et le cercle de rayon R (bissection par
      angle)."""
    th = np.linspace(0, 2 * np.pi, n_ang); lo = np.full(n_ang, 0.5 * R); hi = np.full(n_ang, 2.0 * R)
    for _ in range(60):
```

```
        mid = 0.5 * (lo + hi); val = tri_circle(np.c_[mid * np.cos(th), mid * np.sin(th)], eps) - eps * R ** 2 / 4
        lo = np.where(val < 0, mid, lo); hi = np.where(val < 0, hi, mid)
    return float(np.abs(0.5 * (lo + hi) - R).max())


def verify_B():
    print("\n=== B. Capacite quadrique du triceptron ===")
    x = np.linspace(-3, 3, 301); X1, X2 = np.meshgrid(x, x); G = np.c_[X1.ravel(), X2.ravel()]
    # (i) inclusion : triceptron(s=w, kappa=1) = swish ; kappa -> inf : ReLU, uniformement, ecart sup = 0.2785 / kappa
    z = np.linspace(-5, 5, 2000001)
    for kap in (1.0, 50.0, 1000.0):
        gap = np.abs(sigmoid(kap * z) * z - np.maximum(z, 0)).max()
        print(f"  (i)  s=w : sup_z |sigma(kappa z) z - ReLU(z)| a kappa={kap:6.0f} : {gap:.3e}  (0.2785/kappa = {0.27846/kap:.3e})")
    # (ii) cercle avec 2 unites en regime doux : sigma(eps t) t = t/2 + eps t^2/4 - eps^3 t^4/48 + ...
    for eps in (0.5, 0.2, 0.05):
        f = tri_circle(G, eps); q = eps / 4 * (G[:, 0] ** 2 + G[:, 1] ** 2)
        print(f"  (ii) cercle R=1, 2 unites + saut (kappa={eps}) : ecart relatif a la quadrique sur [-3,3]^2 {norm(f-q)/norm(q):.1e} ; "
              f"ecart radial max de la courbe de niveau {radial_gap(eps):.2e}  (kappa^2 R^3/24 = {eps**2/24:.2e})")
    eps = 0.05
    # XOR : 1 unite + saut (exact en signe pour tout kappa), ou 2 unites sans saut
    f1 = sigmoid(eps * G[:, 0]) * G[:, 1] - 0.5 * G[:, 1]                                  # = (x2/2) tanh(eps x1/2)
    f2 = sigmoid(50 * G[:, 0]) * G[:, 1] - sigmoid(-50 * G[:, 0]) * G[:, 1]               # = x2 tanh(25 x1)
    nz = (G[:, 0] != 0) & (G[:, 1] != 0); truth = (G[:, 0] * G[:, 1]) > 0
    print(f"  (ii) XOR, 1 unite + saut : accord du signe {100*np.mean((f1>0)[nz]==truth[nz]):.2f}% ; 2 unites (regime dur) : {100*np.mean((f2>0)[nz]==truth[nz]):.2f}% (hors axes)")
    # (iv) disque SANS marge (points uniformes dans [-1.5,1.5]^2, classe = ||x|| > 1). Erreur d'encadrement d'une
    #      region D : plus petit e tel que B(0,R-e) c D c B(0,R+e) ; ici e_mes = max | ||x|| - 1 | sur les points
    #      de grille mal classes du domaine d'apprentissage. Borne : e >= R tan^2(pi / 2N), N <= N(m) segments
    #      (N <= 2m si la region est convexe).
    from sklearn.neural_network import MLPClassifier
    rng = np.random.default_rng(0)
    Xtr = rng.uniform(-1.5, 1.5, (4000, 2)); ytr = (norm(Xtr, axis=1) > 1.0).astype(int)
    xg = np.linspace(-1.5, 1.5, 601); A1, A2 = np.meshgrid(xg, xg); Gd = np.c_[A1.ravel(), A2.ravel()]
    rad = norm(Gd, axis=1); truth_g = rad > 1.0
    print("  (iv) disque sans marge, grille 601x601 sur [-1.5,1.5]^2, MLP ReLU a m unites (L-BFGS, 3 graines, graine retenue = meilleure precision d'APPRENTISSAGE) :")
    print(f"        {'m':>3s} {'N(m)':>5s} {'erreur de surface':>18s} {'encadrement mesure':>19s} {'borne tan^2(pi/2N(m))':>22s} {'borne convexe tan^2(pi/4m)':>27s}")
    for m in [2, 3, 4, 6, 10, 20, 40]:
        best = None
        for sd in range(3):
            clf = MLPClassifier((m,), solver="lbfgs", max_iter=3000, random_state=sd).fit(Xtr, ytr); tr = clf.score(Xtr, ytr)
            if best is None or tr > best[0]: best = (tr, clf)
        wrong = best[1].predict(Gd) != truth_g; N = 1 + m + m * (m - 1) // 2
        e_mes = np.abs(rad[wrong] - 1.0).max() if wrong.any() else 0.0
        print(f"        {m:3d} {N:5d} {100*wrong.mean():17.2f}% {e_mes:19.4f} {np.tan(np.pi/(2*N))**2:22.5f} {np.tan(np.pi/(4*m))**2:27.5f}")
    wrong = (tri_circle(Gd, eps) > eps / 4) != truth_g
    print(f"        2 triceptrons doux + saut, poids explicites (kappa={eps}) : erreur de surface {100*wrong.mean():.3f}%, encadrement (ecart radial) {radial_gap(eps):.2e}")
    for eps_t in (1e-2, 1e-3, 1e-4):
        Nmin = int(np.ceil(np.pi / np.arccos((1 - eps_t) / (1 + eps_t)))); m = 1
        while 1 + m + m * (m - 1) // 2 < Nmin: m += 1
        print(f"        precision {eps_t:.0e} : N >= {Nmin} segments, donc m >= {m} unites (m >= {int(np.ceil(Nmin/2))} si la region est convexe)")
    # avec marge (make_circles, facteur 0.5) : un triangle suffit -> la marge, pas la courbure, fixe le besoin
    from sklearn.datasets import make_circles
    Xc, yc = make_circles(2000, noise=0.0, factor=0.5, random_state=0)
    accs = [max(MLPClassifier((m,), solver="lbfgs", max_iter=3000, random_state=sd).fit(Xc, yc).score(Xc, yc) for sd in range(3)) for m in [2, 3]]
    print(f"        (avec marge, make_circles f=0.5, precision d'apprentissage : ReLU m=2 -> {accs
```

```
    [0]*100:.1f}%, m=3 -> {accs[1]*100:.1f}% : un triangle suffit quand il y a une marge)")


# ================================================================ C
def num_jac(F, h, eps=1e-6):
    d = len(h); J = np.zeros((d, d))
    for j in range(d):
        e = np.zeros(d); e[j] = eps; J[:, j] = (F(h + e) - F(h - e)) / (2 * eps)
    return J


def verify_C(seed=0):
    print("\n=== C. Invariance du code et determinants exacts (differences finies) ===")
    rng = np.random.default_rng(seed); d, r = 12, 3
    U = rng.standard_normal((d, r)); W = rng.standard_normal((d, r)); W = W @ np.linalg.inv(U.T @ W)   # W^T U = I_r
    P = U @ W.T; Q = np.eye(d) - P; x = rng.standard_normal(d); b = 0.3
    # invariance du code et collapse de la porte (recursion h <- (1-a) h + a P h)
    def rec(s, T):
        h = x.copy(); a_hist = []
        for _ in range(T):
            a = sigmoid(s @ h + b); a_hist.append(a); h = (1 - a) * h + a * (P @ h)
        return h, np.array(a_hist)
    s_in = W @ rng.standard_normal(r)                                  # porte dans span(W)
    s_out = Q.T @ rng.standard_normal(d); s_out *= 1.5 / abs(s_out @ x)   # porte dans (span U)^perp : lit seulement le residu ; |s^T x| = 1.5
    e_code, e_form = 0.0, 0.0
    for T in (1, 4, 16):
        hT, a_h = rec(s_in, T); a = a_h[0]
        e_code = max(e_code, norm(W.T @ hT - W.T @ x)); e_form = max(e_form, norm(hT - (P @ x + (1 - a) ** T * (Q @ x))))
        e_code = max(e_code, norm(W.T @ rec(s_out, T)[0] - W.T @ x))
    a_out = rec(s_out, 16)[1]
    print(f"  code : max ||W^T h_T - W^T x|| = {e_code:.1e} (T = 1, 4, 16, portes dans et hors de span(W)) ; forme fermee de h_T (porte dans span(W)) : {e_form:.1e}")
    print(f"  porte hors de span(W) : a_t varie de {a_out.max() - a_out.min():.3f} le long de l'orbite (T = 16) ; porte dans span(W) : variation {np.ptp(rec(s_in, 16)[1]):.0e}")
    # determinant du bloc gate scalaire F(h) = h + (a(h)/L)(P - I) h
    def det_formula(h, s, L):
        a = sigmoid(s @ h + b); return (1 - a / L) ** (d - r) * (1 - a * (1 - a) / (L - a) * (s @ Q @ h))
    def F_of(s, L): return lambda h: h + (sigmoid(s @ h + b) / L) * (P @ h - h)
    for L in (1, 2):
        s = 0.5 * rng.standard_normal(d); h = rng.standard_normal(d)
        print(f"  bloc gate scalaire, L={L}, s quelconque : det formule = {det_formula(h, s, L):.7e}, differences finies = {np.linalg.det(num_jac(F_of(s, L), h)):.7e}")
    h = rng.standard_normal(d); a = sigmoid(s_in @ h + b)
    print(f"  s dans span(W), L=1 : det differences finies = {np.linalg.det(num_jac(F_of(s_in, 1), h)):.7e}, (1-a)^(d-r) = {(1 - a) ** (d - r):.7e}")
    # cas construit a s^T (I-P) h = 1 (L = 1) : s = t q avec q = (I-P)h, t solution de sigma(t |q|^2 + b) t |q|^2 = 1
    q = Q @ h; q2 = q @ q; lo, hi = 0.0, 50.0
    for _ in range(200):
        mid = 0.5 * (lo + hi)
        if sigmoid(mid * q2 + b) * mid * q2 < 1: lo = mid
        else: hi = mid
    s_sing = (Q.T @ Q @ h) * lo / (q @ Q.T @ Q @ h) * q2            # s^T h = s^T (I-P) h = lo |q|^2
    a = sigmoid(s_sing @ h + b)
    print(f"  cas construit a * s^T(I-P)h = {a * (s_sing @ Q @ h):.6f} : det differences finies = {np.linalg.det(num_jac(F_of(s_sing, 1), h)):.1e} (formule : 0)")
    # determinant r x r du bloc gate par coordonnee
    d2 = 10; U2 = rng.standard_normal((d2, r)); W2 = rng.standard_normal((d2, r)); W2 = W2 @ np.linalg.inv(U2.T @ W2)
    S2 = 0.5 * rng.standard_normal((d2, r)); b2 = 0.2 * rng.standard_normal(r); L = 2; A2 = S2.T @ U2
    Fv = lambda h: h + (1.0 / L) * U2 @ (sigmoid(S2.T @ h + b2) * (W2.T @ h))
    for _ in range(3):
        h = rng.standard_normal(d2); c = W2.T @ h; a = sigmoid(S2.T @ h + b2)
        small = np.linalg.det(np.eye(r) + (np.diag(a) + np.diag(c * a * (1 - a)) @ A2) / L)
        print(f"  bloc gate par coordonnee (d={d2}, r={r}, L={L}) : det r x r = {small:.8f}, differences finies d x d = {np.linalg.det(num_jac(Fv, h)):.8f}")
    # rang un : contre-exemple a la monotonie hors condition, et constante optimale
    phi = lambda c, L, al, be: c * (1 + sigmoid(al * c + be) / L)
    print(f"  rang un, L=1, alpha=1, b=0, beta=12 : phi(-16) = {phi(-16, 1, 1, 12):.2f}, phi(-12) = {phi(-12, 1, 1, 12):.2f}, phi(-8) = {phi(-8, 1, 1, 12):.2f} (non monotone : alpha c < -4L)")
```

```
    for L in (1, 2, 4):
        t = np.linspace(-30, 0, 30001); u = np.linspace(-30, 30, 6001)[:, None]        # t = alpha c <= 0, u = alpha c + beta + b
        dphi = 1 + sigmoid(u) / L + t[None, :] * sigmoid(u) * (1 - sigmoid(u)) / L
        t_star = -t[np.argmax(dphi.min(0) > 0)]
        print(f"  rang un, L={L} : phi' > 0 pour tout beta des que alpha c > -{t_star:.2f}  (2L+1+2 sqrt(L(L+1)) = {2*L+1+2*np.sqrt(L*(L+1)):.2f} ; condition suffisante 4L = {4*L})")


if __name__ == "__main__":
    verify_A()
    verify_B()
    verify_C()
```

Listing 15: ropn_auto_examples.py — worked examples

```
"""Exemples travailles (petites dimensions, nombres explicites) pour les theoremes de collapse, du determinant, de reduction de la profondeur gatee et de capacite du triceptron."""
import numpy as np
from numpy.linalg import norm

def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

def num_jac(F, h, eps=1e-6):
    d = len(h); J = np.zeros((d, d))
    for j in range(d):
        e = np.zeros(d); e[j] = eps; J[:, j] = (F(h + e) - F(h - e)) / (2 * eps)
    return J

print("=== Exemples : collapse et determinant du bloc gate (d=3, r=1, u=(1,0,0), w=(1,1,0)) ===")
u = np.array([1., 0, 0]); w = np.array([1., 1, 0]); P = np.outer(u, w); I3 = np.eye(3)
M = I3 + P / 2.0
print("T1   L=2, T=2 : (I+P/2)^2 - I = c_2 P avec c_2 = (3/2)^2 - 1 = 1.25 :", np.allclose(M @ M - I3, 1.25 * P),
      "| T=3 : c_3 = (3/2)^3 - 1 =", (1.5 ** 3 - 1), np.allclose(M @ M @ M - I3, (1.5 ** 3 - 1) * P))
print("     h=(1,0.5,2) : (I+P/2)^2 h =", np.round(M @ M @ np.array([1, .5, 2.]), 4), " = h + 1.25 * (1.5, 0, 0)")
s2 = np.array([0., 0, 1]); h2 = np.array([1.0, 0.5, 2.0])
for L in (1.0, 2.0):
    F = lambda h: h + (sigmoid(s2 @ h) / L) * (P @ h - h)
    a = sigmoid(s2 @ h2); t = s2 @ (h2 - P @ h2)
    det_th = (1 - a / L) ** 2 * (1 - a * (1 - a) / (L - a) * t)
    print(f"T2   L={L:.0f}, s=(0,0,1), h=(1,0.5,2) : a = sigma(2) = {a:.4f}, s^T(I-P)h = {t:.1f} -> det (formule) = {det_th:+.6f}, det (diff. finies) = {np.linalg.det(num_jac(F, h2)):+.6f}")
lo, hi = 0.5, 2.0                                       # L=1 : seuil a * h3 = 1
for _ in range(60):
    mid = 0.5 * (lo + hi)
    if sigmoid(mid) * mid < 1: lo = mid
    else: hi = mid
hc = np.array([1.0, 0.5, 0.5 * (lo + hi)]); F1 = lambda h: h + sigmoid(s2 @ h) * (P @ h - h)
print(f"     L=1 : det = 0 quand sigma(h3) h3 = 1, soit h3* = {hc[2]:.4f} ; det (diff. finies) en h3* = {np.linalg.det(num_jac(F1, hc)):.1e} ; h3 = 2 > h3* -> det < 0 (orientation renversee)")
a = sigmoid(2.0)
print(f"     L=2 : det = 0 quand s^T(I-P)h = (L-a)/(a(1-a)) = {(2-a)/(a*(1-a)):.2f} ; condition suffisante du texte |s^T(I-P)h| < L/a = {2/a:.2f} ; ici 2 -> inversible.")

print("\n=== Exemples : reduction de la profondeur gatee ===")
# --- A.1 : d=3, r=1, u=(1,0,0), w=(1,1,0) -> P = u w^T, ker P = {h1+h2=0}
u = np.array([1., 0, 0]); w = np.array([1., 1, 0]); P = np.outer(u, w); Q = np.eye(3) - P
print("A.1  P = u w^T =\n", P, "\n     (I-P) =\n", Q, "\n     P^2 - P =", norm(P @ P - P))
L, T, b = 2.0, 3, 0.0
def step(h, s):                                         # bloc gate par coordonnee, r=1
    a = sigmoid(s @ h + b); return h + (a / L) * u * (w @ h)
def iterate(h, s, T=T):
    for _ in range(T): h = step(h, s)
    return h
s_ker = np.array([0., 0, 1])                            # lit h3 : s^T (I-P) = (0,0,1) != 0  -> m = 1, A = s^T u = 0
print("     s=(0,0,1) : s^T(I-P) =", s_ker @ Q, " A = s^T u =", s_ker @ u)
h  = np.array([1.0, 0.5, 2.0]); hp = np.array([0.2, 1.3, 2.0])    # meme code c = h1+h2 = 1.5, meme beta = h3 = 2
```

```
for hh in (h, hp):
    hT = iterate(hh, s_ker)
    print(f"     h0 = {hh} : c0 = {w@hh:.2f}, beta = {hh[2]:.1f} -> h_T = {np.round(hT, 4)}, c_T = {w@hT
     :.4f}, residu (I-P)h_T = {np.round(Q@hT,4)}")
cT_formula = (w @ h) * (1 + sigmoid(h[2] + b) / L) ** T
print(f"     forme close c_T = c0 (1 + sigma(beta)/L)^T = 1.5 * (1 + {sigmoid(2.0):.4f}/2)^3 = {
     cT_formula:.4f}")
Fk = lambda hh: step(hh, s_ker); a0 = sigmoid(s_ker @ h + b)
print(f"     det nabla F (determinant r x r, A=0) = 1 + a/L = 1 + {a0:.4f}/2 = {1 + a0 / L:.4f} ; diff.
     finies : {np.linalg.det(num_jac(Fk, h)):.4f}")
s_span = w.copy()                                   # s dans span(W) : s^T (I-P) = 0 -> m = 0
print("     s = w=(1,1,0) : s^T(I-P) =", s_span @ Q)
h3a = np.array([1.0, 0.5, 2.0]); h3b = np.array([1.0, 0.5, -7.0])  # meme code, h3 tres different
print(f"     s=w : h3=2 -> c_T = {w@iterate(h3a, s_span):.4f} ; h3=-7 -> c_T = {w@iterate(h3b, s_span):.4
     f}  (le noyau est ignore)")

# --- A.2 : r=1, A != 0 : monotonie et une seule courbe de frontiere psi_T(beta)
s_mix = np.array([1., 0, 1])                        # A = s^T u = 1, beta = s^T (I-P) h = h3 - h2
def cT_of(c, beta, T):
    for _ in range(T): c = c * (1 + sigmoid(1.0 * c + beta + b) / L)
    return c
print("A.2  s=(1,0,1) : A = 1. Table c_T(c0, beta=0.5) :")
for c0 in [-2, -1, 0, 1, 2]:
    print(f"       c0={c0:+d} : T=1 -> {cT_of(c0, 0.5, 1):+.4f}   T=6 -> {cT_of(c0, 0.5, 6):+.4f}")
# frontiere v c_T = theta  <=>  c0 > psi_T(beta) : resolution par bissection
def psi(beta, T, theta=1.0):
    lo, hi = -10.0, 10.0
    for _ in range(60):
        mid = 0.5 * (lo + hi)
        if cT_of(mid, beta, T) < theta: lo = mid
        else: hi = mid
    return 0.5 * (lo + hi)
print("     seuil c_T = 1 :  beta      psi_1(beta)   psi_6(beta)")
for beta in [-2.0, 0.0, 2.0]:
    print(f"                    {beta:+.1f}      {psi(beta,1):+.4f}       {psi(beta,6):+.4f}")

# --- A.3 : d=4, r=2, m=1 vs m=2
U = np.array([[1., 0], [0, 1], [0, 0], [0, 0]]); W = np.array([[1., 0], [0, 1], [1, 0], [0, 1]])
P4 = U @ W.T; Q4 = np.eye(4) - P4
print("A.3  W^T U = I ?", np.allclose(W.T @ U, np.eye(2)), "| P h = (h1+h3, h2+h4, 0, 0)")
def iter4(h, S, T=4, L=2.0):
    for _ in range(T):
        a = sigmoid(S.T @ h); h = h + (1 / L) * U @ (a * (W.T @ h))
    return h
h  = np.array([1.0, 0.5, 0.3, 0.8]); hp = np.array([1.0, 3.3, 0.3, -2.0])   # meme code (1.3, 1.3), meme
     h3, h4 different
S_m1 = np.array([[0., 0], [0, 0], [1, 1], [0, 0]])   # les deux portes lisent h3 -> m = rang(S^T(I-P)) =
     1
S_m2 = np.array([[0., 0], [0, 0], [1, 0], [0, 1]])   # portes lisent h3 et h4 -> m = 2
for name, S in [("m=1 (S lit h3 seulement)", S_m1), ("m=2 (S lit h3 et h4)", S_m2)]:
    print(f"     {name} : rang S^T(I-P) = {np.linalg.matrix_rank(S.T @ Q4)} | c_T(h) = {np.round(W.
     T@iter4(h,S),4)}  c_T(h') = {np.round(W.T@iter4(hp,S),4)}")
# ce que la profondeur change et ne change pas (m = 1) : pour tout T, h et h' restent indiscernables par
     le code ;
# la fonction calculee sur les coordonnees accessibles, elle, depend de T.
h2 = np.array([2.0, 0.5, -1.0, 0.8])                 # autre entree : code (1.0, 1.3), h3 = -1.0
for T in (4, 8, 40):
    cT, cTp, cT2 = W.T @ iter4(h, S_m1, T), W.T @ iter4(hp, S_m1, T), W.T @ iter4(h2, S_m1, T)
    print(f"     m=1, T={T:2d} : c_T(h) = c_T(h') ? {np.allclose(cT, cTp)} | c_T(h)[0] / c_T(h2)[0] = {cT
     [0] / cT2[0]:.4f}  (rapport initial des codes : {1.3 / 1.0:.4f})")

print("\n=== Exemples : capacite quadrique du triceptron ===")
# --- B.1 inclusion
z = 2.0
print(f"B.1  s=w, kappa=1 : sigma(2)*2 = {sigmoid(2.0)*2:.4f} = SiLU(2) ; kappa=50 : {sigmoid(100)*2:.4f}
      = ReLU(2) ; z=-2 : {sigmoid(-100)*(-2):.2e} = ReLU(-2)")
# --- B.2 XOR : 1 unite + saut
k = 0.05
f_xor = lambda x1, x2: sigmoid(k * x1) * x2 - 0.5 * x2
print("B.2  XOR, f = sigma(0.05 x1) x2 - x2/2  (= 0.0125 x1 x2 + O(kappa^3)) :")
for x1, x2 in [(1, 1), (1, -1), (-1, 1), (-1, -1), (2, 0.5), (-2, 0.5)]:
    print(f"       f({x1:+d},{x2:+.1f}) = {f_xor(x1, x2):+.5f}  (0.0125 x1 x2 = {0.0125*x1*x2:+.5f})
     signe {'OK' if np.sign(f_xor(x1,x2))==np.sign(x1*x2) else 'KO'}")
```

```
# --- B.3 cercle : 2 unites + saut, seuil kappa/4 R^2
f_circ = lambda x1, x2: sigmoid(k * x1) * x1 + sigmoid(k * x2) * x2 - 0.5 * (x1 + x2)
print("B.3  cercle R=1, f = sum_i [sigma(0.05 x_i) x_i - x_i/2] (= 0.0125 (x1^2+x2^2)), seuil 0.0125 :")
for x1, x2 in [(0.9, 0), (1.1, 0), (0.7, 0.7), (0.6, 0.9), (-0.5, 0.5)]:
    v = f_circ(x1, x2); print(f"       f({x1:+.1f},{x2:+.1f}) = {v:.5f} -> {'exterieur' if v > 0.0125
     else 'interieur'} (vrai : {'exterieur' if x1**2+x2**2>1 else 'interieur'})")
# --- B.4 inertie : hyperbole, ellipse, selle
def symm(s, w): return 0.5 * (np.outer(s, w) + np.outer(w, s))
print("B.4  hyperbole x1^2 - x2^2 : s=(1,1), w=(1,-1) : sym(s w^T) =\n", symm(np.array([1.,1]), np.array
     ([1.,-1])))
print("     ellipse 2x1^2 + x2^2 : s=w=(sqrt2,0) et s=w=(0,1) : somme =\n", symm(np.array([2**0.5,0]), np
     .array([2**0.5,0])) + symm(np.array([0.,1]), np.array([0.,1])))
Qs = symm(np.array([1.,0,1]), np.array([1.,0,-1])) + symm(np.array([0.,1,0]), np.array([0.,1,0]))
print("     selle x1^2 + x2^2 - x3^2 (inertie (2,1)) : 2 unites, s=(1,0,1),w=(1,0,-1) + s=w=e2 :\n", Qs,
     "\n     valeurs propres :", np.round(np.linalg.eigvalsh(Qs), 6))
# --- B.5 borne ReLU : erreur du N-gone
print("B.5  cercle R=1 : erreur de Hausdorff du N-gone inscrit, N(m) = 1+m+m(m-1)/2 :")
for m in [2, 3, 4, 6, 10, 20]:
    N = 1 + m + m * (m - 1) // 2; print(f"       m={m:2d} -> N<={N:3d} -> erreur >= 1-cos(pi/N) = {1-np.
     cos(np.pi/N):.6f}")
for eps in [1e-2, 1e-3, 1e-4]:
    N = int(np.ceil(np.pi / np.arccos(1 - eps))); m = int(np.ceil((-1 + np.sqrt(1 + 8 * (N - 1))) / 2))
    print(f"       epsilon={eps:.0e} : N >= {N} cotes -> m >= {m} unites ReLU (N({m}) = {1+m+m*(m-1)//2})
      ; triceptrons doux : 2")
# --- B.6 regime dur : le commutateur 1[x1>0] x2
print("B.6  commutateur 1[x1>0]*x2 : triceptron dur s=e1, w=e2 exact ; un reseau ReLU (continu) ne peut
     que l'approcher.")
for x in [(1, 3), (-1, 3), (0.01, 3), (-0.01, 3)]:
    print(f"       x={x} : sigma(50 x1) x2 = {sigmoid(50*x[0])*x[1]:.4f}")
```

Listing 16: ropn_auto_riemann.py — analytic block on the SPD manifold

```
"""
Bloc oblique ferme + geometrie riemannienne (variete SPD des covariances, cadre standard des BCI).

Ce que deviennent les algorithmes quand les entrees sont des matrices de covariance :
  * bloc fast sur l'espace tangent au point moyen riemannien (metrique affine-invariante, TS-AI),
  * bloc fast sur log-euclidien (logm(Sigma), sans moyenne, TS-LE),
  * bloc fast euclidien sur vec(Sigma) (sans geometrie),
  * bloc a metrique de Kronecker sur les essais bruts (canaux x temps, geometrie doublement oblique),
  * MDM (distance riemannienne minimale aux moyennes de classe) et TS + MLP comme references.

Proposition d'invariance par congruence (verifiee ici) : la chaine {moyenne de Karcher -> espace tangent
     -> bloc fast (LW, GEVD C x C)
-> lecture} est EXACTEMENT invariante par congruence Sigma -> A Sigma A^T pour tout A inversible (gains
     de
canaux, re-referencement, melange), y compris avec le retrecissement de Ledoit-Wolf, parce que les
     vecteurs
tangents ne subissent qu'une rotation orthogonale. La version euclidienne vec(Sigma) ne l'est pas.

Deux regimes de donnees synthetiques :
  (MI)  classes = covariances spatiales differentes, signaux centres (imagerie motrice : ERD/ERS),
  (ERP) classes = motifs moyens differents, covariance commune (potentiels evoques).
numpy / scipy / sklearn seulement.
"""
import time
import numpy as np
from numpy.linalg import eigh, norm
from sklearn.neural_network import MLPClassifier
from ropn_auto_fast import fit_block_fast
from ropn_auto_nobp_diag import readout_centroid_fit_predict
from ropn_auto_bioblique import KroneckerMetricBlock


# ------------------------------------------------------------------ SPD utilities
def sym_fun(S, f):
    lam, V = eigh(S); return (V * f(lam)) @ V.T


def logm(S): return sym_fun(S, np.log)
def expm(S): return sym_fun(S, np.exp)
```

```python
def sqrtm(S): return sym_fun(S, np.sqrt)
def invsqrtm(S): return sym_fun(S, lambda l: 1.0 / np.sqrt(l))


def vec_F(S):
    """Vectorisation isometrique (Frobenius) des matrices symetriques : diag, puis sqrt2 * hors-diag."""
    p = S.shape[-1]; iu = np.triu_indices(p); w = np.where(iu[0] == iu[1], 1.0, np.sqrt(2.0))
    return S[..., iu[0], iu[1]] * w


def karcher_mean(Sig, iters=50, tol=1e-10, return_iters=False):
    G = Sig.mean(0)
    for it in range(1, iters + 1):
        Gs, Gis = sqrtm(G), invsqrtm(G)
        Tm = np.mean([logm(Gis @ S @ Gis) for S in Sig], 0)
        G = Gs @ expm(Tm) @ Gs
        if norm(Tm) < tol: break
    return (G, it) if return_iters else G


def tangent_space(Sig, G):
    Gis = invsqrtm(G); return np.stack([vec_F(logm(Gis @ S @ Gis)) for S in Sig])


def riem_dist(A, B):
    Ais = invsqrtm(A); return norm(logm(Ais @ B @ Ais))


class MDM:
    def fit(self, Sig, y):
        self.classes = np.unique(y); self.means = [karcher_mean(Sig[y == c]) for c in self.classes]; return self

    def predict(self, Sig):
        D = np.array([[riem_dist(M, S) for M in self.means] for S in Sig]); return self.classes[D.argmin(1)]


# ---------------------------------------------------------------- data
def make_trials(regime, C=3, p=8, T=64, n_per=20, sep=0.1, gain_sd=0.3, seed=0, A=None):
    """Essais X (n, p, T). MI : covariance spatiale par classe (base + sep * u u^T, rang 2), moyenne nulle.
    ERP : motif moyen par classe (sep * a * onde), covariance commune. Gain d'amplitude aleatoire par essai
    (exp(gain_sd * N(0,1)), non-stationnarite). A : melange/gains de canaux appliques a tous les essais.
    Retourne X, y (labels 0..C-1)."""
    rng = np.random.default_rng(seed)
    def rand_spd(scale):
        Q, _ = np.linalg.qr(rng.standard_normal((p, p))); lam = np.exp(scale * rng.standard_normal(p))
        return Q @ np.diag(lam) @ Q.T
    base = rand_spd(0.5); X, y = [], []
    for c in range(C):
        if regime == "MI":
            u = rng.standard_normal((p, 2)); Rc = sqrtm(base + sep * (u @ u.T)); Mc = 0.0
        else:
            Rc = sqrtm(base); a = rng.standard_normal((p, 1))
            wave = np.sin(2 * np.pi * rng.uniform(1, 3) * np.linspace(0, 1, T)); Mc = sep * a @ wave[None, :]
        for _ in range(n_per):
            g = np.exp(gain_sd * rng.standard_normal()); X.append(g * (Mc + Rc @ rng.standard_normal((p, T)))); y.append(c)
    X, y = np.stack(X), np.array(y); perm = rng.permutation(len(y)); X, y = X[perm], y[perm]
    if A is not None: X = np.einsum("ij,njt->nit", A, X)
    return X, y


def covariances(X):
    Xc = X - X.mean(2, keepdims=True); return np.einsum("npt,nqt->npq", Xc, Xc) / (X.shape[2] - 1)


# ---------------------------------------------------------------- pipelines
def block_on_features(Ftr, ytr, Fte):
    U, W, r, _ = fit_block_fast(Ftr, ytr, shrink=True, rank="C1")
```

```
    return readout_centroid_fit_predict(Ftr @ W, ytr, Fte @ W), W


def run_all(Xtr, ytr, Xte, yte, seed=0, kron=True):
    Ctr, Cte = covariances(Xtr), covariances(Xte); out = {}
    t0 = time.time(); G = karcher_mean(Ctr); Ttr, Tte = tangent_space(Ctr, G), tangent_space(Cte, G)
    pr, _ = block_on_features(Ttr, ytr, Tte); out["TS-AI + bloc fast"] = ((pr == yte).mean(), time.time() - t0)
    t0 = time.time(); Ltr = np.stack([vec_F(logm(S)) for S in Ctr]); Lte = np.stack([vec_F(logm(S)) for S in Cte])
    pr, _ = block_on_features(Ltr, ytr, Lte); out["TS-LE + bloc fast"] = ((pr == yte).mean(), time.time() - t0)
    t0 = time.time(); pr, _ = block_on_features(vec_F(Ctr), ytr, vec_F(Cte)); out["vec(Sigma) + bloc fast"] = ((pr == yte).mean(), time.time() - t0)
    if kron:
        t0 = time.time(); kb = KroneckerMetricBlock().fit(Xtr, ytr); out["Kronecker (essais bruts)"] = ((kb.predict(Xte) == yte).mean(), time.time() - t0)
    t0 = time.time(); out["MDM"] = ((MDM().fit(Ctr, ytr).predict(Cte) == yte).mean(), time.time() - t0)
    t0 = time.time(); mlp = MLPClassifier((64,), max_iter=800, random_state=seed).fit(Ttr, ytr)
    out["TS-AI + MLP(64)"] = (mlp.score(Tte, yte), time.time() - t0)
    return out


if __name__ == "__main__":
    print("=== Invariance exacte par congruence de la chaine TS-AI + bloc fast (regime MI, sep=0.1) ===")
    X, y = make_trials("MI", p=8, n_per=40, seed=0); n = len(y); Xtr, Xte, ytr, yte = X[:n // 2], X[n // 2:], y[:n // 2], y[n // 2:]
    rng = np.random.default_rng(1)
    A = np.diag(np.exp(rng.uniform(-2.3, 2.3, 8))) @ np.linalg.qr(rng.standard_normal((8, 8)))[0]   # gains 0.1..10 + rotation
    print(f"    A : gains de canaux entre 0.1 et 10 puis rotation, conditionnement {np.linalg.cond(A):.1f}")
    Ctr, Cte = covariances(Xtr), covariances(Xte)
    CtrA = np.einsum("ij,njk,lk->nil", A, Ctr, A); CteA = np.einsum("ij,njk,lk->nil", A, Cte, A)
    G = karcher_mean(Ctr); GA = karcher_mean(CtrA)
    print(f"    moyenne de Karcher equivariante : ||G(A Sigma A^T) - A G A^T|| / ||.|| = {norm(GA - A @ G @ A.T) / norm(GA):.1e}")
    for name, F, FA in [("TS-AI", (tangent_space(Ctr, G), tangent_space(Cte, G)), (tangent_space(CtrA, GA), tangent_space(CteA, GA))),
                        ("TS-LE", (np.stack([vec_F(logm(S)) for S in Ctr]), np.stack([vec_F(logm(S)) for S in Cte])),
                                  (np.stack([vec_F(logm(S)) for S in CtrA]), np.stack([vec_F(logm(S)) for S in CteA]))),
                        ("vec(Sigma)", (vec_F(Ctr), vec_F(Cte)), (vec_F(CtrA), vec_F(CteA)))]:
        pr, W = block_on_features(F[0], ytr, F[1]); prA, WA = block_on_features(FA[0], ytr, FA[1])
        gram_gap = norm(F[0] @ F[0].T - FA[0] @ FA[0].T) / norm(F[0] @ F[0].T)      # Gram identique <=> rotation orthogonale
        code_gap = norm(np.sort(F[0] @ W, 0) - np.sort(FA[0] @ WA, 0)) / norm(F[0] @ W)
        print(f"    {name:10s} : Gram des features conserve a {gram_gap:.1e} | codes conserves a {code_gap:.1e} | "
              f"precision {100*(pr==yte).mean():.1f}% -> apres A : {100*(prA==yte).mean():.1f}% | predictions identiques : {np.mean(pr == prA)*100:.1f}%")

    print("\n=== Regimes MI (covariances, sep=0.1) et ERP (moyennes, sep=0.4), p=8, T=64, gain aleatoire par essai, 5 graines ===")
    for regime, n_per, sep in [("MI", 10, 0.1), ("MI", 30, 0.1), ("ERP", 10, 0.4), ("ERP", 30, 0.4)]:
        acc = {}
        for sd in range(5):
            X, y = make_trials(regime, p=8, n_per=2 * n_per, sep=sep, seed=sd); n = len(y)
            res = run_all(X[:n // 2], y[:n // 2], X[n // 2:], y[n // 2:], seed=sd)
            for k, (a, t) in res.items(): acc.setdefault(k, []).append((a, t))
        print(f"  regime {regime}, {n_per} essais/classe en apprentissage (d_tangent = {8*9//2}) :")
        for k, v in acc.items():
            a = np.array([x[0] for x in v]); t = np.mean([x[1] for x in v])
            print(f"      {k:26s} : {100*a.mean():5.1f} +- {100*a.std():4.1f} %   ({t*1000:6.0f} ms)")

    print("\n=== Changement de session : apprentissage avec gains A1, test avec gains A2 = A1 * exp(perturbation) (MI, 30/classe) ===")
    for eps in [0.0, 0.2, 0.5]:
        acc = {}
        for sd in range(5):
            rng = np.random.default_rng(100 + sd)
            A1 = np.diag(np.exp(rng.uniform(-0.7, 0.7, 8))); A2 = A1 @ expm(eps * (lambda M: 0.5 * (M + M
```

```
    .T))(rng.standard_normal((8, 8))))
160         X, y = make_trials("MI", p=8, n_per=60, sep=0.1, seed=sd); n = len(y)      # memes classes, deux sessions
161         Xtr, ytr = np.einsum("ij,njt->nit", A1, X[:n // 2]), y[:n // 2]; Xte, yte = np.einsum("ij,njt->nit", A2, X[n // 2:]), y[n // 2:]
162         Ctr, Cte = covariances(Xtr), covariances(Xte); r = {}
163         G = karcher_mean(Ctr); Ttr = tangent_space(Ctr, G)
164         r["TS-AI, moyenne d'apprentissage"] = (block_on_features(Ttr, ytr, tangent_space(Cte, G))[0] == yte).mean()
165         G2 = karcher_mean(Cte); pr_rc = block_on_features(Ttr, ytr, tangent_space(Cte, G2))[0]
166         r["TS-AI, re-centre par session"] = (pr_rc == yte).mean()
167         # residu apres re-centrage : rotation orthogonale Q = G2^{-1/2} A G1^{1/2}, A = A2 A1^{-1}, G1 = moyenne test sous A1
168         Cte1 = covariances(np.einsum("ij,njt->nit", A1, X[n // 2:])); G1 = karcher_mean(Cte1); Aq = A2 @ np.linalg.inv(A1)
169         Q = invsqrtm(G2) @ Aq @ sqrtm(G1); pr_ref = block_on_features(Ttr, ytr, tangent_space(Cte1, G1))[0]
170         r["  (||Q - I||_F residuel)"] = norm(Q - np.eye(8)); r["  (predictions = sans changement)"] = (pr_rc == pr_ref).mean()
171         r["TS-LE"] = (block_on_features(np.stack([vec_F(logm(S)) for S in Ctr]), ytr, np.stack([vec_F(logm(S)) for S in Cte]))[0] == yte).mean()
172         r["vec(Sigma)"] = (block_on_features(vec_F(Ctr), ytr, vec_F(Cte))[0] == yte).mean()
173         r["MDM"] = (MDM().fit(Ctr, ytr).predict(Cte) == yte).mean()
174         r["TS-AI + MLP(64), re-centre"] = MLPClassifier((64,), max_iter=800, random_state=sd).fit(Ttr, ytr).score(tangent_space(Cte, karcher_mean(Cte)), yte)
175         for k, v in r.items(): acc.setdefault(k, []).append(v)
176     print(f"  perturbation eps = {eps} :")
177     for k, v in acc.items():
178         if k.startswith("  (||Q"): print(f"      {k:32s} : {np.mean(v):5.2f}")
179         else: print(f"      {k:32s} : {100*np.mean(v):5.1f} +- {100*np.std(v):4.1f} %")
180
181 print("\n=== Passage a l'echelle : p=32 canaux (d_tangent=528), 30 essais/classe, T=100, regime MI ( sans Kronecker : ~1 min) ===")
182 X, y = make_trials("MI", p=32, T=100, n_per=60, sep=0.1, seed=0); n = len(y)
183 res = run_all(X[:n // 2], y[:n // 2], X[n // 2:], y[n // 2:], kron=False)
184 for k, (a, t) in res.items(): print(f"      {k:26s} : {100*a:5.1f} %   ({t*1000:6.0f} ms)")
185
186 print("\n=== Diagnostics : plus petite valeur propre des covariances d'essai, iterations de Karcher ( tolerance 1e-10), distance du point de base a l'identite ===")
187 for p_, T_, seeds_ in [(8, 64, range(5)), (32, 100, [0])]:
188     mins, its, dist = [], [], []
189     for sd in seeds_:
190         X, y = make_trials("MI", p=p_, T=T_, n_per=60, sep=0.1, seed=sd); n = len(y); Cv = covariances(X)
191         mins.append(min(np.linalg.eigvalsh(S).min() for S in Cv)); G, it = karcher_mean(Cv[:n // 2], return_iters=True); its.append(it); dist.append(norm(logm(G)))
192     print(f"      p={p_:2d}, T={T_:3d} : plus petite valeur propre {min(mins):.3f} | iterations de Karcher {its} | ||log G||_F = {np.round(dist, 2).tolist()}")
```

Listing 17: ropn_auto_fourier.py — Fourier operators as per-mode analytic problems, numpy FNO, band selection

```
"""
Operateurs de Fourier dans le ROPN auto sans retropropagation.

Un bloc spectral de FNO est K = F^{-1} R F avec R bloc-diagonal par mode : R(k) in C^{CxC}. Sans canaux ( C=1)
c'est un bloc circulant (un gain complexe par frequence) ; il n'est idempotent que si chaque gain
vaut 0 ou 1 (masque de modes). Avec canaux, chaque R(k) peut etre un projecteur oblique de rang r_k <= C :
la geometrie oblique vit DANS chaque mode, et les K problemes sont independants (parallelisme trivial).

Forme fermee : par mode k, un probleme C x C (moindres carres pour un operateur, GEVD de Fisher pour un
classifieur), plus un test de permutation par mode qui met r_k = 0 sur les modes sans information :
n_modes, hyperparametre du FNO souvent mal regle selon Duruisseaux, Kossaifi et Anandkumar (2025),
peut alors etre fixe par les donnees. Attention : dans E1 et E2 les modes ecartes sont ceux d'energie
nulle (condition initiale a bande limitee, de-aliasing 2/3), elimines par le garde-fou d'energie ; le
test de permutation n'en retire aucun de plus. La selection par test n'est demontree que dans E3.

Trois experiences :
  E1. EDP lineaire periodique (advection-diffusion) : le bloc spectral ferme retrouve exactement le
```

```
      propagateur exp((-nu k^2 - i c k) dt) ; FNO (retropropagation, numpy) et MLP l'approchent.
  E2. Burgers (non lineaire) : bloc spectral lineaire ferme, bloc spectral QUADRATIQUE ferme (dictionnaire
      [u, u^2, u^3] de-aliase, = EDMD en base de Fourier, Petrov-Galerkin de rang 1 par mode), FNO numpy
      a gradients manuels (verifies par differences finies), MLP ; erreur a 1 pas et en deroulement.
  E3. Classification : covariances par bande de frequence + bloc fast avec rang
      par permutation : les bandes sans information recoivent r=0 (selection automatique de bande).
numpy / scipy / sklearn seulement, ASCII seulement.
"""
import time
import numpy as np
from numpy.linalg import norm
from scipy.special import erf
from sklearn.neural_network import MLPRegressor, MLPClassifier
from ropn_auto_fast import fit_block_fast, fwer_select
from ropn_auto_nobp_diag import readout_fit_predict


# ============================================================================ EDP spectrales 1D periodiques
def grf_init(rng, N, kmax=6, s=1.5, n=1):
    """Conditions initiales : serie de Fourier aleatoire a spectre decroissant (champ gaussien)."""
    k = np.arange(N // 2 + 1); U = np.zeros((n, N // 2 + 1), complex)
    for i in range(n):
        amp = (1 + k ** 2) ** (-s / 2) * (k <= kmax)
        U[i] = amp * (rng.standard_normal(N // 2 + 1) + 1j * rng.standard_normal(N // 2 + 1)); U[i, 0] = 0
    u = np.fft.irfft(U, n=N, axis=-1) * N
    return u / np.abs(u).max(axis=-1, keepdims=True)


def advdiff_step(u, dt, c=1.0, nu=0.02):
    N = u.shape[-1]; k = np.fft.rfftfreq(N, 1.0 / N)
    return np.fft.irfft(np.fft.rfft(u, axis=-1) * np.exp((-nu * k ** 2 - 1j * c * k) * dt), n=N, axis=-1)


def burgers_step(u, dt, nu=0.05, n_sub=50):
    """u_t + u u_x = nu u_xx, pseudo-spectral, facteur integrant + RK4 sur le terme non lineaire, 2/3 de-aliasing."""
    N = u.shape[-1]; k = np.fft.rfftfreq(N, 1.0 / N); h = dt / n_sub
    dealias = (np.abs(k) < N / 3.0); E = np.exp(-nu * k ** 2 * h / 2); E2 = E ** 2
    def NL(uh):
        ur = np.fft.irfft(uh, n=N, axis=-1)
        return -0.5j * k * np.fft.rfft(ur * ur, axis=-1) * dealias
    uh = np.fft.rfft(u, axis=-1)
    for _ in range(n_sub):
        a = NL(uh); b = NL(E * (uh + h / 2 * a)); c_ = NL(E * uh + h / 2 * b); d = NL(E2 * uh + h * E * c_)
        uh = E2 * uh + h / 6 * (E2 * a + 2 * E * (b + c_) + d)
    return np.fft.irfft(uh, n=N, axis=-1)


def make_pairs(step, rng, n_traj, n_steps, N, dt, **kw):
    u0 = grf_init(rng, N, n=n_traj); X, Y, traj = [], [], [u0]
    u = u0
    for _ in range(n_steps):
        v = step(u, dt, **kw); X.append(u); Y.append(v); traj.append(v); u = v
    return np.concatenate(X), np.concatenate(Y), np.stack(traj, 1)   # (n_traj*n_steps, N), idem, (n_traj, n_steps+1, N)


def rel_l2(pred, true): return norm(pred - true) / norm(true)


def rollout_err(model_step, traj):
    """Erreur relative L2 moyenne sur l'horizon en deroulant depuis traj[:, 0]."""
    u = traj[:, 0]; errs = []
    for t in range(1, traj.shape[1]):
        u = model_step(u); errs.append(rel_l2(u, traj[:, t]))
    return float(np.mean(errs)), float(errs[-1])


# ============================================================================ blocs spectraux en forme fermee
def powers_dealiased(u, m):
```

```
    """[u, u^2, ..., u^m] calculés sur une grille 2x plus fine (zero-padding spectral) puis re-tronqués."""
    N = u.shape[-1]; uh = np.fft.rfft(u, axis=-1); M = 2 * N
    up = np.fft.irfft(uh, n=M, axis=-1) * (M / N)
    feats = []
    for p in range(1, m + 1):
        fh = np.fft.rfft(up ** p, axis=-1) * (N / M)
        feats.append(fh[..., :N // 2 + 1])
    return np.stack(feats, -1)                                        # (n, K_all, m) complexe


class SpectralClosedForm:
    """Par mode k : y_hat_k ~ w_k^H z_k, z_k = Fourier des puissances [u, ..., u^m] (m=1 : lineaire, gain complexe
    par mode = bloc circulant). Moindres carres complexes avec ridge ; test de permutation par mode
    (n_perm appariements aleatoires) : les modes dont la variance expliquee ne depasse pas le quantile 1-alpha
    du nul recoivent w_k = 0 (rang 0). Deroulement : re-lift a chaque pas."""
    def __init__(self, m=1, ridge=1e-6, n_perm=100, alpha=0.05, seed=0):
        self.m, self.ridge, self.n_perm, self.alpha, self.seed = m, ridge, n_perm, alpha, seed

    def fit(self, X, Y):
        rng = np.random.default_rng(self.seed); Z = powers_dealiased(X, self.m); Yh = np.fft.rfft(Y, axis=-1)
        n, K, m = Z.shape; self.N = X.shape[-1]; self.w = np.zeros((K, m), complex); self.keep = np.zeros(K, bool)
        self.r2 = np.zeros(K); self.r2_null = np.zeros(K)
        for k in range(K):
            z, y = Z[:, k, :], Yh[:, k]
            if norm(y) < 1e-12 * max(1.0, norm(Yh)): continue
            G = z.conj().T @ z + self.ridge * np.trace(z.conj().T @ z).real / m * np.eye(m)
            w = np.linalg.solve(G, z.conj().T @ y); r2 = 1 - norm(y - z @ w) ** 2 / norm(y) ** 2
            null = []
            for _ in range(self.n_perm):
                zp = z[rng.permutation(n)]; wp = np.linalg.solve(zp.conj().T @ zp + G - z.conj().T @ z, zp.conj().T @ y)
                null.append(1 - norm(y - zp @ wp) ** 2 / norm(y) ** 2)
            self.r2[k], self.r2_null[k] = r2, np.quantile(null, 1 - self.alpha)
            if r2 > self.r2_null[k]: self.w[k] = w; self.keep[k] = True
        return self

    def predict(self, X):
        Z = powers_dealiased(X, self.m); Yh = np.einsum("nkm,km->nk", Z, self.w)
        return np.fft.irfft(Yh, n=self.N, axis=-1)

    def n_params(self): return int(2 * self.keep.sum() * self.m)
    def n_modes(self): return int(self.keep.sum())


# ============================================================================ FNO 1D numpy, gradients manuels
def gelu_phi(x): return 0.5 * (1 + erf(x / np.sqrt(2)))                  # Phi(x) ; GELU = x Phi(x)
def dgelu(x, phi): return phi + x * np.exp(-x ** 2 / 2) / np.sqrt(2 * np.pi)


class FNO1dNumpy:
    """FNO 1D : lift (2->C) -> L x [conv spectrale (K modes, C x C complexe) + saut lineaire + biais, GELU] ->
    projection (C->1), sortie residuelle y = u + Q(h). DFT reelle explicite par matrices (K x N) pour des
    gradients exacts et verifiables."""
    def __init__(self, N, C=16, K=12, L=2, seed=0):
        rng = np.random.default_rng(seed); self.N, self.C, self.K, self.L = N, C, K, L
        n = np.arange(N); k = np.arange(K)[:, None]
        self.Cm = np.cos(2 * np.pi * k * n / N); self.Sm = -np.sin(2 * np.pi * k * n / N)            # (K, N)
        d = np.where(np.arange(K) == 0, 1.0, 2.0)[:, None]; self.Ci = d * self.Cm / N; self.Si = d * self.Sm / N
        s = 1.0 / np.sqrt(C)
        self.p = {"P": rng.standard_normal((2, C)) * 0.5, "p0": np.zeros(C), "q": rng.standard_normal(C) * s, "q0": np.zeros(1)}
        for l in range(L):
            self.p[f"Rr{l}"] = rng.standard_normal((K, C, C)) * s; self.p[f"Ri{l}"] = rng.standard_normal((K, C, C)) * s
            self.p[f"W{l}"] = rng.standard_normal((C, C)) * s; self.p[f"b{l}"] = np.zeros(C)
```

```
148
149     def forward(self, X, cache=False):
150         p = self.p; a0 = np.stack([X, np.ones_like(X)], 1); v = p["P"].T[None] @ a0 + p["p0"][None, :,
        None]
151         st = [(a0, v)]
152         for l in range(self.L):
153             vr, vi = (v @ self.Cm.T).transpose(2, 0, 1), (v @ self.Sm.T).transpose(2, 0, 1)      # (K, B,
         C)
154             Rr, Ri = p[f"Rr{l}"], p[f"Ri{l}"]                                                    # (K, C,
        C)
155             wr = (vr @ Rr - vi @ Ri).transpose(1, 2, 0); wi = (vr @ Ri + vi @ Rr).transpose(1, 2, 0)   #
        (B, C, K)
156             z = wr @ self.Ci + wi @ self.Si + p[f"W{l}"].T[None] @ v + p[f"b{l}"][None, :, None]
157             phi = gelu_phi(z); v_new = z * phi; st.append((vr, vi, z, v, phi)); v = v_new
158         y = X + np.tensordot(v, p["q"], axes=([1], [0])) + p["q0"]
159         return (y, st, v) if cache else y
160
161     def loss_grads(self, X, Y):
162         y, st, vL = self.forward(X, cache=True); p = self.p; B, N = X.shape
163         loss = np.mean((y - Y) ** 2); dy = 2 * (y - Y) / (B * N); g = {}
164         g["q"] = np.tensordot(vL, dy, axes=([0, 2], [0, 1])); g["q0"] = np.array([dy.sum()]); dv = dy[:,
        None, :] * p["q"][None, :, None]
165         for l in reversed(range(self.L)):
166             vr, vi, z, v, phi = st[l + 1]; dz = dv * dgelu(z, phi)
167             g[f"W{l}"] = np.tensordot(v, dz, axes=([0, 2], [0, 2])); g[f"b{l}"] = dz.sum((0, 2))
168             dwr, dwi = (dz @ self.Ci.T).transpose(2, 0, 1), (dz @ self.Si.T).transpose(2, 0, 1)   # (K, B
        , C)
169             Rr, Ri = p[f"Rr{l}"], p[f"Ri{l}"]; vrT, viT = vr.transpose(0, 2, 1), vi.transpose(0, 2, 1)
170             g[f"Rr{l}"] = vrT @ dwr + viT @ dwi; g[f"Ri{l}"] = -viT @ dwr + vrT @ dwi
171             dvr = dwr @ Rr.transpose(0, 2, 1) + dwi @ Ri.transpose(0, 2, 1); dvi = -dwr @ Ri.transpose(0,
         2, 1) + dwi @ Rr.transpose(0, 2, 1)
172             dv = dvr.transpose(1, 2, 0) @ self.Cm + dvi.transpose(1, 2, 0) @ self.Sm + p[f"W{l}"][None] @
         dz
173         a0, _ = st[0]; g["P"] = np.tensordot(a0, dv, axes=([0, 2], [0, 2])); g["p0"] = dv.sum((0, 2))
174         return loss, g
175
176     def gradcheck(self, X, Y, eps=1e-6):
177         _, g = self.loss_grads(X, Y); worst = 0.0
178         for k, v in self.p.items():
179             idx = np.unravel_index(np.arange(0, v.size, max(1, v.size // 7)), v.shape)
180             for i in zip(*idx):
181                 old = v[i]; v[i] = old + eps; lp = np.mean((self.forward(X) - Y) ** 2); v[i] = old - eps
182                 lm = np.mean((self.forward(X) - Y) ** 2); v[i] = old
183                 worst = max(worst, abs((lp - lm) / (2 * eps) - g[k][i]) / (abs(g[k][i]) + 1e-8))
184         return worst
185
186     def fit(self, X, Y, steps=2000, lr=1e-2, wd=0.0, batch=200, seed=0, verbose=False):
187         rng = np.random.default_rng(seed); m = {k: np.zeros_like(v) for k, v in self.p.items()}; s = {k:
        np.zeros_like(v) for k, v in self.p.items()}
188         b1, b2, eps = 0.9, 0.999, 1e-8; n = len(X); self.hist = []
189         for t in range(1, steps + 1):
190             idx = rng.choice(n, min(batch or n, n), replace=False)
191             loss, g = self.loss_grads(X[idx], Y[idx]); self.hist.append(loss)
192             lr_t = lr * 0.5 * (1 + np.cos(np.pi * t / steps))
193             for k in self.p:
194                 gk = g[k] + wd * self.p[k]; m[k] = b1 * m[k] + (1 - b1) * gk; s[k] = b2 * s[k] + (1 - b2)
        * gk ** 2
195                 self.p[k] -= lr_t * (m[k] / (1 - b1 ** t)) / (np.sqrt(s[k] / (1 - b2 ** t)) + eps)
196             if verbose and t % 500 == 0: print(f"      step {t} loss {loss:.3e}")
197         return self
198
199     def predict(self, X): return self.forward(X)
200     def n_params(self): return int(sum(v.size for v in self.p.values()))
201
202
203 # ============================================================================ E3 : bandes de frequence + bloc
        fast
204 def make_band_trials(C=3, p=8, T=64, n_per=20, band=(8, 12), sep=0.5, gain_sd=0.7, seed=0):
205     """Essais (n, p, T) : bruit de base large bande (covariance spatiale commune) dont le gain spectral
        varie
206     aleatoirement par essai et par bande (nuisance, exp(gain_sd N(0,1))), + composante de classe (motif
        spatial
207     u_c, rang 2) portee UNIQUEMENT par les modes temporels de 'band'. Classes 0..C-1."""
```

```
    rng = np.random.default_rng(seed); k = np.fft.rfftfreq(T, 1.0 / T); mask = (k >= band[0]) & (k < band[1])
    Q, _ = np.linalg.qr(rng.standard_normal((p, p))); base = Q @ np.diag(np.exp(0.5 * rng.standard_normal(p))) @ Q.T
    Rb = np.linalg.cholesky(base); X, y = [], []; nb = 8; bidx = np.minimum(k.astype(int) // 4, nb - 1)
    for c in range(C):
        u = rng.standard_normal((p, 2))
        for _ in range(n_per):
            g = np.exp(gain_sd * rng.standard_normal(nb))[bidx]
            noise = np.fft.irfft(np.fft.rfft(Rb @ rng.standard_normal((p, T)), axis=-1) * g, n=T, axis=-1)
            a = np.fft.irfft(np.fft.rfft(rng.standard_normal((2, T)), axis=-1) * mask, n=T, axis=-1) * np.sqrt(T / (2 * mask.sum()))
            X.append(noise + sep * u @ a); y.append(c)
    X, y = np.stack(X), np.array(y); perm = rng.permutation(len(y)); return X[perm], y[perm]


def band_features(X, bands):
    """Pour chaque bande : matrice cross-spectrale reelle (somme des |X_k|^2 sur la bande), log-euclidienne,
    vectorisee isometriquement. Retourne une liste de (n, p(p+1)/2)."""
    Xh = np.fft.rfft(X - X.mean(2, keepdims=True), axis=-1); p = X.shape[1]; iu = np.triu_indices(p)
    wgt = np.where(iu[0] == iu[1], 1.0, np.sqrt(2.0)); feats = []
    for lo, hi in bands:
        S = np.einsum("nck,ndk->ncd", Xh[:, :, lo:hi], Xh[:, :, lo:hi].conj()).real / X.shape[2]
        S += 1e-3 * np.trace(S, axis1=1, axis2=2)[:, None, None] / p * np.eye(p)
        lam, V = np.linalg.eigh(S); Lg = np.einsum("ncd,nd,ned->nce", V, np.log(lam), V)
        feats.append(Lg[:, iu[0], iu[1]] * wgt)
    return feats


def bands_classify(Xtr, ytr, Xte, yte, bands, mode="auto", seed=0):
    """mode : "broadband" (une seule covariance), "all" (rang C-1 dans chaque bande), "auto" (rang teste par bande,
    controle par bande), "fwer" (une bande n'est retenue que si elle passe le controle global min-p sur les bandes ;
    son rang est alors le rang teste)."""
    Ftr, Fte = band_features(Xtr, bands), band_features(Xte, bands); codes_tr, codes_te, ranks = [], [], []
    fits = []
    for b in range(len(bands)):
        if mode == "broadband": break
        fits.append(fit_block_fast(Ftr[b], ytr, shrink=True, rank="perm" if mode in ("auto", "fwer") else "C1", n_perm=200, seed=seed, min_rank=0))
    keep = fwer_select([f[3] for f in fits], 0.05)[0] if mode == "fwer" else np.ones(len(fits), bool)
    for b, (U, W, r, info) in enumerate(fits):
        r = int(info["r_tested"]) if keep[b] else 0                  # le test de permutation peut donner 0
        ranks.append(r)
        if r > 0: codes_tr.append(Ftr[b] @ W[:, :r]); codes_te.append(Fte[b] @ W[:, :r])
    if mode == "broadband":
        Fb_tr, Fb_te = band_features(Xtr, [(0, Xtr.shape[2] // 2 + 1)])[0], band_features(Xte, [(0, Xte.shape[2] // 2 + 1)])[0]
        U, W, r, _ = fit_block_fast(Fb_tr, ytr, shrink=True, rank="C1"); codes_tr, codes_te, ranks = [Fb_tr @ W], [Fb_te @ W], [int(r)]
    if not codes_tr: return 1.0 / len(np.unique(ytr)), ranks
    # lecture ridge sur la CONCATENATION des codes de plusieurs blocs (et, pour la comparaison a lecture egale, sur le
    # code du bloc large bande) : la regle des centroides n'a la signification de la LDA que sur le code d'UN bloc
    pr = readout_fit_predict(np.hstack(codes_tr), ytr, np.hstack(codes_te), ridge=1e-3)
    return float((pr == yte).mean()), ranks


# ============================================================================ main
if __name__ == "__main__":
    rng = np.random.default_rng(0); N = 64
    print("=== Verification du FNO numpy : gradients manuels vs differences finies ===")
    f = FNO1dNumpy(N=8, C=3, K=3, L=2, seed=1); Xg = rng.standard_normal((3, 8)); Yg = rng.standard_normal((3, 8))
    print(f"    ecart relatif max = {f.gradcheck(Xg, Yg):.1e}")

    # ------------------------------------------------------------------ E0 : correctif du test de
```

```
    permutation
print("\n=== E0. Test de permutation en lot : S_W partage (variante de comparaison) vs scatter total S_T (defaut) ===")
print("    taux de faux positifs (rang teste > 0) sur bruit pur, C=3, 200 tirages, nominal 5% :")
for n_, d_ in [(30, 36), (60, 36), (120, 36)]:
    fp = {"within": 0, "total": 0}
    for t in range(200):
        Xn = rng.standard_normal((n_, d_)); yn = np.repeat(np.arange(3), n_ // 3)
        for pm in fp: fp[pm] += fit_block_fast(Xn, yn, rank="perm", n_perm=100, seed=t, perm_metric=pm)[3]["r_tested"] > 0
    print(f"      n={n_:4d} d={d_:3d} : S_W partage {fp['within']/2:5.1f}%   S_T {fp['total']/2:5.1f}%")

# ---------------------------------------------------------------- E1 : advection-diffusion
print("\n=== E1. Advection-diffusion (c=1, nu=0.02, dt=0.1, N=64) : 10 trajectoires x 20 pas en apprentissage ===")
Xtr, Ytr, _ = make_pairs(advdiff_step, rng, 10, 20, N, 0.1); Xte, Yte, Tte = make_pairs(advdiff_step, rng, 20, 20, N, 0.1)
k = np.fft.rfftfreq(N, 1.0 / N); true_gain = np.exp((-0.02 * k ** 2 - 1j * k) * 0.1)
t0 = time.time(); sl = SpectralClosedForm(m=1).fit(Xtr, Ytr); t_sl = time.time() - t0
kept = np.where(sl.keep)[0]
print(f"    bloc spectral ferme (m=1) : {sl.n_modes()} modes retenus (indices {kept.min()}..{kept.max()}), {sl.n_params()} parametres, {t_sl*1000:.0f} ms")
print(f"      gain estime vs propagateur exact sur les modes retenus : ecart max {np.abs(sl.w[sl.keep, 0] - true_gain[sl.keep]).max():.1e}")
print(f"      erreur 1 pas {rel_l2(sl.predict(Xte), Yte):.2e} ; deroulement 20 pas : moyenne {rollout_err(sl.predict, Tte)[0]:.2e}, final {rollout_err(sl.predict, Tte)[1]:.2e}")
t0 = time.time(); fno = FNO1dNumpy(N, C=16, K=12, L=2, seed=0).fit(Xtr, Ytr); t_f = time.time() - t0
print(f"    FNO numpy (C=16, K=12, L=2, {fno.n_params()} parametres, Adam 2000 pas, {t_f:.1f} s) : erreur 1 pas {rel_l2(fno.predict(Xte), Yte):.2e} ; "
      f"deroulement moyenne {rollout_err(fno.predict, Tte)[0]:.2e}, final {rollout_err(fno.predict, Tte)[1]:.2e}")
t0 = time.time(); mlp = MLPRegressor(hidden_layer_sizes=(256,), max_iter=2000, random_state=0).fit(Xtr, Ytr); t_m = time.time() - t0
print(f"    MLP (256, {sum(c.size for c in mlp.coefs_) + sum(b.size for b in mlp.intercepts_)} parametres, {t_m:.1f} s) : erreur 1 pas {rel_l2(mlp.predict(Xte), Yte):.2e} ; "
      f"deroulement moyenne {rollout_err(mlp.predict, Tte)[0]:.2e}, final {rollout_err(mlp.predict, Tte)[1]:.2e}")

# ---------------------------------------------------------------- E2 : Burgers
for n_traj in (10, 50):
    print(f"\n=== E2. Burgers (nu=0.05, dt=0.1, N=64) : {n_traj} trajectoires x 20 pas en apprentissage ({n_traj*20} paires), test 20 trajectoires ===")
    rng = np.random.default_rng(1)
    Xtr, Ytr, _ = make_pairs(burgers_step, rng, n_traj, 20, N, 0.1); Xte, Yte, Tte = make_pairs(burgers_step, rng, 20, 20, N, 0.1)
    print(f"    {'methode':52s} {'params':>7s} {'temps':>8s} {'1 pas':>9s} {'deroul. moy':>12s} {'deroul. fin':>12s}")
    rows = []
    for m in (1, 2, 3):
        t0 = time.time(); sc = SpectralClosedForm(m=m).fit(Xtr, Ytr); t = time.time() - t0
        rows.append((f"bloc spectral ferme m={m} ({sc.n_modes()} modes auto)", sc.n_params(), t, rel_l2(sc.predict(Xte), Yte), *rollout_err(sc.predict, Tte)))
    for K in (4, 12, 24):
        t0 = time.time(); fno = FNO1dNumpy(N, C=16, K=K, L=2, seed=0).fit(Xtr, Ytr); t = time.time() - t0
        rows.append((f"FNO numpy C=16 L=2 K={K}", fno.n_params(), t, rel_l2(fno.predict(Xte), Yte), *rollout_err(fno.predict, Tte)))
    t0 = time.time(); mlp = MLPRegressor(hidden_layer_sizes=(256,), max_iter=2000, random_state=0).fit(Xtr, Ytr); t = time.time() - t0
    rows.append(("MLP (256)", sum(c.size for c in mlp.coefs_) + sum(b.size for b in mlp.intercepts_), t, rel_l2(mlp.predict(Xte), Yte), *rollout_err(mlp.predict, Tte)))
    for name, npar, t, e1, em, ef in rows:
        print(f"    {name:52s} {npar:7d} {t:7.1f}s {e1:9.2e} {em:12.2e} {ef:12.2e}")

# ---------------------------------------------------------------- E3 : bandes
print("\n=== E3. Classification : information de classe confinee aux modes temporels 8..11, gain spectral de nuisance par essai (p=8, T=64, C=3, sep=0.5), 5 graines ===")
bands = [(4 * b, 4 * b + 4) for b in range(8)]
for n_per in (10, 30):
    res = {"large bande (rang C-1)": [], "8 bandes, rang C-1 chacune": [], "8 bandes, rang par permutation (auto)": [], "8 bandes, controle global min-p": [], "MLP(64) sur les 8 bandes": []}; sel = []; sel_fw = []
```

```
        for sd in range(5):
            X, y = make_band_trials(n_per=2 * n_per, seed=sd); n = len(y); Xtr, ytr, Xte, yte = X[:n //
    2], y[:n // 2], X[n // 2:], y[n // 2:]
            res["large bande (rang C-1)"].append(bands_classify(Xtr, ytr, Xte, yte, bands, "broadband")
    [0])
            res["8 bandes, rang C-1 chacune"].append(bands_classify(Xtr, ytr, Xte, yte, bands, "all")[0])
            a, ranks = bands_classify(Xtr, ytr, Xte, yte, bands, "auto", seed=sd); res["8 bandes, rang
    par permutation (auto)"].append(a); sel.append(ranks)
            a, ranks = bands_classify(Xtr, ytr, Xte, yte, bands, "fwer", seed=sd); res["8 bandes,
    controle global min-p"].append(a); sel_fw.append(ranks)
            Ftr, Fte = np.hstack(band_features(Xtr, bands)), np.hstack(band_features(Xte, bands))
            res["MLP(64) sur les 8 bandes"].append(MLPClassifier((64,), max_iter=800, random_state=sd).
    fit(Ftr, ytr).score(Fte, yte))
        print(f"  {n_per} essais/classe :")
        for kk, v in res.items(): print(f"      {kk:42s} : {100*np.mean(v):5.1f} +- {100*np.std(v):4.1f}
    %")
        print(f"      rangs par bande, controle par bande  : {sel}   (bande 2 = modes 8..11 porte l'
    information)")
        print(f"      rangs par bande, controle global min-p : {sel_fw}")
```

Listing 18: ropn_auto_risques.py — risk audit

```
"""
Audit des sept risques classiques d'un bloc oblique a rang selectionne, applique au bloc analytique :
  R1 amplification du bruit        : ||P||_2, precision sous bruit d'entree et hors distribution ;
  R2 sous-espaces presque degeneres: sigma_min(Q_W^T Q_U) (= 1/||P||_2) ;
  R3 perte de capacite             : rang retenu contre rangs fixes (la reinjection est traitee dans la
     section petits echantillons) ;
  R4 instabilite en profondeur     : conditionnement du Jacobien GLOBAL de T pas (lineaire, porte non
                                     normalisee h <- (1-a) h + a P h, porte normalisee h <- h + (a/T)(P -
      I) h) ;
  R5 selection instable : variabilite du rang (test corrige) et de la configuration du treillis
                                     entre graines ;
  R6 gain attribue a tort a l'obliquite : variante orthogonale et variante bas rang non contrainte, meme
                                     rang, meme lecture ;
  R7 regularisation qui modifie le modele : ||P^2 - P||, et ||P_LW - P_brut|| / ||P_brut|| en fonction de
      n.
numpy / scipy / sklearn seulement.
"""
import numpy as np
from numpy.linalg import norm, qr, svd
from sklearn.model_selection import train_test_split
from ropn_auto_nobp import make_multiclass, make_closed_form_gate, recurse_gated, apply_P
from ropn_auto_fast import fit_block_fast
from ropn_auto_nobp_diag import readout_fit_predict, readout_centroid_fit_predict
from ropn_auto_lattice import fit_lattice_auto


def split(X, y, sd): return train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)


def fit_orthogonal(X, y, r):
    """Variante orthogonale de meme rang : U = W = base orthonormee des r premieres directions
    inter-classes (vecteurs singuliers de D). P = Q Q^T."""
    classes, yi = np.unique(y, return_inverse=True); ns = np.bincount(yi).astype(float); mu = X.mean(0)
    mus = np.stack([X[yi == k].mean(0) for k in range(len(classes))]); D = ((mus - mu).T * np.sqrt(ns))
    Q, _, _ = svd(D, full_matrices=False); Q = Q[:, :r]; return Q, Q


def fit_lowrank_unconstrained(X, y, r, lam=1e-2):
    """Variante bas rang NON contrainte (pas de projecteur) : regression de rang reduit de l'indicatrice
    de classe sur X (ridge), tronquee au rang r ; code = X @ W_r, meme lecture que le bloc."""
    classes, yi = np.unique(y, return_inverse=True); Y = np.eye(len(classes))[yi]; Xc = X - X.mean(0); Yc
      = Y - Y.mean(0)
    B = np.linalg.solve(Xc.T @ Xc + lam * np.trace(Xc.T @ Xc) / X.shape[1] * np.eye(X.shape[1]), Xc.T @
     Yc)
    Uy, s, Vt = svd(Xc @ B, full_matrices=False); return B @ Vt[:r].T                # (d, r)


def acc(Ftr, ytr, Fte, yte, readout="centroid"):
    """lecture de reference : centroides sur le code (regle LDA pour le bloc) ; "ls" : moindres carres.""
```

```
    "
    f = readout_centroid_fit_predict if readout == "centroid" else readout_fit_predict
    return float((f(Ftr, ytr, Fte) == yte).mean())


if __name__ == "__main__":
    C, d, seeds = 4, 20, 20
    print("=== R1/R2/R6 : norme de P, angle entre sous-espaces, robustesse, et variantes de controle (C=4, d=20, 20 graines) ===")
    print("    meme lecture pour les trois variantes : lecture par centroides sur le code (tableau principal), puis")
    print("    lecture par moindres carres sur le code (controle de sensibilite a la lecture)")
    print(f"{'cas':24s} {'||P||_2':>8s} {'s_min(QwQu)':>11s} {'1/s_min':>8s} | precision (%) : oblique / orthogonal / bas-rang libre")
    print(f"{'':24s} {'':8s} {'':11s} {'':8s} |  propre    bruit 0.5   bruit 1.0   dilatation x2")
    tab_ls = []
    for kappa in (10.0, 200.0):
        for n_per in (15, 100):
            Pn, smin, res = [], [], {k: [] for k in ("obl", "ort", "lr", "obl_ls", "ort_ls", "lr_ls")}
            for sd in range(seeds):
                X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=kappa, sep=0.8, seed=sd)
                Xtr, Xte, ytr, yte = split(X, y, sd); rng = np.random.default_rng(100 + sd)
                U, W, r, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); P = U @ W.T
                Pn.append(norm(P, 2)); Qw, Qu = qr(W)[0], qr(U)[0]; smin.append(svd(Qw.T @ Qu, compute_uv=False).min())
                Qo, _ = fit_orthogonal(Xtr, ytr, r); Wl = fit_lowrank_unconstrained(Xtr, ytr, r)
                sig = Xtr.std(0).mean()
                tests = [Xte, Xte + 0.5 * sig * rng.standard_normal(Xte.shape), Xte + 1.0 * sig * rng.standard_normal(Xte.shape),
                         Xte.mean(0) + 2.0 * (Xte - Xte.mean(0))]                  # bruit isotrope ; nuage de test dilate d'un facteur 2 autour de sa moyenne
                res["obl"].append([acc(Xtr @ W, ytr, T @ W, yte) for T in tests])
                res["ort"].append([acc(Xtr @ Qo, ytr, T @ Qo, yte) for T in tests])
                res["lr"].append([acc(Xtr @ Wl, ytr, T @ Wl, yte) for T in tests])
                res["obl_ls"].append([acc(Xtr @ W, ytr, T @ W, yte, "ls") for T in tests])
                res["ort_ls"].append([acc(Xtr @ Qo, ytr, T @ Qo, yte, "ls") for T in tests])
                res["lr_ls"].append([acc(Xtr @ Wl, ytr, T @ Wl, yte, "ls") for T in tests])
            m = {k: 100 * np.mean(v, 0) for k, v in res.items()}
            print(f"kappa={kappa:5.0f} n/cl={n_per:3d}      {np.mean(Pn):8.2f} {np.mean(smin):11.3f} {1/np.mean(smin):8.2f} | "
                  + "  ".join(f"{m['obl'][j]:4.1f}/{m['ort'][j]:4.1f}/{m['lr'][j]:4.1f}" for j in range(4)))
            tab_ls.append(f"kappa={kappa:5.0f} n/cl={n_per:3d}      lecture MC | "
                          + "  ".join(f"{m['obl_ls'][j]:4.1f}/{m['ort_ls'][j]:4.1f}/{m['lr_ls'][j]:4.1f}" for j in range(4)))
    for line in tab_ls: print(line)
    print("    (||P||_2 = 1/sigma_min(Q_W^T Q_U) exactement : les deux colonnes sont le meme risque ; la norme euclidienne de P")
    print("     est 1/sin de l'angle minimal entre im P et ker P, et vaut 1 dans la metrique S_W^{-1} ou P est orthogonal.)")

    print("\n=== R3 : rang retenu contre rangs fixes (kappa=200, sep=0.8, 20 graines) ===")
    for n_per in (5, 15, 100):
        rows = {r: [] for r in (1, 2, 3)}; auto = []; rs = []
        for sd in range(seeds):
            X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=200.0, sep=0.8, seed=sd); Xtr, Xte, ytr, yte = split(X, y, sd)
            U, W, r, info = fit_block_fast(Xtr, ytr, shrink=True, rank="perm", seed=sd); rs.append(info["r_tested"])
            for rf in rows:
                Uf, Wf, _, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); rows[rf].append(acc(Xtr @ Wf[:, :rf], ytr, Xte @ Wf[:, :rf], yte))
            auto.append(acc(Xtr @ W, ytr, Xte @ W, yte))
        print(f"  n/cl={n_per:3d} : r=1 {100*np.mean(rows[1]):.1f}  r=2 {100*np.mean(rows[2]):.1f}  r=3 (C-1) {100*np.mean(rows[3]):.1f}  | auto {100*np.mean(auto):.1f}  rang teste moyen {np.mean(rs):.2f} +- {np.std(rs):.2f}")
    print("  (reglage fin par retropropagation : aucun gain net observe dans le protocole teste, voir ropn_auto_finetune.py.)")

    print("\n=== R4 : Jacobien global de T pas partages (kappa=200, n/cl=100, d=20) ===")
    X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=100, kappa=200.0, sep=0.8, seed=0); Xtr, Xte, ytr, yte = split(X, y, 0)
    U, W, r, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); P = U @ W.T; s, b =
```

```
    make_closed_form_gate(U, W, Xtr)
from ropn_auto_nobp import sigmoid
def rec_norm(h, T):                                    # porte normalisee : h <- h + (a/L)(P - I) h avec L = T
    for _ in range(T):
        a = sigmoid(h @ s + b)[:, None]; h = h + (a / T) * (h @ P.T - h)
    return h
def worst_jac(f):                                      # Jacobien numerique de h0 -> h_T en 5 points test, pire conditionnement
    worst = (0, np.inf, 1, 0)
    for h0 in Xte[:5]:
        Jg = np.zeros((d, d)); eps = 1e-6
        for j in range(d):
            e = np.zeros(d); e[j] = eps; Jg[:, j] = (f((h0 + e)[None])[0] - f((h0 - e)[None])[0]) / (2 * eps)
        sg = svd(Jg, compute_uv=False)
        if sg.max() / sg.min() > worst[2]: worst = (sg.max(), sg.min(), sg.max() / sg.min(), np.linalg.det(Jg))
    return worst
nP = norm(P, 2)
print(f"  ||P||_2 = {nP:.3f}")
print(f"  {'T':>3s} | {'lineaire (I+P/T)^T : cond':>26s} {'borne':>7s} | {'porte non normalisee : cond':>28s} {'det':>10s} | {'porte normalisee (L=T) : cond':>30s} {'det':>10s}")
for T in (1, 2, 4, 8, 32):
    J = np.linalg.matrix_power(np.eye(d) + P / T, T); sv = svd(J, compute_uv=False); cL = (1 + 1 / T) ** T - 1
    bound = (1 + cL * nP) * (1 + cL * nP / (1 + cL))  # cond(I + c_L P) <= (1 + c_L ||P||)(1 + c_L ||P|| / (1 + c_L))
    w1 = worst_jac(lambda h: recurse_gated(U, W, s, b, h, T)); w2 = worst_jac(lambda h: rec_norm(h, T))
    print(f"  {T:3d} | {sv.max()/sv.min():26.2f} {bound:7.2f} | {w1[2]:28.2e} {w1[3]:10.2e} | {w2[2]:30.2f} {w2[3]:10.2e}")
print("  (configuration recommandee : T=1, porte analytique, lecture sur le code ; la prediction ne depend alors pas de T.)")

print("\n=== R5 : stabilite de la selection (rang, configuration du treillis) entre graines ===")
for kappa in (10.0, 200.0):
    for n_per in (5, 15, 50):
        rs = []
        for sd in range(seeds):
            X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=kappa, sep=0.8, seed=sd); Xtr, Xte, ytr, yte = split(X, y, sd)
            rs.append(fit_block_fast(Xtr, ytr, shrink=True, rank="perm", seed=sd)[3]["r_tested"])
        print(f"  rang (test S_T) kappa={kappa:5.0f} n/cl={n_per:3d} : moyenne {np.mean(rs):.2f}, ecart-type {np.std(rs):.2f}, histogramme {np.bincount(rs, minlength=4).tolist()} (rang vrai 3)")
cfgs = []
for sd in range(8):
    X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=30, kappa=200.0, sep=0.8, seed=sd); Xtr, Xte, ytr, yte = split(X, y, sd)
    _, cfg = fit_lattice_auto(Xtr, ytr, seed=sd); cfgs.append((cfg["use_codes_sq"], 0 if not cfg["use_resid_quad"] else cfg["k_quad"]))
print(f"  treillis AUTO (codes^2, k_quad) sur 8 graines, donnees gaussiennes : {cfgs}")

print("\n=== R7 : idempotence et effet du retrecissement sur l'operateur (kappa=200, 10 graines) ===")
for n_per in (15, 100, 1000):
    idem, dP, alphas = [], [], []
    for sd in range(10):
        X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=200.0, sep=0.8, seed=sd); Xtr, Xte, ytr, yte = split(X, y, sd)
        U, W, r, info = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); P = U @ W.T; idem.append(norm(P @ P - P) / norm(P))
        U0, W0, _, _ = fit_block_fast(Xtr, ytr, shrink=False, rank="C1"); P0 = U0 @ W0.T; dP.append(norm(P - P0) / norm(P0))
        from ropn_auto_fast import ledoit_wolf_coef
        yi = np.unique(ytr, return_inverse=True)[1]; mus = np.stack([Xtr[yi == k].mean(0) for k in range(C)]); alphas.append(ledoit_wolf_coef(Xtr - mus[yi])[0])
    print(f"  n/cl={n_per:4d} : ||P^2-P||/||P|| = {np.max(idem):.1e} | ||P_LW - P_brut||/||P_brut|| = {np.mean(dP):.3f} | coefficient LW moyen {np.mean(alphas):.3f}")
print("  (operateur exact : P = U W^T, W = S^{-1} D Q Lambda^{-1/2}, U = D Q Lambda^{-1/2}, S = (1-a) S_W + a mu I (Ledoit-Wolf),")
print("   D Q Lambda Q^T = decomposition de D^T S^{-1} D ; W^T U = I_r par construction.)")
```

Listing 19: ropn_auto_exemple_complet.py — worked example and figures

```
"""
Exemple complet du fonctionnement d'un bloc analytique avec selection du rang (d = 3, C = 3), avec toutes les matrices
affichees, et generation des figures pedagogiques du rapport (fichiers PDF, matplotlib uniquement).

  Exemple : donnees gaussiennes a covariance commune anisotrope, trois classes dont les moyennes vivent dans un
  plan ; le bloc ferme calcule S_W (retreci), D, Z = S^{-1} D, M = D^T Z (3 x 3), ses valeurs propres, le test
  de permutation (rang 2 attendu), U et W, verifie W^T U = I, projette un point test, lit le code et predit.
  Figures :
    fig_oblique_geometry.pdf   geometrie d'un projecteur oblique en 2D (image, noyau, angle, norme)
    fig_worked_example.pdf     l'exemple complet : nuage 3D projete, directions u_i / w_i, codes, test de rang
    fig_recursion.pdf          recursion gatee : le code est invariant, le residu se contracte
    fig_norm_vs_n.pdf          obliquite admissible ||P||_2 contre n et borne metrique (sqrt(kappa(S)) + 1/sqrt(kappa(S)))/2
    fig_triceptron.pdf         XOR avec un triceptron + saut et cercle avec deux triceptrons (Thm 4)
    fig_double_descent.pdf     norme du projecteur brut (pseudo-inverse de S_W) autour du seuil n_train - C ~ d, contre retreci
"""
import numpy as np
from numpy.linalg import norm, eigh, inv, qr, svd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from ropn_auto_fast import fit_block_fast, ledoit_wolf_coef
from ropn_auto_nobp_diag import readout_fit_predict, readout_centroid_fit_predict, centroid_readout_weights
from ropn_auto_nobp import make_multiclass, make_closed_form_gate, recurse_gated

np.set_printoptions(precision=3, suppress=True, linewidth=120)
plt.rcParams.update({"font.size": 9, "axes.titlesize": 9.5, "legend.fontsize": 8, "figure.dpi": 110})
C_OBL, C_ORT, C_KER, C_IMG = "#b2182b", "#2166ac", "#4d4d4d", "#1b7837"


def mat(name, A):
    """Affiche une matrice avec son nom (3 decimales)."""
    A = np.atleast_2d(A); print(f"  {name} =")
    for row in A: print("     [" + " ".join(f"{v:8.3f}" for v in row) + "]")


# ============================================================================ 1. l'exemple complet
def worked_example(seed=0):
    rng = np.random.default_rng(seed); d, C, n_per = 3, 3, 30
    # covariance commune anisotrope : x1, x2 correles (0.7), x3 correle a x1 (0.5) mais sans difference de moyenne
    Sigma = np.array([[1.0, 0.7, 0.5], [0.7, 1.0, 0.2], [0.5, 0.2, 0.6]])
    R = np.linalg.cholesky(Sigma)
    mus = np.array([[0.0, 0.0, 0.0], [3.5, 0.5, 0.0], [0.5, 3.5, 0.0]])
    X = np.concatenate([mus[c] + (R @ rng.standard_normal((d, n_per))).T for c in range(C)]); y = np.repeat(np.arange(C), n_per)
    perm = rng.permutation(len(y)); X, y = X[perm], y[perm]
    Xtr, ytr, Xte, yte = X[:60], y[:60], X[60:], y[60:]
    print("=== Exemple complet : d = 3, C = 3, 30 exemples par classe, decoupage aleatoire non stratifie en 60 (apprentissage) et 30 (test) ===")
    classes, yi = np.unique(ytr, return_inverse=True); ns = np.bincount(yi).astype(float); mu = Xtr.mean(0)
    mus_hat = np.stack([Xtr[yi == k].mean(0) for k in range(C)])
    print("  tailles de classes :", ns.astype(int)); mat("moyennes de classe (lignes)", mus_hat); mat("moyenne globale", mu[None])
    Xc = Xtr - mus_hat[yi]; S_W = Xc.T @ Xc / len(Xtr); a, muLW = ledoit_wolf_coef(Xc); S = (1 - a) * S_W + a * muLW * np.eye(d)
    mat("S_W (intra-classe, denominateur n)", S_W); print(f"  Ledoit-Wolf : a = {a:.3f}, mu = {muLW:.3f}"); mat("S = (1-a) S_W + a mu I", S)
    D = ((mus_hat - mu).T * np.sqrt(ns)) / np.sqrt(C - 1); mat("D = [sqrt(n_c)(mu_c - mu)]/sqrt(C-1)  (d x C)", D)
    Z = np.linalg.solve(S, D); mat("Z = S^{-1} D", Z)
    M = D.T @ Z; M = 0.5 * (M + M.T); mat("M = D^T S^{-1} D  (C x C, rang <= C-1)", M)
    lam, Q = eigh(M); o = np.argsort(lam)[::-1]; lam, Q = lam[o], Q[:, o]
```

```
    print(f"  valeurs propres de M : {np.round(lam, 4)}   (la derniere est nulle : rang structurel C-1 = 2)")
    U_, W_, r, info = fit_block_fast(Xtr, ytr, shrink=True, rank="perm", n_perm=500, seed=seed)
    print(f"  test de permutation (metrique S_T, 500 permutations) : valeurs propres dans la metrique S_T = {np.round(info['eigenvalues_test'][:C-1], 2)}, seuil 95% = {info['null_q95']:.4f}, rang teste = {info['r_tested']}, rang retenu r = {r}")
    W = Z @ Q[:, :r] / np.sqrt(lam[:r]); U = D @ Q[:, :r] / np.sqrt(lam[:r])
    mat("W = Z Q_r Lambda_r^{-1/2}  (mesure, d x r)", W); mat("U = D Q_r Lambda_r^{-1/2}  (representation, d x r)", U)
    mat("W^T U", W.T @ U); P = U @ W.T; mat("P = U W^T", P); print(f"  ||P^2 - P|| = {norm(P @ P - P):.1e}, rang(P) = {np.linalg.matrix_rank(P)}")
    Qu, Qw = qr(U)[0], qr(W)[0]; smin = svd(Qw.T @ Qu, compute_uv=False).min()
    print(f"  ||P||_2 = {norm(P, 2):.3f} = 1/sigma_min(Q_W^T Q_U) = {1/smin:.3f} ; angle minimal image/noyau = {np.degrees(np.arcsin(smin)):.1f} deg ; borne metrique (sqrt(k)+1/sqrt(k))/2 = {0.5*(np.sqrt(np.linalg.cond(S)) + 1/np.sqrt(np.linalg.cond(S))):.2f} <= sqrt(k) = {np.sqrt(np.linalg.cond(S)):.2f}, k = kappa(S)")
    print(f"  ecart entre le bloc 'a la main' et fit_block_fast : ||U - U_fast|| = {norm(np.abs(U) - np.abs(U_)):.1e} (au signe des colonnes pres)")
    # lecture
    # lecture de reference : centroides de classe dans l'espace du code (regle LDA, aucun ajustement)
    Ctr, Cte = Xtr @ W, Xte @ W; _, Wro = centroid_readout_weights(Ctr, ytr)
    mat("centroides de classe dans l'espace du code (C x r)", Wro[:-1].T)
    print(f"  termes constants -||m_k||^2/2 + log pi_k : {np.round(Wro[-1], 3)}")
    pred = classes[(np.c_[Cte, np.ones(len(Cte))] @ Wro).argmax(1)]
    pred_ls = readout_fit_predict(Ctr, ytr, Cte)
    print(f"  precision test : {100*np.mean(pred == yte):.1f}%  (parametres : d r + r C + C = {d*r + r*C + C}) ; "
          f"lecture par moindres carres sur le meme code : {100*np.mean(pred_ls == yte):.1f}%, predictions identiques sur {100*np.mean(pred_ls == pred):.0f}% des points")
    Sinv_mu = np.linalg.solve(S, mus_hat.T)                          # LDA de covariance S, calculee directement
    sc_lda = Xte @ Sinv_mu - 0.5 * np.sum(mus_hat.T * Sinv_mu, 0) + np.log(np.bincount(ytr) / len(ytr))
    print(f"  LDA calculee directement avec la meme covariance S : predictions identiques sur {100*np.mean(classes[sc_lda.argmax(1)] == pred):.0f}% des points de test")
    x = Xte[0]; c = W.T @ x; print(f"  point test x = {np.round(x, 3)} (classe {yte[0]}) : code c = W^T x = {np.round(c, 3)}, P x = {np.round(P @ x, 3)}, residu (I-P) x = {np.round(x - P @ x, 3)}")
    print(f"     distances au carre aux centroides ||c - m_k||^2 = {np.round(np.sum((c[None] - Wro[:-1].T) ** 2, 1), 3)}")
    print(f"     scores = {np.round(np.r_[c, 1] @ Wro, 3)} -> classe predite {pred[0]}")
    # recursion gatee : code invariant
    s, b = make_closed_form_gate(U, W, Xtr)
    print("  recursion gatee partagee (invariance du code) sur ce point :")
    for T in (0, 1, 2, 4):
        hT = recurse_gated(U, W, s, b, x[None], T)[0] if T > 0 else x
        print(f"     T={T} : h_T = {np.round(hT, 3)}, code W^T h_T = {np.round(W.T @ hT, 3)}, ||(I-P) h_T|| = {norm(hT - P @ hT):.3f}")
    # test de rang : distribution nulle pour la figure
    null = []
    rng2 = np.random.default_rng(1); Xbar = Xtr - mu
    aT, mT = ledoit_wolf_coef(Xbar); ST = (1 - aT) * Xbar.T @ Xbar / len(Xtr) + aT * mT * np.eye(d)
    MT = D.T @ np.linalg.solve(ST, D); lamT = np.sort(eigh(0.5 * (MT + MT.T))[0])[::-1]
    for _ in range(500):
        yp = rng2.permutation(yi); musp = np.stack([Xtr[yp == k].mean(0) for k in range(C)]); Dp = ((musp - mu).T * np.sqrt(ns)) / np.sqrt(C - 1)
        Mp = Dp.T @ np.linalg.solve(ST, Dp); null.append(eigh(0.5 * (Mp + Mp.T))[0].max())
    return dict(Xtr=Xtr, ytr=ytr, Xte=Xte, yte=yte, U=U, W=W, P=P, mus=mus_hat, mu=mu, lamT=lamT, null=np.array(null), s=s, b=b, Wro=Wro, S=S)


# ============================================================================ 2. figures
def fig_geometry():
    """Projecteur oblique en 2D : image = span(u), noyau = span(w)^perp, point x, Px, Q_S x."""
    u = np.array([1.0, 0.35]); u /= norm(u); w = np.array([1.0, -1.2]); w /= norm(w); w /= (w @ u)
    P = np.outer(u, w); Q = np.outer(u, u); x = np.array([0.6, 1.35])
    kdir = np.array([-w[1], w[0]]); kdir /= norm(kdir)                    # direction du noyau (w^perp)
    fig, ax = plt.subplots(figsize=(5.0, 4.0))
    t = np.linspace(-2.4, 2.4, 2)
    ax.plot(t * u[0], t * u[1], color=C_IMG, lw=1.6, label=r"image $\mathrm{im}\,P=\mathrm{span}(u)$")
    ax.plot(t * kdir[0], t * kdir[1], color=C_KER, lw=1.6, ls="--", label=r"kernel $\ker P=\mathrm{span}(w)^\perp$")
    wn = w / norm(w)
    ax.annotate("", xy=wn * 1.0, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=C_OBL, lw=1.4)); ax.text(wn[0] * 1.05, wn[1] * 1.05 - 0.1, r"$w$ (measurement direction, $\perp$ kernel)", color=C_OBL
```

```
    , fontsize=8)
    ax.annotate("", xy=u * 1.0, xytext=(0, 0), arrowprops=dict(arrowstyle="->", color=C_IMG, lw=1.4)); ax.text(u[0] * 0.55, u[1] * 0.55 - 0.3, r"$u$ (representation)", color=C_IMG, fontsize=8)
    Px, Qx = P @ x, Q @ x; xw = wn * (wn @ x)                                  # projection orthogonale de x sur span(w) : etape 1
    ax.plot([x[0], xw[0]], [x[1], xw[1]], color=C_OBL, lw=0.9, ls="-."); ax.plot(*xw, "D", color=C_OBL, ms=4)
    ax.text(xw[0] + 0.1, xw[1] - 0.3, r"$\hat x_W$ (step 1: orthogonal onto span$(w)$)", color=C_OBL, fontsize=7)
    ax.annotate("", xy=Px, xytext=xw, arrowprops=dict(arrowstyle="->", color=C_OBL, lw=0.9, ls="-."))
    ax.plot(*x, "ko"); ax.text(x[0] + 0.06, x[1] + 0.04, r"$x$")
    ax.plot([x[0], Px[0]], [x[1], Px[1]], color=C_OBL, lw=1.2); ax.plot(*Px, "o", color=C_OBL); ax.text(Px[0] - 0.9, Px[1] - 0.42, r"$Px$ (oblique, along the kernel)", color=C_OBL, fontsize=8)
    ax.plot([x[0], Qx[0]], [x[1], Qx[1]], color=C_ORT, lw=1.2, ls=":"); ax.plot(*Qx, "s", color=C_ORT, ms=5); ax.text(Qx[0] + 0.12, Qx[1] + 0.22, r"$Q_Sx$ (orthogonal)", color=C_ORT, fontsize=8)
    th = np.degrees(np.arccos(abs(u @ kdir)))
    ax.text(-2.3, -1.9, rf"$\theta_{{\min}}={th:.0f}^\circ,\ \|P\|_2=1/\sin\theta_{{\min}}=1/\cos\angle(u,w)={1/np.sin(np.radians(th)):.2f}$", fontsize=8)
    ax.text(-2.3, -2.15, "two steps: project orthogonally onto span$(w)$, then stretch by $1/\\cos\\angle(u,w)$ along $u$ (step 2)", fontsize=7)
    ax.set_xlim(-2.5, 2.5); ax.set_ylim(-2.2, 2.2); ax.set_aspect("equal"); ax.legend(loc="upper left", frameon=False)
    ax.set_title("Oblique projection: onto the image, parallel to the kernel", fontsize=9); ax.axis("off")
    fig.tight_layout(); fig.savefig("fig_oblique_geometry.pdf"); plt.close(fig)


def fig_worked(ex):
    Xtr, ytr, Xte, yte, U, W, mus, mu = ex["Xtr"], ex["ytr"], ex["Xte"], ex["yte"], ex["U"], ex["W"], ex["mus"], ex["mu"]
    fig, axs = plt.subplots(1, 3, figsize=(11.5, 3.6)); cols = ["#1b9e77", "#d95f02", "#7570b3"]
    ax = axs[0]
    for k in range(3):
        ax.scatter(Xtr[ytr == k, 0], Xtr[ytr == k, 1], s=14, color=cols[k], alpha=0.7, label=f"class {k}")
    for i in range(U.shape[1]):                                    # directions affichees a longueur 2.2 (echelle du nuage)
        ud = U[:2, i] / norm(U[:2, i]) * 2.2; wd = W[:2, i] / norm(W[:2, i]) * 2.2
        ax.annotate("", xy=mu[:2] + ud, xytext=mu[:2], arrowprops=dict(arrowstyle="->", color=C_IMG, lw=1.8))
        ax.annotate("", xy=mu[:2] + wd, xytext=mu[:2], arrowprops=dict(arrowstyle="->", color=C_OBL, lw=1.8, ls="--"))
        ax.text(*(mu[:2] + ud * 1.12), rf"$u_{i+1}$", color=C_IMG, fontsize=10); ax.text(*(mu[:2] + wd * 1.12), rf"$w_{i+1}$", color=C_OBL, fontsize=10)
    ax.set_aspect("equal"); ax.legend(frameon=False, loc="lower right"); ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
    ax.set_title("Data (first two coordinates), $u_i$ in the span of the means,\n$w_i=S^{-1}u_i$ tilted by the within-class covariance")
    ax = axs[1]; Cte = Xte @ W; Ctr = Xtr @ W
    for k in range(3):
        ax.scatter(Ctr[ytr == k, 0], Ctr[ytr == k, 1], s=14, color=cols[k], alpha=0.5); ax.scatter(Cte[yte == k, 0], Cte[yte == k, 1], s=30, color=cols[k], marker="^", edgecolor="k", lw=0.5)
    ax.set_xlabel(r"$c_1=w_1^\top x$"); ax.set_ylabel(r"$c_2=w_2^\top x$"); ax.set_title("Oblique code $c=W^\\top x$ (train: dots, test: triangles)\nthe read-out is a linear map of this $r=2$ vector")
    ax = axs[2]; null, lamT = ex["null"], ex["lamT"]
    ax.hist(null, bins=30, color="#bbbbbb", label="null (500 permutations)"); q = np.quantile(null, 0.95); ax.axvline(q, color="k", ls="--", label="95th percentile")
    for i, l in enumerate(lamT[:2]): ax.axvline(l, color=C_OBL, lw=2); ax.text(l, ax.get_ylim()[1] * (0.85 - 0.12 * i), rf"$\lambda_{i+1}$", color=C_OBL, ha="right")
    ax.set_xlabel(r"largest eigenvalue of $D_p^\top S_T^{-1}D_p$"); ax.set_title("Rank test: two eigenvalues above the null,\n$r=2=C-1$ retained"); ax.legend(frameon=False, fontsize=7.5)
    fig.tight_layout(); fig.savefig("fig_worked_example.pdf"); plt.close(fig)


def fig_recursion(ex):
    """Invariance du code en 2D : trajectoires h_t de la recursion gatee, code invariant, residu contracte."""
    U, W, P, s, b = ex["U"], ex["W"], ex["P"], ex["s"], ex["b"]
    fig, axs = plt.subplots(1, 2, figsize=(9, 3.6))
    ax = axs[0]; Xte, yte = ex["Xte"], ex["yte"]; cols = ["#1b9e77", "#d95f02", "#7570b3"]
    for x, k in zip(Xte[:9], yte[:9]):
        traj = [x] + [recurse_gated(U, W, s, b, x[None], T)[0] for T in range(1, 7)]; traj = np.array(traj)
```

```
        ax.plot(traj[:, 0], traj[:, 2], "-", color=cols[k], lw=1, alpha=0.8); ax.plot(traj[0, 0], traj[0, 2], "o", color=cols[k], ms=4); ax.plot(traj[-1, 0], traj[-1, 2], "s", color=cols[k], ms=4)
    ax.set_xlabel(r"$h_1$"); ax.set_ylabel(r"$h_3$ (kernel-dominated coordinate)"); ax.set_title("Gated recursion $h_{t+1}=(1-a)h_t+aPh_t$, $T=0\\to6$\n(dots: $h_0$, squares: $h_6$): motion along the kernel only")
    ax = axs[1]; Ts = np.arange(0, 13)
    for x, k in zip(Xte[:6], yte[:6]):
        codes = [norm(W.T @ (recurse_gated(U, W, s, b, x[None], T)[0] if T > 0 else x) - W.T @ x) for T in Ts]
        res = [norm((np.eye(3) - P) @ (recurse_gated(U, W, s, b, x[None], T)[0] if T > 0 else x)) for T in Ts]
        ax.semilogy(Ts, np.maximum(res, 1e-17), "-", color=cols[k], lw=1); ax.semilogy(Ts, np.maximum(codes, 1e-17), ":", color=cols[k], lw=1)
    ax.set_xlabel(r"$T$"); ax.set_ylabel("norm"); ax.set_title("Residual $\\|(I-P)h_T\\|$ decays as $(1-a)^T$ (solid);\ncode change $\\|W^\\top h_T-W^\\top h_0\\|$ stays at $10^{-16}$ (dotted)")
    fig.tight_layout(); fig.savefig("fig_recursion.pdf"); plt.close(fig)


def fig_norm_vs_n():
    from sklearn.model_selection import train_test_split
    ns, raw, lw, bound = [5, 10, 15, 30, 100, 300, 1000], [], [], []
    for n_per in ns:
        r_, l_, b_ = [], [], []
        for sd in range(10):
            X, y, *_ = make_multiclass(n_classes=4, d=20, n_per=n_per, kappa=200.0, sep=0.8, seed=sd)
            Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
            U1, W1, _, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); l_.append(norm(U1 @ W1.T, 2))
            yi = np.unique(ytr, return_inverse=True)[1]; mus = np.stack([Xtr[yi == k].mean(0) for k in range(4)]); Xc = Xtr - mus[yi]
            a, mu = ledoit_wolf_coef(Xc); S = (1 - a) * Xc.T @ Xc / len(Xtr) + a * mu * np.eye(20); k_ = np.sqrt(np.linalg.cond(S)); b_.append(0.5 * (k_ + 1 / k_))
            if len(Xtr) > 24:
                U0, W0, _, _ = fit_block_fast(Xtr, ytr, shrink=False, rank="C1"); r_.append(norm(U0 @ W0.T, 2))
        raw.append(np.mean(r_) if r_ else np.nan); lw.append(np.mean(l_)); bound.append(np.mean(b_))
    fig, ax = plt.subplots(figsize=(5, 3.4))
    ax.semilogx(ns, lw, "o-", color=C_OBL, label=r"$\|P\|_2$, shrunk metric (Ledoit--Wolf)"); ax.semilogx(ns, raw, "s--", color=C_KER, label=r"$\|P\|_2$, raw $S_W$")
    ax.semilogx(ns, bound, "^:", color=C_ORT, label=r"metric bound $\frac{1}{2}(\sqrt{\kappa(S)}+1/\sqrt{\kappa(S)})$")
    ax.set_xlabel("training samples per class"); ax.set_ylabel("spectral norm"); ax.set_title("Admissible obliquity grows with $n$ as the shrinkage vanishes\n($d=20$, $C=4$, $\\kappa(S_W)=200$, 10 seeds)")
    ax.legend(frameon=False); fig.tight_layout(); fig.savefig("fig_norm_vs_n.pdf"); plt.close(fig)


def fig_triceptron():
    sig = lambda z: 1 / (1 + np.exp(-z)); g = np.linspace(-2, 2, 201); X1, X2 = np.meshgrid(g, g)
    fig, axs = plt.subplots(1, 2, figsize=(8.6, 3.6))
    f_xor = sig(0.05 * X1) * X2 - 0.5 * X2
    cs = axs[0].contourf(X1, X2, f_xor / 0.0125, levels=21, cmap="RdBu_r"); axs[0].contour(X1, X2, f_xor, levels=[0], colors="k")
    axs[0].set_title("XOR with ONE triceptron $+$ skip:\n$f=\\sigma(0.05x_1)x_2-x_2/2\\approx 0.0125\\, x_1x_2$"); fig.colorbar(cs, ax=axs[0], shrink=0.85)
    f_c = sig(0.05 * X1) * X1 + sig(0.05 * X2) * X2 - 0.5 * (X1 + X2)
    cs = axs[1].contourf(X1, X2, f_c / 0.0125, levels=21, cmap="viridis"); axs[1].contour(X1, X2, f_c, levels=[0.0125], colors="w")
    th = np.linspace(0, 2 * np.pi, 200); axs[1].plot(np.cos(th), np.sin(th), "r--", lw=1, label="unit circle")
    axs[1].set_title("Unit circle with TWO triceptrons $+$ skip:\n$f\\approx0.0125(x_1^2+x_2^2)$, level set (white) vs circle (red)"); axs[1].legend(frameon=False, loc="lower left"); fig.colorbar(cs, ax=axs[1], shrink=0.85)
    for ax in axs: ax.set_aspect("equal"); ax.set_xlabel(r"$x_1$"); ax.set_ylabel(r"$x_2$")
    fig.tight_layout(); fig.savefig("fig_triceptron.pdf"); plt.close(fig)


def fig_double_descent():
    """Norme du projecteur de Fisher brut (pseudo-inverse de S_W) et retreci autour du seuil d'interpolation."""
    from numpy.linalg import pinv
    from sklearn.model_selection import train_test_split
    def block_metric(Xtr_, ytr_, Smat):
        cl, yi_ = np.unique(ytr_, return_inverse=True); Cn = len(cl); ns_ = np.bincount(yi_).astype(float
```

```
    ); mu_ = Xtr_.mean(0)
208         mus_ = np.stack([Xtr_[yi_ == k].mean(0) for k in range(Cn)]); D_ = ((mus_ - mu_).T * np.sqrt(ns_)
    ) / np.sqrt(Cn - 1)
209         Z_ = pinv(Smat) @ D_; M_ = D_.T @ Z_; M_ = 0.5 * (M_ + M_.T); lam_, Q_ = eigh(M_); o_ = np.
    argsort(lam_)[::-1]; lam_, Q_ = lam_[o_], Q_[:, o_]
210         lr_ = np.clip(lam_[:Cn - 1], 1e-12, None); return D_ @ Q_[:, :Cn - 1] / np.sqrt(lr_), Z_ @ Q_[:,
    :Cn - 1] / np.sqrt(lr_)
211     d, C = 20, 4; ratios, nr, nl, ar, al = [], [], [], [], []
212     for n_per in list(range(3, 21)) + [25, 30, 40, 50]:
213         rows = []
214         for sd in range(20):
215             X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=200.0, sep=0.8, seed=sd)
216             Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)
217             yi = np.unique(ytr, return_inverse=True)[1]; mus = np.stack([Xtr[yi == k].mean(0) for k in
    range(C)]); Xc = Xtr - mus[yi]
218             S_W = Xc.T @ Xc / len(Xtr); a, mu = ledoit_wolf_coef(Xc); S = (1 - a) * S_W + a * mu * np.eye
    (d)
219             Ur, Wr = block_metric(Xtr, ytr, S_W); Ul, Wl = block_metric(Xtr, ytr, S)
220             rows.append(((len(Xtr) - C) / d, norm(Ur @ Wr.T, 2), norm(Ul @ Wl.T, 2),
221                          (readout_centroid_fit_predict(Xtr @ Wr, ytr, Xte @ Wr) == yte).mean(), (
    readout_centroid_fit_predict(Xtr @ Wl, ytr, Xte @ Wl) == yte).mean()))
222         m = np.mean(rows, 0); ratios.append(m[0]); nr.append(m[1]); nl.append(m[2]); ar.append(m[3]); al.
    append(m[4])
223     fig, axs = plt.subplots(1, 2, figsize=(9, 3.4))
224     axs[0].plot(ratios, nr, "s-", color=C_KER, label=r"raw Fisher projector ($S_W^{+}$)"); axs[0].plot(
    ratios, nl, "o-", color=C_OBL, label="shrunk metric (Ledoit--Wolf)")
225     axs[0].axvline(1.0, color="k", ls=":", lw=1); axs[0].set_xlabel(r"$(n_{\mathrm{train}}-C)/d$"); axs
    [0].set_ylabel(r"$\|P\|_2$"); axs[0].set_xlim(0, 7.2)
226     axs[0].set_title("Norm of the block across the interpolation threshold\n(dotted: $S_W$ becomes full
    rank)"); axs[0].legend(frameon=False)
227     axs[1].plot(ratios, 100 * np.array(ar), "s-", color=C_KER, label="raw"); axs[1].plot(ratios, 100 * np
    .array(al), "o-", color=C_OBL, label="shrunk")
228     axs[1].axvline(1.0, color="k", ls=":", lw=1); axs[1].set_xlabel(r"$(n_{\mathrm{train}}-C)/d$"); axs
    [1].set_ylabel("test accuracy (%)"); axs[1].set_xlim(0, 7.2)
229     axs[1].set_title("Test accuracy of the code read-out\n($d=20$, $C=4$, $\\kappa(S_W)=200$, 20 seeds)")
    ; axs[1].legend(frameon=False)
230     fig.tight_layout(); fig.savefig("fig_double_descent.pdf"); plt.close(fig)
231     print("  seuil d'interpolation : (n_train-C)/d, ||P|| brut, ||P|| LW, acc brut, acc LW")
232     for r_, a_, b_, c_, d_ in zip(ratios, nr, nl, ar, al): print(f"     {r_:5.2f}  {a_:6.2f}  {b_:5.2f}
    {100*c_:5.1f}  {100*d_:5.1f}")
233
234
235 if __name__ == "__main__":
236     ex = worked_example()
237     fig_geometry(); fig_worked(ex); fig_recursion(ex); fig_norm_vs_n(); fig_triceptron();
    fig_double_descent()
238     print("\n  figures ecrites : fig_oblique_geometry.pdf, fig_worked_example.pdf, fig_recursion.pdf,
    fig_norm_vs_n.pdf, fig_triceptron.pdf, fig_double_descent.pdf")
```

Listing 20: ropn_auto_stabilite.py — stability limits of oblique projectors and their control

```
1  """
2  Limites de stabilite des projecteurs obliques et leur controle dans le bloc ferme : verifications
       numeriques.
3
4    S1  norme et angle : ||P||_2 = 1/sin(theta_min) = 1/sigma_min(Q_W^T Q_U) ; ||I - P||_2 = ||P||_2 ;
         la borne ||P(x+dx) - Px|| <= ||P||_2 ||dx|| est atteinte dans la direction singuliere dominante ;
5        exemple 2x2 P_a = [[1, a], [0, 0]] : valeurs propres {0, 1}, ||P_a||_2 = sqrt(1 + a^2).
6    S2  le conditionnement du cross-Gram ne suffit pas : W^T U = I_r (kappa = 1) alors que ||P||_2 est
       arbitraire.
7    S3  borne metrique : pour un projecteur M-orthogonal, ||P||_2 <= (sqrt(kappa) + 1/sqrt(kappa))/2 <=
       sqrt(kappa),
8        kappa = kappa(M) (inegalite de Wielandt ; la premiere borne est atteinte au rang un pour u = v_min/
       sqrt(l_min) + v_max/sqrt(l_max)) ; avec M = S^{-1},
9        S = (1-a) S_W + a mu I (Ledoit-Wolf), ||P||_2 <= sqrt(kappa(S)) <= sqrt(1 + (1-a) lambda_max(S_W) /
         (a mu)) :
10       le retrecissement BORNE l'obliquite admissible ; mesure de ||P||_2 brut / retreci en fonction de n.
11   S4  meilleure approximation : pas en norme euclidienne (||x - Px|| <= ||P||_2 ||x - Q_S x||, borne
       atteinte),
12       mais oui dans la metrique de Fisher ||.||_M (P est M-orthogonal).
13   S5  perte d'information : une perturbation dans ker(W^T) ne change ni le code ni la prediction ;
14
```

```
      la forme residuelle I + P/L n'est pas idempotente mais inversible, d'inverse I - P/(L+1).
  S6  regulariser le cross-Gram, (W^T U + lambda I)^{-1}, detruit l'idempotence ; regulariser la METRIQUE
      (retrecissement de S_W) la conserve exactement.
  S7  profondeur : ||T||_2 <= prod ||P_l||_2 (borne lache) ; inversibilite, stabilite et conditionnement
     sont
      trois proprietes distinctes (det != 0 avec sigma_min -> 0).
  S8  spectre singulier : les r valeurs singulieres non nulles de P sont 1/cos(theta_i), theta_i = angles
      principaux entre span(U) et span(W) (lecture en deux etapes : projection orthogonale sur span(W),
     puis
      re-expression dans span(U) avec un facteur 1/cos(theta_i) par direction) ; le code ne depend de x
     que par
      sa projection orthogonale sur span(W).
  S9  seuil d'interpolation : ||P||_2 du projecteur de Fisher BRUT (pseudo-inverse de S_W) presente un
     pic
      quand n_train - C ~ d (S_W presque singuliere) ; le retrecissement supprime le pic.
numpy / scipy / sklearn seulement.
"""
import numpy as np
from numpy.linalg import norm, qr, svd, eigvalsh, inv
from sklearn.model_selection import train_test_split
from ropn_auto_nobp import make_multiclass, make_closed_form_gate, recurse_gated
from ropn_auto_fast import fit_block_fast, ledoit_wolf_coef
from ropn_auto_nobp_diag import readout_centroid_fit_predict   # lecture de reference : centroides sur le
      code (regle LDA)


def split(X, y, sd): return train_test_split(X, y, test_size=0.3, stratify=y, random_state=sd)


def block(X, y, shrink=True):
    U, W, r, info = fit_block_fast(X, y, shrink=shrink, rank="C1"); return U, W, U @ W.T


def shrunk_metric(X, y):
    """S = (1-a) S_W + a mu I (convention sklearn, denominateur n), identique au solveur du bloc fast."""
    classes, yi = np.unique(y, return_inverse=True); mus = np.stack([X[yi == k].mean(0) for k in range(
     len(classes))])
    Xc = X - mus[yi]; a, mu = ledoit_wolf_coef(Xc); S_W = Xc.T @ Xc / len(X)
    return (1 - a) * S_W + a * mu * np.eye(X.shape[1]), a, mu, S_W


if __name__ == "__main__":
    rng = np.random.default_rng(0); C, d = 4, 20
    print("=== S1. Norme, angle, borne d'amplification atteinte, exemple 2x2 ===")
    for a in (0.0, 1.0, 10.0):
        Pa = np.array([[1.0, a], [0.0, 0.0]]); print(f"  P_a, a={a:4.1f} : P^2 = P ? {np.allclose(Pa @ Pa
     , Pa)} | valeurs propres {np.round(np.linalg.eigvals(Pa), 6)} | ||P_a||_2 = {norm(Pa, 2):.4f} = sqrt
     (1+a^2) = {np.sqrt(1 + a * a):.4f}"
              f" | perturbation (0, eps) -> {np.round(Pa @ np.array([0, 1e-3]), 5)}")
    for kappa in (10.0, 200.0):
        X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=100, kappa=kappa, sep=0.8, seed=0); Xtr, Xte,
     ytr, yte = split(X, y, 0)
        U, W, P = block(Xtr, ytr); Qu, Qw = qr(U)[0], qr(W)[0]; smin = svd(Qw.T @ Qu, compute_uv=False).
     min()
        # angle minimal entre im P = span(U) et ker P = span(W)^perp : sin(theta_min) = sigma_min(Q_W^T
     Q_U)
        Nk = qr(np.eye(d) - Qw @ Qw.T)[0][:, :d - U.shape[1]]           # base de ker P (= (span W)^perp)
        cos_max = svd(Qu.T @ Nk, compute_uv=False).max(); theta_min = np.arccos(min(cos_max, 1.0))
        _, sv, Vt = svd(P); dx = Vt[0]                                  # direction singuliere dominante
        x = Xte[0]; amp = norm(P @ (x + dx) - P @ x) / norm(dx)
        print(f"  bloc ferme kappa={kappa:5.0f} : ||P||_2 = {norm(P, 2):.4f} | 1/sigma_min(Q_W^T Q_U) =
     {1/smin:.4f} | 1/sin(theta_min) = {1/np.sin(theta_min):.4f} "
              f"| ||I-P||_2 = {norm(np.eye(d) - P, 2):.4f} | amplification mesuree le long de v_1 = {amp
     :.4f} (borne atteinte)")

    print("\n=== S2. kappa(W^T U) = 1 par construction ne borne pas ||P||_2 (rang 1) ===")
    u = np.array([1.0, 0.0])
    for t in (0.0, 1.0, 5.0, 50.0):
        w = u + t * np.array([0.0, 1.0]); P1 = np.outer(u, w) / (w @ u)   # w^T u = 1 : cross-Gram = [1],
      kappa = 1
        print(f"  w = u + {t:4.1f} v_perp : W^T U = {w @ u:.1f} (kappa = 1) | ||P||_2 = {norm(P1, 2):.3f}
      = sqrt(1 + t^2) = {np.sqrt(1 + t * t):.3f} | P^2 = P ? {np.allclose(P1 @ P1, P1)}")

```

```
print("\n=== S3. Borne metrique ||P||_2 <= (sqrt(kappa)+1/sqrt(kappa))/2 <= sqrt(kappa), kappa = kappa(S), et controle de l'obliquite par le retrecissement (kappa=200, 10 graines) ===")
print(f"  {'n/cl':>5s} {'a (LW)':>7s} {'||P||_2 brut':>12s} {'||P||_2 LW':>10s} {'borne fine (S_LW)':>18s} {'sqrt(kappa(S_LW))':>18s} {'sqrt(1+(1-a)lmax/(a mu))':>24s} {'sqrt(kappa(S_W))':>16s}")
for n_per in (10, 15, 30, 100, 1000):
    rows = []
    for sd in range(10):
        X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=200.0, sep=0.8, seed=sd); Xtr, Xte, ytr, yte = split(X, y, sd)
        S, a, mu, S_W = shrunk_metric(Xtr, ytr); ev = eigvalsh(S); evw = eigvalsh(S_W)
        _, _, P_lw = block(Xtr, ytr, shrink=True); _, _, P_raw = block(Xtr, ytr, shrink=False)
        rows.append((a, norm(P_raw, 2), norm(P_lw, 2), np.sqrt(ev.max() / ev.min()), np.sqrt(1 + (1 - a) * evw.max() / (a * mu)),
                     np.sqrt(evw.max() / max(evw.min(), 1e-300)), norm(inv(S) @ P_lw - P_lw.T @ inv(S)) / norm(inv(S) @ P_lw)))
    m = np.mean(rows, 0); fine = np.mean([0.5 * (r_[3] + 1 / r_[3]) for r_ in rows]); viol = max(r_[2] - 0.5 * (r_[3] + 1 / r_[3]) for r_ in rows)
    print(f"  {n_per:5d} {m[0]:7.3f} {m[1]:12.2f} {m[2]:10.2f} {fine:18.2f} {m[3]:18.2f} {m[4]:24.2f} {m[5]:16.2f}   | ||M P - P^T M||/||M P|| = {m[6]:.1e} | max(||P||_2 - borne fine) = {viol:.2f}")
print("  (la borne fine est respectee pour chaque graine ; le coefficient a decroit avec n et l'obliquite admissible croit.)")
# la borne fine est atteinte au rang un : u = e_1 + e_d dans la base propre de M
rng = np.random.default_rng(0); worst = 0.0
for _ in range(2000):
    dd = int(rng.integers(2, 8)); A_ = rng.standard_normal((dd, dd)); Mm = A_ @ A_.T + 0.05 * np.eye(dd); rr = int(rng.integers(1, dd))
    Uu = rng.standard_normal((dd, rr)); Pp = Uu @ np.linalg.solve(Uu.T @ Mm @ Uu, Uu.T @ Mm); k_ = np.sqrt(np.linalg.cond(Mm))
    worst = max(worst, norm(Pp, 2) / (0.5 * (k_ + 1 / k_)))
ev_, V_ = np.linalg.eigh(Mm); u_ = V_[:, [0]] / np.sqrt(ev_[0]) + V_[:, [-1]] / np.sqrt(ev_[-1]); P1 = u_ @ (u_.T @ Mm) / (u_.T @ Mm @ u_); k_ = np.sqrt(ev_[-1] / ev_[0])
print(f"  2000 couples (M, U) aleatoires : max ||P||_2 / borne fine = {worst:.4f} ; rang un, u = v_min/sqrt(l_min) + v_max/sqrt(l_max) : ||P||_2 = {norm(P1, 2):.4f}, borne fine = {0.5 * (k_ + 1 / k_):.4f} (atteinte)")

print("\n=== S4. Meilleure approximation : euclidienne non, dans la metrique de Fisher oui (kappa=200, n/cl=100) ===")
X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=100, kappa=200.0, sep=0.8, seed=0); Xtr, Xte, ytr, yte = split(X, y, 0)
U, W, P = block(Xtr, ytr); S, a, mu, _ = shrunk_metric(Xtr, ytr); M = inv(S); Qu = qr(U)[0]; Qs = Qu @ Qu.T
Lm = np.linalg.cholesky(M)                                          # ||v||_M = ||L^T v||
r_eu = [norm(x - P @ x) / norm(x - Qs @ x) for x in Xte]; r_M = [norm(Lm.T @ (x - P @ x)) / norm(Lm.T @ (x - Qs @ x)) for x in Xte]
print(f"  ||x - Px|| / ||x - Q_S x|| (euclidien) : min {min(r_eu):.3f}, max {max(r_eu):.3f} <= ||P||_2 = {norm(P, 2):.3f} ; ratio > 1 sur {100*np.mean(np.array(r_eu) > 1):.0f}% des points")
print(f"  meme ratio en norme M = S^-1 (Fisher)  : min {min(r_M):.3f}, max {max(r_M):.3f} (<= 1 partout : P est la meilleure approximation M-orthogonale)")

print("\n=== S5. Perte d'information et forme residuelle ===")
s_, b_ = make_closed_form_gate(U, W, Xtr); pred0 = readout_centroid_fit_predict(Xtr @ W, ytr, Xte @ W)
Nk = qr(np.eye(d) - qr(W)[0] @ qr(W)[0].T)[0][:, :d - W.shape[1]]   # base de ker(W^T)
sig = Xtr.std(0).mean(); dz = (Nk @ rng.standard_normal((Nk.shape[1], len(Xte)))).T; dz *= 3 * sig / norm(dz, axis=1, keepdims=True)
du = (rng.standard_normal((len(Xte), U.shape[1])) @ U.T); du *= 3 * sig / norm(du, axis=1, keepdims=True)
pk = readout_centroid_fit_predict(Xtr @ W, ytr, (Xte + dz) @ W); pu = readout_centroid_fit_predict(Xtr @ W, ytr, (Xte + du) @ W)
print(f"  perturbation de norme 3 sigma dans ker(W^T) : code change de {norm((Xte + dz) @ W - Xte @ W):.1e}, predictions identiques {100*np.mean(pk == pred0):.1f}%")
print(f"  perturbation de meme norme dans span(U)     : predictions identiques {100*np.mean(pu == pred0):.1f}%  (le noyau est invisible, l'image ne l'est pas)")
for L in (1, 2, 4):
    T = np.eye(d) + P / L; Tinv = np.eye(d) - P / (L + 1)
    print(f"  I + P/{L} : idempotent ? ||T^2 - T||/||T|| = {norm(T @ T - T)/norm(T):.2f} | inverse I - P/{L+1} : ||T (I - P/(L+1)) - I|| = {norm(T @ Tinv - np.eye(d)):.1e} | det = (1+1/L)^r = {np.linalg.det(T):.4f} vs {(1 + 1/L)**U.shape[1]:.4f}")

print("\n=== S6. Regulariser le cross-Gram detruit l'idempotence ; regulariser la metrique la conserve ===")
Uf, Wf, _, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1"); G = Wf.T @ Uf
for lam in (0.0, 1e-2, 1e-1, 1.0):
```

```
        Pl = Uf @ inv(G + lam * np.eye(G.shape[0])) @ Wf.T
        print(f"  (W^T U + {lam:4.2f} I)^-1 : ||P^2 - P||/||P|| = {norm(Pl @ Pl - Pl)/norm(Pl):.1e}")
    print(f"  metrique retrecie (Ledoit-Wolf, a = {a:.3f}) : ||P^2 - P||/||P|| = {norm(P @ P - P)/norm(P)
     :.1e}, ||W^T U - I|| = {norm(Wf.T @ Uf - np.eye(G.shape[0])):.1e}")

    print("\n=== S7. Profondeur : borne produit et inversibilite != conditionnement ===")
    for L in (2, 4, 8):
        T = np.linalg.matrix_power(np.eye(d) + P / L, L); cL = (1 + 1 / L) ** L - 1
        print(f"  L={L} : ||(I+P/L)^L||_2 = {norm(T, 2):.3f} | borne produit (1 + ||P||_2/L)^L = {(1 +
     norm(P, 2)/L)**L:.3f} | exact ||I + c_L P||_2 = {norm(np.eye(d) + cL * P, 2):.3f}")
    for T_ in (1, 4, 8):
        h0 = Xte[0]; J = np.zeros((d, d)); eps = 1e-6
        for j in range(d):
            e = np.zeros(d); e[j] = eps
            J[:, j] = (recurse_gated(U, W, s_, b_, (h0 + e)[None], T_)[0] - recurse_gated(U, W, s_, b_, (
     h0 - e)[None], T_)[0]) / (2 * eps)
        sv = svd(J, compute_uv=False)
        print(f"  porte, T={T_} : det = {np.linalg.det(J):.2e} (non nul) | sigma_min = {sv.min():.3e} |
     sigma_max = {sv.max():.3f} | conditionnement = {sv.max()/sv.min():.1f}")

    print("\n=== S8. Spectre singulier d'un projecteur oblique et lecture en deux etapes (kappa=200, n/cl
     =100) ===")
    sv = svd(P, compute_uv=False)[:U.shape[1]]; cosang = svd(qr(W)[0].T @ qr(U)[0], compute_uv=False)
    print(f"  valeurs singulieres non nulles de P : {np.round(sv, 4)}")
    print(f"  1/cos(theta_i), angles principaux span(U)/span(W) = {np.round(np.degrees(np.arccos(cosang))
     , 2)} deg : {np.round(np.sort(1 / cosang)[::-1], 4)}")
    Qw = qr(W)[0]; x = Xte[0]; print(f"  W^T x - W^T (Q_W Q_W^T x) = {norm(W.T @ x - W.T @ (Qw @ Qw.T @ x
     ))):.1e} : le code ne lit que la projection orthogonale de x sur span(W)")

    print("\n=== S9. Seuil d'interpolation : ||P||_2 brut (pseudo-inverse de S_W) contre retreci, en
     fonction de (n_train - C)/d (kappa=200, 10 graines) ===")
    from numpy.linalg import pinv, eigh
    def block_metric(Xtr_, ytr_, Smat):
        cl, yi_ = np.unique(ytr_, return_inverse=True); Cn = len(cl); ns_ = np.bincount(yi_).astype(float
     ); mu_ = Xtr_.mean(0)
        mus_ = np.stack([Xtr_[yi_ == k].mean(0) for k in range(Cn)]); D_ = ((mus_ - mu_).T * np.sqrt(ns_)
     ) / np.sqrt(Cn - 1)
        Z_ = pinv(Smat) @ D_; M_ = D_.T @ Z_; M_ = 0.5 * (M_ + M_.T); lam_, Q_ = eigh(M_); o_ = np.
     argsort(lam_)[::-1]; lam_, Q_ = lam_[o_], Q_[:, o_]
        lr_ = np.clip(lam_[:Cn - 1], 1e-12, None); return D_ @ Q_[:, :Cn - 1] / np.sqrt(lr_), Z_ @ Q_[:,
     :Cn - 1] / np.sqrt(lr_)
    print(f"  {'n/cl':>5s} {'(n_train-C)/d':>14s} {'||P|| brut':>10s} {'||P|| LW':>9s} {'acc brut':>9s}
     {'acc LW':>7s}")
    for n_per in (4, 6, 7, 8, 9, 10, 12, 15, 30):
        rows = []
        for sd in range(10):
            X, y, *_ = make_multiclass(n_classes=C, d=d, n_per=n_per, kappa=200.0, sep=0.8, seed=sd); Xtr
     , Xte, ytr, yte = split(X, y, sd)
            S_, a_, mu_, S_W_ = shrunk_metric(Xtr, ytr); Ur, Wr = block_metric(Xtr, ytr, S_W_); Ul, Wl =
     block_metric(Xtr, ytr, S_)
            rows.append((len(Xtr), norm(Ur @ Wr.T, 2), norm(Ul @ Wl.T, 2), (readout_centroid_fit_predict(
     Xtr @ Wr, ytr, Xte @ Wr) == yte).mean(), (readout_centroid_fit_predict(Xtr @ Wl, ytr, Xte @ Wl) ==
     yte).mean()))
        m = np.mean(rows, 0); print(f"  {n_per:5d} {(m[0] - C) / d:14.2f} {m[1]:10.2f} {m[2]:9.2f} {100 *
      m[3]:9.1f} {100 * m[4]:7.1f}")
    print("  (le pic de ||P|| brut est a (n_train - C)/d ~ 0.9 ; le retrecissement le supprime et la
     precision brute ne montre pas de double descente ici.)")
```

Listing 21: ropn_auto_stats.py — paired comparisons and equivalence margin

```
"""
Comparaisons appariees entre methodes evaluees sur les MEMES realisations (memes tirages ou memes
    decoupages).

paired_summary(a, b) : difference moyenne a - b, intervalle de confiance de Student a 95 % sur les
    differences
appariees, et verdict par rapport a une marge d'equivalence pratique fixee a l'avance (1 point de
    precision par
defaut) :
    "equivalent"   : l'intervalle est contenu dans ]-marge, +marge[ ;
    "A > B", "A < B" : l'intervalle exclut 0 (et n'est pas contenu dans la marge) ;
```

```
    "indetermine"  : l'intervalle contient 0 et deborde de la marge (les donnees ne permettent pas de conclure).
Une difference inferieure a l'ecart-type n'est donc jamais declaree nulle : elle est "indeterminee" tant que
l'intervalle n'est pas dans la marge. Les decoupages repetes d'un meme jeu de donnees ne sont pas des realisations
independantes du probleme : l'intervalle mesure alors la variabilite du decoupage, pas celle de la population.
"""
import numpy as np
from scipy.stats import t as student_t

MARGIN = 0.01          # marge d'equivalence pratique : 1 point de precision, fixee avant les experiences


def paired_summary(a, b, margin=MARGIN, level=0.95):
    d = np.asarray(a, float) - np.asarray(b, float); n = len(d); m = float(d.mean())
    se = float(d.std(ddof=1) / np.sqrt(n)) if n > 1 else float("nan")
    if not np.isfinite(se) or se == 0.0:
        lo = hi = m
    else:
        q = float(student_t.ppf(0.5 + level / 2, n - 1)); lo, hi = m - q * se, m + q * se
    if -margin < lo and hi < margin: verdict = "equivalent"
    elif lo > 0: verdict = "A > B"
    elif hi < 0: verdict = "A < B"
    else: verdict = "indetermine"
    return dict(mean=m, lo=lo, hi=hi, n=n, verdict=verdict)


def fmt_paired(a, b, margin=MARGIN, scale=100.0):
    r = paired_summary(a, b, margin)
    return f"{scale*r['mean']:+5.1f} [{scale*r['lo']:+5.1f}, {scale*r['hi']:+5.1f}] {r['verdict']}"
```

Listing 22: ropn_auto_rang.py — rank selection: calibration, recovery, predictive risk, family-wise control

```
"""
Selection du rang du bloc discriminant analytique : ce qui est garanti, ce qui est mesure.

Trois rangs sont distingues : r_struct = min(C-1, d) (borne exacte), r_tested (regle de test, peut valoir 0),
r_used = max(1, r_tested) (rang effectivement utilise par le classifieur). Regles comparees :
  perm     : chaque valeur propre contre le quantile 95 % du MAXIMUM sous permutation (metrique S_T) ;
  seq      : test de permutation sequentiel avec deflation (metrique S_T) ;
  bartlett : test sequentiel de Bartlett sur le Lambda de Wilks (asymptotique) ;
  C-1      : borne structurelle, aucun test.

Experiences (donnees gaussiennes a covariance commune, C = 4, d = 20, kappa = 10) :
  0. calibration sous l'hypothese nulle globale (2000 tirages) : metrique S_T, metrique S_W partagee, Bartlett,
     en bruit isotrope puis anisotrope ;
  A. alternatives de rang vrai q = 0, 1, 2, 3 avec des forces egales ou graduees par direction : loi du rang
     teste (sous-estimation, exactitude, sur-estimation), separement du risque predictif (precision avec le rang
     utilise, avec C-1 et avec le rang vrai) ;
  B. famille de tests (8 blocs independants, par exemple des bandes de frequence) : probabilite de retenir au
     moins un bloc sous l'hypothese nulle globale avec un test par bloc, et avec le controle global min-p ;
     puissance quand un seul bloc porte l'information.
numpy / scipy seulement.
"""
import numpy as np
from numpy.linalg import qr, cholesky
from ropn_auto_fast import fit_block_fast, fwer_select
from ropn_auto_nobp_diag import readout_centroid_fit_predict
from ropn_auto_stats import fmt_paired

RULES = ("perm", "seq", "bartlett")

```

```
30
31  def make_rank_data(q, strengths, n_per, C=4, d=20, kappa=10.0, seed=0, n_test=2000):
32      """Moyennes de classe de rang q exactement : mu_c = sqrt(C) * sum_j H[c, j] s_j v_j, H base
         orthonormee des
33      contrastes entre classes, v_j directions orthonormees ; s_j = ecart-type inter-classes le long de v_j
         , en
34      unites de l'ecart-type moyen du bruit. Bruit gaussien de covariance commune (conditionnement kappa)."
         ""
35      rng = np.random.default_rng(seed)
36      eig = np.logspace(0, np.log10(kappa), d); eig = eig / eig.mean()
37      Qm, _ = qr(rng.standard_normal((d, d))); L = cholesky(Qm @ np.diag(eig) @ Qm.T)
38      H = qr(np.eye(C) - 1.0 / C)[0][:, :C - 1]; V, _ = qr(rng.standard_normal((d, C - 1)))
39      mus = np.sqrt(C) * (H[:, :q] * np.asarray(strengths[:q], float)) @ V[:, :q].T if q > 0 else np.zeros
         ((C, d))
40      def draw(n):
41          y = np.repeat(np.arange(C), n); return rng.standard_normal((C * n, d)) @ L.T + mus[y], y
42      Xtr, ytr = draw(n_per); Xte, yte = draw(n_test // C)
43      return Xtr, ytr, Xte, yte
44
45
46  def acc_rank(W, r, Xtr, ytr, Xte, yte):
47      # lecture par centroides dans l'espace du code = regle LDA restreinte aux r premieres directions
48      r = max(1, int(r)); return float((readout_centroid_fit_predict(Xtr @ W[:, :r], ytr, Xte @ W[:, :r])
         == yte).mean())
49
50
51  if __name__ == "__main__":
52      C, d, REPS = 4, 20, 200
53      print("=== 0. Calibration sous l'hypothese nulle globale (bruit pur), 2000 tirages, 199 permutations,
          niveau nominal 5 % (erreur-type 0.5 point) ===")
54      print(f"    {'n':>4s} {'d':>4s} {'C':>3s} | {'permutation, S_T':>17s} | {'permutation, S_W partage
         ':>25s} | {'Bartlett':>9s}")
55      for n_per, d0, C0 in [(10, 36, 3), (20, 36, 3), (40, 36, 3), (10, 20, 4), (30, 20, 4), (100, 20, 4)]:
56          fp = {"total": 0, "within": 0, "bartlett": 0}; R0 = 2000
57          for rep in range(R0):
58              rng = np.random.default_rng(900000 + rep); y0 = np.repeat(np.arange(C0), n_per); X0 = rng.
         standard_normal((C0 * n_per, d0))
59              fp["total"] += fit_block_fast(X0, y0, rank="perm", n_perm=199, seed=rep, min_rank=0)[3]["
         r_tested"] > 0
60              fp["within"] += fit_block_fast(X0, y0, rank="perm", n_perm=199, seed=rep, min_rank=0,
         perm_metric="within")[3]["r_tested"] > 0
61              fp["bartlett"] += fit_block_fast(X0, y0, rank="bartlett", min_rank=0)[3]["r_tested"] > 0
62          print(f"    {C0*n_per:4d} {d0:4d} {C0:3d} | {100*fp['total']/R0:16.1f}% | {100*fp['within']/R0
         :24.1f}% | {100*fp['bartlett']/R0:8.1f}%")
63      # meme verification avec le bruit ANISOTROPE (kappa = 10) et les tailles de la partie A : l'
         echangeabilite ne
64      # depend pas de la covariance du bruit, le niveau doit donc rester <= 5 % ; 2000 tirages au lieu de
         200
65      print("    bruit anisotrope de la partie A (C = 4, d = 20, kappa = 10), 2000 tirages, 200
         permutations :")
66      for n_per in (10, 30, 100):
67          fp = {"perm": 0, "seq": 0}; R0 = 2000
68          for rep in range(R0):
69              X0, y0, _, _ = make_rank_data(0, (1.2, 1.2, 1.2), n_per, seed=50000 + rep, n_test=8)
70              for rule in fp:
71                  fp[rule] += fit_block_fast(X0, y0, rank=rule, n_perm=200, seed=rep, min_rank=0)[3]["
         r_tested"] > 0
72          print(f"    {4*n_per:4d} {20:4d} {4:3d} | regle perm {100*fp['perm']/R0:4.1f}% | regle seq {100*
         fp['seq']/R0:4.1f}%")
73      print()
74      print(f"=== A. Recuperation du rang sous des alternatives de rang q (C={C}, d={d}, kappa=10, {REPS}
         tirages independants, 200 permutations, niveau 5 %) ===")
75      print("    colonnes : P(rang teste < q) / P(= q) / P(> q), rang teste moyen ; precision (%) sur 2000
         points de test, lecture par centroides, avec le rang UTILISE max(1, rang teste)")
76      for prof_name, strengths in [("forces egales (1.2, 1.2, 1.2)", (1.2, 1.2, 1.2)), ("forces graduees
         (1.6, 0.8, 0.4)", (1.6, 0.8, 0.4))]:
77          for n_per in (10, 30, 100):
78              print(f"\n  {prof_name}, n = {n_per}/classe (n_train = {C*n_per}) :")
79              print(f"    {'q':>2s} | " + " | ".join(f"{r_:^34s}" for r_ in RULES) + " | precision : " + "
         ".join(f"{r_:>8s}" for r_ in RULES) + f" {'C-1':>7s} {'rang q':>7s}")
80              for q in range(C):
81                  rt = {r_: [] for r_ in RULES}; ac = {k: [] for k in RULES + ("C1", "oracle")}
82                  for rep in range(REPS):
```

```
                    Xtr, ytr, Xte, yte = make_rank_data(q, strengths, n_per, C, d, seed=1000 * q + rep)
                    U, W, _, _ = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")
                    for r_ in RULES:
                        info = fit_block_fast(Xtr, ytr, shrink=True, rank=r_, n_perm=200, seed=rep,
    min_rank=0)[3]
                        rt[r_].append(info["r_tested"])
                        if q > 0: ac[r_].append(acc_rank(W, info["r_tested"], Xtr, ytr, Xte, yte))
                    if q > 0:
                        ac["C1"].append(acc_rank(W, C - 1, Xtr, ytr, Xte, yte)); ac["oracle"].append(
    acc_rank(W, q, Xtr, ytr, Xte, yte))
                cells = []
                for r_ in RULES:
                    v = np.array(rt[r_]); cells.append(f"{100*np.mean(v < q):5.1f} /{100*np.mean(v == q)
    :5.1f} /{100*np.mean(v > q):5.1f}  moy {v.mean():4.2f}")
                accs = " ".join(f"{100*np.mean(ac[k]):8.1f}" if q > 0 else f"{'--':>8s}" for k in RULES)
    + (f" {100*np.mean(ac['C1']):7.1f} {100*np.mean(ac['oracle']):7.1f}" if q > 0 else f" {'--':>7s}
    {'--':>7s}")
                print(f"    {q:2d} | " + " | ".join(cells) + " |             " + accs)
                if q > 0:    # risque predictif, differences appariees sur les memes tirages
                    print(f"         appariees [IC 95 %] : rang teste (seq) - (C-1)  {fmt_paired(ac['seq
    '], ac['C1'])} | (C-1) - rang vrai q  {fmt_paired(ac['C1'], ac['oracle'])}")
    print("\n  (q = 0 : P(> q) est le taux de faux positifs de l'hypothese nulle globale, niveau nominal
     5 % ; erreur-type binomiale <= 3.5 points.)")

    print("\n=== B. Famille de 8 tests (blocs independants de dimension 36, C = 3, n = 30) : controle par
      bloc contre controle global min-p, 200 tirages ===")
    for label, sep in [("hypothese nulle globale (aucun bloc informatif)", 0.0), ("un bloc informatif sur
      8 (bloc 2, force 1.5)", 1.5)]:
        any_pb, any_fw, hit_pb, hit_fw, fp_pb, fp_fw = [], [], [], [], [], []
        for rep in range(200):
            rng = np.random.default_rng(5000 + rep); y = np.repeat(np.arange(3), 10); infos = []
            for b in range(8):
                Xb = rng.standard_normal((30, 36))
                if b == 2 and sep > 0:
                    Hc = qr(np.eye(3) - 1.0 / 3)[0][:, :2]; Vb, _ = qr(rng.standard_normal((36, 2))); Xb
    = Xb + np.sqrt(3) * sep * (Hc @ Vb.T)[y]
                infos.append(fit_block_fast(Xb, y, shrink=True, rank="perm", n_perm=200, seed=rep,
    min_rank=0)[3])
            pb = np.array([i["r_tested"] > 0 for i in infos]); fw, _ = fwer_select(infos, 0.05)
            null_idx = np.array([b != 2 or sep == 0 for b in range(8)])
            any_pb.append(pb[null_idx].any()); any_fw.append(fw[null_idx].any()); fp_pb.append(pb[
    null_idx].mean()); fp_fw.append(fw[null_idx].mean())
            hit_pb.append(pb[2]); hit_fw.append(fw[2])
        line = f"  {label} :\n      test par bloc : P(au moins un bloc nul retenu) = {100*np.mean(any_pb)
    :5.1f} %, taux par bloc nul = {100*np.mean(fp_pb):4.1f} %"
        line += f"\n      controle min-p : P(au moins un bloc nul retenu) = {100*np.mean(any_fw):5.1f} %,
     taux par bloc nul = {100*np.mean(fp_fw):4.1f} %"
        if sep > 0: line += f"\n      puissance (bloc informatif retenu) : test par bloc {100*np.mean(
    hit_pb):5.1f} %, controle min-p {100*np.mean(hit_fw):5.1f} %"
        print(line)
    print("  (attendu sous l'hypothese nulle globale pour 8 tests independants au niveau 5 % : 1 - 0.95^8
      = 33.7 %.)")
```

Listing 23: ropn_auto_central.py — central benchmark with symmetric comparisons

```
"""
Benchmark central : le bloc discriminant analytique face a des references, avec un protocole symetrique.

Question testee : le bloc (metrique de Fisher retrecie, rang C-1, lecture sur le code) reste-t-il utile
     quand ses
hypotheses (classes gaussiennes de covariance commune) ne sont qu'approximatives, et a petit n ?

Deux lectures analytiques du code sont comparees : par centroides (c'est exactement la regle de la LDA de
      meme
covariance, voir readout_centroid_fit_predict) et par moindres carres sur les indicatrices de classe (la
     lecture
utilisee dans les autres scripts, sujette au masquage quand les classes sont nombreuses).

Jeux de donnees
  synthetiques, tirages INDEPENDANTS, test sur 2000 points (C = 4, d = 20, n = 15 ou 60 par classe) :
    gauss   : covariance commune (hypotheses satisfaites) ;
```

```
    hetero  : une covariance par classe (covariance commune violee) ;
    student : bruit de Student a 3 degres de liberte, dispersion commune (gaussianite violee) ;
    echelle : covariances proportionnelles, facteurs 0.4, 0.8, 1.4, 2.4 (violation moderee) ;
  reel : digits de scikit-learn (1797 images 8 x 8, 10 classes), sous-echantillons stratifies repetes de 10, 30 ou
    100 exemples par classe, test sur tout le reste (les repetitions mesurent la variabilite du decoupage, ce ne
    sont pas des realisations independantes du probleme).

Deux comparaisons distinctes
  SANS reglage : une recette par defaut par methode, fixee a l'avance ;
  APRES selection : 4 configurations par methode, choisies par validation croisee interne a 3 plis sur les MEMES
    plis pour toutes les methodes (budget de selection comparable).

Statistiques : differences appariees au bloc sur les memes realisations, intervalle de confiance a 95 %, marge
d'equivalence de 1 point (ropn_auto_stats.py). Temps : pretraitement, selection, ajustement final et inference
mesures separement. numpy / scipy / sklearn seulement.
"""
import time, warnings
import numpy as np
from numpy.linalg import qr, cholesky
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_digits
from ropn_auto_fast import fit_block_fast
from ropn_auto_crossgram import fit_crossgram_block
from ropn_auto_nobp_diag import readout_fit_predict, readout_centroid_fit_predict
from ropn_auto_stats import fmt_paired

warnings.filterwarnings("ignore")


# ---------------------------------------------------------------- generateurs synthetiques
def make_synth(kind, n_per, n_test=2000, C=4, d=20, kappa=50.0, sep=0.8, seed=0):
    rng = np.random.default_rng(seed); eig = np.logspace(0, np.log10(kappa), d); eig = eig / eig.mean()
    def cov():
        Q, _ = qr(rng.standard_normal((d, d))); return Q @ np.diag(eig) @ Q.T
    mus = sep * rng.standard_normal((C, d)); mus -= mus.mean(0, keepdims=True)
    base = cov(); Ls = [cholesky(cov() if kind == "hetero" else base) for _ in range(C)]
    scale = np.array([0.4, 0.8, 1.4, 2.4]) if kind == "echelle" else np.ones(C)
    def draw(n):
        X, y = [], []
        for c in range(C):
            Z = rng.standard_normal((n, d))
            if kind == "student": Z = Z / np.sqrt(rng.chisquare(3, (n, 1)) / 3.0)
            X.append(np.sqrt(scale[c]) * Z @ Ls[c].T + mus[c]); y.append(np.full(n, c))
        return np.concatenate(X), np.concatenate(y)
    Xtr, ytr = draw(n_per); Xte, yte = draw(n_test // C)
    return Xtr, ytr, Xte, yte


# ---------------------------------------------------------------- methodes : (ajuste, predit) pour une configuration
def block_fit_predict(cfg, Xtr, ytr, Xte, readout="centroid"):
    if cfg == "LW": W = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")[1]
    else: W = fit_crossgram_block(Xtr, ytr, cfg)[1]
    return (readout_centroid_fit_predict if readout == "centroid" else readout_fit_predict)(Xtr @ W, ytr, Xte @ W)

def block_ls_fit_predict(cfg, Xtr, ytr, Xte): return block_fit_predict(cfg, Xtr, ytr, Xte, "ls")

def lda_fit_predict(cfg, Xtr, ytr, Xte): return LDA(solver="lsqr", shrinkage=cfg).fit(Xtr, ytr).predict(Xte)
def logreg_fit_predict(cfg, Xtr, ytr, Xte): return LogisticRegression(C=cfg, max_iter=2000).fit(Xtr, ytr).predict(Xte)
def mlp_fit_predict(cfg, Xtr, ytr, Xte, seed=0):
    h, alpha, solver = cfg
```

```
    return MLPClassifier(hidden_layer_sizes=(h,), alpha=alpha, solver=solver, max_iter=500, random_state=seed).fit(Xtr, ytr).predict(Xte)

METHODS = {   # nom : (fonction, recette par defaut, grille de 4 configurations)
    "bloc analytique": (block_fit_predict, "LW", ["LW", 0.01, 0.1, 1.0]),           # lecture par centroides (regle LDA)
    "bloc, lecture MC": (block_ls_fit_predict, "LW", ["LW", 0.01, 0.1, 1.0]),        # lecture par moindres carres
    "LDA retrecie":    (lda_fit_predict, "auto", ["auto", 0.01, 0.1, 0.5]),
    "reg. logistique": (logreg_fit_predict, 1.0, [0.01, 0.1, 1.0, 10.0]),
    "MLP":             (mlp_fit_predict, (100, 1e-4, "adam"), [(64, 1e-4, "adam"), (64, 1e-1, "adam"), (64, 1e-4, "lbfgs"), (64, 1e-1, "lbfgs")]),
}


def evaluate(Xtr, ytr, Xte, yte, seed, select):
    """retourne {methode: (precision, t_selection, t_ajustement+prediction, configuration)} ; select=False : recette par defaut."""
    out = {}
    skf = list(StratifiedKFold(n_splits=3, shuffle=True, random_state=seed).split(Xtr, ytr)) if select else None
    for name, (fn, default, grid) in METHODS.items():
        t_sel = 0.0; cfg = default
        if select:
            t0 = time.perf_counter(); best = -1.0
            for g in grid:
                a = np.mean([(fn(g, Xtr[tr], ytr[tr], Xtr[va]) == ytr[va]).mean() for tr, va in skf])
                if a > best + 1e-12: best, cfg = a, g
            t_sel = time.perf_counter() - t0
        t0 = time.perf_counter(); pred = fn(cfg, Xtr, ytr, Xte); t_fit = time.perf_counter() - t0
        out[name] = ((pred == yte).mean(), t_sel, t_fit, cfg)
    return out


def time_phases(name, cfg, Xtr, ytr, Xte, n_rep=3):
    """ajustement final et inference chronometres SEPAREMENT (minimum de n_rep executions), en secondes."""
    best_fit, best_pred = np.inf, np.inf
    for _ in range(n_rep):
        if name.startswith("bloc"):
            t0 = time.perf_counter()
            W = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")[1] if cfg == "LW" else fit_crossgram_block(Xtr, ytr, cfg)[1]
            Ctr = Xtr @ W; classes = np.unique(ytr)
            if name == "bloc analytique":
                M = np.stack([Ctr[ytr == c].mean(0) for c in classes]); lp = np.log(np.array([(ytr == c).mean() for c in classes]))
            else:
                V, *_ = np.linalg.lstsq(np.c_[Ctr, np.ones(len(Ctr))], np.eye(len(classes))[np.searchsorted(classes, ytr)], rcond=None)
            t1 = time.perf_counter()
            Cte = Xte @ W
            if name == "bloc analytique": pred = classes[(Cte @ M.T - 0.5 * (M ** 2).sum(1) + lp).argmax(1)]
            else: pred = classes[(np.c_[Cte, np.ones(len(Cte))] @ V).argmax(1)]
            t2 = time.perf_counter()
        else:
            if name == "LDA retrecie": model = LDA(solver="lsqr", shrinkage=cfg)
            elif name == "reg. logistique": model = LogisticRegression(C=cfg, max_iter=2000)
            else: model = MLPClassifier(hidden_layer_sizes=(cfg[0],), alpha=cfg[1], solver=cfg[2], max_iter=500, random_state=0)
            t0 = time.perf_counter(); model.fit(Xtr, ytr); t1 = time.perf_counter(); pred = model.predict(Xte); t2 = time.perf_counter()
        best_fit, best_pred = min(best_fit, t1 - t0), min(best_pred, t2 - t1)
    return best_fit, best_pred


def report(title, runs_def, runs_sel):
    print(f"\n  {title}")
    print(f"    {'methode':16s} | SANS reglage ({len(runs_def)} realisations) : precision, difference appariee au bloc [IC 95 %] verdict | APRES selection ({len(runs_sel)} realisations) : idem")
    for name in METHODS:
        cells = []
```

```
        for runs in (runs_def, runs_sel):
            a = np.array([r[name][0] for r in runs]); b = np.array([r["bloc analytique"][0] for r in runs
    ])
            cells.append(f"{100*a.mean():5.1f} +/- {100*a.std():4.1f}   " + (fmt_paired(a, b) if name !=
    "bloc analytique" else " " * 34))
        print(f"    {name:16s} | {cells[0]:62s} | {cells[1]}")


if __name__ == "__main__":
    R_DEF, R_SEL = 20, 10
    print(f"=== 1. Donnees synthetiques : tirages independants, test sur 2000 points ; {R_DEF} tirages
     sans reglage, {R_SEL} avec selection ===")
    for kind, label in [("gauss", "gauss (hypotheses satisfaites)"), ("hetero", "hetero (covariance
     commune violee)"),
                        ("student", "student t3 (gaussianite violee)"), ("echelle", "echelle (covariances
      proportionnelles)")]:
        for n_per in (15, 60):
            runs_def, runs_sel = [], []
            for rep in range(R_DEF):
                Xtr, ytr, Xte, yte = make_synth(kind, n_per, seed=rep)
                sc = StandardScaler().fit(Xtr); Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)
                runs_def.append(evaluate(Xtr, ytr, Xte, yte, rep, False))
                if rep < R_SEL: runs_sel.append(evaluate(Xtr, ytr, Xte, yte, rep, True))
            report(f"{label}, n = {n_per}/classe", runs_def, runs_sel)

    print(f"\n=== 2. Cas reel central : digits (d = 64 dont 61 variables non constantes, C = 10), sous-
     echantillons stratifies repetes, test sur le reste ===")
    D = load_digits(); X0, y0 = D.data, D.target
    timing = None
    for n_per in (10, 30, 100):
        runs_def, runs_sel = [], []
        for rep in range(R_DEF):
            rng = np.random.default_rng(rep); tr = np.concatenate([rng.choice(np.where(y0 == c)[0], n_per
     , replace=False) for c in range(10)])
            te = np.setdiff1d(np.arange(len(y0)), tr)
            t0 = time.perf_counter(); keep = X0[tr].std(0) > 1e-8; sc = StandardScaler().fit(X0[tr][:,
     keep])
            Xtr, Xte = sc.transform(X0[tr][:, keep]), sc.transform(X0[te][:, keep]); t_pre = time.
     perf_counter() - t0
            runs_def.append(evaluate(Xtr, y0[tr], Xte, y0[te], rep, False))
            if rep < R_SEL: runs_sel.append(evaluate(Xtr, y0[tr], Xte, y0[te], rep, True))
            if n_per == 100 and rep == 0: timing = (t_pre, len(te), runs_sel[0], Xtr, y0[tr], Xte)
        report(f"digits, n = {n_per}/classe (n_train = {10*n_per}, n_test = {len(te)})", runs_def,
     runs_sel)
    t_pre, n_te, r0, Xtr_t, ytr_t, Xte_t = timing
    print(f"\n  Temps par phase sur digits, n = 100/classe, une realisation (ms) : pretraitement commun (
     standardisation) {1000*t_pre:.1f}")
    print(f"    {'methode':16s} {'selection (4 config. x 3 plis)':>32s} {'ajustement final':>18s} {'
     inference de ' + str(n_te) + ' points':>26s}   (configuration selectionnee ; ajustement et inference
     : min de 3)")
    for name in METHODS:
        t_fit, t_pred = time_phases(name, r0[name][3], Xtr_t, ytr_t, Xte_t)
        print(f"    {name:16s} {1000*r0[name][1]:32.1f} {1000*t_fit:18.2f} {1000*t_pred:26.2f}   {r0[name
    ][3]}")

    # variabilite d'entrainement contre variabilite du decoupage (MLP par defaut, digits, n = 30/classe)
    rng = np.random.default_rng(0); tr = np.concatenate([rng.choice(np.where(y0 == c)[0], 30, replace=
     False) for c in range(10)]); te = np.setdiff1d(np.arange(len(y0)), tr)
    keep = X0[tr].std(0) > 1e-8; sc = StandardScaler().fit(X0[tr][:, keep]); Xtr, Xte = sc.transform(X0[
     tr][:, keep]), sc.transform(X0[te][:, keep])
    a_seed = [(mlp_fit_predict((100, 1e-4, "adam"), Xtr, y0[tr], Xte, seed=s) == y0[te]).mean() for s in
     range(10)]
    print(f"\n  MLP par defaut, digits n = 30/classe : ecart-type sur 10 graines d'entrainement a
     decoupage fixe = {100*np.std(a_seed):.2f} point(s) (moyenne {100*np.mean(a_seed):.1f}) ; "
          f"le bloc analytique est deterministe a decoupage fixe.")

    print("\n=== 2 bis. Equivalence bloc + lecture par centroides = LDA de meme covariance (digits, 10
     sous-echantillons) ===")
    from ropn_auto_fast import ledoit_wolf_coef
    for n_per in (10, 30, 100):
        ag_own, ag_sk = [], []
        for rep in range(10):
            rng = np.random.default_rng(rep); tr = np.concatenate([rng.choice(np.where(y0 == c)[0], n_per
```

```
    , replace=False) for c in range(10)]); te = np.setdiff1d(np.arange(len(y0)), tr)
191         keep = X0[tr].std(0) > 1e-8; sc = StandardScaler().fit(X0[tr][:, keep]); Xtr, Xte, ytr = sc.
    transform(X0[tr][:, keep]), sc.transform(X0[te][:, keep]), y0[tr]
192         W = fit_block_fast(Xtr, ytr, shrink=True, rank="C1")[1]; pc = readout_centroid_fit_predict(
    Xtr @ W, ytr, Xte @ W)
193         mus = np.stack([Xtr[ytr == k].mean(0) for k in range(10)]); Xc = Xtr - mus[ytr]; al, mu_ =
    ledoit_wolf_coef(Xc)
194         Si = np.linalg.inv((1 - al) * Xc.T @ Xc / len(Xtr) + al * mu_ * np.eye(Xtr.shape[1]))     #
    meme S que le bloc
195         sco = Xte @ Si @ mus.T - 0.5 * np.einsum("kd,de,ke->k", mus, Si, mus)[None, :] + np.log(np.
    bincount(ytr) / len(ytr))
196         ag_own.append((pc == sco.argmax(1)).mean()); ag_sk.append((pc == LDA(solver="lsqr", shrinkage
    ="auto").fit(Xtr, ytr).predict(Xte)).mean())
197       print(f"  n = {n_per:3d}/classe : predictions identiques a la LDA calculee avec le meme S : {100*
    np.mean(ag_own):.1f} % ; a LDA(lsqr, shrinkage='auto') de scikit-learn : {100*np.mean(ag_sk):.1f} %")
198
199   print("\n=== 3. Rang teste sur digits (regle sequentielle, 200 permutations) et precision (lecture
    par centroides) avec le rang teste ou avec C-1 = 9, 20 sous-echantillons ===")
200   for n_per in (5, 10, 30):
201       rt, a_t, a_c = [], [], []
202       for rep in range(R_DEF):
203           rng = np.random.default_rng(rep); tr = np.concatenate([rng.choice(np.where(y0 == c)[0], n_per
    , replace=False) for c in range(10)]); te = np.setdiff1d(np.arange(len(y0)), tr)
204           keep = X0[tr].std(0) > 1e-8; sc = StandardScaler().fit(X0[tr][:, keep]); Xtr, Xte = sc.
    transform(X0[tr][:, keep]), sc.transform(X0[te][:, keep])
205           U, W, r, info = fit_block_fast(Xtr, y0[tr], shrink=True, rank="seq", n_perm=200, seed=rep,
    min_rank=0); Wf = fit_block_fast(Xtr, y0[tr], shrink=True, rank="C1")[1]
206           rt.append(info["r_tested"]); k = max(1, info["r_tested"])
207           a_t.append((readout_centroid_fit_predict(Xtr @ Wf[:, :k], y0[tr], Xte @ Wf[:, :k]) == y0[te])
    .mean()); a_c.append((readout_centroid_fit_predict(Xtr @ Wf, y0[tr], Xte @ Wf) == y0[te]).mean())
208       print(f"  n = {n_per:3d}/classe (n_train = {10*n_per}) : rang teste moyen {np.mean(rt):.2f} (min
    {min(rt)}, max {max(rt)}) | precision au rang utilise {100*np.mean(a_t):.1f} | au rang C-1 {100*np.
    mean(a_c):.1f} | difference appariee {fmt_paired(a_t, a_c)}")
```

Acknowledgments

The author used Claude (Anthropic) for assistance with code generation, mathematical formatting, and discussion of conceptual interpretations during the preparation of this work.

References

[1] Olivier Ledoit and Michael Wolf. A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2):365–411, 2004.

[2] Jason W. Rocks and Pankaj Mehta. The geometry of over-parameterized regression and adversarial perturbations, 2021. arXiv:2103.14108.

[3] Roger A. Horn and Charles R. Johnson. *Matrix Analysis*. Cambridge University Press, 2nd edition, 2013.

[4] Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 933–941, 2017.

[5] Noam Shazeer. GLU variants improve transformer, 2020. arXiv:2002.05202.

[6] Thomas Zaslavsky. Facing up to arrangements: face-count formulas for partitions of space by hyperplanes. *Memoirs of the American Mathematical Society*, 1(154), 1975.

[7] David E. Rumelhart, Geoffrey E. Hinton, and James L. McClelland. A general framework for parallel distributed processing. In David E. Rumelhart, James L. McClelland, and

the PDP Research Group, editors, *Parallel Distributed Processing: Explorations in the Microstructure of Cognition, Volume 1: Foundations*, chapter 2, pages 45–76. MIT Press, Cambridge, MA, 1986.

[8] Ronald A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936.

[9] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition, 2009.

[10] Belinda Phipson and Gordon K. Smyth. Permutation P-values should never be zero: Calculating exact P-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1):Article 39, 2010.

[11] Maurice S. Bartlett. Multivariate analysis. *Supplement to the Journal of the Royal Statistical Society*, 9(2):176–197, 1947.

[12] Mati Wax and Thomas Kailath. Detection of signals by information theoretic criteria. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 33(2):387–392, 1985.

[13] Peter H. Westfall and S. Stanley Young. *Resampling-Based Multiple Testing: Examples and Methods for p-Value Adjustment*. John Wiley & Sons, New York, 1993.

[14] Daniel B. Szyld. The many proofs of an identity on the norm of oblique projections. *Numerical Algorithms*, 42(3–4):309–323, 2006.

[15] Ilse C. F. Ipsen and Carl D. Meyer. The angle between complementary subspaces. *American Mathematical Monthly*, 102(10):904–911, 1995.

[16] Tsung-Han Chan, Kui Jia, Shenghua Gao, Jiwen Lu, Zinan Zeng, and Yi Ma. PCANet: a simple deep learning baseline for image classification? *IEEE Transactions on Image Processing*, 24(12):5017–5032, 2015.

[17] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. Extreme learning machine: theory and applications. *Neurocomputing*, 70(1–3):489–501, 2006.

[18] Pierre Dutilleul. The MLE algorithm for the matrix normal distribution. *Journal of Statistical Computation and Simulation*, 64(2):105–123, 1999.

[19] Benjamin Blankertz, Ryota Tomioka, Steven Lemm, Motoaki Kawanabe, and Klaus-Robert Müller. Optimizing spatial filters for robust EEG single-trial analysis. *IEEE Signal Processing Magazine*, 25(1):41–56, 2008.

[20] Sebastian Mika, Gunnar Rätsch, Jason Weston, Bernhard Schölkopf, and Klaus-Robert Müller. Fisher discriminant analysis with kernels. In *Neural Networks for Signal Processing IX: Proceedings of the 1999 IEEE Signal Processing Society Workshop*, pages 41–48, 1999.

[21] Jerome H. Friedman. Regularized discriminant analysis. *Journal of the American Statistical Association*, 84(405):165–175, 1989.

[22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[23] Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. SliceGPT: compress large language models by deleting rows and columns. In *International Conference on Learning Representations*, 2024.

[24] Matthew Barker and William Rayens. Partial least squares for discrimination. *Journal of Chemometrics*, 17(3):166–173, 2003.

[25] Paramveer S. Dhillon, Dean P. Foster, and Lyle H. Ungar. Eigenwords: spectral word embeddings. *Journal of Machine Learning Research*, 16:3035–3078, 2015.

[26] Xavier Pennec, Pierre Fillard, and Nicholas Ayache. A Riemannian framework for tensor computing. *International Journal of Computer Vision*, 66(1):41–66, 2006.

[27] Vincent Arsigny, Pierre Fillard, Xavier Pennec, and Nicholas Ayache. Log-Euclidean metrics for fast and simple calculus on diffusion tensors. *Magnetic Resonance in Medicine*, 56(2):411–421, 2006.

[28] Alexandre Barachant, Stéphane Bonnet, Marco Congedo, and Christian Jutten. Multiclass brain–computer interface classification by Riemannian geometry. *IEEE Transactions on Biomedical Engineering*, 59(4):920–928, 2012.

[29] Paolo Zanini, Marco Congedo, Christian Jutten, Salem Said, and Yannick Berthoumieu. Transfer learning: a Riemannian geometry framework with applications to brain–computer interfaces. *IEEE Transactions on Biomedical Engineering*, 65(5):1107–1116, 2018.

[30] Pedro L. C. Rodrigues, Christian Jutten, and Marco Congedo. Riemannian Procrustes analysis: transfer learning for brain–computer interfaces. *IEEE Transactions on Biomedical Engineering*, 66(8):2390–2401, 2019.

[31] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

[32] Valentin Duruisseaux, Jean Kossaifi, and Anima Anandkumar. Fourier neural operators explained: a practical perspective, 2025. arXiv:2512.01421.

[33] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015.

[34] Kai Keng Ang, Zheng Yang Chin, Haihong Zhang, and Cuntai Guan. Filter bank common spatial pattern (FBCSP) in brain–computer interface. In *Proceedings of the IEEE International Joint Conference on Neural Networks*, pages 2390–2397, 2008.